

Daniel Costea



Microsoft Agent Framework in .NET

Build production-ready AI agents
and multi-agent systems in C#

Microsoft Agent Framework in .NET

Build production-ready AI agents
and multi-agent systems in C#

Early Access Edition

Release 2026.09.18

Daniel Costea

Copyright and Legal Notice

Copyright © 2026 Daniel Costea

All rights reserved.

No part of this book may be reproduced, distributed, stored in a retrieval system, or transmitted in any form or by any means—electronic, mechanical, photocopying, recording, or otherwise—without the prior written permission of the copyright holder, except for brief quotations used in reviews or scholarly references.

Early Access edition

Release 2026.09.18

Published independently by Daniel Costea.

Complete book and early access updates:

<https://leanpub.com/agentframework>

This book is provided for educational and informational purposes only. Every effort has been made to ensure that the information in this book is accurate at the time of publication. However, software, APIs, services, pricing, product names, and capabilities can change frequently. The author makes no warranties, express or implied, and assumes no responsibility for errors, omissions, or outcomes resulting from use of the information in this book.

Examples in this book are provided as-is. Before using them in production, review and adapt them for your requirements, including security, privacy, compliance, reliability, performance, cost, and operational monitoring.

The complete source code for this book is available at:

<https://github.com/dcostea/AgentFrameworkBook>

Most code listings in this book include a code path, such as

`/ProjectName/File.cs`, which identifies the corresponding file in the source repository.

Microsoft, .NET, C#, Azure, Azure OpenAI, Semantic Kernel, AutoGen, OpenTelemetry, and other product or company names mentioned in this book may be trademarks or registered trademarks of their respective owners. Their use in this book does not imply endorsement by, affiliation with, or sponsorship by those owners.

Book Status and Release Notes

This is an Early Access edition of Microsoft Agent Framework in .NET.

This release includes Chapters 1–12. Chapters 13–16 are in development and will be published in future releases.

Purchasers will receive access to updated editions as new content, examples, corrections, and framework updates are published.

Current release

Release 2026.09.18

- Removed obsolete terminology, refreshed API usage, corrected code and setup issues, improved explanations.
- Includes Chapters 1–12
- Covers Agent Framework fundamentals, Microsoft.Extensions.AI, agents, model providers, context, tools, observability, and enterprise-ready patterns.

Planned content

The following chapters are planned for future releases:

- Chapter 13 — Iterative reasoning using the group chat pattern
- Chapter 14 — Conditional routing using the handoff pattern
- Chapter 15 — Orchestrating agents dynamically using AI skills
- Chapter 16 — Building custom orchestration using workflows

Planned chapter titles, ordering, and scope may change as Microsoft Agent Framework evolves and the book is refined.

Welcome

Thank you for purchasing Microsoft Agent Framework in .NET: Build production-ready AI agents and multi-agent systems in C#.

This book helps .NET developers bridge the gap between conventional application development and agentic AI. You'll learn how to use Microsoft Agent Framework—the unified SDK that brings together strengths from Semantic Kernel and AutoGen—to build practical, reliable AI agents and multi-agent systems in C#.

I wrote this book for intermediate and advanced C# developers who want to apply generative AI without first becoming AI specialists. Through clear explanations and hands-on examples, you'll learn to design agents that integrate models, tools, context, workflows, and enterprise concerns such as observability, reliability, and governance.

This book goes beyond simple LLM integration. It equips you with practical patterns for building reliable, scalable agentic applications that address real-world needs—whether you are developing enterprise systems, experimenting with personal projects, or creating new products.

Happy coding and AI exploration!

—Daniel Costea

.NET software engineer, AI specialist, technical author, and Microsoft MVP for AI and .NET

Table of Contents

PART 1: BUILD YOUR FIRST AGENT

1 From conventional code to agents

2 Building your first agent step by step

PART 2: CHAT CLIENTS

3 Building chat clients for model providers

4 Building responses and self-hosted clients

PART 3: AGENTS

5 Crafting agents from scratch

6 Equipping agents with tools

7 Adding agent memory with context and chat message history

8 Standardizing tools using MCP (Model Context Protocol)

9 Building enterprise-ready agents with ChatClient middleware

10 Building enterprise-ready agents with Agent middleware

11 Preparing reliable agents with observability and evaluation

PART 4: MULTI-AGENTS

12 Orchestrating agents using sequential and concurrent patterns

13 Iterative reasoning using the group chat pattern [coming soon]

14 Conditional routing using the handoff pattern [coming soon]

15 Orchestrating agents dynamically using AI skills [coming soon]

16 Building custom orchestration using workflows [coming soon]

From conventional code to agents

This chapter covers

- The rise of Generative AI and Large Language Models (LLMs)
- Key features of Agent Framework for AI integration
- Comparison with other AI tools such as LangChain and ML.NET
- Overview of Agent Framework architecture and components

Generative AI and Large Language Models (LLMs) are increasingly integrated into software development workflows. Here, we explore how Microsoft Agent Framework enables .NET developers to incorporate agents and agentic AI into their applications in the .NET ecosystem. Through intuitive analogies, we'll discuss the architecture and components of these tools so we can become proficient practitioners in the AI-driven era of programming.

1.1 Introducing Microsoft Agent Framework

Let's look at Agent Framework, an SDK for building AI agents and multi-agent workflows in .NET. Instead of wiring models, tools, and orchestration by hand, we use Agent Framework to define agents, connect them to LLMs and other services, and coordinate them into reliable workflows that solve real problems in our applications.

NOTE We'll use Agent Framework instead of Microsoft Agent Framework for brevity. The official name is still Microsoft Agent Framework.

Agent Framework brings together ideas from Semantic Kernel and AutoGen and turns them into a single, production-ready foundation for agentic AI. It provides first-class support for popular model providers, graph-based workflows, observability, checkpointing for long-running processes, and modern multi-agent patterns such as sequential and concurrent execution, group chat, and handoff orchestration. You'll see how AF fits into the broader .NET AI stack and how to start using it to add agents to your own applications.

NOTE Semantic Kernel is Microsoft's SDK for integrating LLMs into applications with plugins, memory, and connectors, and it focuses on production features such as state management and telemetry. AutoGen is a research-driven framework for building multi-agent systems that excel at

dynamic collaboration patterns like group chat and handoff. Agent Framework combines strengths from both, so you do not have to choose between experimentation and enterprise readiness.

1.1.1 The Rise of Generative AI and LLMs

Over the past few years, we've seen remarkable progress in artificial intelligence, which is helping solve complex challenges across many industries. From predictive analytics and natural language processing to computer vision and autonomous systems, AI is driving innovation at an unprecedented pace.

Earlier AI systems typically focused on classification and predictions, such as identifying spam, recommending products, or forecasting trends, by recognizing patterns in existing data. In contrast, generative AI systems are designed to create new content, such as text, images, music, and video, enabling developers to build applications with capabilities such as automated drafting, creative generation, and conversational interfaces.

Generative AI refers to a category of AI systems that can create new content such as text, images, voice, music, video, and code, based on patterns learned from existing data. This technology is redefining the boundaries of what's possible, from enhancing creativity in art studios to accelerating research in scientific labs.

At the heart of many generative AI applications are *Large Language Models* (LLMs), which act as core components enabling these capabilities. LLMs are transformer-based neural networks trained on large text corpora, typically billions to trillions of tokens. They predict the most likely next token given a context, enabling tasks like text generation, summarization, and code completion.

NOTE Although this book continues to use the term Large Language Models (LLMs) in a broad sense, it is important to note that the field increasingly distinguishes Small Language Models (SLMs) as a separate category. These models are sometimes considered to have fewer than 10 billion parameters, and we will do the same. They are fine-tuned for domain-specific tasks, local inference, and deployment efficiency.

The importance of generative AI and LLMs lies in their potential to augment human capabilities across many areas: enhancing productivity through task automation, creating personalized user experiences, accelerating innovation in fields like drug discovery and new materials, improving decision-making by analyzing large datasets, and supporting creativity in artistic work. These technologies are reshaping industries and addressing complex challenges in new ways.

Generative AI agents can provide strong context awareness and predict user needs, but integrating these models into production systems poses several challenges. We must manage diverse model providers and versions, handle rapidly evolving APIs, and ensure robust error handling and security. We also need to respect rate limits and cost constraints. Each provider may differ in supported features, operational limits, and update cycles, which adds friction to day-to-day development.

To address these integration and lifecycle management challenges and help developers harness the full power of generative AI, Microsoft introduced *Agent Framework*. This toolkit streamlines the process of connecting, orchestrating, and monitoring AI models from various providers, so we can focus more on building solutions and less on infrastructure complexity.

Get ready, because we're about to embark on a journey in which Agent Framework becomes our companion and organizing guide for design agents in real applications.

1.1.2 Why Agent Framework?

Agent Framework is an open-source SDK released in April 2026, intended to help .NET and Python developers design, deploy, and manage single- or multi-agent AI applications in enterprise environments. It provides a unified solution for orchestrating intelligent agents on a scale, supporting both creative, LLM-driven workflows and deterministic business processes.

IMPORTANT This book is limited to Agent Framework for .NET (C#), even though Agent Framework is also available for Python and Go.

Think of Agent Framework as a modern kitchen designed for a chef (the developer). In this kitchen, the chef doesn't have to build every tool from scratch or worry about the plumbing. Instead, there's a team of specialized cooks (AI agents) that can work together seamlessly. The framework provides the recipes (orchestration), universal appliance hookups (open standards), food safety inspectors (governance), and overhead cameras (observability) so we can create reliable, production-ready dishes (agentic AI applications).

ORCHESTRATION AND INTEGRATION

At its core, the framework manages and coordinates agents and tools.

Key features:

- **Unified Orchestration Engine:** The framework combines AutoGen's dynamic, multi-agent orchestration capabilities with Semantic Kernel's enterprise-grade foundations in a single SDK and runtime. This allows developers to use one interface for both creative, LLM-driven agent workflows and predictable, deterministic business processes.
- **Multi-agent collaboration patterns:** It supports sophisticated collaboration among agents through various patterns:
- **Open standards and interoperability:** The Agent Framework is built on an open-by-design philosophy, ensuring agents are portable and not locked into a single vendor.

NOTE Model Context Protocol (MCP) is an open standard that lets agents discover and call external tools and data sources through a common contract, rather than requiring custom wiring for each integration. Agent Framework uses MCP so tools can be shared across agents and even across runtimes that support the protocol. Agent-to-Agent (A2A) communication refers to agents talking directly to each other to exchange information or delegate work, often across different runtimes or platforms. Agent Framework uses structured A2A messaging so agents can collaborate safely while being still observable and governable. OpenAPI is an industry-standard way to describe HTTP APIs in a machine-readable format. When an API has an OpenAPI description, Agent Framework can import it and expose its operations as callable tools, letting agents interact with existing REST services with minimal glue code.

ENTERPRISE READINESS AND GOVERNANCE

These features are designed to make agents reliable, secure, and manageable in production environments. Key enterprise features include:

- **Comprehensive security and governance:** The framework integrates deeply with Microsoft's security stack to provide enterprise-grade controls.

- **Integrated observability:** Native support for OpenTelemetry (using the OpenTelemetry NuGet packages) provides deep visibility into agent behavior. Developers can trace and visualize every agent's action, tool call, and workflow step. This telemetry data flows into services like Azure Monitor and Application Insights, making agent systems as observable as traditional microservices.
- **Long-running durability:** Workflows are designed for reliability at scale. Agent processes can be paused, resumed, and recovered after interruptions using sessions, with built-in retry and error-handling logic. This keeps long-running tasks robust.
- **Human-in-the-Loop (HITL):** Tools can be marked as requiring approval for critical or sensitive operations. When invoked, an approval-required tool returns an approval request without executing. The application or hosting integration must route that request to an authorized human, obtain a decision, and submit the approval response before execution can continue.

NOTE OpenTelemetry is an open standard and set of libraries for collecting traces, metrics, and logs from applications consistently. Agent Framework uses OpenTelemetry so that agent invocations, tool calls, and workflow steps appear as standard telemetry that can be routed to Azure Monitor and Application Insights for analysis and alerting. Azure Monitor is Microsoft's unified observability platform for collecting and analyzing telemetry from applications and infrastructure across Azure, on-premises, and other clouds. Application Insights is the application performance monitoring part of Azure Monitor that focuses on web and service telemetry such as requests, dependencies, exceptions, and custom events. In practice, this means we can send Agent Framework telemetry to Azure Monitor and Application Insights and use familiar dashboards, queries, and alerts to understand and troubleshoot our agents.

Agent Framework simplifies building and orchestrating agentic AI applications by abstracting the complexity of messaging, orchestration, and coordination. Its modular architecture enables developers to integrate LLMs, tools, and deterministic components under a single runtime with minimal boilerplate. This approach lets developers focus on creating intelligent, context-aware agents instead of dealing with API variations and provider-specific implementations.

The following brief comparison highlights the key differences between Agent Framework, Microsoft.Extensions.AI (MEAI), LangChain, and ML.NET. It also acknowledges that other frameworks, such as CrewAI and LlamaIndex, offer alternative paradigms worth exploring. Our goal is not to prescribe a one-size-fits-all solution, but to empower developers to make informed decisions in an evolving landscape.

1.2 *Agent Framework and Other Frameworks*

Let's look at Agent Framework in relation to MEAI (Microsoft.Extensions.AI), LangChain, and ML.NET. Not because these are the only available solutions, but because they're among the most frequently discussed options for developers exploring AI integration and orchestration. While frameworks such as CrewAI, LangGraph, and LlamaIndex offer alternative paradigms worth considering, our goal here is to clarify the most common questions readers encounter in practice. By mapping the strengths, architectures, and best-use scenarios of these leading frameworks, we aim not to prescribe a single "right" choice, but to give you the context and criteria to navigate this rapidly evolving ecosystem.

1.2.1 Agent Framework and MEAI (*Microsoft.Extensions.AI*)

Before diving deeper into Agent Framework's capabilities, it's essential to understand the foundation on which it is built. Agent Framework (`Microsoft.Agents.AI`) does not operate in isolation; it is built on top of MEAI (`Microsoft.Extensions.AI`), a set of .NET libraries that provide unified abstractions for working with AI services. Understanding this architectural relationship is crucial to seeing how Agent Framework achieves flexibility, portability, and seamless integration with the .NET ecosystem.

MEAI is a foundational library that provides standardized abstractions for interacting with generative AI services from different providers. Released as part of the broader .NET ecosystem, it addresses a common challenge: each AI provider (OpenAI, Azure OpenAI, Anthropic, Ollama, Google Gemini, and more) typically has its own SDK, with unique APIs, patterns, and response formats. This fragmentation makes it difficult to write portable code or switch providers without significant refactoring.

`Microsoft.Extensions.AI` solves this problem by introducing core interfaces that standardize how .NET applications communicate with AI models:

`IChatClient`: The primary abstraction for chat-based interactions with large language models. Any provider that implements this interface can be used interchangeably in your code.

`IEmbeddingGenerator<TInput, TEmbedding>`: An abstraction for generating embeddings from text or other inputs, enabling semantic search and vector operations.

`AIFunction` and `AIFunctionFactory`: Support for tool-calling and function invocation, allowing models to request execution of .NET methods during a conversation.

By programming with these interfaces rather than concrete implementations, developers can change AI providers, compose middleware pipelines for caching, telemetry, and retries, and test their code with mock implementations, all without rewriting business logic.

Agent Framework is designed as a layered stack. At its core, it's built directly on top of MEAI, a set of .NET libraries that provide unified abstractions for connecting to various AI service providers. As shown in figure 1.1, `Microsoft.Agents.AI` leverages these abstractions, while MEAI connects to specific provider libraries such as OpenAI or Ollama. This design ensures modularity, flexibility, and easy integration across the evolving AI ecosystem.

Microsoft AI Stack Conceptual Layers

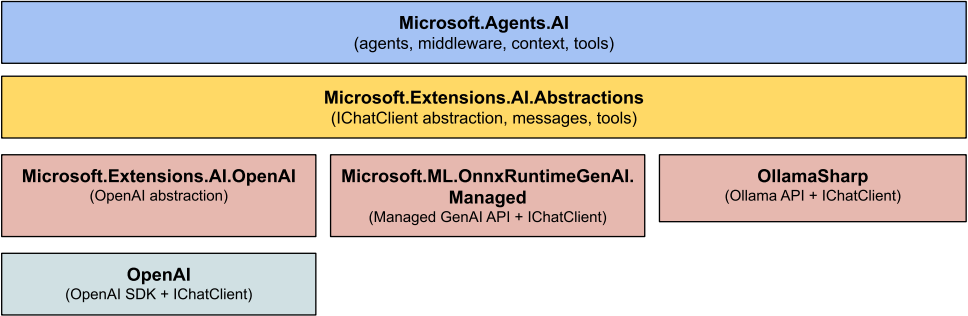


Figure 1.1 Simplified conceptual layers of the .NET AI stack.

`Microsoft.Extensions.AI.Abstractions` defines shared contracts for chat clients, messages, and tools. `Microsoft.Extensions.AI.OpenAI` adapts the OpenAI .NET SDK, while ONNX Runtime GenAI’s managed package and `OllamaSharp` provide their own `IChatClient` implementations. Above these contracts, `Microsoft.Agents.AI` provides model-backed agents, agent middleware, and context providers. Middleware packages and native runtime dependencies are omitted for clarity. MEAI is an independent foundational library that provides AI abstractions for .NET applications. It was designed to be used with or without an agent framework, as shown in the sample in listing 1.1.

Requirements (NuGet packages):

```
dotnet add package Microsoft.Extensions.AI.OpenAI
```

Listing 1.1 Prompting an LLM with a user prompt

```
using Microsoft.Extensions.AI; //❶
using OpenAI;
using OpenAI.Chat;

IChatClient chatClient = new OpenAIClient("your-openai-api-key")
    .GetChatClient("your-openai-model-name") //❷
    .AsIChatClient(); //❸

var prompt = """
You are an AI assistant controlling a robot car capable of performing
basic moves: forward, backward, turn left, turn right, and stop.
You have to break down the provided complex commands into basic
moves you know.

Complex command:
"There is a tree directly in front of the car.
Avoid it and then come back to the original path."
"""; //❹

var response = await chatClient.GetResponseAsync(prompt); //❺
```

```
Console.WriteLine(response.Text); // 6
```

- ❶ Imports the `Microsoft.Extensions.AI` namespace for creating chat clients
- ❷ Instantiates an OpenAI chat client
- ❸ Abstracts the chat client to `IClient`
- ❹ Declares a user prompt
- ❺ Runs the chat client with the provided prompt
- ❻ Prints the response text

This code efficiently uses `Microsoft.Extensions.AI` to interact with an LLM, handling user prompts and output responses.

IMPORTANT Never hard-code API keys. Use more secure solutions such as secrets management systems, environment variables, or vault secrets to protect sensitive credentials.

`Microsoft.Extensions.AI` (MEAI) can handle AI model interactions on their own, without `Microsoft.Agents.AI`. Beyond basic chat completions, it supports advanced features such as streaming responses, tools (function invocation), embedding generation, dependency injection, and middleware composition for cross-cutting concerns like logging, caching, and telemetry.

NOTE If all you need is to get responses to prompt from an AI provider, you likely don't need Agent Framework. `Microsoft.Extensions.AI` on its own is sufficient.

1.2.2 Agent Framework and LangChain

The market offers numerous frameworks for agentic AI development and orchestration. Popular choices include *LangChain*, *LangGraph*, *CrewAI*, *LlamaIndex*, and smaller, lightweight tools like *Ollama* or *Llama.cpp*. To understand Microsoft's direction, let's look at how Agent Framework compares with *LangChain*, one of the most established frameworks in the Python community.

LangChain is a Python-centered ecosystem for integrating language models, tools, retrieval, and agents. It supports flexible agent and multi-agent patterns (including subagents, routing, handoffs, and parallel execution) often using *LangGraph* for stateful, graph-based orchestration. Microsoft Agent Framework provides first-class .NET APIs and Microsoft-oriented integration options, making it a natural choice for teams building agentic applications in the .NET ecosystem.

Agent Framework is built with first-class support for C# and the .NET ecosystem, making it a natural fit for .NET developers. It provides facilities for enterprise applications, a modular approach, and advanced multi-agent orchestration. Agent Framework integrates with services such as OpenAI and Anthropic, as well as self-hosted models such as Mistral, Phi, and the LLaMA family, via Ollama and ONNX providers.

It's worth noting that both frameworks are under active development, and their features and capabilities may change over time. The choice between *LangChain* and Agent Framework (or similar frameworks) depends on your project requirements, preferred programming languages, and the desired level of integration and flexibility.

1.2.3 Agent Framework and Microsoft ML.NET

Since we're talking comparisons, let's look at another frequently asked question: "What's the difference between Agent Framework and ML.NET?" ML.NET allows you to run some LLMs locally, but it lacks the flexibility and composability that Agent Framework supports for complex AI orchestration scenarios. Agent Framework excels at integrating and orchestrating various AI services, including

large language models, but it doesn't provide the extensive machine learning capabilities and AutoML features that ML.NET offers. This complementary relationship allows developers to use both frameworks together, choosing the most appropriate tool for specific AI tasks within their applications.

1.2.4 A Glimpse into Agent Framework Code

Now that we understand what Agent Framework is, let's continue with a simple example to show how straightforward it is to work with this powerful tool.

We'll create a chat-completion client and use it to initialize an agent. Then we'll send a prompt to the agent and print the response to the console.

The sample code in listing 1.2 illustrates how to use Agent Framework to create an AI agent that solves a very simple task, such as breaking down a complex action into a predefined list of basic actions. More precisely, we use a large language model to act as a robot car assistant, converting high-level actions, such as avoiding obstacles or performing complex movement patterns, into a sequence of basic movements, such as turning left, turning right, going forward, going backward, or stopping. We use these basic movements because the robot car is limited to them. Essentially, we aim to simulate complex actions through sequences of simpler ones.

Requirements (NuGet packages):

```
dotnet add package Microsoft.Agents.AI.OpenAI
```

Listing 1.2 Prompting the LLM with a user prompt using an agent

```
using Microsoft.Agents.AI;
using OpenAI; //❶
using OpenAI.Chat; //❶

var chatClient = new OpenAIClient("your-openai-api-key")
    .GetChatClient("your-openai-model-name"); //❷

AIAgent agent = chatClient.AsAIAgent("""
    You are an AI assistant controlling a robot car capable of performing
    basic moves: forward, backward, turn left, turn right, and stop.
    You have to break down the provided complex commands into basic
    moves you know.
    """)
); //❸

var prompt = """
    Complex command:
    "There is a tree directly in front of the car.
    Avoid it and then come back to the original path."
    """; //❹

AgentResponse response = await agent.RunAsync(prompt); //❺
Console.WriteLine(response.Text); //❻
```

❶ Imports the OpenAI namespaces for creating clients and agents

❷ Instantiates an OpenAI chat client

❸ Instantiates an OpenAI agent

❹ Declares a user prompt

❺ Runs the agent with the provided prompt

❻ Prints the response text

We know that large language models can sometimes hallucinate a response, and some older models may not respond at all. Here's a response, hopefully similar to what you'll see in the console:

To execute the command, break down the required actions into the following basic steps:

1. ****Stop****: Come to a complete stop to assess the situation.
2. ****Turn Right****: Rotate the car to the right to start moving around the tree.
3. ****Forward****: Move forward while passing alongside the tree.
4. ****Turn Left****: After getting past the tree, turn left to re-align with the original path.
5. ****Forward****: Continue moving forward to return to the original path.
6. ****Turn Left****: Align back in the direction of the original path.
7. ****Forward****: Resume moving forward on the original path.

Make sure to adjust distances and turning angles as needed based on the car's proximity to the tree and the width of the path.

Modifying a prompt can dramatically change how a large language model responds, refining prompts is often a trial-and-error process.

This simple but powerful example shows how Agent Framework can create an AI-powered app with just a few lines of code. It highlights how easy it is to add AI capabilities to your projects and sets the stage for the more detailed explorations later in this chapter.

To deepen your understanding, go back to the sample and change the `Complex command` part of the prompt using other complex commands, such as `Navigate around a parked car`, `Perform an evasive maneuver`, or `Make a U-turn safely`. Testing different scenarios will help you explore how Agent Framework handles diverse tasks. For more inspiration, try creating your own complex commands.

Now imagine you can parameterize prompts for dynamic, reusable content in code, connect seamlessly to different AI providers, and invoke local or external tools as agent actions. With Agent Framework, you can intercept and control agent flow at any stage of the lifecycle and orchestrate complex scenarios in which multiple agents collaborate toward a shared goal. The possibilities are vast, and this is only the beginning of what Agent Framework can do.

1.3 How Agent Framework Works

Understanding Agent Framework's functional architecture is essential to understanding how it works. It's a bit like understanding how the human body works.

1.3.1 Human Body Analogy

To showcase how Agent Framework works, consider this analogy. Imagine waking up with no memories of the past, but with basic functions such as walking, breathing, and seeing. Most importantly, you can reason and learn. Likely, the first thing you do is observe the environment and start gathering information to build context and reconstruct your short-term memory. You need to be ready to react to whatever happens next. You do all these actions using human skills, but let's explore how human systems map to Agent Framework components by looking at figure 1.2.

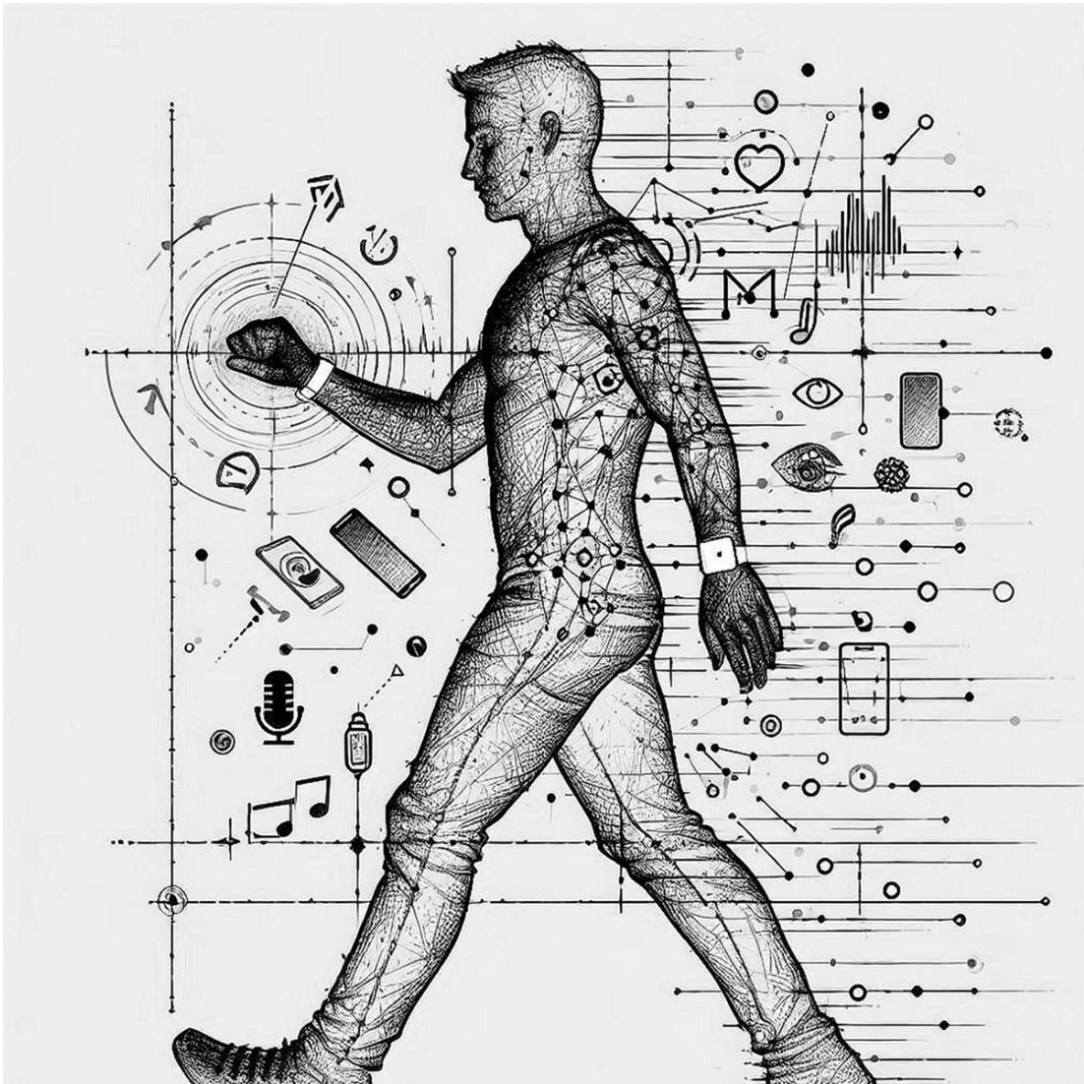


Figure 1.2 This image compares human cognitive processes with Agent Framework's architecture, showing how sensory systems (such as eyes and ears) gather data, how the brain processes it and forms memories, and how the mind filters irrelevant stimuli while focusing on important details—simulating planning and adaptation. (Image generated using Bing Copilot)

Let's see how human body skills map to Agent Framework components:

- **Sensing (Providers):** Like human senses (sight, hearing, touch), Agent Framework uses providers and connectors to interact with data sources and inputs. A vision connector works like eyes, ears, and skin, and can flexibly interface with domains far beyond human perception (e.g., system health, API events).
- **Nervous System (Middleware and Orchestration):** The middleware pipeline transmits and processes agent actions, much as the nerves relay and coordinate impulses. Agents interpret incoming signals, execute logic, and maintain communication among tools, services, and human users.

- Muscle coordination (tool invocation and actions): Tools and external actions act like muscles, translating agent decisions into real-world effects—calling APIs, updating databases, or triggering workflows—much like muscle fibers respond to neural commands.
- Brain Reasoning (Agent Collaboration and Decision Making): Individual agents reason independently. But Agent Framework supports multi-agent workflows (like distributed brain regions): agents collaborate, delegate subtasks, and assemble goal-oriented plans using graph-based orchestration (workflow graphs), comparable to distributed reasoning in the brain.
- Planning (Workflow Engine): The workflow engine and orchestrators serve as the brain's executive center, analyzing available tools and agents, sequencing complex plans, and adapting paths in real time, like how the prefrontal cortex handles strategic behavior and problem-solving in humans.
- Selective focus (middleware, observability): Middleware components can filter, intercept, and regulate agent actions, akin to selective attention in the human brain, ensuring that only relevant information passes through and discarding noise or inappropriate content.
- Memory (state management, context providers): Agent sessions and context providers act like working and long-term memory, storing conversation history, task context, and prior results so agents maintain awareness over extended interactions.
- Information Exchange (Request/Response & Integration): Prompt-response handling and integration across MCP (Model Context Protocol), A2A (Agent-to-Agent), and OpenAPI enable agents to seek, process, and exchange information with external systems, much like humans do through observation, questioning, and dialogue.
- Thinking (Model Clients & Reasoning): Agents' use of model clients for LLMs and reasoning functions echoes human cognition: pattern recognition, creative association, and adaptive learning.

This analogy has its limitations, but it offers a quick way to familiarize yourself with Agent Framework concepts. In practice, Agent Framework's capabilities are more structured and programmatic, designed to integrate seamlessly into software applications and provide consistent, scalable, AI-driven functionality.

1.3.2 Primary Components of an Agent

A production-ready agent relies on a set of core building blocks that support its ability to reason, make decisions, and take actions. These building blocks (see figure 1.3) are organized into three primary resource groups:

- Context, which provides access to relevant memory or state.
- Tools which offer external capabilities such as APIs or integrations.
- A model provider, which is responsible for the underlying AI reasoning via a chat client interface.

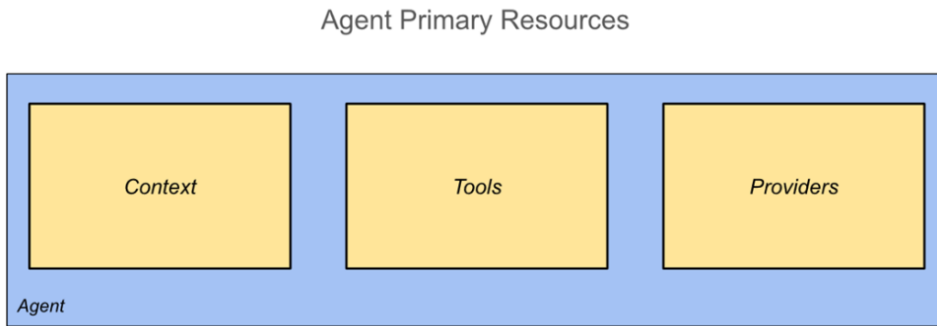


Figure 1.3 Agent components showing context, tools, and providers as core resource modules.

An agent is composed of three primary resource categories: **context** that informs decisions, **tools** that enable actions, and **providers** that supply pluggable runtime capabilities such as models, storage, and retrieval.

1.3.3 Stateless Agent Architecture

Agent Framework's architecture is designed to efficiently manage natural language queries through a system of interconnected components.

A minimal but functional workflow follows these steps:

1. Prompt input: The *Agent* receives a prompt that defines the user's goal and initial instructions.
2. Model provider prompt: The *Agent* sends a prompt through the *Chat client* to the model *Provider* to obtain reasoning or generated text.
3. Response handling: The *Agent* composes the result by integrating model output.
4. Result formulation: The *Agent* returns the result to the caller, completing the request-response cycle.

We will call this simplified agent architecture the Stateless Agent (figure 1.4).

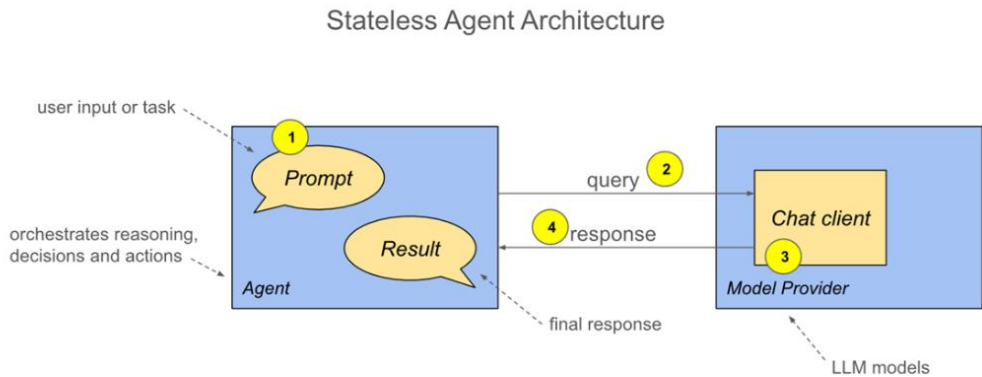


Figure 1.4 The Stateless Agent Architecture diagram shows how an agent queries a model provider via a chat client to produce a result.

This lightweight design is ideal for single-turn tasks or scenarios where context and memory aren’t needed.

NOTE A stateless interaction does not retain conversation state between requests. Each request is processed independently, using only the current input and any context supplied for that invocation.

This is a compact flow, ideal for explaining basic concepts like text generation, reasoning, and response handling. In the next section, we’ll discuss a more complex flow.

1.3.4 Standard Agent Architecture

A real-world agent needs a few more key ingredients to fulfill its goals. The diagram in figure 1.5 presents the core components of a standard agent architecture.

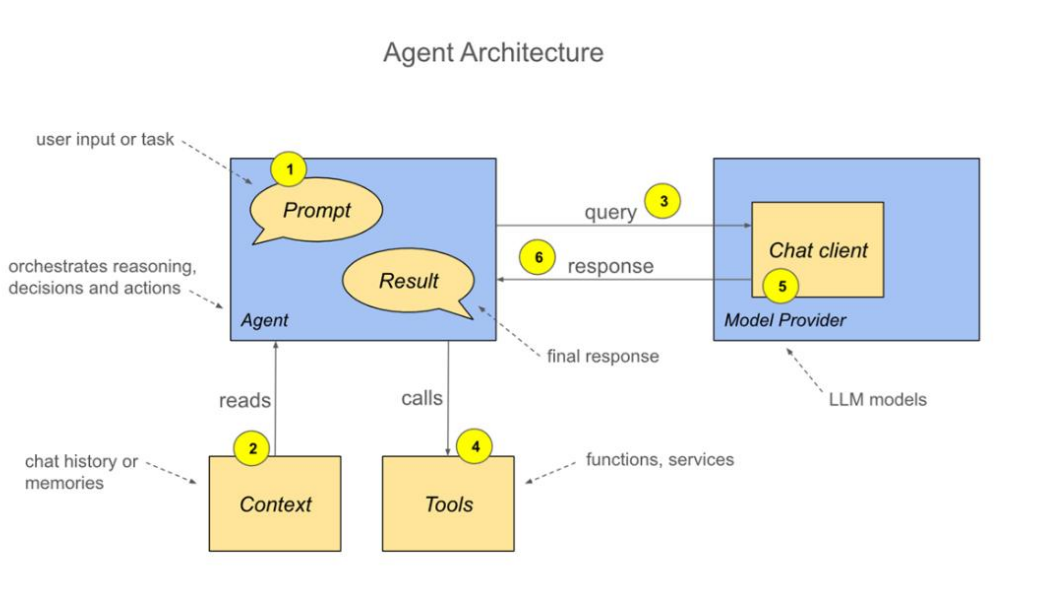


Figure 1.5 The Agent Architecture diagram shows how an Agent enriches a Prompt with Context, queries a Model Provider via a Chat client, optionally calls Tools, and iterates until it produces a Result.

The functional flow follows these steps:

1. **Prompt input:** The *Agent* receives a *Prompt* that defines the user's goal and initial instructions.
2. **Context enrichment:** The *Agent* reads *Context* (memory, message history, or knowledge) to ground the request before reasoning.
3. **Model provider prompt:** The *Agent* sends a prompt through the *Chat client* to the *Model Provider* to obtain reasoning or text generation.
4. **Tool execution:** If needed, the *Agent* calls external tools (APIs, databases, actions) to gather facts or perform operations. The *Agent* updates the context with the tool result and loops back, repeating this process until it has enough information to complete the task.
5. **Response Handling:** The *Agent* composes the final *Result*, integrating the model output and any tool-derived data.
6. **Result Formulation:** The *Agent* returns the *Result* to the caller, completing the request-response cycle.

This is a standard flow, ideal for practical agents, but an agent can still include more advanced components such as middleware, sessions, observability, and multi-agent collaboration.

1.3.5 Enterprise-Ready

Agent Framework delivers key enterprise-level features:

- **Middleware** acts as an intermediary layer between your agents and handles key operational tasks such as logging, security checks, and usage tracking. Think of it as a central control room that monitors and manages every interaction without requiring you to rebuild your agents from scratch. Instead of hardcoding security or compliance rules into each agent, middleware lets you add, update, or remove these checks dynamically. This makes it easier for different teams

(developers, security, and operations) to work together, ensuring that all agents follow the same rules and standards while keeping the system flexible and maintainable.

- *Chat and AI Providers* ensure that each conversation or task your agent handles retains everything that happened before. Even if the system restarts or pauses, the agent picks up where it left off, maintaining full context across hours or even days. This memory isolation also means that different conversations don't accidentally interfere with each other, keeping workflows organized and reliable.
- *Sessions* isolate each conversation or task, preventing state from one workflow from affecting another.
- *Chat-history providers* load and store conversation messages, while *AI context providers* add relevant context such as memories, retrieved documents, or dynamic instructions. This state is not automatically preserved after a process restart: persist and restore session state or use durable storage. For workflows, enable durable checkpointing and explicitly resume a saved checkpoint.
- *Checkpointing* is like creating saving points in a video game. The framework automatically saves progress at key moments so that if something goes wrong—such as a timeout or failure—or if it needs to wait for human approval, the agent can resume from the last checkpoint instead of starting over. This makes long-running processes more dependable and user-friendly.
- *Observability* means you can see what your agents are doing at every step. Using tools like OpenTelemetry, the framework tracks every decision, tool use, and interaction in real time. Teams can visualize this data through dashboards, making it easy to spot problems, understand costs, and improve performance. This transparency brings AI agents into line with how modern software systems are monitored and managed.
- *Governance* brings together role-based access, human approval workflows, secure network setups, and detailed audit trails that record every agent's decision. This helps teams keep control and meet regulatory standards. These measures let leaders verify that agents operate safely and consistently with compliance requirements.

We'll take a step-by-step approach to building intelligent chatbots and agents. Along the way, we'll add more features and explore how the Agent Framework orchestrates AI agents to tackle complex tasks. Then we'll dive deeper into its capabilities and put generative AI to work in your applications.

Conclusion

This book is for .NET developers who want to build agentic AI systems, not just call LLM APIs. We assume you're comfortable with C#, async programming, and basic software development, but not necessarily with AI or ML theory.

It will be especially useful if you work in enterprise or cloud-hosted systems and need to integrate agents into existing services, APIs, and workflows. Architects, tech leads, and senior engineers can use it to design agentic features that are observable, secure, and production ready. Curious developers who mainly build side projects are also welcome, as long as they're ready to work through practical, code-first examples.

Summary

- What generative AI, LLMs, and agentic AI mean, and how they differ from traditional machine learning.
- Why production integration is difficult: fragmented providers, reliability, security, governance, and lifecycle complexity.

- What Agent Framework is and how it supports single- and multi-agent applications in .NET.
- How Agent Framework builds on Microsoft.Extensions.AI to stay provider-agnostic across multiple model backends.
- Where Agent Framework fits compared to LangChain and ML.NET in the broader AI and .NET ecosystem.
- The core building blocks of agents are tools, context, providers, and how they map human cognitive functions.
- Key orchestration patterns include sequential, concurrent, group chat, and handoff patterns, represented as workflows and graphs.
- Enterprise capabilities for agents are:
 - observability through OpenTelemetry, Azure Monitor, and Application Insights;
 - durability through configured session/checkpoint storage;
 - human-in-the-loop execution through approval requests and application-provided approval handling.

2

Building your first agent step by step

This chapter covers

- Building a simple AI-powered console application with Agent Framework
- Obtaining and securing API keys for OpenAI and Azure OpenAI
- Creating a basic console application for prompting a GPT model
- Creating your first AI agent

Let's dive into practical applications with Agent Framework and see how to build AI-powered apps that use large language models. We'll start with a simple console app that interacts with OpenAI's GPT models, then move on to an agent that can control a robot car. You'll also learn how to obtain and secure API keys, set up your development environment, connect Agent Framework to GPT models, and expose native (C#) functions as tools.

2.1 A Robot Car Story

Imagine a robot car that can sense fire danger and escape independently. While this concept may sound like something out of science fiction, it represents a real pet project that sparked the journey behind this book. The idea of creating an autonomous vehicle capable of detecting and responding to potential fire hazards highlights the fascinating intersection of robotics, sensor technology, and machine learning. This project was built on a very low budget (less than 100 USD), with a focus on exploring how these technologies could run in the .NET ecosystem on an IoT (Internet of Things) device.

2.1.1 A Fire-Detecting Robot Car

The story began several years ago when I decided to build a robotic car as part of my academic exploration of machine learning. This project was meant to power demonstrations for upcoming speeches, not to be a commercially viable product. The primary function of the robot car is to read its surroundings and react to potential fire hazards.

I based the project on a low-cost model, using a Raspberry Pi (a low-power mini-PC with an ARM architecture) and a motor controller add-on board. The motor controller drives the wheels through a software API (Application Programming Interface). While it may not be sophisticated, this setup lets the car move forward and backward, turn left and right, and stop. To improve its awareness of its surroundings, I added a camera and sensors for distance, light, infrared, and temperature.

Next, I developed software to control the robot car and trained a machine learning model to classify inputs into two states: *Fire* and *Safe*. The Fire state indicates danger, while the Safe state signifies the absence of such a threat.

IMPORTANT You don't need a physical robot car to build an agent. The robot car is a visual aid to help illustrate concepts. Your agent will operate entirely in the digital realm.

With this machine learning model in place, the car could make predictions from sensor readings and react accordingly, showing the potential of low-cost robotics to address real-world challenges.

2.1.2 Challenges in Programming Complex Movements

Initially, I programmed basic steps supported by the robot car API into a step-by-step sequence that simulates an escape maneuver. As I assembled these steps, I immediately realized a limitation: this approach doesn't scale well to new movement patterns. Each new movement would require programming a new sequence of steps. An escape maneuver can be coded manually, but more complex patterns—following a circular path, zigzagging, imitating a dance, or avoiding obstacles—quickly lead to more hard-coded, user-defined step sequences. Figure 2.1 suggests how easy it is for a robot car to do simple moves, and how difficult it can become to do complex ones.

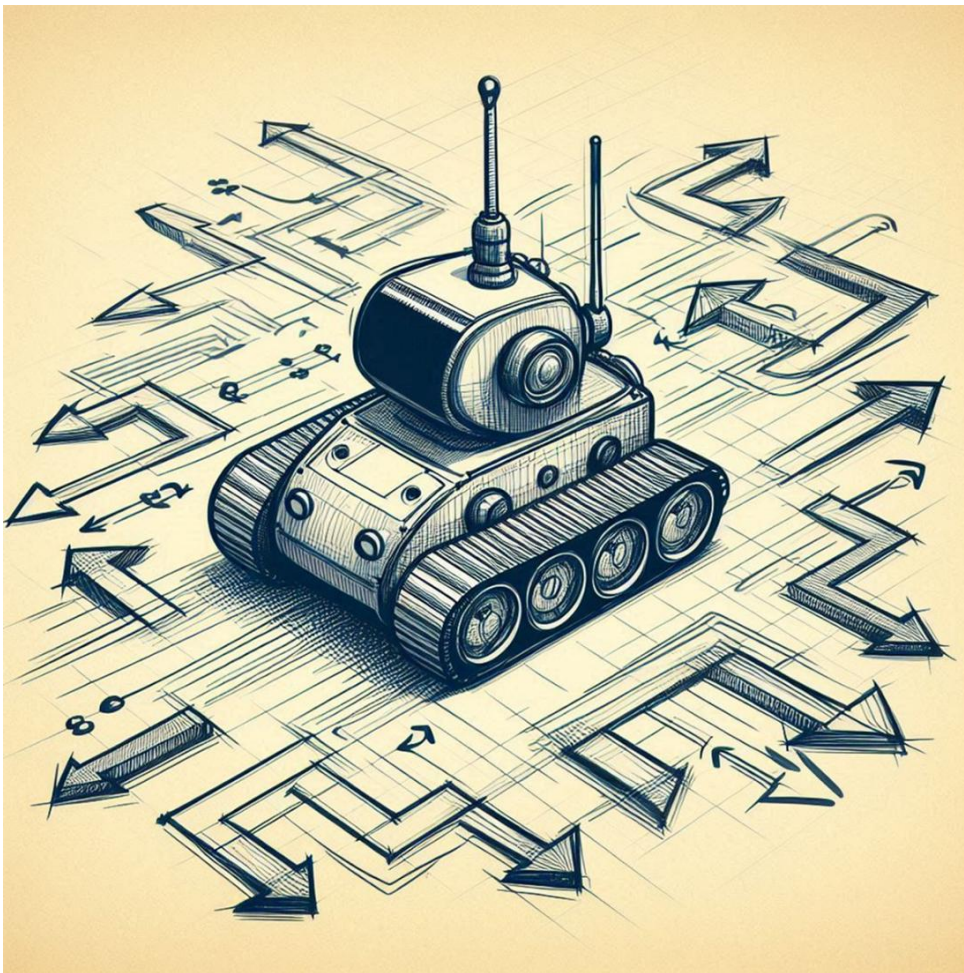


Figure 2.1 The robot car understands five basic commands: forward, backward, left turn, right turn, and stop. Any complex instruction must be broken down into a sequence of these fundamental movements, allowing the vehicle

to execute more sophisticated maneuvers by combining its core functions.

Even though I recognized a key element of this challenge, the semantic link between simple and complex commands, I lacked an elegant solution at the time and had to abandon my efforts to automate motion patterns until a better opportunity arose.

2.1.3 *Revisiting the Problem with Generative AI*

A few years later, the emergence of generative artificial intelligence reawakened my interest in solving the complex movement decomposition problem. Through experimentation with ChatGPT, I discovered LLMs' remarkable ability to translate complex, and thus more abstract, commands into executable sequences, a capability rooted in their functional nature. This inspired me to use generative AI as an intermediary layer between the user and the robot's API to create motion models on the fly.

EXPERIMENTATION WITH CHATGPT

Eager to explore this possibility, I experimented with ChatGPT to break down complex movements into basic commands that the robot car API could understand. Here are the findings, which you can also verify using ChatGPT or Copilot.

User prompt:

You are an AI assistant controlling a robot car capable of performing basic moves: forward, backward, turn left, turn left, turn right, and stop. You have to break down the complex command "There is a tree directly in front of the car. Avoid it and then return to the original path." into basic moves you know.

The prompt is fluent and concise, written in the style we're used to with ChatGPT, so the answer should be coherent and to the point.

ChatGPT response:

Let's break down the complex command into a series of basic moves:

1. ****Stop****: If the car is moving, first make it stop.
2. ****Turn Left****: Turn the car left to start moving around the tree.
3. ****Move Forward****: Move forward a bit to clear the tree.
4. ****Turn Right****: Turn the car right to straighten it out.
5. ****Move Forward****: Continue moving forward to clear the tree completely.
6. ****Turn Right****: Turn the car right again to start moving back towards the original path.
7. ****Move Forward****: Move forward a bit to align the car with the original path.
8. ****Turn Left****: Turn the car left to return to the original path.
9. ****Move Forward****: Continue moving forward along the original path.

This way, the car can avoid the tree and get back on its original path!

If you have any specific parameters like distances or turning angles, I can refine the steps further.

The result was good enough. Much of the conditional logic that had instructed the robot car to move in complex patterns was now delegated to the AI.

Let’s run it again!

ChatGPT response:

Absolutely. Let’s break it down:

1. ****Turn right**** to start avoiding the tree.

2. ****Move forward**** for a short distance to bypass the tree.

3. ****Turn left**** to align parallel to the original path.

4. ****Move forward**** to pass the tree.

5. ****Turn left**** again to face the original path.

6. ****Move forward**** to return to the original path.

7. ****Turn right**** to get back on the original path.

8. ****Move forward**** to continue forward.

9. ****Stop**** to stop the car.

Feel free to ask if you need further assistance or have other tasks for the robot car!

Whether the model decides to start by turning right or left, the maneuver is pretty much the same. This shows how an LLM can introduce random variation in its responses.

Sometimes, it’s much easier to grasp these steps when you can visualize them, so I created a simple diagram. See figure 2.2 for a clearer picture of the robot car’s journey around the tree.

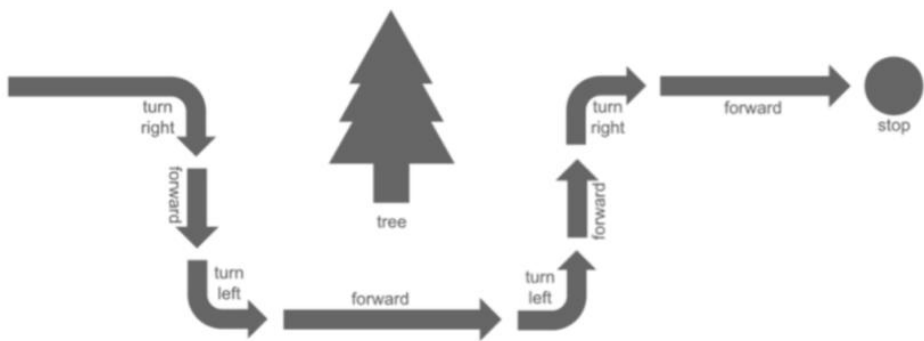


Figure 2.2 The LLM generates a textual response that details the car robot's movements around the tree, but visualizing this sequence as a diagram makes the detour easier to understand.

These steps clearly describe a path around the tree to avoid an imminent collision.

THE NON-DETERMINISTIC NATURE OF LLMs

The result from ChatGPT was promising, delegating much of the conditional logic for complex movement patterns to AI. Resending the same prompt reveals a well-known limitation: LLMs are non-deterministic and can produce varied outputs for identical inputs. While this flexibility is valuable for creative applications, machine-to-machine communication needs more predictable responses.

This functional decomposition stems from key architectural features of LLMs:

- *Task decomposition through pattern recognition:* LLMs can semantically interpret instructions such as `avoid the tree` and map them to sequences of actions. The model captures the intent behind the command and produces step-by-step outputs grounded in learned semantic patterns. This creates a key link between complex commands and basic moves.
- *Contextual Understanding Through Attention:* Although LLMs are inherently stateless, transformer-based attention mechanisms let them approximate conversational memory. By attending to earlier tokens within the context window, they can adapt later reasoning or actions based on prior information.
- *Structured Output with Prompt Engineering:* Through carefully designed prompts, LLMs can be guided to produce structured, machine-readable outputs such as JSON or XML. Methods like the *PACT framework* help enforce schema alignment and output consistency.

This example shows functional translation from natural language into structured data, helping integration with robotic control systems. With this paradigm shift, large language models are no longer simple text generators; they become functional components that link human instructions to system execution.

Despite discovering these LLM capabilities, I still wasn't convinced to return to my old problem of decomposing complex motions into basic moves. There were no .NET libraries available yet, and working directly with the OpenAI API wasn't appealing.

2.1.4 The Spark that Ignites My Robot Car

A few weeks later, during a discussion with a colleague who shared my passion for .NET, I was asked whether I had started using Microsoft Semantic Kernel for C#. I had seen a few posts about this new tool, but that conversation sparked a deeper exploration. What I discovered transformed my approach to the robot machine project: Semantic Kernel provided the missing link between the semantic understanding of generative AI and the deterministic execution of the system, which is what I needed to solve the motion decomposition problem.

Since Semantic Kernel joined forces with AutoGen to create the newer, more powerful Agent Framework, we'll use Agent Framework for all subsequent examples.

2.2 Acquiring the API Keys

Let's look at how to transform the original ChatGPT prompt, through a series of incremental steps, into an agent capable of communicating semantically with the robot-car API commands. It's worth noting that working with advanced GPT models is not as straightforward as you might assume. These powerful language models aren't freely accessible; instead, they're hosted as services provided by OpenAI or Azure OpenAI services. Access typically comes with usage-based and operational costs, which can vary by usage and by the service plan you choose.

2.2.1 How to Get an OpenAI API Key

OpenAI is a pioneer in generative AI, known for its GPT family of language models. Many of the innovations we've seen came first from OpenAI. To use OpenAI's capabilities, you need an API key, which lets you integrate AI features into your applications. Next, we'll walk through how to get an OpenAI API key:

1. Go to <https://platform.openai.com> and sign in or create an OpenAI Platform account.
2. Create a project or select an existing project.
3. Configure API billing for the project or organization if your account requires it.
4. Open the project's API Keys page and select Create new secret key.
5. Give the key a recognizable name, select appropriate permissions, and create it.
6. Copy the key immediately. OpenAI displays secret value only once.
7. Store the key securely, for example, in .NET User Secrets during local development or in a managed secret store such as Azure Key Vault for deployed applications.

WARNING Never commit an API key to source control, embed it in client-side applications, or include it in documentation. Use a separate key for each application or environment and revoke any key that may have been exposed.

2.2.2 How to Get an Azure OpenAI API Key

Azure OpenAI provides OpenAI models through Azure services and Microsoft Foundry. To use it, create or select an Azure subscription, create an Azure OpenAI or Microsoft Foundry resource, and deploy an available model. Then obtain the resource endpoint and credentials, such as an API key or Microsoft Entra ID authentication details, for your application.

Most Azure OpenAI models do not require a general service-access application. Before creating a deployment, check the selected model's current regional availability, subscription quota, and any limited-access requirements. Some models, or requests to modify guardrails or abuse-monitoring settings, require separate approval.

2.2.3 Securing API Keys

In .NET, you can store API keys securely using several best practices and options:

ENVIRONMENT VARIABLES

1. Store API keys as environment variables on the server or development machine.

For Windows:

```
setx OPENAI_API_KEY "YOUR_OPENAI_API_KEY"
setx OPENAI_MODEL_ID "YOUR_OPENAI_MODEL_ID"
```

For Linux:

```
export OPENAI_API_KEY="YOUR_OPENAI_API_KEY"
export OPENAI_MODEL_ID="YOUR_OPENAI_MODEL_ID"
```

2. Access environment variables in code by reading the corresponding OpenAI key.

```
var apiKey = Environment.GetEnvironmentVariable("OPENAI_API_KEY");
var modelId = Environment.GetEnvironmentVariable("OPENAI_MODEL_ID");
```

The API key is retrieved from environment variables. We may need to open a new terminal and restart any already-running IDE before launching the program.

SECRET MANAGER

1. Use the Secret Manager tool provided by .NET for development only. This tool helps store secrets outside the application's folders, preventing accidental commits to Git or other repositories. Secret Manager stores secrets unencrypted in a JSON file within the user's profile folder, making it suitable for local development but not for production, so never use it in production.
2. Store secrets locally using the `dotnet` tool.

```
dotnet user-secrets init
dotnet user-secrets set "OpenAI:ApiKey" "api-key"
dotnet user-secrets set "OpenAI:ModelId" "model-id"
```

`init` creates the secret file, `set` sets the values.

3. Build and access the secrets through the `IConfiguration` interface.

```
var configuration = new ConfigurationBuilder()
    .AddUserSecrets<Program>()
    .Build();
var apiKey = configuration["OpenAI:ApiKey"];
var modelId = configuration["OpenAI:ModelId"];
```

The API key is programmatically retrieved from the local secrets store.

More techniques to deal with API keys

We've covered several methods for handling API keys, but production environments often call for more advanced techniques, including:

Azure Key Vault: A cloud service for securely storing and accessing secrets, including API keys.

AWS Secrets Manager: A service that helps protect access to your applications, services, and IT resources without needing to hardcode sensitive information.

Database Storage: Store encrypted API keys in a secure database and access them only when needed.

Containerized secrets management: Tools such as Docker Secrets or Kubernetes Secrets for managing sensitive data in containerized environments.

Advanced API key management techniques are critical in production environments because they enhance security through encryption and access control, improve scalability through centralized management, ensure compliance with audit trails, and integrate seamlessly with DevOps practices. These methods significantly reduce security risks in large-scale, complex applications.

While these solutions offer robust security for API key management in production settings, a detailed discussion of how to implement them is beyond the scope of this book. For most applications, the methods discussed earlier should provide sufficient security when implemented properly.

2.3 Your First AI Agent

Ready to return to the problem of decomposing complex movements into basic movements that the robot car API can understand? Agent Framework makes it easy to integrate into a .NET console application. We'll start with a simple example and gradually enhance it to create a dynamic, interactive AI assistant for a robot car. You'll have an agent that can break down complex movements into basic commands, and you'll be eager to expand its capabilities even further.

Open Visual Studio Code to create a new console application and add the necessary NuGet packages. If you prefer a more full-featured IDE, you can use Visual Studio instead. Open a terminal and run the commands shown in listing 2.1.

NOTE The examples were tested with `Microsoft.Agents.AI 1.21.0` and `Microsoft.Extensions.AI 10.10.0`. Commands without `--version` install the current release, which may require code changes.

Listing 2.1 Creating a new console application – Terminal

```
dotnet new console -n RobotCarAssistant ❶  
cd RobotCarAssistant ❷  
dotnet add package Microsoft.Agents.AI.OpenAI ❸  
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
```

❶ Creates a new console application

❷ Navigates to the newly created folder and get ready to add some packages

❸ Adds Agent Framework library dependency

This approach ensures your project is set up correctly, with the required dependencies for working with Agent Framework and securely managing secrets during development.

2.3.1 Crafting a Good Prompt

I know you're eager to send your first prompt to an LLM from your C# application, but I encourage you to have a bit of patience. When working with generative AI, nothing is more important than crafting a well-designed prompt. Before jumping into implementation, it's beneficial to experiment with tools like ChatGPT, Copilot, or any GPT-powered sandbox to refine your prompts. As shown in section 2.1.3 with the robot car AI assistant prompt, an unrefined prompt can lead to inconsistent or suboptimal results. Structuring and improving prompts are essential to guide the model toward accurate, reliable responses.

Structuring a prompt into distinct parts enhances a model's ability to understand and execute tasks by breaking the prompt into specific components, making it easier to interpret and generate effective responses. It also helps separate the original input into key elements such as *user messages*, *system messages*, and *chat message history* for maintaining context. Message types and chat history are covered in more detail in later chapters.

PACT (Persona-Action-Context-Template) Prompt Engineering Framework

To create effective prompts, I use the PACT framework, my adaptation of the Role-Task-Format (RTF) framework. PACT is for Agent Framework because it aligns with its modular, structured approach. The framework consists of four components:

Persona (Who): Defines the role or identity the model should assume when generating a response. For example, you might ask the model to act as a teacher, a journalist, or an AI assistant.

Action (What): Specify the task or action you want the model to perform. Clear instructions guide its behavior (for example, "write an article" or "break down a command").

Context (With What): Provides relevant background information or details that help the model generate accurate, contextually appropriate responses.

Template (How): Defines the desired structure or format of the response, such as JSON arrays, tables, or plain text.

This structured approach makes PACT easy to use and highly effective for crafting precise, reliable prompts. For more complex scenarios, PACT can be enriched with few-shot learning and zero-shot chain-of-thought techniques.

Let's revisit the first unstructured prompt and its generated response, then analyze its structure. By focusing on the text's semantics, we can identify three main components:

1. *The Persona* (or role or actor): You are an AI assistant controlling a robot car, highlighted in light gray.
2. *The Action* (or task): You have to break down complex commands, the unhighlighted text.
3. *Context*: "There is a tree directly in front of the car...", highlighted in dark gray.

Let's analyze the user prompt:

User prompt:

You are an AI assistant controlling a robot car that can perform basic moves: forward, backward, turn left, turn right, and stop. Break down the complex command "There is a tree directly in front of the car. Avoid it and then return to the original path." into the basic moves you know.

While this prompt provides all the necessary information in a single block of text, it lacks a clear structure. By applying *PACT* principles (*Persona*, *Action*, *Context*, and *Template*) and separating key elements into distinct prompt parts, we can improve clarity and guide the model more effectively.

Our analysis of the original prompt identified the *Persona*, *Action*, and *Context* components of *PACT*, but it was missing a crucial element: the *Template*. The *Template* part of *PACT* specifies the desired output format, which helps guide the model to produce a structured, consistent response.

We can improve that by adding something like *Use a JSON array for the response* to the prompt to force structured output. This provides an immediate benefit: asking for a JSON response promotes more deterministic output. JSON's strict structure guides the model to follow specific syntax and key-value arrangements, reducing variability and leading to a more deterministic response.

The user prompt looks like this:

User prompt:

You are an AI assistant controlling a robot car capable of performing basic moves: forward, backward, turn left, turn left, turn right, and stop.

You have to break down the provided complex commands into the basic moves you know.

Complex command: "There is a tree directly in front of the car. Avoid it and then return to the original path.

Use a JSON array for the response.

Requesting the response in JSON format makes it easier for Agent Framework to parse the output into a native type. This is key to seamlessly integrating AI-generated content into your application's workflow.

For learning purposes, we can keep the labels for the prompt parts (Persona, Action, Context, Template) in the prompt. Note, though, that explicitly labeling these components in the prompt isn't necessary.

Let's look at the refined user prompt in the PACT framework, with the prompt-part labels:
User prompt:

User prompt:

Persona

You are an AI assistant controlling a robot car capable of performing basic moves: forward, backward, turn left, turn right, and stop.

Action

You have to break down the provided complex commands into the basic moves you know.

Context

Complex command: "There is a tree directly in front of the car. Avoid it and then return to the original path."

Template

Use a JSON array for the response.

Let's look at the generated response:

ChatGPT response:

Here's a breakdown of the complex command into basic moves for the robot car:

```
```json
```

```
[
 {"move": "turn left"},
 {"move": "forward"},
 {"move": "turn right"},
 {"move": "forward"},
 {"move": "turn right"},
 {"move": "forward"},
 {"move": "turn left"}
]
```

```
```
```

These steps should help the robot car avoid the tree and return to its original path.

If we run the prompt again, we'll get similar responses. Try running it a few times to see how similar they are.

ChatGPT response:

We can break the complex command down into basic moves:

```
```json
[
 {"action": "stop"},
 {"action": "turn right"},
 {"action": "forward"},
 {"action": "turn left"},
 {"action": "forward"},
 {"action": "turn left"},
 {"action": "forward"},
 {"action": "turn right"}
]
```
```

I've broken down the complex command into basic moves for the robot car. The car will stop, turn right to avoid the tree, move forward to pass it, then turn left and continue forward in the original direction.

We're making substantial progress. The JSON part of the response is what we want, but most of the time it's wrapped in explanatory text. Let's clean that up by adding a new rule: `Respond only with the moves, without any additional explanations.` This will instruct the model to return only the JSON output.

User prompt:

Persona

You are an AI assistant controlling a robot car capable of performing basic moves: forward, backward, turn left, turn right, and stop.

Action

You have to break down the provided complex commands into the basic moves you know.

Context

Complex command: "There is a tree in front of the car. Avoid it, then return to the original path."

Template

Use a JSON array for the response.

Respond only with the moves, without any additional explanations.

Response:

ChatGPT response:

```
```json
["turn right", "forward", "turn left", "forward", "turn left", "forward", "turn right"]
```
```

It looks much better: the JSON content is now isolated. But let's not celebrate yet! Run it a few more times (as I advised) to see how stable the response is:

ChatGPT response:

```
[
  "turn right",
  "forward",
  "turn left",
  "forward",
  "turn left",
  "forward",
  "turn right"
]
```

Almost there! The result is now cleanly formatted as a JSON array, without unnecessary explanatory text. During testing across multiple runs, some variation in output formatting may still occur. Sometimes, items in the JSON array are key-value pairs, and even the key names can vary. Other times, the JSON response is formatted on a single line or split across multiple lines.

TIP Include explicit instructions in the prompt, such as `Respond only with JSON with properties: name, age, profession,` or `Respond only with JSON arrays like [item1, item2, item3],` or like `[key1:value1, key2:value2, key3:value3]`.

Let's fix this by providing a formal schema: `Use a JSON array like [move1, move2, move3] for the response.`

User prompt:

Persona

You are an AI assistant controlling a robot car capable of performing basic moves: forward, backward, turn left, turn right, and stop.

Action

You have to break down the provided complex commands into the basic moves you know.

Context

Complex command: "There is a tree directly in front of the car. Avoid it and then return to the original path."

Template

Use a JSON array like [move1, move2, move3] for the response.

Respond only with the moves, without any additional explanations.

Let's see what the response is now:

ChatGPT response:

```
```json
[
 "turn right", "forward", "turn left", "forward", "turn left", "forward", "turn right"
]
```
```

I ran it a few times, and it's obvious that the response becomes more stable and adheres nicely to the desired JSON schema.

While I recommend the PACT framework for its clarity, modularity, and Agent Framework readiness, other prompt engineering frameworks, such as *RTF* (Role, Task, Format), can also be effective, depending on the task and context.

2.3.2 Structuring Prompts for Reusability and Scalability

The core of Agent Framework is the agent. The agent's `Instructions` property defines the agent's persona, a foundational element that remains constant and helps characterize the agent. This property acts as the system message. Breaking down the prompt into semantic parts (a method introduced earlier with the PACT prompting framework) is especially beneficial here.

Let's decompose the prompt into smaller semantic parts, as shown in figure 2.3.

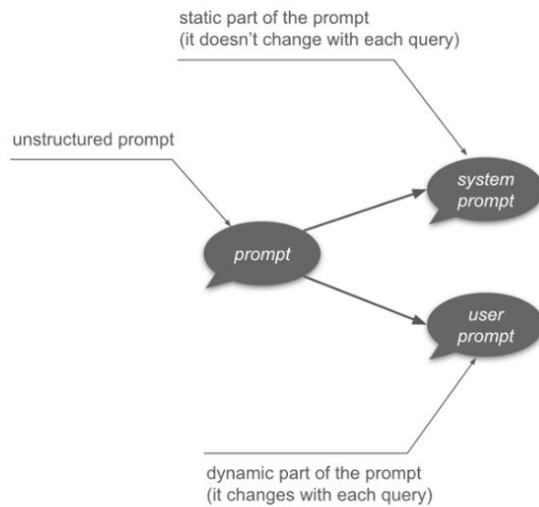


Figure 2.3 A prompt split into user prompt (the dynamic part that changes with each new interaction with an AI model) and system prompt (the static part that doesn't change during interaction with an AI model)

On one hand, the static parts—such as Persona, Actions, and Output Template—aren't expected to change when you send multiple queries to the model. On the other hand, Context is the dynamic part of the prompt and is expected to change with each prompt. The static (or general, if you prefer) parts go into the system prompt (a.k.a. instructions in the agents' world), and the dynamic part goes into the user prompt.

NOTE To help distinguish the components of a prompt, such as the user prompt, system prompt, and instructions, I use the term *prompt* for the dynamic portion (the user prompt). The static portion (the system prompt) corresponds to *instructions*, and I'll keep this convention throughout the book.

Now that we've split the well-structured prompt into a prompt and instructions, we're ready to use them in our first Agent Framework application.

2.3.3 Building a Stateless Agent with Agent Framework

When creating a new application with Agent Framework, the first step is to choose an AI provider and create a chat client. The provider might be OpenAI, Azure OpenAI, Ollama, or others. Most providers require a private API key for authentication. It's crucial to understand that these keys are often tied to your account's billing system. If someone intercepts your key, they could misuse it and run up charges against your account. You don't want that.

There are several ways to securely manage sensitive information like API keys. For development purposes, I recommend using the .NET Secret Manager tool because it's straightforward to implement and maintain. We'll use it in examples in this chapter and the next. Although Secret Manager is

convenient during development, it isn't suitable for production environments because of its limited security features.

In production, use more secure options such as environment variables, Azure Key Vault, or AWS Secrets Manager. These tools manage sensitive information in live environments and help protect it from unauthorized access. If you want to use these methods during development, adapt the examples as needed. The right choice depends on your project requirements and deployment environment.

IMPORTANT Never underestimate the importance of protecting API keys. Strong security measures help prevent accidental exposure. Common risks include editing solution files, committing keys to Git, and showing them during screen sharing or code demos. Exposed API keys can lead to unauthorized access, financial losses, and compromised data. Use secure methods such as environment variables or secret-management tools to handle API keys.

Now that we have the necessary setup, agent instructions, and prompt, let's start coding our console application to demonstrate how to use the OpenAI API to break complex movements into basic commands the robot car API can understand.

Step-by-step explanation:

1. **Namespaces:** Import the `Microsoft.Extensions.Configuration` namespace to manage user secrets, `Microsoft.Agents.AI` to work with agents, and `OpenAI` to initialize the OpenAI services provider.
2. **Configuration setup:** Uses `ConfigurationBuilder` to load user secrets, a secure way to store sensitive information such as API keys.
3. **OpenAI Client Initialization:** An `OpenAIClient` instance is created using the model ID and API key stored in user secrets. This chat client will handle the agent's internal communication with OpenAI services.
4. **Agent initialization:** Creates an AI agent using the previously created chat client with the provided instructions.
5. **Prompt (User Prompt) Definition:** The prompt is declared using a raw string literal. This feature is particularly useful because it allows more readable and maintainable multiline strings without escape characters, especially double quotes. The prompt string defines the task, or the goal we expect the agent to complete; more specifically, it instructs the AI model to control a robot car using basic moves.
6. **Prompting the API.** The `RunAsync` method is called with the prompt argument, sending a request to OpenAI for a chat completion.
7. **Output:** The `AgentResponse` text response from the AI model is printed to the console.

Open the `Program.cs` file and replace its contents with the following code (listing 2.2):

Requirements (NuGet packages):

```
dotnet add package Microsoft.Agents.AI.OpenAI
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
```

Code path: `/FirstAgent/Program.cs`

Listing 2.2 Agent sends a prompt to an AI model

```
using Microsoft.Extensions.Configuration; // ❶
using Microsoft.Agents.AI; // ❷
```

```

using OpenAI; //❷
using OpenAI.Chat; //❷

var configuration = new ConfigurationBuilder()
    .AddUserSecrets<Program>() //❸
    .Build();

var model = configuration["OpenAI:ModelId"];
var apiKey = configuration["OpenAI:ApiKey"];
var chatClient = new OpenAIClient(apiKey)
    .GetChatClient(model); //❹

AIAgent agent = chatClient.AsAIAgent("""
    ## Persona
    You are an AI assistant controlling a robot car capable of performing
    basic moves: forward, backward, turn left, turn right, and stop.

    ## Action
    You have to break down the provided complex commands into the basic
    moves you know.

    ## Template
    Use a JSON array like [move1, move2, move3] for the response.
    Respond only with the moves, without any additional explanations.
    """); //❺

var prompt = """
    ## Context
    Complex command:
    "There is a tree directly in front of the car. Avoid it and then
    return to the original path."
    """; //❻

AgentResponse response = await agent.RunAsync(prompt);
Console.WriteLine(response.Text); //❼

```

- ❶ Imports namespace for reading the user secrets
- ❷ Imports namespaces for working with OpenAI provider
- ❸ Reads the secrets to configuration object
- ❹ Creates the chat client
- ❺ Creates the AI agent using the chat client and the instructions
- ❻ Declares the prompt
- ❼ Prints the text response to the console

Let's run the code. Open the Terminal and run the following command.

```
dotnet run
```

Are you getting a response like this?

Assistant:

ChatGPT response:

```

```json
[
 "turn right",
 "forward",
 "turn left",
 "forward",
 "turn left",
 "forward",
 "turn right"
]
```

```

If your response is similar, everything is fine because you just replicated what ChatGPT does when a user sends a prompt to the model, except this time you did it through the OpenAI API with an AI agent running in a console application.

I encourage you to try a few prompt variations to see how large language models “understand” the intent behind a user prompt.

Try these queries (one at a time!):

- a straightforward command for basic movements:

```
"Go like: turning left, forward, turning right, backward, stop"
```

- This relies on semantic understanding of geometric paths:

```
"Go on a semi-circle"
```

- This relies, again, on semantic understanding of geometric paths:

```
"Go on a square path"
```

- The word "randomly" introduces variability in the output:

```
"Go 10 steps where each step is a randomly selected step"
```

- a longer command combining multiple steps into a sequence:

```
"Do a full circle by turning left, followed by a full circle by turning right"
```

- This challenges the LLM to understand the initial position and navigate back to it:

```
"Move forward, turn left, forward, and return to the same place where it started"
```

- Let's get creative:

```
"Do the moonwalk dancing"
```

- Let's look at some stops and moves that mimic jellyfish movement:

```
"Move like a jellyfish"
```

2.3.4 Troubleshooting the application

There are relatively few things that can go wrong with the previous example. If problems arise, they're most likely related to API key management or endpoint configuration. Double-check these elements before running your application to minimize problems. Mishandling API keys or endpoints can result in errors such as `ClientResultException`. Error descriptions are usually self-explanatory and help you solve the problem (table 2.1):

Table 2.1 Troubleshooting the Application

| ClientResultException | Description |
|---|--|
| HTTP 401
(invalid_request_error:
invalid_api_key) | Access denied due to invalid subscription key or wrong API endpoint. Make sure to provide a valid key for an active subscription and use a correct regional API endpoint for your resource. |
| HTTP 404
(invalid_request_error:
model_not_found) | The model `...` does not exist or you do not have access to it |
| HTTP 429
(insufficient_quota:
insufficient_quota) | You exceeded your current quota, please check your plan and billing details. For more information on this error, read the docs:
https://platform.openai.com/docs/guides/ |

Hopefully, everything worked as expected! If so, congratulations on completing this example of a generative AI application using Agent Framework. Whether this is your first experience with Agent Framework or you're already familiar with it, this marks an important first step toward building more advanced AI-driven applications.

2.3.5 Building a stateful agent with memory

The code sends a prompt to the model and then prints the result. This behavior is stateless and follows a request-response pattern, which works well when we need the model to respond to a single request. Simply put, it doesn't remember anything from previous messages. What if we need to create an agent that remembers?

Let's turn our sample into an interactive agent that recalls conversation history (see listing 2.3) by adding the `AgentSession` object, which stores the message history.

Requirements (NuGet packages):

```
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
dotnet add package Microsoft.Agents.AI.OpenAI
```

Code path: `/FirstAgentWithHistory/Program.cs`

Listing 2.3 Agent sends a prompt to an AI model using `AgentSession`

```
using Microsoft.Extensions.Configuration; //❶
using Microsoft.Agents.AI; //❷
using OpenAI; //❷
using OpenAI.Chat; //❷

var configuration = new ConfigurationBuilder()
    .AddUserSecrets<Program>().Build();

var model = configuration["OpenAI:ModelId"];
var apiKey = configuration["OpenAI:ApiKey"];
var chatClient = new OpenAIClient(apiKey)
    .GetChatClient(model); //❸

AIAgent agent = chatClient.AsAIAgent("""
## Persona
You are an AI assistant controlling a robot car capable of performing
basic moves: forward, backward, turn left, turn right, and stop.

## Action
You have to break down the provided complex commands into the basic
moves you know.

## Template
Use a JSON array like [move1, move2, move3] for the response.
Respond only with the moves, without any additional explanations.
"""); //❹

AgentSession session = await agent.CreateSessionAsync(); //❺

var prompt1 = "go 10 meters forward then turn back";
Console.WriteLine($"User: {prompt1}");
var response1 = await agent.RunAsync(prompt1, session); //❻
Console.WriteLine($"Assistant: {response1.Text}");

var prompt2 = "repeat the last maneuvers";
Console.WriteLine($"User: {prompt2}");
var response2 = await agent.RunAsync(prompt2, session); //❼
Console.WriteLine($"Assistant: {response2.Text}");
Console.WriteLine("\nHistory:");

if (session.TryGetInMemoryChatHistory(out var history)) //❽
{
    foreach (var message in history) //❾
    {
        Console.WriteLine($"{{message.Role}}: {{message.Text}}");
    }
}
```

- ❶ Imports namespace for reading the user secrets
- ❷ Imports namespaces for working with Agent Framework agents OpenAI provider
- ❸ Creates the chat client
- ❹ Creates the agent with its instructions
- ❺ Creates an agent session
- ❻ Calls the agent with first prompt
- ❼ Calls the agent with the second prompt
- ❽ Reads the chat messages from the chat session
- ❾ Iterates each message for printing to console

This enhanced version creates an interactive experience where the agent remembers previous interactions and maintains context throughout the conversation. Notice how the session object is built and passed back to the agent.

When you run the code from listing 2.3, your agent will remember every previous interaction, as shown in listing 2.4. For example:

Listing 2.4 Console output from code in Listing 2.3

```
User: go 10 meters forward then turn back
Assistant: ["forward", "turn left", "turn left"]
User: repeat the last maneuvers
Assistant: ["forward", "turn left", "turn left"]

History:
user: go 10 meters forward then turn back
assistant: ["forward", "turn left", "turn left"]
user: repeat the last maneuvers
assistant: ["forward", "turn left", "turn left"]
```

Congratulations! You've just created an agent that can intelligently break down complex movements for a robot car and that remembers previous interactions!

We can see that the agent uses the previous chat messages in its history to recall earlier interactions with the LLM and repeat the last maneuvers. Without an agent session, LLM would have no knowledge of those previous maneuvers. `TryGetInMemoryChatHistory`, on the other hand, lets us retrieve all prior chat messages from the agent session.

2.3.6 Streaming Agent Responses

To capture the essence of an interactive agent, we need to implement response streaming. This feature enhances the user experience and improves perceived response speed. With streaming, users can start reading the AI's response as it's generated, instead of waiting for the entire message to complete.

Let's upgrade our agent from listing 2.3 with this streaming capability. Locate this line:

```
AgentResponse response1 = await agent.RunAsync(prompt1, session);
Console.WriteLine($"Assistant: {response1.Text}");
```

And replace it with:

```
Console.Write("Assistant: ");
await foreach (AgentResponseUpdate update in agent
    .RunStreamingAsync(prompt1, session)) //❶
{
    Console.Write(update.Text);
}
Console.WriteLine();
```

- ❶ Streams the response from the AI model in chunks

Run the application again and observe the response as it is produced:

```
Assistant: ["forward", "turn right", "turn right"]
```

The response should be like what we saw earlier, but streaming responses improve user experience by reducing perceived latency, since users see the AI's reply almost immediately. This creates a more natural, interactive flow that mimics human conversation.

IMPORTANT When responding with structured information, streaming is less effective because you need the full response to ensure clarity and coherence. For example, a partial JSON response is not very useful. In those cases, a non-streaming response is more suitable.

While our agent can now hold a conversation and maintain context, it's still a passive system. It responds to user input, but it doesn't take any independent actions. Next, we'll enhance the agent's capabilities by enabling it to directly control the robot car based on its understanding of the conversation.

2.3.7 Building a Tool Agent with Actions

While having a responsive agent is impressive, the real power lies in taking action based on AI's responses. This lets the vehicle make decisions more independently, rather than relying on preprogrammed handling of the model's output. We can use Agent Framework's ability to automate actions as the model generates responses, enabling the agent to execute those actions based on its language understanding.

DEFINING TOOLS WITH NATIVE FUNCTIONS

Native functions let you define custom tools that agents can use to interact with your application's logic or data. A tool is typically a native function (conventional code like C#) that's wrapped and registered so an agent can invoke it as needed.

Listing 2.5 shows the `MotorTools` class, which shows how to create tools from native C# methods:

Code path: `/FirstAgentWithTools/MotorTools.cs`

Listing 2.5 Motor AI tools class

```
using Microsoft.Extensions.AI; //❶
using System.ComponentModel;

public static class MotorTools //❷
{
    private const int Delay = 1000; //❸

    static public IEnumerable<AITool> AsAITools() //❹
    {
        yield return AIFunctionFactory.Create(TurnRight);
        yield return AIFunctionFactory.Create(TurnLeft);
        yield return AIFunctionFactory.Create(Stop);
        yield return AIFunctionFactory.Create(Forward);
        yield return AIFunctionFactory.Create(Backward);
    }

    [Description("Basic command: Moves the robot car backward.")]
    public static async Task<string> Backward(
        [Description("The distance (in meters) backward.")] int distance)
    {
        Console.WriteLine($"[{DateTime.Now:hh:mm:ss:fff}] MOTORS: Backward:
{distance}m");
        await Task.Delay(Delay); //❺
        return $"moved backward for {distance} meters.";
    }
}
```

```

    }

    [Description("Basic command: Moves the robot car forward.")]
    public static async Task<string> Forward(
        [Description("The distance (in meters) to move the robot car forward.")]
        int distance)
    {
        Console.WriteLine($"[{DateTime.Now:hh:mm:ss:fff}] MOTORS: Forward:
{distance}m");
        await Task.Delay(Delay);
        return $"moved forward for {distance} meters.";
    }

    [Description("Basic command: Stops the robot car.")]
    public static async Task<string> Stop()
    {
        Console.WriteLine($"[{DateTime.Now:hh:mm:ss:fff}] MOTORS: Stop");
        await Task.Delay(Delay);
        return "stopped.";
    }

    [Description("Basic command: Turns the robot car anticlockwise.")]
    public static async Task<string> TurnLeft(
        [Description("The angle (in degrees) to turn the robot car anticlockwise.")]
        int angle)
    {
        Console.WriteLine($"[{DateTime.Now:hh:mm:ss:fff}] MOTORS: TurnLeft:
{angle}°");
        await Task.Delay(Delay);
        return $"turned anticlockwise {angle}°.";
    }

    [Description("Basic command: Turns the robot car clockwise.")]
    public static async Task<string> TurnRight(
        [Description("The angle (in degrees) to turn the robot car clockwise.")]
        int angle)
    {
        Console.WriteLine($"[{DateTime.Now:hh:mm:ss:fff}] MOTORS: TurnRight:
{angle}°");
        await Task.Delay(Delay);
        return $"turned clockwise {angle}°.";
    }
}

```

- ❶ Imports the namespace for AITool class used in user-defined AsAITools method
- ❷ Defines the MotorTools class that keep the domain-specific functions together
- ❸ Declares a property that simulates 1000 milliseconds delay
- ❹ Defines an iterator method that exposes the native functions as AI tools
- ❺ Introduces a delay to simulate an action that takes time to execute

AsAITools we created is an iterator method that helps us expose the motor commands as an `IEnumerable<AITool>`. Each `yield return` wraps one command method in an `AIFunction` and supplies it as the next tool when the returned sequence is enumerated.

Native functions in Agent Framework use method annotations such as `DescriptionAttribute`. Classes and parameters also have their own descriptions. In generative AI, these semantic descriptions are as crucial as syntax is in traditional programming.

IMPORTANT If no explicit description or `DescriptionAttribute` is supplied, the function description is empty. The model still receives the tool name and the parameter names and type

schema, so meaningful names remain useful. Add descriptions whenever names alone do not clearly communicate the tool's purpose, argument constraints, units, accepted values, or side effects.

Advanced large language models use these descriptions (or, if descriptions aren't provided, their names) to:

- understand a function's purpose
- use functions correctly
- identify appropriate functions to call.

Comprehensive descriptions of native functions wrapped as agent tools are essential for effective AI interactions. The LLM uses each tool's name, description, and parameter metadata together with the conversation context to decide whether to invoke it and which arguments to supply.

When LLMs prepare a response, they may decide to call none, one, or multiple tools associated with the prompt to provide a more accurate, informed response, depending on the agent setting. For example, if we ask the model about the current weather in a specific location, it likely doesn't have real-time weather data. Without access to relevant tools, it might hallucinate a response or state that it can't provide accurate information. But when the prompt includes well-described functions, the LLM can fill information gaps by calling the right ones.

I propose the term *semantic signature* to describe this set of contextual conventions, analogous to a traditional method's syntax signature, which defines its name, parameters, and data types. A semantic signature extends this notion by capturing a function's meaning, intent, and capability, so the AI can use it more intelligently in suitable contexts.

IMPORTANT Semantic signatures capture both the structural and descriptive details of a function, encompassing its name, arguments, and descriptions. This information helps the AI model understand what each function can do, decide when to select it as a tool, and generate appropriate arguments. Agent Framework can then validate and invoke the selected function, improving the reliability of tool use and enabling more capable agent workflows.

While LLMs can decide whether to call tools based on context, the underlying decision process varies across implementations and model architectures.

IMPORTING AI TOOLS FROM A CLASS

To build an agent with AI tools, use `Tools` property:

Listing 2.6 Create a tool agent

```
var agent = chatClient.AsAIAgent(
    instructions: """
    You are an AI assistant controlling a robot car capable of performing
    basic moves: forward, backward, turn left, turn right, and stop.
    You have to break down the provided complex commands into the basic
    moves you know.
    Respond only with the moves, without any additional explanations.
    Execute the corresponding basic moves using the available tools.
    """,
    tools: [.. MotorTools.AsAITools()]
);
```

With these enhancements (history, streaming, and tools), we've transformed our agent into a stateful, tool-using agent. It can now:

- Engage in natural language conversations
- Stream responses for more fluid interaction
- Access external data and services
- Execute real-world actions through native functions

This foundation opens exciting possibilities. Imagine expanding this system to control various aspects of a smart home, assist in complex data analysis, or even guide autonomous systems through challenging scenarios.

IMPORTANT Some AI models may not call tools even when tools are attached to an agent. It is worth noting that you can nudge the model by adding explicit instructions, such as: `Execute the corresponding basic moves using the available tools.`

BRINGING ALL CHANGES TOGETHER

Listing 2.7 shows the full code for creating an agent with tools. It uses the class defined already in listing 2.5.

Requirements (NuGet packages):

```
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
dotnet add package Microsoft.Agents.AI.OpenAI
```

Code path: `/FirstAgentWithTools/Program.cs`

Listing 2.7 Agent sends a prompt to an AI model using tools

```
using Microsoft.Agents.AI;
using Microsoft.Extensions.Configuration;
using OpenAI;
using OpenAI.Chat;

var configuration = new ConfigurationBuilder()
    .AddUserSecrets<Program>().Build();

var model = configuration["OpenAI:ModelId"];
var apiKey = configuration["OpenAI:ApiKey"];
var chatClient = new OpenAIClient(apiKey)
    .GetChatClient(model); //❶

AIAgent agent = chatClient.AsAIAgent("""
    You are an AI assistant controlling a robot car capable of performing
    basic moves: forward, backward, turn left, turn right, and stop.
    You have to break down the provided complex commands into the basic
    moves you know.
    Respond only with the moves, without any additional explanations.
    Execute the corresponding basic moves using the available tools.
    """, //❷
    tools: [..MotorTools.AsAITools()] //❸
);

AgentSession session = await agent.CreateSessionAsync();

while (true)
{
    Console.Write("User: ");
```

```

var input = Console.ReadLine(); //❹
if (string.IsNullOrEmpty(input)) break; //❹

var prompt = $"""
    Complex command:
    "{input}"
    """; //❺

AgentResponse response = await agent.RunAsync(prompt, session);
Console.WriteLine($"Assistant: {response.Text}"); //❻
}

```

- ❶ Creates the chat client
- ❷ Supplies the agent instructions
- ❸ Registers the MotorTools functions as tools
- ❹ Reads the input from user until the input is empty
- ❺ Wraps the input into a prompt
- ❻ Prints the text response to the console

Run the code with the user input `There is a tree directly in front of the car. Avoid it and then return to the original path.` For the second user input, type `Enter` to finish the loop. The output should look like this:

Listing 2.8 Console output for listing 2.7

```

User: There is a tree directly in front of the car.
Avoid it and then return to the original path.
[06:01:43:339] MOTORS: TurnRight: 90°
[06:01:44:361] MOTORS: Forward: 5m
[06:01:45:366] MOTORS: TurnLeft: 90°
[06:01:46:369] MOTORS: Forward: 10m
[06:01:47:372] MOTORS: TurnLeft: 90°
[06:01:48:378] MOTORS: Forward: 5m
[06:01:49:384] MOTORS: TurnRight: 90°
Assistant: The robot car successfully avoided the tree and returned to the
original path.

```

Notice the tool responses (for example, `MOTORS: TurnRight: 90°`) and the assistant response produced by the AI model. Each step in the sequence triggered the corresponding tool from `MotorTools`, as expected.

IMPORTANT Unlike listing 2.4 that shows the output when there are no tools involved, this response from listing 2.7 was produced after the agent invoked one or more provided tools to complete the task.

2.4 Conclusion

As we've seen so far, we've only scratched the surface of Agent Framework's capabilities. Our simple console application is the first step in an exciting journey into generative AI development. Agent Framework streamlines interactions with large language models, abstracting away complex API calls and prompt engineering.

Summary

- Motivation for agents and LLMs: a robot car story that links complex commands to low-level device APIs.
- Limitations of hardcoded movement sequences and the need for a more scalable approach to complex robot-car behaviors.

- Use assistants such as ChatGPT or Copilot to translate natural language movement descriptions into executable step sequences for the robot car API.
- An understanding of LLM nondeterminism and its effects on machine-to-machine scenarios that require predictable, structured outputs.
- The PACT prompt engineering framework is a practical method for designing clear, reusable prompts for control scenarios.
- Techniques for shaping free-form prompts into JSON responses that are easy to parse and integrate into .NET applications.
- Steps for obtaining OpenAI and Azure OpenAI API keys, plus core options for securing them across environments.
- Build your first stateless Agent Framework console agent that sends queries to an LLM and prints structured results.
- Evolving to a stateful agent with `AgentSession` so conversation history and prior context are preserved across interactions.
- Separating static instructions from dynamic queries, and mapping them to Agent Framework concepts for modular, maintainable prompts.

3

Building chat clients for model providers

This chapter covers

- Understanding chatbots and chat clients in `Microsoft.Extensions.AI` (MEAI)
- Using Chat Completions with major AI model providers
- Working with `IChatClient` as a unified abstraction
- Managing conversation history and basic streaming

The `Microsoft.Extensions.AI` (MEAI) library is the foundational layer for any AI agent, connecting your applications to large language model providers. We explore MEAI and the building blocks you'll need. You'll use this foundation to dive deeper into Agent Framework and build complete agent systems.

NOTE From now on, *MEAI* refers to `Microsoft.Extensions.AI`; we use the full name when clarity requires.

3.1 *Introducing Robby, the Robot Car Assistant*

Most of the time, analogies work best when dealing with abstract concepts, so we'll imagine things from a robot-car perspective. Let's name our robot car Robby. As we go, we'll see how Robby evolves from a simple chatbot to an advanced AI agent and even benefits from a team of agents and an agentic AI architecture.

Let's start with what a *chatbot* is. A chatbot is an AI application that interacts with users in a conversational way. It takes user input, runs it against a chat client, and generates a response using an AI model. Developers can quickly build chatbots by combining prompts, chat clients with chat options, and chat history, which makes it easy to adjust the chatbot's style and memory. This turns a basic language model into a context-sensitive assistant that can answer questions and automate simple tasks with minimal code.

With this high-level understanding of chatbots, we can start creating one by writing good prompt.

3.2 What is Chat Completion API?

The *Chat Completion* API is a foundational way to interact with large language models across major providers. This stateless, message-based architecture gives developers complete control over conversation management while keeping things simple and scalable.

Figure 3.1 shows what happens when a client submits a prompt to the *Chat Completions* API, waits for the model to finish, and then receives the final output in a single response.

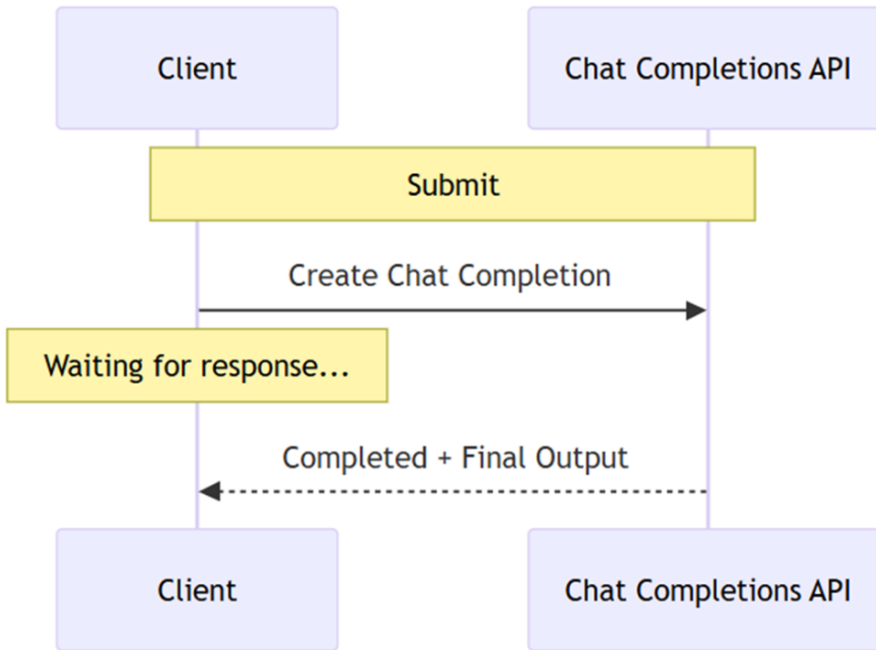


Figure 3.1 The client sends a Create Chat Completion request to the Chat Completions API, waits while the service generates a result, then receives a response containing the final output in one step.

You'll implement chat clients for OpenAI, Ollama, and Anthropic using the *Chat Completion* API and see practical applications.

3.2.1 Understanding the Request-Response Mechanism

Prompting using chat completion clients operates on a stateless request-response pattern with the OpenAI API. This means that each prompt is processed independently, without retaining memory of previous interactions. You can overcome this limitation using short-term memories.

This approach offers the following advantages:

- **Flexibility:** each prompt can have a fresh start without being influenced by past interactions.
- **Efficiency:** the system doesn't need to manage or store session data.
- **Scalability:** statelessness simplifies architecture and enhances the ability to handle multiple queries.

This stateless approach has a trade-off: developers must explicitly manage the state across multiple interactions. Agent Framework via MEAI (Microsoft.Extensions.AI) provides tools to address this through chat message history. These features help manage the stateless nature of large language model interactions but require deliberate implementation by developers.

It's important to understand that the *Chat Completion* API used by Agent Framework is inherently stateless. While other APIs, such as the *Responses* API, offer built-in conversation state management through persistent sessions that store message history and it may automatically truncate it when conversations exceed the model's context length (disabled as the default), the Chat Completion API requires developers to manage message history and state themselves.

When a prompt is crafted within Agent Framework, it is transformed into an HTTP request and sent to the selected large language model chat completion API. The APIs provided by OpenAI, Azure, Ollama, or other providers process the request and generate an HTTP response. The response is then received and interpreted by the chat client, which integrates it into the application's workflow.

Listing 3.1 shows a `curl` command running in a Bash terminal that sends an HTTP request to the OpenAI API service. The request must include the API endpoint and authorization (OpenAI API key). The body of the HTTP request typically includes:

- The specific model identifier (e.g., "gpt-4o, gpt-5.2")
- An array of messages, including system instructions, user prompts, and any chat history
- Additional parameters such as `temperature`, `max_tokens`, or `top_p`, and many others, can fine-tune the model's output.

Let's examine listing 3.1:

Listing 3.1 Example HTTP request using the Bash terminal

```
curl 'https://api.openai.com/v1/chat/completions' \
--ssl-no-revoke \ ❶
-H 'Authorization: Bearer $YOUR_OPENAI_KEY' \ ❷
-H 'Content-Type: application/json' \
-d '{
  "model": "gpt-4o", ❸
  "messages": [ ❹
    {
      "role": "system",
      "content": "You are a helpful assistant."
    },
    {
      "role": "user",
      "content": "Hello!"
    }
  ]
}'
```

❶ Disables SSL certificate revocation check (Windows-specific)

❷ Replaces \$YOUR_OPENAI_KEY with your OpenAI key

❸ Specifies the model id

❹ Messages with roles: system, user, assistant, or tool

After you send the request, the OpenAI API processes it and returns a response consisting of one or more choices, as shown in listing 3.2:

Listing 3.2 Example of an HTTP response

```
{
  "id": "chatcmpl-AWRvydbeq4Sne4LepluwsYp3wJIho",
  "object": "chat.completion",
  "created": 1732297238,
```

```

"model": "gpt-4o-2024-08-06",
"choices": [
  {
    "index": 0,
    "message": { //❶
      "role": "assistant",
      "content": "Hello! How can I assist you today?",
      "refusal": null
    },
    "logprobs": null,
    "finish_reason": "stop"
  }
],
"usage": { //❷
  "prompt_tokens": 19,
  "completion_tokens": 9,
  "total_tokens": 28,
  "prompt_tokens_details": {
    "cached_tokens": 0,
    "audio_tokens": 0
  },
  "completion_tokens_details": {
    "reasoning_tokens": 0,
    "audio_tokens": 0,
    "accepted_prediction_tokens": 0,
    "rejected_prediction_tokens": 0
  }
},
"system_fingerprint": "fp_831e067d82"
}

```

❶ Response choices

❷ Token usage metrics

Remember that this request-response interaction is stateless. Later, we'll explore how managing state with chat history can further enhance the user experience, enabling more coherent and continuous conversations as you use MEAI and Agent Framework.

3.3 Introduction to Chat Clients and Model Providers

A chat client, such as `OpenAIClient` or `OllamaChatClient`, is a strongly typed class built for a specific model provider. It enables an application to send messages to an AI model and receive conversational responses. The chat client bridges application code and the underlying AI service (for example, OpenAI or Ollama) by managing authentication, configuration, and communication with the remote chat completion endpoint. The model provider, in contrast, is the service that exposes those AI capabilities for chat clients to use.

The terms "chat client" and "model provider" are closely related but represent different concepts in the MEAI and Agent Framework ecosystems.

3.3.1 IChatClient Abstraction Interface

`IChatClient` is a standardized interface in `Microsoft.Extensions.AI` that provides a unified way to interact with AI chat services. Think of it as a contract or specification that defines how your code communicates with a chat-based AI service, regardless of the underlying provider.

`IChatClient` acts as a wrapper around a model provider's SDK. Most providers have their own SDKs (such as `OpenAI`, `OllamaSharp`, or `OnnxRuntimeGenAI`), and you can adapt them to `IChatClient`.

The MEAI layer defines core contracts, such as `IChatClient`, that standardize how we interact with different models. Whether we use cloud-based models (via `Microsoft.Extensions.AI.OpenAI`) or local models (via `OllamaSharp` or

`OnnxRuntimeGenAI`), the implementations live in the lower layer. These provider-specific packages adapt their respective SDKs to the MEAI standard.

Figure 3.2 illustrates the layered conceptual stack we use to build AI applications in .NET. At the top, `Microsoft.Agents.AI` provides the orchestration layer where we define agents, workflows, and multi-agent interactions. These agents don't communicate directly with model providers; instead, they rely on the `Microsoft.Extensions.AI` (MEAI) abstraction layer below.

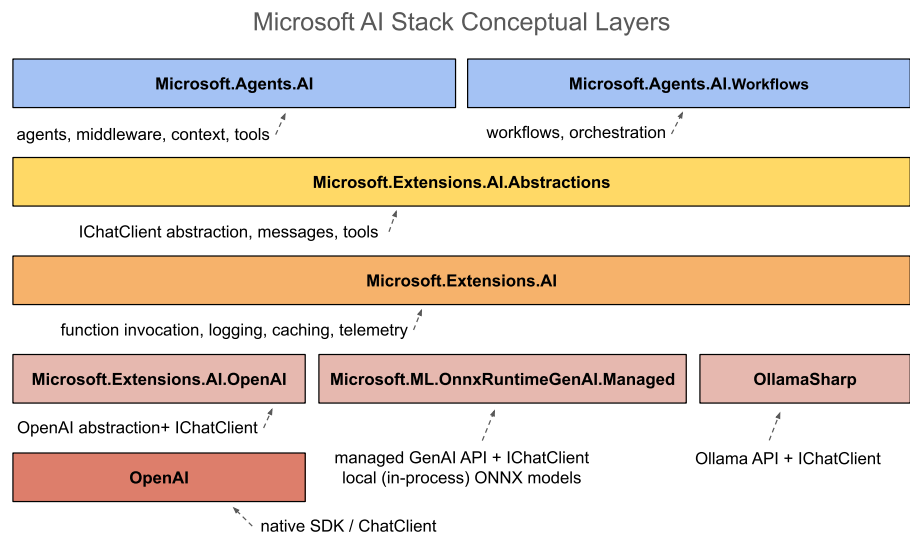


Figure 3.2 shows the Microsoft AI stack conceptual layers. Agents built with Agent Framework use chat clients built with MEAI (`Microsoft.Extensions.AI`), which provides an abstraction over concrete model provider SDKs.

The `IChatClient` abstraction addresses several real-world challenges developers face when building AI-powered applications:

- **Model Provider Independence:** Your application code isn't tightly coupled to a specific AI service. Need to switch from OpenAI to Ollama, or from *Chat Completions API* to *Responses API*? Change just the few lines of code in your chat client.
- **Composability:** This interface supports middleware patterns, so we can add cross-cutting concerns (logging, caching, retry logic, and rate limiting) that work across all model providers.
- **Future-proofing:** As new model providers emerge, they can implement `IChatClient`, and your existing code will work with the new provider.
- **Testability:** We can create mock implementations of `IChatClient` for unit testing without calling actual AI services.

3.3.2 Model Provider Implementation

A model provider is an AI service or implementation that fulfills the `IChatClient` contract. It's the AI service or SDK that performs inference.

Examples of model providers include:

- OpenAI: OpenAI service (using `OpenAI` library)

- Anthropic: Anthropic service (using `Anthropic` library)
- Ollama: Local model hosting (using `OllamaSharp` library)
- ONNX: In-process model hosting (using `Microsoft.ML.OnnxRuntimeGenAI` library)

Think of `IChatClient` as a standard contract that defines how your application communicates with any chat-based AI service. Instead of learning different APIs for OpenAI, Anthropic, or local models that run on Ollama, we write our code once against the `IChatClient` interface and can swap model providers with minimal changes.

NOTE In Agent Framework, agents are built either on concrete chat clients or on chat client abstractions, `IChatClient`. Using `IChatClient` ensures consistency in how we send and receive messages, message formats, tool-calling conventions, and streaming patterns.

All members of `IChatClient` are thread-safe for concurrent use. Implementations may mutate the `ChatOptions` arguments passed to `GetResponseAsync()` and `GetStreamingResponseAsync()`. Avoid sharing `ChatOptions` instances across concurrent method calls. Instead, create new options instances for each invocation to prevent race conditions.

Implementations are expected to support multiple requests running simultaneously, making them suitable for high-concurrency scenarios like web applications serving multiple users.

IMPORTANT All members of `IChatClient` are thread-safe for concurrent use. Implementations are expected to support multiple simultaneous requests, making them suitable for high-concurrency scenarios such as web applications serving multiple users.

3.3.3 Multi-Modal Support

Modern AI models support more than just text. The `IChatClient` interface and `ChatMessage` types support multimodal content such as:

- Text content: Standard conversational text
- Data content: Inline image or audio byte arrays (always check if the model is capable of vision or audio)
- URL content: URLs of image or audio files (always check whether the model supports vision or audio)

Supported media types are as follows, and we expect support for more types in the future:

- PNG → image/png
- JPEG → image/jpeg
- WEBP → image/webp
- Non-animated GIF → image/gif
- MP3 → audio/mpeg
- MP4 → video/mp4 (audio track)
- MPEG → audio/mpeg
- MPGA → audio/mpeg

- M4A → audio/m4a
- WAV → audio/wav
- WEBM → audio/webm

Here's a sample that shows how to mix these messages:

```
ChatMessage message = new(ChatRole.User, [
    new TextContent("Read the task from the audio file and use it with the image."),
    new UriContent(@"http://apexcode.ro/task.mp3", "audio/mpeg"),
    new UriContent(@"http://apexcode.ro/path.jpg", "image/jpeg")
]);

var response = await client.GetResponseAsync(message);
```

Or the same message using byte arrays instead of URLs:

```
byte[] audioBytes = File.ReadAllBytes(@"task.mp3");
byte[] imageBytes = File.ReadAllBytes(@"path.jpg");

ChatMessage message = new(ChatRole.User, [
    new TextContent("Read the task from the audio file and use it with the image."),
    new DataContent(audioBytes, "audio/mpeg"),
    new DataContent(imageBytes, "image/jpeg")
]);

var response = await client.GetResponseAsync(message);
```

3.4 Creating and Configuring Chat Clients

`IChatClient` provides a unified abstraction, but you still need to connect the interface to real-world AI services. While `IChatClient` standardizes how we interact with chat models, setup and configuration are specific to each model provider.

Different model providers, such as OpenAI, Anthropic, ONNX, and Ollama, offer unique connection methods, authentication requirements, and configuration options.

Create and configure chat clients for different model providers and use `IChatClient` as a unified way to work with them.

3.4.1 OpenAI Chat Completion Client

The OpenAI Chat Completion client is an implementation of OpenAI's Chat Completion API. Each request is independent: you send a message or the complete conversation history and receive a response.

Architecture and characteristics:

- Stateless: No server-side conversation memory
- Message-based: Send a list of messages representing the entire conversation
- Client-Managed History: Your application stores and manages all conversation states
- Synchronous and Streaming: Supports both complete responses and token streaming

NOTE Obtain an OpenAI API key by signing up at <https://platform.openai.com>. After creating an account and setting up billing (if required), go to the API keys section in the dashboard and generate a new secret key. Store this key securely (for example, in user secrets or environment variables) and reference it from your configuration.

PRACTICAL EXAMPLE

Listing 3.3 shows the OpenAI chat client in two variants: the concrete model provider, `ChatClient`, and `IChatClient`, the MEAI abstraction. `AsIChatClient` converts the concrete class into an abstract client.

IMPORTANT Use the MEAI `AsIChatClient` method to convert a concrete class to the abstract client `IChatClient`.

Requirements (NuGet packages):

```
dotnet add package Microsoft.Extensions.AI.OpenAI
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
dotnet add package OpenAI
```

Code path: `/OpenAIChatClient/Program.cs`

Listing 3.3 OpenAI chat client

```
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.AI;
using OpenAI;
using OpenAI.Chat;
using System.ClientModel;

var configuration = new ConfigurationBuilder()
    .AddUserSecrets<Program>().Build();

var model = configuration["OpenAI:ModelId"];
var apiKey = configuration["OpenAI:ApiKey"];

var prompt = """
    You are an AI assistant controlling a robot car capable of performing
    basic moves: forward, backward, turn left, turn right, and stop.
    You have to break down the provided complex commands into the basic
    moves you know.
    There is a tree in front of the car.
    Avoid it and resume the original path.
    Respond with a JSON array like [move1, move2, move3].
    Do not respond with reasoning, comments, or any additional text.
    """;
Console.WriteLine($"User: {prompt}");

ChatClient openAIChatClient = new OpenAIClient(apiKey)
    .GetChatClient(model); //❶
ClientResult<ChatCompletion> response = openAIChatClient
    .CompleteChat(prompt); //❷
Console.WriteLine($"Assistant (OpenAI ChatClient): "
    + $"{response.Value.Content.First().Text}");

IChatClient chatClient = new OpenAIClient(apiKey)
    .GetChatClient(model)
    .AsIChatClient(); //❸
ChatResponse chatResponse = await chatClient
    .GetResponseAsync(prompt); //❹
Console.WriteLine($"Assistant (IChatClient): {chatResponse.Text}");
```

- ❶ Gets the native chat client
- ❷ Invokes the native chat client with the prompt
- ❸ Converts the native chat client to abstract chat client `IChatClient`
- ❹ Invokes the abstract chat client with the prompt

Code output:

```
User: You are an AI assistant controlling a robot car capable of performing
```

```
basic moves: forward, backward, turn left, turn right, and stop.
You have to break down the provided complex commands into the basic
moves you know.
There is a tree in front of the car. Avoid it and resume the original path.
Respond with a JSON array like [move1, move2, move3].
Do not respond with reasoning, comments, or any additional text.
```

```
Assistant (OpenAI ChatClient): ["turn right", "forward", "turn left",
"forward", "turn left", "forward", "turn right"]
```

```
Assistant (IChatClient): ["turn left", "forward", "turn right", "forward",
"turn right", "forward", "turn left"]
```

When to Use Chat Completion

- You need complete control over conversation history.
- You're building stateless APIs or serverless functions
- You need to store conversations in your own database
- You want to manipulate or filter the conversation context

By supporting both the `ChatClient` implementation and the `IChatClient` abstraction, developers can switch model providers while keeping code patterns consistent.

EXERCISE

Locate, in listing 3.3, the lines that use the chat client as `IChatClient`:

```
IChatClient chatClient = new OpenAIClient(apiKey)
    .GetChatClient(model)
    .AsIChatClient();
ChatResponse chatResponse = await chatClient.GetResponseAsync(prompt);
Console.WriteLine($"Assistant (IChatClient): {chatResponse.Text}");
```

and replace them with a direct chat client (without converting to `IChatClient`):

```
ChatClient openAIChatClient = new OpenAIClient(apiKey)
    .GetChatClient(model);
ClientResult<ChatCompletion> response = openAIChatClient
    .CompleteChat(prompt);
Console.WriteLine($"Assistant (OpenAI ChatClient):
{response.Value.Content.First().Text}");
```

Run the code and compare the output with the original. They should be similar, even though one uses the abstraction and the other calls the provider directly.

3.4.2 OpenAI Chat Completion Client with Azure Credentials

Unsurprisingly, when using the OpenAI client with Azure credentials, the approach is the same as for the OpenAI client.

NOTE Azure OpenAI does not use a single global API key. First, create an Azure OpenAI resource in Azure. You can use Microsoft Foundry to deploy a model to that resource and assign the deployment a name. Your application then uses the Azure OpenAI resource endpoint and either an access key or Microsoft Entra ID authentication, together with the model deployment name. You can retrieve the resource endpoint (we need to append `openai/v1/` to the resource root) and API keys from Keys & Endpoint in the Azure portal; the endpoint is also visible from the deployment experience in Microsoft Foundry. API keys belong to the Azure OpenAI resource, whereas the deployment name identifies the particular model deployment to invoke.

PRACTICAL EXAMPLE

Listing 3.4 uses the concrete `OpenAI ChatClient` with Azure credentials and the `IChatClient` abstraction.

Requirements (NuGet packages):

```
dotnet add package Microsoft.Extensions.AI.OpenAI
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
```

Code path: `/AzureOpenAIChatClient/Program.cs`

Listing 3.4 OpenAI chat client with Azure credentials

```
// usings and apikey reading are omitted

var endpoint = configuration["AzureOpenAI:Endpoint"];
var apiKey = configuration["AzureOpenAI:ApiKey"];
var deploymentName = configuration["AzureOpenAI:DeploymentName"];

var prompt = """
    You are an AI assistant controlling a robot car capable of performing
    basic moves: forward, backward, turn left, turn right, and stop.
    You have to break down the provided complex commands into the basic
    moves you know.
    There is a tree in front of the car.
    Avoid it and resume the original path.
    Respond with a JSON array like [move1, move2, move3].
    Do not respond with reasoning, comments, or any additional text.
    """;
Console.WriteLine($"User: {prompt}");

OpenAIClient client = new(
    new ApiKeyCredential(apiKey),
    new OpenAIClientOptions { Endpoint = new Uri(endpoint) }); //❶

ChatClient azureOpenAIChatClient = client
    .GetChatClient(deploymentName); //❷

ClientResult<ChatCompletion> response = azureOpenAIChatClient
    .CompleteChat(prompt); //❸
Console.WriteLine($"Assistant (Azure ChatClient): "
    + $"{response.Value.Content.First().Text}");

IChatClient chatClient = new ApiKeyCredential(apiKey),
    new OpenAIClientOptions { Endpoint = new Uri(endpoint) })
    .GetChatClient(deploymentName)
    .AsIChatClient(); //❹

ChatResponse chatResponse = await chatClient.GetResponseAsync(prompt); //❺
Console.WriteLine($"Assistant (IChatClient): {chatResponse.Text}");
```

❶ Gets the native OpenAI client

❷ Gets the native chat client

❸ Invokes the native chat client with the prompt

❹ Converts the native chat client to abstract chat client `IChatClient`

❺ Invokes the abstract chat client with the prompt

Output:

```
User: You are an AI assistant controlling a robot car capable of performing
basic moves: forward, backward, turn left, turn right, and stop.
You have to break down the provided complex commands into the basic
moves you know.
There is a tree in front of the car. Avoid it and resume the original path.
Respond with a JSON array like [move1, move2, move3].
Do not respond with reasoning, comments, or any additional text.

Assistant (Azure ChatClient): ["stop", "turn left", "forward", "turn right",
"forward"]
```

```
Assistant (IChatClient): ["stop", "turn left", "forward", "turn right", "forward"]
```

The OpenAI Chat Completion API provides a flexible, stateless approach to building conversational AI applications. This example shows how Robby can use cloud-based AI models to break down complex navigation commands into basic moves that it can execute, highlighting the value of prompt engineering.

EXERCISE

In listing 3.4, find the lines where the chat client is used as `IChatClient`:

```
IChatClient chatClient = new(
    new ApiKeyCredential(apiKey),
    new OpenAIClientOptions { Endpoint = new Uri(endpoint) })
    .GetChatClient(model)
    .AsIChatClient();
ChatResponse chatResponse = await chatClient.GetResponseAsync(prompt);
Console.WriteLine($"Assistant (IChatClient): {chatResponse.Text}");
```

and replace them with a direct chat client (without converting to `IChatClient`):

```
ChatClient azureOpenAIChatClient = new(
    new ApiKeyCredential(apiKey),
    new OpenAIClientOptions { Endpoint = new Uri(endpoint) })
    .GetChatClient(deploymentName);
ClientResult<ChatCompletion> response = azureOpenAIChatClient
    .CompleteChat(prompt);
Console.WriteLine($"Assistant (Azure ChatClient):
    {response.Value.Content.First().Text}");
```

Run the code and compare the output with the original. They should be similar, even though one uses the abstraction and the other calls the provider directly.

3.4.3 Anthropic Client

`AnthropicClient` is the client implementation provided by Anthropic. Like the OpenAI implementation, each request is independent: you send a message or the complete conversation history and receive a response.

PRACTICAL EXAMPLE

Listing 3.5 shows `AnthropicClient`, which is compatible with the `IChatClient` abstraction.

NOTE Obtain an API key by signing up at <https://platform.claude.com>. After creating an account, go to the API Keys section of the console to generate a new key. The process is similar to other model providers: creating an account, verifying your email, and generating credentials in the developer console.

Requirements (NuGet packages):

```
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
dotnet add package Anthropic
```

Code path: `/AnthropicChatClient/Program.cs`

Listing 3.5 Anthropic chat client

```
using Anthropic;
using Microsoft.Extensions.AI;
using Microsoft.Extensions.Configuration;

var configuration = new ConfigurationBuilder()
    .AddUserSecrets<Program>().Build();

var model = configuration["Anthropic:ModelId"];
var apiKey = configuration["Anthropic:ApiKey"];
```

```

var prompt = """
    You are an AI assistant controlling a robot car capable of performing
    basic moves: forward, backward, turn left, turn right, and stop.
    You have to break down the provided complex commands into the basic
    moves you know.
    There is a tree in front of the car.
    Avoid it and resume the original path.

    Respond with a JSON array like [move1, move2, move3].
    Do not respond with reasoning, comments, or any additional text.
    """;
Console.WriteLine($"USER: {prompt}");

IChatClient chatClient = new AnthropicClient() { ApiKey = apiKey }
    .AsIChatClient(defaultModelId: model); //❶

ChatResponse chatResponse = await chatClient
    .GetResponseAsync(prompt); //❷
Console.WriteLine($"Assistant (IChatClient): {chatResponse.Text}");

```

❶ Converts to IChatClient abstraction for provider independence

❷ Uses abstraction IChatClient for chat client general features

Code output:

```

USER: You are an AI assistant controlling a robot car capable of performing
basic moves: forward, backward, turn left, turn right, and stop.
You have to break down the provided complex commands into the basic
moves you know.
There is a tree in front of the car. Avoid it and resume the original path.

Respond with a JSON array like [move1, move2, move3].
Do not respond with reasoning, comments, or any additional text.

Assistant (IChatClient): ````json
["turn left", "forward", "turn right", "forward", "turn right", "forward", "turn
left"]
````

```

Anthropic's integration completes the picture of MEAI as a provider-agnostic chat layer. Anthropic is the company behind the Claude family of LLMs, which play a similar role to OpenAI's GPT models in many architectures. As with OpenAI, we work with a native Anthropic client when we need vendor-specific capabilities, then wrap it in `IChatClient` so the rest of the codebase does not care which model answers the request. Swapping in a Claude model is mostly a configuration decision, not an architectural change, which aligns with the broader goal of keeping Robby's behavior consistent as we vary models, vendors, or hosting environments.

---

**WARNING** Anthropic library does not provide a concrete chat client to directly work with it. Anthropic chat client inherits `IChatClient`.

---

## EXERCISE

Locate the lines in listing 3.5 that use the chat client as `IChatClient`:

```

IChatClient chatClient = new AnthropicClient() { ApiKey = apiKey }
 .AsIChatClient(defaultModelId: model);

```

and try to replace them with a direct chat client (with no conversion to `IChatClient`):

```

AnthropicClient? anthropicClient = new() { ApiKey = apiKey };

```

You will notice that the Anthropic client cannot be used directly, so it must be converted to `IChatClient` first.



### 3.4.4 Conversation History Management

The *Chat Completions* `IChatClient` used in this example is stateless, so this application maintains and resends the history. Other `IChatClient` implementations can maintain service-side state and use `ConversationId`; follow the provider's state-management contract. Your app is responsible for managing the message list and providing context with each chat request. Listing 3.6 shows how to maintain a simple message collection and use it to build a stateful conversation.

#### Listing 3.6 Conversation history management

```
List<ChatMessage> conversation =
[
 new(ChatRole.System, "You are a robot car assistant."),
 new(ChatRole.User, "Tell me what you can do for me.")
];

conversation.Add(new(ChatRole.User, prompt)); //❶

ChatResponse response = await chatClient
 .GetResponseAsync(conversation); //❷

conversation.AddRange(response.Messages); //❸
```

❶ Adds user message to conversation

❷ Gets response

❸ Adds assistant's response to conversation

Effective conversation history management transforms a stateless chat API into a contextual, multi-turn conversational experience. By maintaining conversation history and storing model responses, developers can create chatbots with short-term memory and multi-turn coherent interaction with the AI model.

#### PRACTICAL EXAMPLE

Let's redesign the chatbot example (listing 3.7) using a persistent conversation list that evolves with each user request.

Requirements (NuGet packages):

```
dotnet add package Microsoft.Extensions.AI.OpenAI
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
```

Code path: `/Chatbot/Program.cs`

#### Listing 3.7 Chatbot

```
using OpenAI;
using Microsoft.Extensions.AI;
using Microsoft.Extensions.Configuration;
using System.ClientModel;

var configuration = new ConfigurationBuilder()
 .AddUserSecrets<Program>().Build();

var endpoint = configuration["AzureOpenAI:Endpoint"];
var apiKey = configuration["AzureOpenAI:ApiKey"];
var deploymentName = configuration["AzureOpenAI:DeploymentName"];

IChatClient chatClient = OpenAIClient(
 new ApiKeyCredential(apiKey),
 new OpenAIClientOptions { Endpoint = new Uri(endpoint) })
 .GetChatClient(deploymentName)
 .AsIChatClient();

var system = """
You are an AI assistant controlling a robot car capable of performing
basic moves: forward, backward, turn left, turn right, and stop.
```

```

 You have to break down the provided complex commands into the basic
 moves you know.
 Use a JSON array like [move1, move2, move3] for the response.
 Respond only with the moves, without any additional explanations.
 """

List<ChatMessage> conversation = [
 new ChatMessage(ChatRole.System, system)
];
Console.WriteLine($"System:\n{system}\n");

while (true)
{
 Console.Write("User: ");
 var input = Console.ReadLine();
 if (string.IsNullOrEmpty(input)) break;
 var prompt = $"
 Complex command:
 \"{input}\"
 ";

 conversation.Add(new ChatMessage(ChatRole.User, prompt)); //❶

 ChatResponse response = await chatClient.GetResponseAsync(conversation); //❷
 Console.WriteLine($"Assistant (Azure ChatClient): {response.Text}");
 conversation.AddRange(response.Messages); //❸
}

```

- ❶ Adds user message
- ❷ Requests response
- ❸ Appends assistant output

Code output:

```

System:
You are an AI assistant controlling a robot car capable of performing
basic moves: forward, backward, turn left, turn right, and stop.
You have to break down the provided complex commands into the basic
moves you know.
Respond with a JSON array like [move1, move2, move3].
Do not respond with reasoning, comments, or any additional text.

User: There is a tree directly in front of the car.
Avoid it and then come back to the original path.

Assistant (Azure ChatClient): ["turn right", "forward", "turn left",
"forward", "turn left", "forward", "turn right"]

```

Effective conversation history management transforms a stateless chat API into a contextual, multi-turn conversational experience.

#### EXERCISE

Place both statements inside the while loop immediately after `GetResponseAsync` in listing 3.7:

```

Console.WriteLine($"Input tokens: {response.Usage.InputTokenCount}");
Console.WriteLine($"Output tokens: {response.Usage.OutputTokenCount}");

```

Run the code and see the input/output token consumption for each response.

### 3.4.5 Chat Options

When we prompt a chat client for a response, several optional parameters can affect how the model generates output. You provide them through the `ChatOptions` argument passed to `GetResponseAsync`.

Table 3.1 lists the most common `ChatOptions` properties and their purpose.

Table 3.1 Common `ChatOptions` properties

| Property         | Purpose                            | Typical Range                                |
|------------------|------------------------------------|----------------------------------------------|
| Temperature      | Controls randomness and creativity | 0.0 (deterministic) to 2.0 (highly creative) |
| MaxOutputTokens  | Limits response length             | 1 to model maximum                           |
| TopP             | Nucleus sampling threshold         | 0.0 to 1.0                                   |
| FrequencyPenalty | Reduces token repetition           | -2.0 to 2.0                                  |
| PresencePenalty  | Encourages new topics              | -2.0 to 2.0                                  |
| ResponseFormat   | Controls output structure          | Text, JSON, or structured JSON               |

`ChatOptions` maps closely to model parameters offered by model providers. Some model providers expose additional configuration options, which can be supplied through the `AdditionalPropertiesDictionary`, a dictionary of key-value pairs that lets us extend configuration beyond the standard properties. For example, `ResponseFormat` uses the built-in `ForJsonSchema` method to read the response schema from a C# type, record, or class.

**WARNING** For reasoning models like GPT-5 and the o-series (o1, o3, o4-mini) some `ChatOptions` parameters are not supported. Setting them will throw an HTTP 400 at runtime. For `Temperature`: only the default value `1` is accepted. For `TopP`, `FrequencyPenalty`, `PresencePenalty` only their respective defaults are accepted, and `StopSequences`, `Seed` are not supported at all. Other parameters work but behave differently than expected, so it is strongly recommended to test the code with models from each family you plan to support.

PRACTICAL EXAMPLE

Listing 3.8 shows the updated request with `ChatOptions`.

Requirements (NuGet packages):

```
dotnet add package Microsoft.Extensions.AI.OpenAI
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
```

Code path: `/ChatbotWithChatOptions/Program.cs`

Listing 3.8 Chatbot with ChatOptions

```
using OpenAI;
using Microsoft.Extensions.AI;
using Microsoft.Extensions.Configuration;
using System.ClientModel;

var configuration = new ConfigurationBuilder()
 .AddUserSecrets<Program>().Build();

var endpoint = configuration["AzureOpenAI:Endpoint"];
var apiKey = configuration["AzureOpenAI:ApiKey"];
var deploymentName = configuration["AzureOpenAI:DeploymentName"];

IChatClient chatClient = new OpenAIClient(
 new ApiKeyCredential(apiKey),
 new OpenAIClientOptions { Endpoint = new Uri(endpoint) })
 .GetChatClient(deploymentName)
 .AsIChatClient();

var system = """
 You are an AI assistant controlling a robot car capable of performing
 basic moves: forward, backward, turn left, turn right, and stop.
```

```

 You have to break down the provided complex commands into the basic
 moves you know.
 Use a JSON array like [move1, move2, move3] for the response.
 Respond only with the moves, without any additional explanations.
 """;

List<ChatMessage> conversation = [
 new ChatMessage(ChatRole.System, system)
];
Console.WriteLine($"System:\n{system}\n");

ChatOptions options = new()
{
 Temperature = 0.4f,
 MaxOutputTokens = 500,
 TopP = 0.9f,
 FrequencyPenalty = 0.5f,
 PresencePenalty = 0.3f,
 StopSequences = ["END"],
 ResponseFormat = ChatResponseFormat.ForJsonSchema<StepsResponse>(),
};

while (true)
{
 Console.Write("User: ");
 var input = Console.ReadLine();
 if (string.IsNullOrEmpty(input)) break;
 var prompt = $$$$
 Complex command:
 "{input}"
 """; //❶

 conversation.Add(new ChatMessage(ChatRole.User, prompt)); //❷

 ChatResponse response = await chatClient
 .GetResponseAsync(conversation, options); //❸
 Console.WriteLine($"Assistant (Azure ChatClient): {response.Text}");
 conversation.AddRange(response.Messages); //❹
}

record StepsResponse(string[] Steps); //❺
❶ Reads the input and prepares a prompt
❷ Adds user message
❸ Requests response with ChatOptions
❹ Appends assistant message output
❺ Declares StepsResponse record used in ForJsonSchema

```

Code output (enter this prompt when asked: *There is a tree directly in front of the car. Avoid it and then come back to the original path.*):

```

System: You are an AI assistant controlling a robot car capable of performing
basic moves: forward, backward, turn left, turn right, and stop.
You have to break down the provided complex commands into the basic moves
you know.
Respond with a JSON array like [move1, move2, move3].
Do not respond with reasoning, comments, or any additional text.

User: There is a tree directly in front of the car.
Avoid it and then come back to the original path.

Assistant (Azure ChatClient): {"steps":["stop","turn right","forward","turn
left","forward","turn left","forward","turn right"]}

```

By maintaining a message collection and using `ChatOptions` to control model behavior, developers can create sophisticated chatbots with short-term memory and customizable personalities.

**EXERCISE**

Update the `ChatOptions` used with `GetResponseAsync` (see listing 3.8) so that:

```
MaxOutputTokens = 5
```

Run the program and, when prompted, enter:

```
There is a tree directly in front of the car. Avoid it.
```

You should see a truncated response similar to:

```
Assistant: {"steps":["turn right
```

Inspect `response.FinishReason` and confirm that its value is `length`. This means the model stopped because it hit the `MaxOutputTokens` limit, not because it completed the full response.

### 3.4.6 Streaming Prompt Response

Unlike traditional request-response mechanisms that wait for complete responses before answering, prompt streaming allows partial responses to be processed in real time. This is particularly useful for applications requiring immediate feedback or for handling large responses incrementally, but less so for structured information such as JSON, where we need the entire content to determine its schema.

#### HOW STREAMING WORKS

Prompt response streaming operates by establishing a continuous connection with a model provider, allowing data to be sent and received in partial responses (figure 3.3).

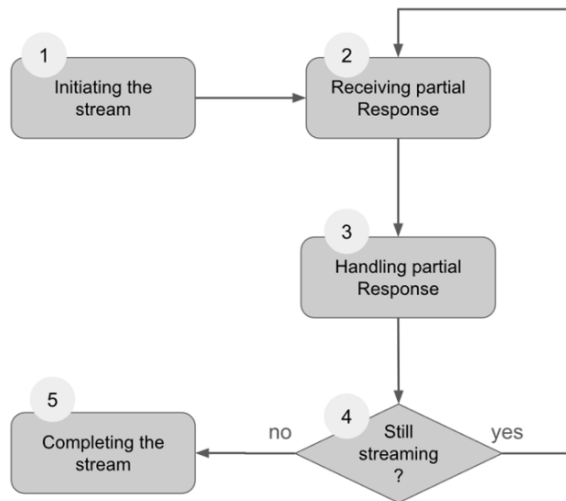


Figure 3.3 Asynchronous streaming from a large language model. After stream initiation (1), the process enters a loop of receiving (2) and handling partial responses (3). When streaming ends (4), it's followed by stream completion (5).

Here's how it works, step by step:

1. **Initiating the stream:** The process begins when we send the initial prompt to the large language model and open a streaming connection.
2. **Receiving partial responses:** As the large language model processes the prompt, it starts sending partial responses asynchronously. These responses arrive in real time, allowing the application to begin processing the data immediately.

3. Handling partial responses: Each partial response is handled as it arrives. This can involve updating the user interface, processing the data for further use, or storing it for later analysis.
4. Completing the Stream: Once a large language model finishes processing the prompt, the streaming connection closes.
5. The final partial response is received, and any necessary cleanup or final processing is completed.

Streaming ensures continuous processing of incoming data until the stream is exhausted.

With asynchronous streaming, the user experience is better because the user doesn't have to wait for the entire response to be generated at once. On the other hand, a formatted response such as JSON usually needs to be fully generated before you can use it.

### STREAMING RESPONSES

In scenarios where real-time updates are beneficial, we can use streaming methods. Here's how to stream text generation:

```
await foreach (ChatResponseUpdate update in chatClient
 .GetStreamingResponseAsync(conversation, options))
{
 Console.Write(update);
}
```

This method provides a streaming list of completion updates, allowing for more responsive, interactive applications. Its return type is `IAsyncEnumerable<ChatResponseUpdate>`.

This approach is particularly useful for simpler applications or when quick, straightforward language model interactions are required.

To make Robby more versatile, we'll introduce chat clients from various model providers and learn how to use an abstract chat client for greater flexibility and reusability.

While we've explored the conceptual benefits of streaming, implementing it in practice requires careful handling of asynchronous updates and state management.

### PRACTICAL EXAMPLE

Listing 3.9 shows a complete streaming implementation for a conversational chatbot, showing how to accumulate partial responses and maintain conversation history. This pattern ensures Robby can provide real-time feedback while preserving the context needed for multi-turn dialogs.

Requirements (NuGet packages):

```
dotnet add package Microsoft.Extensions.AI.OpenAI
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
```

Code path: `/ChatbotWithStreaming/Program.cs`

#### Listing 3.9 Chatbot with streaming

```
using OpenAI;
using Microsoft.Extensions.AI;
using Microsoft.Extensions.Configuration;
using System.Text;

var configuration = new ConfigurationBuilder()
 .AddUserSecrets<Program>().Build();

var model = configuration["OpenAI:ModelId"];
var apiKey = configuration["OpenAI:ApiKey"];

IChatClient chatClient = new OpenAIClient(apiKey)
 .GetChatClient(model)
 .AsIChatClient();
```

```

var system = """
You are an AI assistant controlling a robot car capable of performing
basic moves: forward, backward, turn left, turn right, and stop.
You have to break down the provided complex commands into the basic
moves you know.
Use a JSON array like [move1, move2, move3] for the response.
Respond only with the moves, without any additional explanations.
""";

StringBuilder response = new();

List<ChatMessage> conversation = [
 new ChatMessage(ChatRole.System, system)
];
Console.WriteLine($"System:\n{system}\n");

while (true)
{
 Console.Write("User: ");
 var input = Console.ReadLine();
 if (string.IsNullOrEmpty(input)) break;
 var prompt = $"""
 Complex command:
 "{input}"
 """;

 conversation.Add(new ChatMessage(ChatRole.User, prompt)); //❶

 ChatOptions options = new()
 {
 Temperature = 0.4F,
 ResponseFormat = ChatResponseFormat.ForJsonSchema<StepsResponse>(),
 };

 Console.Write($"Assistant (Azure ChatClient): ");
 await foreach (ChatResponseUpdate update in chatClient.
 GetStreamingResponseAsync(conversation, options)) //❷
 {
 response.Append(update); //❸
 Console.Write(update);
 }
 Console.WriteLine();
 conversation.Add(
 new ChatMessage(ChatRole.Assistant, response.ToString())); //❹
 response.Clear();
}

record StepsResponse(string[] Steps);

```

❶ Adds user prompt to conversation

❷ Streams response

❸ Accumulate the partial response in StringBuilder

❹ Adds complete response to conversation and clear buffer for next iteration

Code output:

```

System:
You are an AI assistant controlling a robot car capable of performing
basic moves: forward, backward, turn left, turn right, and stop.
You have to break down the provided complex commands into the basic
moves you know.
Respond with a JSON array like [move1, move2, move3].
Do not respond with reasoning, comments, or any additional text.

User: There is a tree in front of the car.
Avoid it and resume the original path.

```

```
Assistant (Azure ChatClient): {"steps":["stop","turn right","forward","turn
left","forward"]}
```

Streaming responses can dramatically improve the user experience by providing immediate feedback as the model generates text. Instead of waiting for a complete response, applications can display content token by token, creating a more natural and responsive interaction. This implementation shows proper stream handling with `await foreach` and `ChatResponseUpdate`, along with careful message reconstruction for the conversation history.

---

**IMPORTANT** For LLM request-response workflows, streaming and JSON responses serve different purposes. Use streaming for machine-to-user interactions (for example, progressively rendering assistant text in a chat UI). For machine-to-machine integrations, prefer nonstreamed calls that return a complete JSON object. Streaming JSON is rarely practical because the client must buffer chunks and reassemble them into valid JSON before parsing.

---

While streaming offers superior interactivity for UI applications, developers should keep in mind that structured formats like JSON require complete responses for parsing. That makes streaming better suited to narrative or explanatory content than to programmatic output.

#### EXERCISE

Add a short delay inside the streaming loop in listing 3.9 after the line:

```
response.Append(update);
```

Like this:

```
await Task.Delay(50);
```

Run the code again and see how the assistant's answer appears token by token in the console. The delay slows the rendering so you can see each partial update arrive, making the streaming behavior easier to notice when models respond quickly.

### 3.5 Adding Logging Using Middleware

As the prompt gets more complex, we need more feedback about what happens, and logging is a powerful, highly configurable tool for that. In MEAI, logging is registered through *middleware*. Middleware is a pipeline that lets you decompose a complex module. We'll do the same with our chat client using the `ChatClientBuilder` class.

For learning and debugging, setting the log level to `Trace` provides the most granular diagnostic information, which is invaluable during development and early troubleshooting. In production, we may choose a less verbose level, such as `Debug` or `Information`, to strike a balance between detail, performance, and verbosity.

#### PRACTICAL EXAMPLE

See listing 3.10 for a chatbot with logging implemented via middleware.

Requirements (NuGet packages):

```
dotnet add package Microsoft.Extensions.AI
dotnet add package Microsoft.Extensions.AI.OpenAI
dotnet add package Microsoft.Extensions.Logging.Console
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
```

Code path: `/ChatClientWithMiddleware/Program.cs`

#### Listing 3.10 Chat Client with logging middleware

```
using Microsoft.Extensions.AI;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.Logging;
```



```

using OpenAI;

var configuration = new ConfigurationBuilder()
 .AddUserSecrets<Program>().Build();

var model = configuration["OpenAI:ModelId"];
var apiKey = configuration["OpenAI:ApiKey"];

var prompt = """
 You are an AI assistant controlling a robot car capable of performing
 basic moves: forward, backward, turn left, turn right, and stop.
 You have to break down the provided complex commands into the basic
 moves you know.
 There is a tree in front of the car.
 Avoid it and resume the original path.
 Respond with a JSON array like [move1, move2, move3].
 Do not respond with reasoning, comments, or any additional text.
 """;
Console.WriteLine($"User: {prompt}");

IChatClient baseClient = new OpenAIClient(apiKey)
 .GetChatClient(model)
 .AsIChatClient(); //❶

using ILoggerFactory loggerFactory = LoggerFactory.Create(builder =>
{
 builder.AddConsole();
 builder.SetMinimumLevel(LogLevel.Trace);
}); //❷

IChatClient chatClient = new ChatClientBuilder(baseClient)
 .UseLogging(loggerFactory)
 .Build(); //❸

ChatResponse chatResponse = await chatClient.GetResponseAsync(prompt);
Console.WriteLine($"Assistant (IChatClient): {chatResponse.Text}");

```

❶ Creates base `IChatClient` from `OpenAI`

❷ Configures the logging factory with console output and Trace level

❸ Builds chat client with logging middleware pipeline

#### Code output:

```

User: You are an AI assistant controlling a robot car capable of performing
basic moves: forward, backward, turn left, turn right, and stop.
You have to break down the provided complex commands into the basic moves
you know.
There is a tree in front of the car. Avoid it and resume the original path.
Respond with a JSON array like [move1, move2, move3].
Do not respond with reasoning, comments, or any additional text.
trce: Microsoft.Extensions.AI.LoggingChatClient[805843669]
 GetResponseAsync invoked: [
 {
 "role": "user",
 "contents": [
 {
 "$type": "text",
 "text": "You are an AI assistant controlling a robot car
capable of performing basic moves: forward, backward, turn left, turn
right, and stop.\r\n\r\nYou have to break down the provided complex
commands into the basic moves you know.\r\n\r\nThere is a tree in front of
the car. Avoid it and resume the original path.\r\n\r\nRespond with a JSON
array like [move1, move2, move3].\r\nDo not respond with reasoning,
comments, or any additional text."
 }
]
 }
]

```

```

]. Options: null. Metadata: {
 "providerName": "openai",
 "providerUri": "https://api.openai.com/v1",
 "defaultModelId": "gpt-4o"
 }.
trce: Microsoft.Extensions.AI.LoggingChatClient[384896670]
 GetResponseAsync completed: {
 "messages": [
 {
 "createdAt": "2025-11-13T15:21:55+00:00",
 "role": "assistant",
 "contents": [
 {
 "$type": "text",
 "text": "[\"stop\", \"turn right\", \"forward\", \"turn
left\", \"forward\", \"turn left\", \"forward\", \"turn right\"]"
 }
],
 "messageId": "chatcmpl-CbTQxgVYflbNdf7t15an8NGk9juuL"
 }
],
 "responseId": "chatcmpl-CbTQxgVYflbNdf7t15an8NGk9juuL",
 "modelId": "gpt-4o-2024-08-06",
 "createdAt": "2025-11-13T15:21:55+00:00",
 "finishReason": "stop",
 "usage": {
 "inputTokenCount": 113,
 "outputTokenCount": 28,
 "totalTokenCount": 141,
 "additionalCounts": {
 "InputTokenDetails.AudioTokenCount": 0,
 "InputTokenDetails.CachedTokenCount": 0,
 "OutputTokenDetails.ReasoningTokenCount": 0,
 "OutputTokenDetails.AudioTokenCount": 0,
 "OutputTokenDetails.AcceptedPredictionTokenCount": 0,
 "OutputTokenDetails.RejectedPredictionTokenCount": 0
 }
 }
 }.

Assistant (IChatClient): ["stop", "turn right", "forward", "turn left",
"forward", "turn left", "forward", "turn right"]

```

This configuration registers a console logging provider to capture all logs at the `Trace` level. Adjust the minimum log level to suit your deployment context, so your application can effectively capture and diagnose problems as they arise.

Logging isn't just useful for debugging; it's a critical feature for enterprise-ready applications that need to maintain robust observability and streamlined troubleshooting over time.

We can use logs to track metrics such as:

- **Token Usage:** Captures the number of tokens used in prompts and responses.
- **Response Times:** Measures the duration of API calls.

**Errors:** Identifies issues such as timeouts or rate limits.

#### EXERCISE

In example 3.10, locate the logger factory configuration:

```
builder.SetMinimumLevel(LogLevel.Trace);
```

Change the minimum level to another value, such as `LogLevel.Debug` or `LogLevel.Information`, and run the program again. At `Debug`, the detailed message/options payloads disappear but invocation/completion events remain. At `Information`, successful-call `Debug/Trace` events are filtered out; `Error` logs are still emitted on failure.

## Summary

- A chatbot is an application that combines prompts, chat clients, options, and history to turn raw models into task-focused assistants.
- Use Chat Completions as a stateless request-response API and rely on explicit message history when we need multi-turn conversations.
- Treat OpenAI, Anthropic, Ollama, and ONNX as interchangeable model providers that we access through provider-specific clients or `IChatClient`.
- Standardize on `IChatClient` so your code doesn't depend on a particular SDK, endpoint, or hosting model.
- Understand that `IChatClient` is thread-safe and supports high concurrency, which fits web backends and other multi-user workloads.
- Use multimodal messages when models support them by combining text, URLs, and binary data such as images and audio in a single `ChatMessage`.
- Implement provider-specific configuration (keys, endpoints, models) at the edge and keep the rest of the code focused on business behavior.
- Start with simple, non-streaming calls, then add streaming responses when we want lower perceived latency or more interactive user experiences.

# 4

## *Building responses and self-hosted clients*

### **This chapter covers**

- Using the *Responses API* for single and background calls
- Creating `ResponsesClient` for OpenAI and for Azure OpenAI
- Running Robby on self-hosted Ollama and ONNX models
- Handling failures with finish reasons and exceptions

We already introduced Robby as a simple agent. Now we focus on how Robby talks to models in production. The *Responses API* extends traditional chat interfaces with server-managed state and background processing, making it easier to handle long-running AI tasks without blocking your application. Self-hosted clients give you complete control over the model infrastructure, running locally through Ollama or embedded in your process via ONNX.

### **4.1 What is Responses API?**

One of the most powerful features of the *Responses API* is its support for background response processing through *continuation tokens*. A continuation token is state: a property in data returned by an agent that the client stores and sends back on later requests to check the status of the same long-running operation. The client does not interpret or modify the token. It simply passes it back so the service can locate and resume the in-progress response. This capability allows applications to handle computationally intensive queries that may exceed typical request timeout limits. Instead of waiting synchronously for a potentially long-running response, the application receives an initial acknowledgment with a continuation token, then polls for completion. This pattern is essential for production systems that require resilience and scalability when dealing with complex AI workloads.

Figure 4.1 illustrates how the *Responses API* handles background responses when we set `AllowBackgroundResponses = true`. Instead of blocking until the model finishes, the client gets an initial acknowledgment and then polls for completion by checking the response status.

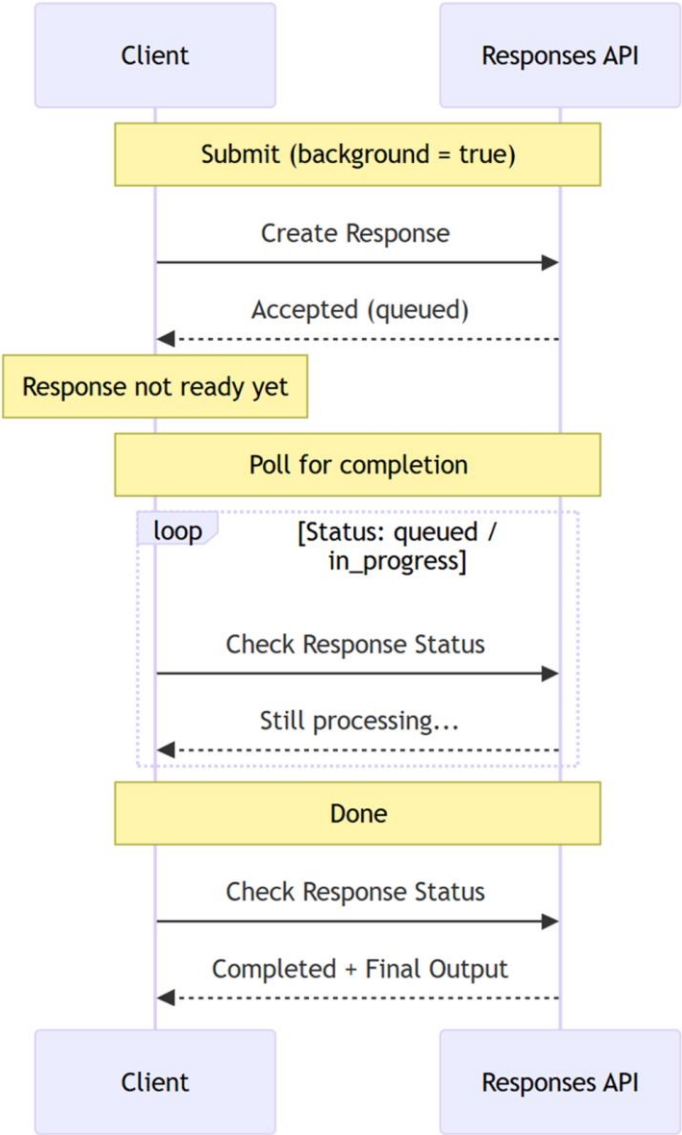


Figure 4.1 The client sends a background response request to the Responses API, receives an accepted but not-yet-complete result, then polls the response status until processing finishes and the final output is returned.

The client sends a background response request to the *Responses API*, receives an accepted but not yet complete result, then repeatedly polls the response status until processing finishes and the final output is returned.

#### 4.1.1 Introduction to ResponsesClient

`GetResponsesClient` is a concrete way of using the *Responses API* from code. A responses client can be provider specific, like `ResponsesClient`, or wrapped behind the `IChatClient` abstraction that MEAI uses everywhere else. In practice, this means we can start with a provider SDK and

gradually converge on the unified `IChatClient` surface without re-architecting our agents. We will also look at how responses clients differ from classic chat completion clients and why they fit better with long-running, stateful interactions.

### 4.2 Creating and Configuring ResponsesClient

The *Responses API* provides multiple client implementations that support direct, provider-specific access as well as the unified `IChatClient` abstraction. While OpenAI's *Chat Completion API* served conversational needs for years, the *Responses API* is now the recommended approach for modern AI applications. It offers stateful conversation management, built-in support for long-running operations, and simplified input models that reduce boilerplate code.

Table 4.1 presents the key differences between *Chat Completion API* and *Responses API*.

Table 4.1 Key differences between Chat Completion API and Responses API

| Feature              | Chat Completion API | Responses API                      |
|----------------------|---------------------|------------------------------------|
| Conversation History | Client-managed      | Service-managed (optional)         |
| State Management     | Application handles | Service can handle                 |
| Background Responses | Not supported       | Supported with continuation tokens |
| API Style            | Stateless           | Can be stateful                    |
| Complexity           | Lower-level control | Higher-level convenience           |

When to use *Chat Completion API*: Choose it when you need complete control over conversation history, are building stateless APIs or serverless functions, want to store conversations in your own database, or need to manipulate or filter the conversation context.

When to use *Responses API*: Use it for long-running tasks that may exceed typical timeout limits, scenarios where service-managed state simplifies your architecture, background processing with polling patterns, or when you want reduced boilerplate for common conversational workflows.

#### 4.2.1 OpenAI Responses Client

The OpenAI *Responses API* keeps a simple request-response mode but adds server-side state management and support for background processing and other advanced scenarios.

Here we use it in a straightforward single request-response mode, so it behaves much like the classic *Chat Completions API*. Next, we switch to a long-running task and see how the same client shines when we enable background responses.

**PRACTICAL EXAMPLE**

Listing 4.1 shows the OpenAI chat client `ResponsesClient` and the MEAI chat client abstraction `IChatClient`.

Requirements (NuGet packages):

```
dotnet add package Microsoft.Extensions.AI.OpenAI
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
```

Code path: `/OpenAIResponses/Program.cs`

**Listing 4.1 OpenAI Responses**

```
using Microsoft.Extensions.Configuration;
using OpenAI;
using Microsoft.Extensions.AI;
using System.ClientModel;
using OpenAI.Responses;
```

```

var configuration = new ConfigurationBuilder()
 .AddUserSecrets<Program>().Build();

var model = configuration["OpenAI:ModelId"];
var apiKey = configuration["OpenAI:ApiKey"];

var prompt = """
 You are an AI assistant controlling a robot car capable of performing
 basic moves: forward, backward, turn left, turn right, and stop.
 You have to break down the provided complex commands into the basic
 moves you know.
 There is a tree in front of the car.
 Avoid it and resume the original path.
 Respond with a JSON array like [move1, move2, move3].
 Do not respond with reasoning, comments, or any additional text.
 """;
Console.WriteLine($"User: {prompt}");

#pragma warning disable OPENAI001

IChatClient chatClient = new OpenAIClient(apiKey)
 .GetResponsesClient()
 .AsIChatClient(model); //❶
ChatResponse chatResponse = await chatClient.GetResponseAsync(prompt);
Console.WriteLine($"Assistant (IChatClient): {chatResponse.Text}");

```

#### ❶ Converts to IChatClient abstraction for unified interface

#### Code output:

```

User: You are an AI assistant controlling a robot car capable of performing
basic moves: forward, backward, turn left, turn right, and stop.
You have to break down the provided complex commands into the basic
moves you know.
There is a tree in front of the car. Avoid it and resume the original path.
Respond with a JSON array like [move1, move2, move3].
Do not respond with reasoning, comments, or any additional text.

Assistant (IChatClient): ``json
["turn right", "forward", "turn left", "forward"]
``

```

*OpenAI Responses API* represents an evolution in conversational AI, offering a streamlined approach to chat interactions. While maintaining the core capabilities of the *Chat Completion API*, it introduces a simplified input model that can reduce boilerplate code in common scenarios. This example shows how both the provider-specific `ResponsesClient` and the unified `IChatClient` abstraction can be used interchangeably, reinforcing the architectural principle of model provider independence in MEAI.

#### EXERCISE

Locate the lines in listing 4.1 that use the chat client as `IChatClient`:

```

IChatClient chatClient = new OpenAIClient(apiKey)
 .GetResponsesClient()
 .AsIChatClient(model);
ChatResponse chatResponse = await chatClient.GetResponseAsync(prompt);
Console.WriteLine($"Assistant (IChatClient): {chatResponse.Text}");

```

and replace them with direct chat clients (with no conversion to `IChatClient`):

```

ResponsesClient responsesClient = new OpenAIClient(apiKey)
 .GetResponsesClient();
ClientResult<ResponseResult> response = responsesClient
 .CreateResponse(model, prompt);
Console.WriteLine($"Assistant (OpenAI Responses): "
 + $"{response.Value.GetOutputText()}");

```

Run the code and compare its output with the original version. They should be similar, even though one uses the abstraction and the other calls the provider directly.

### 4.2.2 OpenAI Responses Client with Background Response

You can use `ResponsesClient` as a simple request-response client, like *Chat Completion* API, but OpenAI's *Responses API* becomes most valuable when you stop thinking in terms of a single request-response cycle and start treating responses as long-running jobs, like in the most production-ready use-cases. In many real systems, you don't know how complex a question will be, how much context the model will need, or whether the answer will fit within typical HTTP timeouts. Background responses solve this by turning a single blocking call into a short initial request plus a polling loop that resumes work using a continuation token. You keep the same `IChatClient` abstraction and the same mental model for prompts and responses, but you gain a predictable pattern you can reuse anywhere you expect heavy workloads: submit, acknowledge, then poll until the work is done.

#### PRACTICAL EXAMPLE

Listing 4.2 shows a chat client with background responses enabled by setting `AllowBackgroundResponses = true` in the `ChatOptions` object. Note the pragma directive `#pragma warning disable OPENAI001`, which suppresses the warning because `GetResponsesClient` is still experimental at the time of writing.

Requirements (NuGet packages):

```
dotnet add package Microsoft.Extensions.AI.OpenAI
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
```

Code path: `/OpenAIResponsesWithBackgroundResponses/Program.cs`

#### Listing 4.2 OpenAI Responses with Background Responses

```
using Microsoft.Extensions.AI;
using Microsoft.Extensions.Configuration;
using OpenAI;

var configuration = new
ConfigurationBuilder().AddUserSecrets<Program>().Build();

var model = configuration["OpenAI:ModelId"]!;
var apiKey = configuration["OpenAI:ApiKey"]!;

var prompt = """
You are an AI assistant controlling a robot car capable of performing
basic moves: forward, backward, turn left, turn right, and stop.
You have to break down the provided complex commands into the basic
moves you know.
Respond only with the moves and their parameters (angle or distance),
and provide additional explanations.

Complex command:
"There is a tree directly in front of the car.
Avoid it and then return to the original path."
""";
Console.WriteLine($"User: {prompt}");

#pragma warning disable OPENAI001
IChatClient chatClient = new OpenAIClient(apiKey)
 .GetResponsesClient()
 .AsIChatClient(model);

try
{
 ChatOptions options = new() { AllowBackgroundResponses = true }; //❶
 ChatResponse chatResponse = await chatClient
 .GetResponseAsync(prompt, options); //❷
 Console.ForegroundColor = ConsoleColor.Yellow;
 Console.WriteLine($"\"\\n[INITIAL] Assistant:");
```



```

Console.WriteLine($"Finish Reason: {chatResponse.FinishReason?.Value
?? "(null)"}");
Console.WriteLine("Has Continuation Token: "
+ $"{chatResponse.ContinuationToken is not null}");
Console.ResetColor();
Console.WriteLine(chatResponse.Text);

var token = chatResponse.ContinuationToken;
if (token is not null) //❸
{
 Console.ForegroundColor = ConsoleColor.Yellow;
 Console.Write("POLLING");
 Console.ResetColor();

 const int maxPollingAttempts = 60;
 const int pollingDelayMs = 1000;
 int attempts = 0;

 while (token is not null && attempts < maxPollingAttempts)
 {
 await Task.Delay(pollingDelayMs);
 Console.Write(".");
 attempts++;

 ChatOptions resumeOptions = new() { ContinuationToken = token };
 chatResponse = await chatClient
 .GetResponseAsync([], resumeOptions); //❹
 token = chatResponse.ContinuationToken;
 }

 if (token is not null)
 {
 Console.WriteLine($"\\n\\nPolling timeout after {maxPollingAttempts}
attempts.");
 Console.WriteLine("The background response did not complete in time.");
 }
 else
 {
 Console.ForegroundColor = ConsoleColor.Yellow;
 Console.WriteLine($"\\n\\n[FINAL] Assistant:");
 var finishReason = chatResponse.FinishReason?.Value ?? "(null)";
 Console.WriteLine($"Finish Reason: {finishReason}");
 Console.ResetColor();
 Console.WriteLine(chatResponse.Text);
 }
}
else
{
 Console.ForegroundColor = ConsoleColor.Yellow;
 Console.WriteLine(string.IsNullOrEmpty(chatResponse.Text)
? "\\nNo continuation token - (no text)."
: $"\\nNo continuation token\\n[COMPLETED]Assistant: {chatResponse.Text}");
 Console.ResetColor();
}
}
catch (Exception ex)
{
 Console.ForegroundColor = ConsoleColor.Red;
 Console.WriteLine($"\\n\\nError: {ex.GetType().Name}");
 Console.WriteLine($"Message: {ex.Message}");
 Console.ResetColor();
}

```

❶ Enables background processing with continuation tokens

❷ Queries for a response but does not expect a generated result

❸ Polls for completion using continuation token with timeout

### ❶ Handles final response after background processing completes

#### Code output:

```
User:
You are an AI assistant controlling a robot car capable of performing
basic moves: forward, backward, turn left, turn right, and stop.
You have to break down the provided complex commands into the basic
moves you know.
Respond only with the moves and their parameters (angle or distance),
and provide additional explanations.

Complex command:
"There is a tree directly in front of the car.
Avoid it and then return to the original path."

[INITIAL] Assistant:
Finish Reason: (null)
Has Continuation Token: True

POLLING....

[FINAL] Assistant:
Finish Reason: (null)
1. **Backward** (distance: 1 meter)
 - Move back to gain space for maneuvers.
2. **Turn Right** (angle: 90 degrees)
 - Turn to move perpendicular to the original path.
3. **Forward** (distance: 2 meters)
 - Move to clear the obstacle.
4. **Turn Left** (angle: 90 degrees)
 - Turn to align parallel to the original path.
5. **Forward** (distance: 2 meters)
 - Move forward past the tree.
6. **Turn Left** (angle: 90 degrees)
 - Turn back toward the original path direction.
7. **Forward** (distance: 1 meter)
 - Move to regain original path line.
8. **Turn Right** (angle: 90 degrees)
 - Final adjustment to return to the original direction.
```

Background response processing with continuation tokens is a useful approach for handling long-running AI operations. It allows applications to submit complex queries and poll for results asynchronously, avoiding timeouts and improving the user experience. The implementation uses a robust polling pattern with configurable delays and a maximum number of attempts, allowing you to handle detailed task descriptions without blocking the application. This pattern is especially valuable for enterprise applications that require high availability and resilient AI integration.

#### EXERCISE

In listing 4.2, locate the lines that activate background responses:

```
ChatOptions options = new() { AllowBackgroundResponses = true };
```

and turn it off:

```
ChatOptions options = new() { AllowBackgroundResponses = false };
```

Run the code and see how the response is generated in one shot.

### 4.2.3 OpenAI Responses Client in Azure

While OpenAI provides direct access to cutting-edge models, many organizations require additional governance, security, and compliance features offered by Azure. The Azure implementation of the *Responses API* offers the same capabilities as the direct OpenAI service but hosted within Microsoft's cloud infrastructure. This enables features such as virtual network integration, managed identity authentication, regional deployment, and compliance with industry-specific regulations.

**PRACTICAL EXAMPLE**

Listing 4.3 shows an implementation of the *Responses API* with OpenAI using Azure credentials: first as a concrete chat client, `ResponsesClient`, and second using the MEAI `IChatClient` abstraction.

Requirements (NuGet packages):

```
dotnet add package Microsoft.Extensions.AI.OpenAI
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
dotnet add package OpenAI
```

Code path: `/AzureOpenAIResponses/Program.cs`

**Listing 4.3 Azure OpenAI Responses**

```
using Microsoft.Extensions.Configuration;
using System.ClientModel;
using Microsoft.Extensions.AI;
using OpenAI.Responses;
using OpenAI;

var configuration = new ConfigurationBuilder()
 .AddUserSecrets<Program>().Build();

var endpoint = configuration["AzureOpenAI:Endpoint"]!;
var apiKey = configuration["AzureOpenAI:ApiKey"]!;
var deploymentName = configuration["AzureOpenAI:DeploymentName"]!;

var prompt = """
You are an AI assistant controlling a robot car capable of performing
basic moves: forward, backward, turn left, turn right, and stop.
You have to break down the provided complex commands into the basic
moves you know.

"There is a tree in front of the car.
Avoid it and resume the original path."

Respond with a JSON array like [move1, move2, move3].
Do not respond with reasoning, comments, or any additional text.
""";
Console.WriteLine($"User: {prompt}");

OpenAIClient client = new OpenAIClient(
 new ApiKeyCredential(apiKey),
 new OpenAIClientOptions { Endpoint = new Uri(endpoint) });

#pragma warning disable OPENAI001
IChatClient chatClient = client
 .GetResponsesClient() //❶
 .AsIChatClient(deploymentName); //❷
ChatResponse chatResponse = await chatClient.GetResponseAsync(prompt);
Console.WriteLine($"Assistant (IChatClient): {chatResponse.Text}");
```

❶ Experimental and subject to change or removal

❷ Converts to `IChatClient` for provider independence

Code output:

```
User: You are an AI assistant controlling a robot car capable of performing basic
moves:
forward, backward, turn left, turn right, and stop.
You have to break down the provided complex commands into the basic moves you
know.
There is a tree in front of the car. Avoid it and resume the original path.
Respond with a JSON array like [move1, move2, move3].
Do not respond with reasoning, comments, or any additional text.

Assistant (IChatClient): [stop, turn_right, forward, turn_left, forward]
```

The *OpenAI Responses API* in Azure provides enterprise-grade hosting for OpenAI models with enhanced security, compliance, and regional availability. This implementation mirrors the *OpenAI Responses API* structure while using Azure's infrastructure, showing how the `IChatClient` abstraction enables seamless transitions between model providers. Again, the comparable output (within the limits of LLM nondeterministic behavior) across both concrete and abstracted implementations validates the effectiveness of MEAI's provider-agnostic design.

#### EXERCISE

In listing 4.3, locate the lines that use the chat client as `IChatClient`:

```
IChatClient chatClient = client
 .GetResponsesClient()
 .AsIChatClient(deploymentName);
ChatResponse chatResponse = await chatClient
 .GetResponseAsync(prompt);
Console.WriteLine($"Assistant (IChatClient): {chatResponse.Text}");
```

and replace them with a direct chat client (with no conversion to `IChatClient`):

```
ResponsesClient azureOpenAIChatClient = client
 .GetResponsesClient();
ClientResult<ResponseResult> response = azureOpenAIChatClient
 .CreateResponse(deploymentName, prompt);
Console.WriteLine("Assistant (Azure Responses): "
 + $"{response.Value.GetOutputText()}");
```

Run the code and compare the output to the original version. The results should be similar, even though one uses the abstraction and the other calls the provider directly.

### 4.3 Creating and Configuring Self-Hosted Clients

Not every agent has to rely on a cloud model. In many scenarios, we run the model where our application runs and keep all data on our own infrastructure. Self-hosted clients in MEAI cover this space by treating local runtimes like Ollama and ONNX as first-class chat providers, while still flowing through the same `IChatClient` abstraction as cloud services. Robby can be powered entirely by local models—from an Ollama-hosted LLM to an ONNX model embedded in the process—without changing the surrounding agent architecture.

---

**IMPORTANT** If you want to explore self-hosted models in more depth, including Ollama-based setups and ONNX runtimes, I recommend *Domain-Specific Small Language Models* by Guglielmo Iozzia (Manning): <https://www.manning.com/books/domain-specific-small-language-models>.

---

#### 4.3.1 Ollama Client

For developers seeking complete control over their AI infrastructure, self-hosted models offer compelling advantages. Ollama provides an elegant solution for running large language models locally, eliminating cloud dependencies and associated costs while ensuring data privacy. This approach is particularly valuable for development environments, air-gapped systems, or applications with strict data residency requirements. Ollama supports a wide range of open-source models and integrates seamlessly with MEAI through the `OllamaSharp` library.

---

**NOTE** To use the Ollama-based samples, we first install the Ollama runtime on the same machine where our .NET application will run. Start at the official Ollama site, download the installer for your operating system, and run it using the default options so the background service is registered correctly. After installation completes, open a terminal and confirm that the CLI is available by

running `curl http://localhost:11434/api/version`. If that command returns a version number instead of an error, the server is installed and the local API is listening on `http://localhost:11434`. 11434 is the default port number. Next, pull at least one model, for example `ollama pull minstral-3`, then call `ollama list` to verify that the model is available and ready to be used by your `OllamaApiClient` implementation.

## PRACTICAL EXAMPLE

Listing 4.4 shows the Ollama chat client, available as an `IChatClient` abstraction in MEAI.

Requirements (Ollama server):

```
ollama pull gemma4
```

Requirements (NuGet packages):

```
dotnet add package OllamaSharp
```

Code path: `/OllamaChatClient/Program.cs`

### Listing 4.4 Ollama Chat Client

```
using Microsoft.Extensions.AI;
using OllamaSharp;

var systemMessage = """
 You are an AI assistant controlling a robot car capable of performing
 basic moves: forward, backward, turn left, turn right, and stop.
 You have to break down the provided complex commands into basic
 moves you know.
 Respond only with the permitted moves, without any additional
 explanations.
 """;

var userMessage = """
 Complex command:
 "There is a tree directly in front of the car.
 Avoid it and then come back to the original path."
 """;

var modelName = "gemma4";
var ollamaServer = "http://localhost:11434";

IChatClient ollamaApiClient = new OllamaApiClient(
 new Uri(ollamaServer), modelName); //❶

List<ChatMessage> messages = [
 new(ChatRole.System, systemMessage),
 new(ChatRole.User, userMessage)
]; //❷

ChatResponse response = await ollamaApiClient
 .GetResponseAsync(messages); //❸
messages.AddRange(response.Messages); //❹

Console.WriteLine(response);
```

- ❶ Initializes Ollama API client with local server endpoint
- ❷ Builds conversation with system and user messages
- ❸ Gets response
- ❹ Updates conversation history

Code output:

```
turn left
forward
stop
turn right
```

```
backward
stop
```

The Ollama implementation shows the power of self-hosted AI models, enabling developers to run large language models locally without cloud dependencies. This approach offers significant advantages for privacy-sensitive applications, compliance with GDPR-like data privacy laws, offline scenarios, and development environments. With self-hosted models, applications can deliver responsive AI interactions without external API calls or usage fees. The `IChatClient` abstraction seamlessly integrates local models alongside cloud model providers, allowing Robby to function identically whether powered by local or remote AI services.

#### EXERCISE

In example 4.4, replace the AI model name with `gemma3`:

```
var modelName = "gemma3:4b";
```

Then pull the new model from Ollama library.

```
ollama pull gemma3
```

Run the code again, and you should get a valid response. I didn't say "similar" because small language models may perform very differently depending on their parameters and accuracy.

### 4.3.2 ONNX Client

The *ONNX* (Open Neural Network Exchange) format represents a platform-independent standard for neural network models, enabling deployment across diverse runtime environments. Through *ML.NET* and *OnnxRuntimeGenAI*, developers can embed AI models directly into their applications, achieving the lowest possible latency and complete independence from external services. This approach excels in scenarios requiring offline operation, edge computing, or when network connectivity is unreliable. Next, we'll integrate an ONNX-based chat client into Robby's architecture.

To use local ONNX models, you can use *Foundry Local*.

*Foundry Local* is a runtime environment from Microsoft that runs AI models entirely on our machine instead of in the cloud. It provides a local model catalog, a CLI for managing and serving models, and an on-device cache where downloaded models are stored. With *Foundry Local*, we can experiment with small and medium language models, keep data on our own hardware, and wire those models into applications through libraries like `Microsoft.ML.OnnxRuntimeGenAI` without depending on external services.

Install *Foundry Local* using the official distribution. For example, on Windows:

```
winget install Microsoft.FoundryLocal
```

We can also download an installer from the official *Foundry Local* releases page:

<https://github.com/microsoft/Foundry-Local/releases>

Use the *Foundry CLI* to list available models:

```
foundry model list
```

For this CPU-based example, select a CPU-compatible chat model and download it. For example:

```
foundry model download Phi-4-generic-cpu
```

We can also use:

```
foundry model run <model-name>
```

which downloads the model when necessary and starts an interactive session.

Use the following commands to inspect local storage:

```
foundry cache location
foundry cache list
```

It prints the cache root, which may contain multiple models and model variants. Do not pass that root directory directly as `modelPath`. Instead, locate the subdirectory for the downloaded model variant and set `modelPath` to that specific directory. It must contain `genai_config.json` and all model, tokenizer, and auxiliary files referenced by that configuration.

For example, the final model directory may resemble:

```
C:\Users\<user>\.foundry\Microsoft\Phi-4-generic-cpu\cpu-int4-rtn-block-32-acc-level-4
```

The exact directory name depends on the selected model and variant. A command such as `foundry model download` does not create an unrelated path such as `C:\LLMs`; use that location only if you explicitly create it and configure or copy a complete compatible model directory there.

#### PRACTICAL EXAMPLE

Listing 4.5 shows the ONNX chat client using the `Microsoft.ML.OnnxRuntimeGenAI` library as an `IChatClient` MEAI abstraction.

Requirements (ONNX models):

```
foundry model download phi-4-mini
```

Requirements (NuGet packages):

```
dotnet add package Microsoft.ML.OnnxRuntimeGenAI
```

Code path: `/OnnxChatClient/Program.cs`

#### Listing 4.5 ONNX Chat Client

```
using Microsoft.Extensions.AI;
using Microsoft.ML.OnnxRuntimeGenAI;

var prompt = """
 You are an AI assistant controlling a robot car capable of performing
 basic moves: forward, backward, turn left, turn right, and stop.
 You have to break down the provided complex commands into the basic
 moves you know.

 "There is a tree in front of the car.
 Avoid it and resume the original path."

 Respond with a JSON array like [move1, move2, move3].
 Do not respond with reasoning, comments, or any additional text.
 """;
Console.WriteLine($"User: {prompt}");

var modelPath = @"c:\LLMs\ONNX\" +
 @"phi-4-mini-instruct\cpu and mobile\" +
 @"cpu-int4-rtn-block-32-acc-level-4";

var config = new Config(modelPath);
var model = new Model(config); //❶

using var onnxChatClient = new OnnxRuntimeGenAIChatClient(model); //❷
ChatMessage message = new(ChatRole.User, prompt);
ChatResponse response = await onnxChatClient
 .GetResponseAsync(message); //❸
Console.WriteLine($"Assistant (ONNX ChatClient): {response.Text}");
```

❶ Loads ONNX model from local file path

❷ Creates ONNX chat client with loaded model

❸ Gets response from in-process ONNX model

Code output:

```
User: You are an AI assistant controlling a robot car capable of performing
basic moves: forward, backward, turn left, turn right, and stop.
You have to break down the provided complex commands into the basic
moves you know.
There is a tree in front of the car. Avoid it and resume the original path.
Respond with a JSON array like [move1, move2, move3].
Do not respond with reasoning, comments, or any additional text.

Assistant (ONNX ChatClient): ``json
[turn left, forward, turn right, forward, stop]
```

ONNX Runtime with ML.NET enables in-process model hosting, bringing AI capabilities directly into the application runtime without external dependencies. This is the most self-contained approach to AI integration, making it a good fit for edge computing, embedded systems, or environments with strict security requirements. By using optimized ONNX models like phi-4-mini-instruct, applications can achieve low-latency inference with minimal resource overhead. The seamless integration through `IChatClient` shows how MEAI abstracts away the complexity of different hosting approaches, allowing developers to focus on application logic rather than infrastructure.

#### EXERCISE

In example 4.5, replace the AI model path with a model of your choice:

```
var modelPath = "path-to-your-model";
```

Then pull the new model from Foundry Local.

```
foundry model download your-model
```

Run the code again, and you should get a valid response. I didn't say "similar" because small language models may perform very differently depending on the model parameters and accuracy.

## 4.4 Exception Management and Error Handling

The `IChatClient` interface in MEAI can encounter several failure scenarios during chat operations. Applications must implement appropriate error-handling strategies to ensure robust AI interactions.

The response type is `ChatResponse`, and its `FinishReason` property of type `ChatFinishReason` shows why a completion ended; not all reasons are successful completions. Agent Framework defines four primary finish reasons:

- **Stop**: Indicates the model reached a natural stopping point or encountered a provided stop sequence. This represents a successful completion where the model finished its response normally.
- **Length**: Signals the model reached the maximum token limit for the request or response. Your application should handle this by either requesting a continuation, summarizing the partial response, or notifying the user that the response was truncated.
- **ContentFilter**: Indicates that content was filtered due to safety policies, prohibited content, or sensitivity concerns. This may require special handling, since it can indicate policy violations that need to be logged or communicated to users.
- **ToolCalls**: Indicates the model is requesting tool execution before completing its response. This is not an error; your application should execute the requested tools and then continue the conversation.
- **null**: `FinishReason` is not available or was not supplied by the client or provider. Do not interpret null as meaning that a background response is still running. Poll only while `ContinuationToken` is non-null (as shown for the null finish reason in listing 4.2). To distinguish `Completed`, `Failed`, `Cancelled`, or `Incomplete Responses API` operations, inspect the provider's `ResponseResult.Status`.

### 4.4.1 Exception Types and Scenarios

When calling `GetResponseAsync` or `GetStreamingResponseAsync`, several exception scenarios can occur:

- Network and connectivity failures manifest as standard .NET exceptions such as `HttpRequestException` or `TaskCanceledException`. These typically show transient issues with the underlying AI service.
- Timeout exceptions occur when operations exceed allowed durations. Always pass a



`CancellationToken` to enable proper timeout handling and allow graceful shutdown during long-running operations.

- Content policy violations may throw provider-specific exceptions (like `RequestFailedException` for Azure OpenAI) with error codes such as `content_filter`. These exceptions often contain detailed information about which content category triggered it (hate speech, self-harm, sexual content, violence).
- Rate limiting errors show the application exceeded the provider's throttling limits. These need exponential backoff retry strategies for resilient operation.
- Invalid request exceptions occur when messages are malformed, exceed size limits, or violate API contracts. Validate inputs before sending requests to catch these early.

#### 4.4.2 Best Practices for Exception Handling

Let's wrap all `IChatClient` operations in comprehensive try/catch blocks (listing 4.6):

##### Listing 4.6 Best practices suggestion for exception handling

```
try
{
 var response = await chatClient.GetResponseAsync(
 messages,
 options,
 cancellationToken);

 if (response.FinishReason == ChatFinishReason.ContentFilter) // ❶
 {
 HandleContentFiltering(response);
 }
 else if (response.FinishReason == ChatFinishReason.Length) // ❷
 {
 HandleOutputLimit(response);
 }
}
catch (OperationCanceledException)
 when (cancellationToken.IsCancellationRequested) // ❸
{
 HandleCancellation();
}
catch (ClientResultException ex) when (ex.Status == 0) // ❹
{
 HandleTransportFailure(ex);
}
catch (ClientResultException ex) when (ex.Status == 429) // ❺
{
 HandleRateLimit(ex);
}
catch (ClientResultException ex) // ❻
{
 HandleServiceError(ex);
}
catch (Exception ex) // ❼
{
 logger.LogError(ex, "Unexpected error while getting the response");
 throw;
}
```

- ❶ Handles a returned response that was stopped by content filtering
- ❷ Handles a response that reached its maximum input or output length
- ❸ Handles cancellation requested through the supplied `CancellationToken`
- ❹ Handles failures for which no HTTP response was received; inspect `InnerException` for details

- ⑤ Handles HTTP 429 rate-limiting responses
- ⑥ Handles unsuccessful service responses; inspect `Status`, `Message`, and `GetRawResponse()`
- ⑦ Logs and rethrows unexpected exceptions so they are not silently hidden

Let's look at a few best practices.

- Always pass `CancellationToken` to enable request cancellation and prevent resource leaks during shutdown. This allows deployed applications to end cleanly and prevents long-polling operations from hanging.
- Validate inputs before you send them to catch malformed requests early. Check message content, tool arguments, and option values before calling the AI service to provide better error messages to users.
- Implement retry policies for transient failures, but only for idempotent operations. Avoid retrying tool calls that mutate state, as this could cause unintended side effects.
- Use middleware for cross-cutting concerns like logging, rate limiting, and security checks. Agent Framework supports middleware patterns that intercept requests and responses, enabling centralized error handling.
- Handle partial responses gracefully by checking the `FinishReason` property. If `Length` is returned, decide whether to request a continuation, summarize what you received, or prompt the user to reformulate the request with fewer tokens.
- Monitor and log errors with sufficient context for troubleshooting. Include request IDs, message counts, token usage, and finish reasons in logs to diagnose problems effectively.

## Summary

- Use the *Responses API* for both simple calls and long-running background operations with continuation tokens and polling.
- Centralize provider configuration behind `IChatClient` so Robby can switch between OpenAI and Ollama without changing the code logic.
- Run Robby on self-hosted models through Ollama or ONNX when data residency, offline operation, or lower latency is more important than cloud convenience.
- Add logging and telemetry middleware with `ChatClientBuilder` to capture prompts, responses, and token usage for debugging and production monitoring.
- Handle finish reasons explicitly: `Stop`, `Length`, `ContentFilter`, `ToolCalls`, and `null` to show an in-progress background response in the *Responses API*.
- Implement structured try/catch blocks around `IChatClient` calls and include correlation details in logs for troubleshooting distributed systems.
- Apply retry policies for transient failures only for idempotent calls and validate inputs before sending requests to avoid provider-side validation errors.
- Standardize a provider-agnostic chat client factory so new models or providers can be added through configuration instead of modifying business logic. JSON response format can be used to structure AI model outputs for easy integration with other systems and applications.

# 5

## *Crafting agents from scratch*

### **This chapter covers**

- From chatbots to agents
- Building a `ChatClientAgent`
- Configuring agent instructions, output format, streaming
- Working with agent sessions

With a conventional chatbot that powers Robby, we can generate smart answers, but each response is isolated. Robby can reason about driving, describe his surroundings, or read a weather report, one question at a time, with no real memory and no built-in way to act in the physical world. We can manually carry chat history around and pass extra options to the model, but that is plumbing we build ourselves, not a first-class abstraction. This approach starts to break down once we handle dynamic, multi-step problems that need persistent context, tool calls, and coordinated reasoning across several steps or several specialized components.

Now consider a single objective such as *safely traversing difficult terrain*. That is not one prompt and one answer. It is a chain of decisions, checks, and actions: read sensors, choose a path, adjust speed, re-check safety, and so on. At that point we do not want Robby to behave like a stateless completion engine. We want him to have memory across turns, tools he can call, and the ability to work in a team of focused agents.

Imagine Robby splitting the work into specialized roles. One agent plans the route. Another reads and interprets sensor data. A third enforces safety rules and can veto risky moves. Each agent focuses on a narrower responsibility, and together they form a system that is more robust and easier to reason about than a single, oversized chatbot.

A good mental model is a flight crew. The pilot focuses on flying the plane. The co-pilot double-checks instruments and procedures. Air traffic control manages airspace and clearances. Ground crew takes care of fuel, maintenance, and safety checks on the ground. Each role has clear responsibilities and authority, and the overall system works because the crew shares information and keeps context over time. Agents play a similar part for Robby: separate, specialized roles that still contribute to one shared goal.

### **5.1 Introducing Agents**

All agents share the same foundation: they have an identity (name and description), a set of instructions that shape behavior, and integration points for tools and memory. What varies is where they run, how they store state, and which capabilities they expose. Some agents keep state in local

memory or a store we control. Others rely on a service that owns the conversation history. Some can call tools that act as wrappers for conventional code. Others are pure planners.

Traditional LLM-based chatbots built directly on chat clients (including the `Microsoft.Extensions.AI` clients from chapter 3) or raw completion APIs run into several problems in richer scenarios:

- Having to orchestrate conversation state and message history by hand across turns.
- Repeating boilerplate around tool invocation, result handling, and formatting.
- Solving problems from a single perspective instead of coordinating multiple specialized capabilities.

As interactions grow, and as we start to mix tools, retrieval, and different reasoning strategies, this ad-hoc code gets harder to maintain. It also hides the real structure of the system: which parts think, which parts act, and which parts remember.

We treat an agent as a component that uses an AI model to make decisions and take actions based on those decisions. It is more than a prompt: it has a defined role, access to tools, and a way to keep context over time.

5.1.1 What is ChatClientAgent?

A `ChatClientAgent` is the simplest way to turn an `IChatClient` abstraction or a concrete `ChatClient` into a first-class agent with instructions, tools, and conversation state. We start from any compatible chat client, wrap it in a `ChatClientAgent`, and then work at the agent level: we set instructions, attach tools, and run conversations through sessions.

Note Under the hood, Microsoft Agent Framework (MAF) relies on `Microsoft.Extensions.AI` to communicate with LLMs. A `ChatClientAgent` always sits on top of an abstract `IChatClient` or concrete `ChatClient` implementation, which handles the actual model calls.

At its core, a `ChatClientAgent` does three jobs. It combines the agent’s instructions, chat messages, and additional context into a request to the underlying model. It interprets model outputs as agent responses, including tool calls and structured output. It works with sessions and providers so the agent can keep and reuse history across multiple turns.

The diagram in figure 5.1 illustrates the architecture of a `ChatClientAgent`.

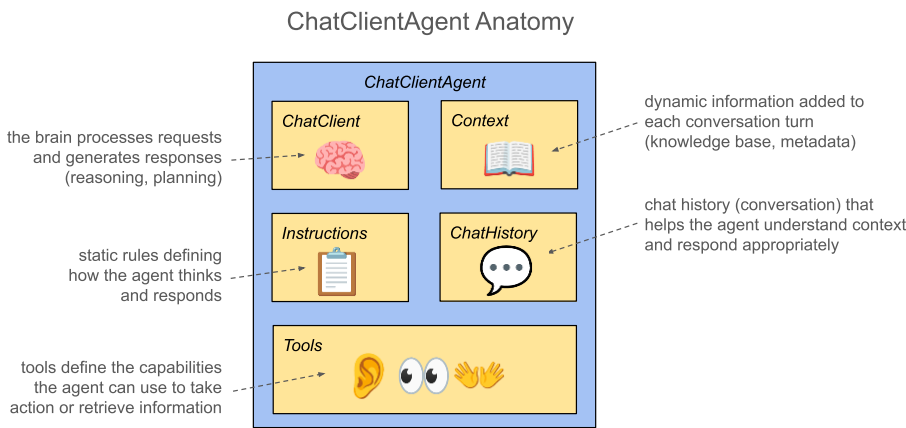


Figure 5.1 ChatClientAgent anatomy, showing the core elements: a processing brain that manages conversations with the model, static rules defining agent behavior, tools offering action capabilities, dynamic context added per turn, and chat history for contextual understanding.

ChatClientAgent building blocks:

- Chat client. The concrete `IChatClient` that connects to the AI model and sends the merged context containing instructions, messages, and any additional context. This can be a client backed by OpenAI, Azure OpenAI, Ollama, Anthropic, ONNX, or another supported provider.
- Instructions. The stable configuration that defines the agent's role, behavior, and guardrails. Here, we specify things like `You are a travel assistant` or `Always answer in JSON`. The agent applies these instructions on every call, so they act like a profile.
- Context. Additional AI context is injected into each invocation beyond the raw chat messages. This may include user data, retrieved documents, or external signals.
- Tools. The set of callable operations the model is allowed to invoke, such as `SearchOrders`, `ReadTemperature`, or `TurnLeft`. The model decides when to call these tools. `ChatClientAgent` executes them and feeds the results back into the conversation.
- Chat history. The conversation state as the agent perceives it across runs: user messages, assistant replies, tool calls, tool results, and any persisted system messages. This state lives in a context provider or service-backed conversation store and is updated after each turn so later calls can reason over the full history.

Robby will use all of them. His identity comes from instructions. His memory comes from context and chat history via sessions. His actions come from tools. Together, these turn a stateless language model into an agent that can reason, act, and remember across long-running interactions.

We start with instructions, which set up Robby's identity.

### 5.1.2 Instructions

Imagine Robby wakes up with a blank mind. Without guidance, he might respond like a chef or a poet. To be useful, Robby needs a permanent identity card stamped into his circuits: *You are Robby, an AI assistant controlling a robot car capable of performing basic moves*. This immutable identity ensures that no matter what unexpected situations he sees on the road, he never forgets his core mission.

Instructions define the identity, behavior, and boundaries (guardrails) of your AI agent. In the large language model (LLM) world, this is often referred to as the system prompt or system message. Think of instructions as agent profiling: they tell the model who it is (e.g., "You are a senior SQL database administrator") and how it should act (e.g., "Always explain your reasoning before providing an answer").

#### PRACTICAL EXAMPLE

Unlike user messages, which change every *turn*, instructions typically remain static throughout the conversation (a turn is a request-response interaction between the agent and the AI model). They act as the foundational layer of the context window, ensuring the agent adheres to its persona and business rules regardless of the user's input. These instructions are automatically prepended to every interaction, consistently guiding the model and are supplied on runs, but do not guarantee compliance. The rules requiring enforcement must be checked in application/tool code rather than relying only on the prompt.

**WARNING** Instructions guide model behavior but cannot guarantee compliance. Enforce security, authorization, and other mandatory rules in application or tool code.

Listing 5.1 shows a minimalist `ChatClientAgent` that uses the chat client and declares its instructions, name, and description.

Required NuGet packages:

```
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
dotnet add package Microsoft.Agents.AI.OpenAI
```

Code path: `/Agent/Program.cs`

### Listing 5.1 Simple agent

```
using Microsoft.Extensions.Configuration;
using Microsoft.Agents.AI;
using OpenAI;
using OpenAI.Chat;
using Microsoft.Extensions.AI;

var configuration = new ConfigurationBuilder()
 .AddUserSecrets<Program>().Build();
var model = configuration["OpenAI:ModelId"];
var apiKey = configuration["OpenAI:ApiKey"];

ChatClientAgent agent = new OpenAIClient(apiKey)
 .GetChatClient(model)
 .AsIAgent(instructions: """
 You are an AI assistant controlling a robot car capable of performing
 basic moves: forward, backward, turn left, turn right, and stop.
 You have to break down the provided complex commands into the basic
 moves you know.
 Use a JSON array like [move1, move2, move3] for the response.
 Respond only with the moves and their parameters (angle or distance),
 without any additional explanations.
 """,
 name: "RobotCarAgent",
 description: "An AI agent that controls a robot car."
); // ❶

var prompt = """
 Complex command:
 "There is a tree directly in front of the car.
 Avoid it and then return to the original path."
 """; // ❷

AgentResponse response = await agent.RunAsync(prompt); // ❸
Console.WriteLine(response.Text); // ❹

❶ Creates agent with name, description, instructions
❷ Defines the prompt
❸ Calls the agent with the prompt
❹ Prints the agent response to console
```

Code output:

```
```json
[{"turn": "right", "angle": 90}, {"forward": 5}, {"turn": "left", "angle": 90},
{"forward": 5}, {"turn": "left", "angle": 90}, {"forward": 5}, {"turn": "right",
"angle": 90}]
```
```

The code output represents a sequence of robot car movement commands that form a path for avoiding an obstacle using the permitted basic movements.

For a minimalist agent, we can optionally declare a name and a description alongside the instructions. While not strictly needed for execution, defining these properties is best practice when the agent will be part of a larger orchestration or when observability is required.

The response is produced in a single call, which is right when we need the complete output at once, such as a JSON response that is invalid if partial. When we need to print out tokens as soon as they are generated, streaming is a better approach.

#### EXERCISE

```
var followupPrompt = ""
Complex command:
"What was your second last move?"
"";
AgentResponse followupResponse = await agent.RunAsync(followupPrompt);
Console.WriteLine(followupResponse.Text);
```

Run the code and see the follow-up response. It may look like an empty array (`[]`). The agent does not remember its previous answer, which is expected because we have not enabled any memory mechanism. We'll explore memory later through sessions, advanced context, and chat history.

### 5.1.3 Streaming

When working with enterprise systems, the complete response is usually needed for communication between agents or modules. In scenarios such as machine-to-human communication, or when the use case involves partial responses (tokens), streaming is a better choice.

Let's have Robby report a status update in a human-friendly format. From a user-experience perspective, streaming is exactly what we need, as it reduces perceived latency by displaying information the moment it becomes available.

The streaming relies on the `RunStreamingAsync` method, which returns an `IAsyncEnumerable<AgentResponseUpdate>` instead of waiting for the complete response to be generated. This lets the agent's output flow incrementally, token by token, as the LLM generates it.

Each update represents a chunk of the response: typically, a `TextContent` item containing a partial string, followed optionally by metadata items such as `UsageContent` at the end of the stream. In other words, we do not buffer the entire message and then render it; we process and display each piece as soon as it arrives, which is ideal for interactive or UI-driven scenarios.

#### PRACTICAL EXAMPLE

Listing 5.2 shows how `RunStreamingAsync` enables real-time token display, which is critical for interactive scenarios where users benefit from immediate feedback instead of waiting for complete responses. Replace the last two lines in listing 5.1, which invoke the full response, with a streamed invocation.

Requirements (NuGet packages):

```
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
dotnet add package Microsoft.Agents.AI.OpenAI
```

Code path: `/AgentWithStreaming/Program.cs`

#### Listing 5.2 Agent with Streaming

```
// 'usings' and API key fetching omitted for brevity

// agent creation omitted for brevity

await foreach (AgentResponseUpdate update in agent
 .RunStreamingAsync(prompt)) //❶
{
 Console.Write(update.Text); //❷
 if (update.Contents.FirstOrDefault() is UsageContent usageContent)
 {
 Console.WriteLine("\n\nInput Tokens: "
 + $"{usageContent.Details.InputTokenCount}");
 Console.WriteLine("Output Tokens: "
 + $"{usageContent.Details.OutputTokenCount}");
```

```

 Console.WriteLine("Total Tokens: "
 + $"{usageContent.Details.TotalTokenCount}");
 } //❸
}

```

- ❶ Calls the agent with the prompt in streaming mode
- ❷ Prints out the streamed item text
- ❸ If item is UsageContent prints out the details

Code output:

```

``json
[{"command": "turn right", "angle": 90}, {"command": "forward", "distance": 5},
{"command": "turn left", "angle": 90}, {"command": "forward", "distance": 10},
{"command": "turn left", "angle": 90}, {"command": "forward", "distance": 5},
{"command": "turn right", "angle": 90}]
``

Input Tokens: 117
Output Tokens: 93
Total Tokens: 210

```

When running the application, we notice the tokens streaming in real time, providing immediate feedback for the user. The stream primarily yields `TextContent` items, which contain the partial response text. This sequence is followed by a final `UsageContent` item. It is optional, but I chose to print it because it provides telemetry details such as the input, output, and total token counts for the interaction.

#### EXERCISE

Compare the values across multiple streaming updates and see which properties remain constant throughout a single response and which change. We may need these fields when debugging multi-turn conversations and multi-agent systems.

### 5.1.4 Structured Output

We've mentioned several times that when agents communicate with machines, we need reliable parsing. Instead of parsing unstructured JSON strings, enforce structure at the model level. To control the response format, set the `ResponseFormat` property in the `ChatOptions` object within the `ChatClientAgentOptions` argument passed to the agent constructor.

The `ResponseFormat` typically accepts either text or JSON. Text is the default behavior, which we've seen in all previous examples. The alternative is JSON, which can enforce a specific structure via a JSON schema, as shown in the following example:

```

{"$schema": "https://json-schema.org/draft/2020-12/schema", "type": "object", "properties": {"steps": {"type": "array", "items": {"type": "object", "properties": {"move": {"type": "string"}, "value": {"type": ["string", "null"]}}, "required": ["move", "value"]}}, "required": ["steps"]}

```

This is nothing but the serialized form of the following types:

```

record StepsResponse(StepItem[] Steps);
record StepItem(string Move, string? Value);

```

#### PRACTICAL EXAMPLE

Listing 5.3 shows how to enforce type-safe responses using the `ResponseFormat` property of `ChatOptions`, eliminating string parsing. The schema constrains supported model output and `Result` deserializes it at runtime into a statically typed object; runtime generation/deserialization failures still need handling.

Requirements (NuGet packages):

```

dotnet add package Microsoft.Extensions.Configuration.UserSecrets
dotnet add package Microsoft.Agents.AI.OpenAI

```

Code path: `/AgentWithStructuredOutput/Program.cs`

#### Listing 5.3 Agent with structured output

```

// 'usings' and API key fetching omitted for brevity

```



```

ChatClientAgent agent = new OpenAIClient(apiKey)
 .GetChatClient(model)
 .AsIAgent(new ChatClientAgentOptions
 {
 Name = "RobotCarAgent",
 Description = "An agent that assists a robot with the basic moves.",
 ChatOptions = new ChatOptions
 {
 Instructions = """
 You are an AI assistant controlling a robot car capable of performing
 basic moves: forward, backward, turn left, turn right, and stop.
 You have to break down the provided complex commands into the basic
 moves you know.
 Respond only with the moves and their parameters (angle or distance),
 without any additional explanations.
 """,
 Temperature = 0.4F, //❶
 MaxOutputTokens = 300, //❷
 ResponseFormat = Microsoft.Extensions.AI.ChatResponseFormat
 .ForJsonSchema<StepsResponse>() //❸
 }
 });

var prompt = """
Context
Complex command:
"There is a tree directly in front of the car.
Avoid it and then return to the original path."
""";

AgentResponse<StepsResponse> response = await agent
 .RunAsync<StepsResponse>(prompt); //❹
Console.WriteLine(JsonSerializer.Serialize(response.Result)); //❺

record StepsResponse(StepItem[] Steps);
record StepItem(string Move, string? Value); //❻

```

- ❶ Reduces the response "creativity"
- ❷ Limits the response max output tokens
- ❸ Sets the response format to StepsResponse type
- ❹ Calls the agent with the prompt
- ❺ Prints the serialized response of the agent
- ❻ Declares the response format data models

Code output:

```

{"steps":[{"move":"turn right","value":"90 degrees"}, {"move":"forward","value":"5
meters"}, {"move":"turn left","value":"90 degrees"}, {"move":"forward","value":"10
meters"}, {"move":"turn left","value":"90 degrees"}, {"move":"forward","value":"5
meters"}, {"move":"turn right","value":"90 degrees"}]}

```

By using the generic `agent.RunAsync<T>` method, we ensure that the response conforms to the specified schema and deserializes directly into the `StepsResponse` type. We access the deserialized object via `response.Result` instead of `response.Text`, and the framework handles both schema enforcement at the model level and type-safe deserialization at the client level. This pattern eliminates fragile string parsing, catches schema mismatches early, and gives us compile-time safety when working with agent responses in production code.

#### EXERCISE

Extend the structured output in listing 5.3 to include a human-readable explanation for each step. Add an `Explanation` property to the `StepItem` class, and adjust the `Instructions`, replacing *"Respond only with the moves and their parameters (angle or distance), without any additional explanations"* with *"Respond only with the moves and their parameters (angle or distance, and explanation)"*.

Run the agent and verify that `response.Result.Steps` contains `Move`, `Value`, and `Explanation` (which is the newly added property).

## 5.2 Chat Messages

So far, we have seen how an agent manages input and output using a request-response pattern. While this approach inherently provides stateless behavior, it only covers a small subset of real-world use cases where follow-up interactions are unnecessary. We now need to understand how to stitch these individual exchanges into a coherent conversation, transforming our stateless implementation into a stateful agent (see figure 5.2).

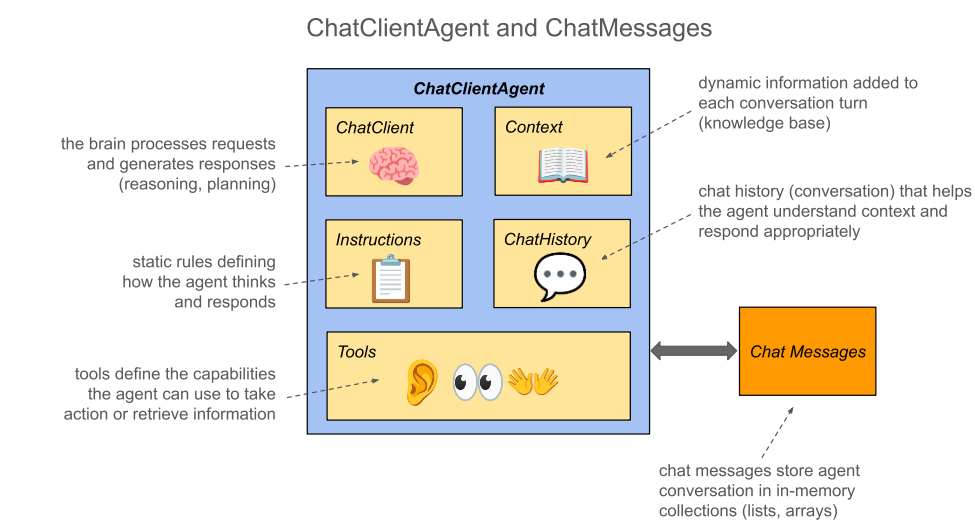


Figure 5.2 ChatClientAgent with chat messages for storing agent conversations in memory (lists, arrays).

By retaining recent interactions between the user and the agent, we allow the conversation to carry forward context and continuity. Instead of processing messages in isolation, the agent can recall previous responses, understand the flow of dialogue, and make decisions that feel coherent across multiple turns.

### 5.2.1 Conversations with Chat Messages

Practically, in its basic form, agent interaction with AI models is managed through chat messages, with each interaction representing a step in the ongoing conversation. This low-level approach, inherited from MEAI (Microsoft.Extensions.AI), allows us to send one or multiple chat messages to the agent at once. But it is our responsibility to manage the history of interactions with the AI model. We must collect the agent’s responses and maintain the sequence of request-response pairs to preserve conversational context during the session. The overloaded signatures of the `RunAsync` method allow us to wrap that prompt string in a `ChatMessage` instance (listing 5.4).

#### Listing 5.4 Single ChatMessage

```
var prompt = """
Complex command:
"There is a tree directly in front of the car.
Avoid it and then return to the original path."

```

```
""";
ChatMessage userMessage = new(ChatRole.User, prompt);
AgentResponse response = await agent
 .RunAsync(userMessage); //❶
```

❶ **Calls the agent with the prompt as Chat Message**

Often, instead of having a single message, we may have a series of request-response pairs: a conversation that we send to the AI model at once. When generative AI emerged and the first models such as GPT-3 were officially released, this was all we had: stateless interactions, and we had to handle the history of interactions ourselves, much like this (listing 5.5).

Requirements (NuGet packages):

```
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
dotnet add package Microsoft.Agents.AI.OpenAI
```

Code path: `AgentWithChatMessages/Program.cs`

### Listing 5.5 Agent with chat messages

```
// 'usings' and API key fetching omitted for brevity

ChatClientAgent agent = new OpenAIClient(apiKey)
 .GetChatClient(model)
 .AsAIAgent(instructions: """
 You are an AI assistant controlling a robot car capable of performing
 basic moves: forward, backward, turn left, turn right, and stop.
 You have to break down the provided complex commands into the basic
 moves you know.
 Use a JSON array like [move1, move2, move3] for the response.
 Respond only with the moves and their parameters (angle or distance),
 without any additional explanations.
 """)
);

List<ChatMessage> conversation = [
 new(ChatRole.User, """
 Complex command:
 "There is a tree directly in front of the car. Avoid it and then return to the
 original path.
 """),
 new(ChatRole.Assistant, """
 ``json
 [{"move": "stop"},
 {"move": "turn", "direction": "right", "angle": 90},
 {"move": "forward", "distance": "5" },
 {"move": "turn", "direction": "left", "angle": 90},
 {"move": "forward", "distance": "5"},
 {"move": "turn", "direction": "left", "angle": 90},
 {"move": "forward", "distance": "5"},
 {"move": "turn", "direction": "right", "angle": 90}]
 """),
 new(ChatRole.User, """
 What was your second last basic move?
 """),
]; //❶

AgentResponse response = await agent
 .RunAsync(conversation); //❷
Console.WriteLine(response.Text);
```

❶ **Declares prior conversation as list of ChatMessage**

❷ **Calls the agent with the conversation**

Code output:

```
``json
{"move": "forward", "distance": "5"}
```

The conversation object keeps all previous chat messages, queries, and responses, including the system message. The agent considers all chat messages when responding to the last user message and correctly identifies the second-to-last movement, as requested.

In Agent Framework, while you could manually manage chat messages, as we just did, sessions, a more advanced concept, handle this automatically.

### 5.2.2 Multi-Modal Input

Next, we will explore other message types that can appear in a conversation.

AI models that support *multimodal input* (i.e., the ability to process multiple data types such as text, images, and audio in the same prompt) are becoming standard, so let us see how we can handle images and audio content. This is straightforward because all content types inherit from the `AIContent` class. We will focus on the `UriContent` and `DataContent` types, but there are additional content types as well.

The `UriContent` type handles binary content that is compatible with the multimodal AI model you are using. We must check the MIME types each AI model supports ourselves.

```
ChatMessage userMultiModelMessage = new(ChatRole.User, [
 new TextContent("Look at the image of the map and proceed safely."),
 new UriContent(@"http://apexcode.ro/path.jpg", "image/jpeg")
]); //❶
```

❶ Packs two contents as one `ChatMessage`

Or, if we would rather use a local binary resource, `DataContent` is the type to use.

```
byte[] imageBytes = File.ReadAllBytes(@"Data\Map.png"); //❶
ChatMessage userMultiModelMessage = new(ChatRole.User, [
 new TextContent("Look at the image of the map and proceed safely."),
 new DataContent(imageBytes, "image/png")
]); //❷
```

❶ Reads the bytes array from the image file

❷ Packs two contents as one `ChatMessage`

Using an audio resource is not much different, but we must identify its supported MIME type.

```
ChatMessage userMultiModelMessage = new(ChatRole.User, [
 new UriContent(new Uri(@"https://apexcode.ro/task.mp3"), "audio/mpeg")
]); //❶
```

❶ Packs audio file in `ChatMessage`

And for a local audio file:

```
byte[] audioBytes = File.ReadAllBytes(@"Data\Task.mp3"); //❶
ChatMessage userMultiModelMessage = new(ChatRole.User, [
 new DataContent(audioBytes, "audio/mpeg")
]); //❷
```

❶ Reads the bytes array from the image file

❷ Packs audio file in `ChatMessage`

Multi-modal input lets Robby consume the same world that humans see and hear, not just what they type. By combining `TextContent` with `UriContent` or `DataContent`, we can send images and audio as first-class parts of the conversation, so the model can interpret a map screenshot, a road photo, or a spoken instruction alongside the textual prompt.

In agentic systems, we typically do not construct and manage individual `ChatMessage` objects manually for each turn. Instead, we use an `AgentSession`, which automatically tracks the full conversation history, including multi-modal content, across multiple interactions. This abstraction simplifies state management, enables persistence and resumption of conversations, and ensures that all message types flow seamlessly through the agent lifecycle without requiring explicit collection handling.

## 5.3 Agent Session

Imagine Robby performing a sequence of commands. Often we need to refer to earlier commands, and we expect Robby to remember and make decisions accordingly.

At the same time, Robby's agent can be instantiated multiple times, like a fleet of specialized *Robbies*, each focusing on a different goal.

If *RobbyAgent\_1*, *RobbyAgent\_2*, and *RobbyAgent\_3* all pushed their messages into the same conversation history, the AI model would see a single blended conversation. *RobbyAgent\_1*'s safety instructions could mix with *RobbyAgent\_2*'s exploration notes and *RobbyAgent\_3*'s retreat plans.

Agent sessions solve this by giving each Robby its own conversation history: a separate session object that holds only its messages and state. The AI model receives three independent conversations (from each *RobbyAgent*) instead of mixed and confusing entries, so each agent instance stays aligned with its specific goal.

`AgentSession` represents a dedicated conversation channel for agents. It correlates a specific user with a specific sequence of chat messages over time.

Fundamentally there are two main ways to manage these channels:

- **In-Memory Session:** You manage the channel yourself. `AgentSession` relies on your agent's `ChatHistoryProvider` to keep Robby's conversations organized in your robot car.

```
AgentSession session = await agent.CreateSessionAsync();
```

- **In-Service Session:** You let a cloud service manage the channel. Using the *Conversations API*, Robby just sends a `ConversationID` for subsequent identification of the conversation in the provider service. OpenAI service or Azure OpenAI service instantly loads the full context, letting Robby pick up right where the user left off without you managing any local files or databases. `RunAsync` does not automatically download the full remote history.

```
ConversationClient conversationClient = openAIClient
 .GetConversationClient(); //❶
AgentSession session = await agent
 .CreateSessionAsync(conversationId); //❷
```

- ❶ Instantiates an in-server conversation client
- ❷ Initializes the session with conversationId

In this variant, you don't implement a custom conversation history as chat messages. `AgentSession` acts as a pointer to a conversation living in the OpenAI (or Azure OpenAI) service. The service itself handles persistence, durability, and message history. This is useful for building stateless agent services that offload memory management entirely to the cloud.

### 5.3.1 What is ChatClientAgentSession?

`ChatClientAgentSession` is an `AgentSession` implementation designed specifically for `ChatClientAgent`.

It has three key members:

- `ConversationId`: a string id that points to a service-managed conversation. When non-null, the AI service (e.g., OpenAI *Responses API*) owns the chat history. When null, `StateBag` stores local chat history (conversation messages) via a `ChatHistoryProvider`, or enriched context and knowledge via `AIContextProvider`.
- `StateBag` is of type `AgentSessionStateBag`, a thread-safe key-value store for providers associated with a session, such as `ChatHistoryProvider` and `AIContextProvider`.
- `GetService<T>`: a service locator that lets callers probe the session capabilities (e.g., if it exposes `ChatHistoryProvider`)

Figure 5.3 shows the dependencies between agent session types. The `ChatClientAgentSession` inherits from the abstract `AgentSession` class.

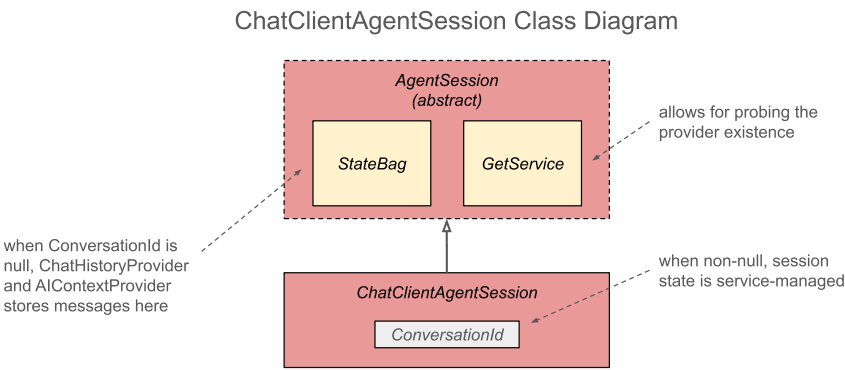


Figure 5.3 `ChatClientAgentSession` manages conversation state for `ChatClientAgent`, handling the `StateBag` and `ConversationId`.

`ChatClientAgentSession` represents the conversation state for `ChatClientAgent` in Agent Framework. It holds the messages and related metadata that let an agent reference prior turns and preserve context across runs. The concrete `ChatClientAgentSession` type exposes a single session abstraction that can work with different storage mechanisms, including in-memory and service-backed options. `ChatHistoryProvider` and `AIContextProvider` are introduced later; here, the focus is on using sessions without unpacking how history is stored internally.

5.3.2 Multiple Turns on the Same Session

Talking about conversation history, it is built from multiple conversation rounds, and the session is where it’s preserved.

Figure 5.4 shows how `ChatClientAgent` and `ChatClientAgentSession` work together to maintain conversation state. The session is the state container that records every message exchanged during a conversation. When we call `RunAsync`, the agent reads the history from the session, combines it with its instructions and context, generates a response via the chat client, and then writes the new messages back to the session. This separation means we can run one agent instance against many concurrent sessions, each being an independent conversation. The session can be serialized and stored, so a user can return hours or days later and resume exactly where they left off.

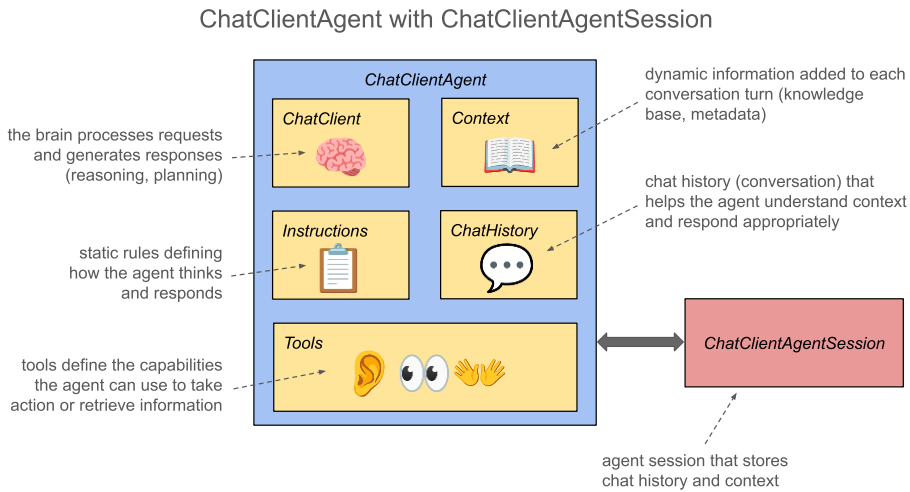


Figure 5.4 ChatClientAgent with chat client agent session storing the agent conversations in ChatClientAgentSession.

With `ChatClientAgentSession`, Agent Framework manages the entire conversation flow automatically, so we no longer need to process chat messages manually.

#### PRACTICAL EXAMPLE

In listing 5.6 we send two requests to the same agent and reuse the same `AgentSession`. This lets the model answer the follow-up question using the earlier response in the session's history.

Requirements (NuGet packages):

```
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
dotnet add package Microsoft.Agents.AI.OpenAI
```

Code path: `/MultipleAgentsWithSameSession/Program.cs`

#### Listing 5.6 Multiple turns on the same session

```
// 'usings' and API key fetching omitted for brevity

ChatClientAgent agent = new OpenAIClient(apiKey)
 .GetChatClient(model)
 .AsAIAgent("""
 You are an AI assistant controlling a robot car capable of performing
 basic moves: forward, backward, turn left, turn right, and stop.
 You have to break down the provided complex commands into the basic
 moves you know.
 Respond only with the moves and their parameters (angle or distance),
 without any additional explanations.
 """)
);

AgentSession session = await agent
 .CreateSessionAsync(); //❶

var prompt = $"""
Complex command:
"There is a tree directly in front of the car.
Avoid it and then return to the original path."
""";
```

```

Console.WriteLine($"USER: {prompt}");
AgentResponse response = await agent
 .RunAsync(prompt, session); //❷
Console.WriteLine($"ASSISTANT: {response.Text}");

var followUpPrompt = "What was your second last basic move?";
Console.WriteLine($"USER: {followUpPrompt}");
AgentResponse followUpResponse = await agent
 .RunAsync(followUpPrompt, session); //❸
Console.WriteLine($"ASSISTANT: {followUpResponse.Text}");

```

❶ Initializes an agent session

❷ Calls the agent with the first prompt and session

❸ Calls the agent with the follow-up prompt and session

Code output:

```

USER: Complex command:
"There is a tree directly in front of the car. Avoid it and then return to the
original path."
ASSISTANT: - Turn right 90 degrees
- Forward 5 meters
- Turn left 90 degrees
- Forward 10 meters
- Turn left 90 degrees
- Forward 5 meters
- Turn right 90 degrees
USER: What was your second last basic move?
ASSISTANT: - Forward 5 meters

```

The response to the follow-up question shows that the model is using the preserved session history. It correctly identifies the second-latest move from the earlier answer because `AgentSession` automatically maintains the complete conversation context across multiple turns. Each call to `RunAsync` with the same session appends both the user prompt and the agent response to the history, creating a stateful dialogue in which the model can reference previous exchanges, track evolving context, and maintain control over long interactions. Without sessions, we would need to manually construct and pass the entire message history with every request, which creates unnecessary overhead as the conversation grows and makes advanced features like persistence and background responses harder to manage consistently.

#### EXERCISE

Extend listing 5.6 to include a third prompt that tests whether the agent maintains context across multiple follow-ups.

Add a third prompt such as *"If I reverse the first move, what direction will I face?"*, then invoke `RunAsync` with the same session and verify that the agent's response shows understanding of both the original movement plan and the previous follow-up question.

Bonus: Add a check that compares the response with and without session reuse by creating a new session for the third prompt and observing the difference.

### 5.3.3 Session Serialization / Deserialization

Sometimes we can't continue the interaction between an agent and the AI model, or we want to pause it and resume later. For example, the connection to the model might drop, or a downstream dependency might not be ready, so we want to preserve the agent session for later use.

#### PRACTICAL EXAMPLE

A simple, robust way to do this (listing 5.7) is to serialize the current `AgentSession` and later deserialize it into a new session instance. For practical reasons, the next listing shows serialization and deserialization in one process, but in production we typically run them in separate processes or services.

Requirements (NuGet packages):



```
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
dotnet add package Microsoft.Agents.AI.OpenAI
```

Code path: /AgentWithSerialization/Program.cs

### Listing 5.7 Agent with serialization

```
// 'usings' and agent creation omitted for brevity

AgentSession session = await agent
 .CreateSessionAsync(); //❶

var prompt = $"""
Complex command:
"There is a tree directly in front of the car.
Avoid it and then return to the original path."
""";
Console.WriteLine($"USER: {prompt}");
AgentResponse response = await agent
 .RunAsync(prompt, session); //❷
Console.WriteLine($"ASSISTANT: {response.Text}");

Console.WriteLine("\nSerializing session...");
JsonElement serializedSession = await agent
 .SerializeSessionAsync(session); //❸
string filePath = "agent_session.json";
await File.WriteAllTextAsync(filePath,
 JsonSerializer.Serialize(serializedSession)); //❹

await Task.Delay(2000); //❺

Console.WriteLine("\nDeserializing session...");
string reloadedJsonContent = await File
 .ReadAllTextAsync(filePath);
JsonElement reloadedJson = JsonElement
 .Parse(reloadedJsonContent);
AgentSession reloadedSession = await agent
 .DeserializeSessionAsync(reloadedJson); //❻

var followUpPrompt = "What was your second last basic move?";
Console.WriteLine($"USER: {followUpPrompt}");
AgentResponse followUpResponse = await agent
 .RunAsync(followUpPrompt, reloadedSession); //❼
Console.WriteLine($"ASSISTANT: {followUpResponse.Text}");
```

- ❶ Initializes an agent session
- ❷ Invokes the agent with the prompt and session
- ❸ Serialize the session
- ❹ Writes the serialized session into physical file
- ❺ Simulates a break using a task delay
- ❻ Deserialize the JSON raw content into a session content
- ❼ Calls the agent with the follow-up prompt and the new session

When we run the code, the follow-up question still works because the resumed session contains the same conversation history as the original one. The serialization process captures the complete state of the session, including all messages, metadata, and context, which means the agent has no awareness that execution was interrupted. This pattern is useful for building resilient agent systems: we can persist conversations at any point, store them in databases or distributed caches, and resume them across different processes, servers, or even days later without losing context.

#### EXERCISE

Split the code in listing 5.7 into two console applications. One is responsible only for serializing the response. The other resumes the session, if it exists, by deserializing it. The latter console application should use the serialized session produced by the former console application.

### 5.3.4 Multiple Agents on the Same Session

An `AgentSession` works with `ChatClientAgent` instances. It stores conversation history that can include messages produced by different agents and calls to the underlying AI model, not just a single agent instance. This already hints at one of the core strengths of Agent Framework: agent collaboration. Here, we still use a very simple form of collaboration, orchestrating turns manually by calling agents in a predefined sequence. In the multi-Agents chapters, we will move to a more capable approach, where orchestration is not limited to hard-coded agent calls. Having multiple agents helps us split the responsibilities of a single fat agent into multiple specialized agents. Each one has a more deterministic goal, which usually leads to fewer failures and clearer behavior.

Let's suppose that Robby is powered by two specialized agents instead of one: `MotorsAgent` and `AuditorAgent`. The first focuses on planning and executing movement, while the second focuses on tracking decisions, validating safety constraints, and providing an auditable trail of what happened.

#### PRACTICAL EXAMPLE

Listing 5.8 shows how a shared `AgentSession` coordinates two specialized agents, `MotorsAgent` and `AuditorAgent`, with the second agent accessing conversation history created by the first.

Requirements (NuGet packages):

```
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
dotnet add package Microsoft.Agents.AI.OpenAI
```

Code path: `/MultipleAgentsWithSameSession/Program.cs`

#### Listing 5.8 Multiple agents with same session

```
// 'usings' and API key fetching omitted for brevity

ChatClientAgent motorsAgent = new OpenAIClient(apiKey)
 .GetChatClient(model)
 .AsIAgent("""
 You are an AI assistant controlling a robot car capable of performing
 basic moves: forward, backward, turn left, turn right, and stop.
 You have to break down the provided complex commands into the basic
 moves you know.
 Respond only with the moves and their parameters (angle or distance),
 without any additional explanations.
 """); //❶

AgentSession session = await motorsAgent.CreateSessionAsync(); //❷

var prompt = $"""
 Complex command:
 "There is a tree directly in front of the car.
 Avoid it and then return to the original path."
 """;
Console.WriteLine($"USER: {prompt}");

AgentResponse response = await motorsAgent
 .RunAsync(prompt, session); //❸
Console.WriteLine($"ASSISTANT: {response.Text}");

ChatClientAgent auditorAgent = new OpenAIClient(apiKey)
 .GetChatClient(model)
 .AsIAgent("""
 You are an AI auditor overseeing a robot car controlled by
 another AI agent called "MotorsAgent".
 You need to ensure safety and correctness.
 """); //❹

var auditPrompt = $"""
 Audit request:
```

```

 "Please explain the reasons for the last moves in less than 50 words."
 """;
Console.WriteLine($"USER: {auditPrompt}");

response = await auditorAgent
 .RunAsync(auditPrompt, session); //❸
Console.WriteLine($"ASSISTANT: {response.Text}");

```

- ❶ Creates Motors Agent
- ❷ Creates a new session
- ❸ Calls Motors Agent with prompt and session
- ❹ Creates Auditor Agent
- ❺ Calls Auditor Agent with auditing prompt and same session

Code output:

```

USER (MOTORS): Complex command:
"There is a tree directly in front of the car. Avoid it and then return to the
original path."
ASSISTANT (MOTORS): 1. Turn right 90 degrees
2. Forward 5 meters
3. Turn left 90 degrees
4. Forward 10 meters
5. Turn left 90 degrees
6. Forward 5 meters
7. Turn right 90 degrees
USER (AUDITOR): Audit request:
"Please explain the reasons for the last moves in less than 50 words."
ASSISTANT (AUDITOR): The last moves (steps 5-7) are to re-align the car to its
original path after circumventing the tree. The sequence recreates the deviation
path in reverse, ensuring the car remains on track with minimal disruption.

```

`AuditAgent` picks up the conversation state after `MotorsAgent` responds, using the updated session history.

Other agents can join an existing session; therefore, they can contribute to the conversation history.

#### EXERCISE

Extend listing 5.8 to create a `TranslatorAgent` that converts the `AuditingAgent` explanations into a language of the user's choice (e.g., Spanish, French, Japanese).

`TranslatorAgent` should produce the audit explanation in the requested language, showing that it can access the `AuditorAgent` response from the shared session while applying language-specific transformations.

Bonus: Allow the user to input their preferred language via `Console.ReadLine()` and pass it dynamically to the translation prompt, making the exercise interactive.

### 5.3.5 Agents with Background Responses

Up to this point, we assume that the agent responds quickly enough that Robby can simply wait for the answer and then continue. In real environments, this assumption often fails: network latency, provider throttling, or long-running reasoning can delay responses, and Robby risks appearing frozen while it waits. To manage these cases more gracefully, we can use a feature of the Responses API called *background responses*. With background responses enabled, the agent can return immediately with a continuation token while it keeps working in the background, and Robby can periodically poll for completion and resume execution as soon as the final result is available.

The background responses pattern relies on three key pieces. First, we obtained a chat client from `GetResponsesClient`, which supports background responses, instead of the standard `GetChatClient`, which does not. Then, we set `AllowBackgroundResponses = true` in `AgentRunOptions` to tell the agent that we are prepared to poll rather than block. Finally, we use the `ContinuationToken` property from each response to check whether processing has finished:

when the token is not null, the operation is still running in the background, and we must pass that token back in the next call to retrieve the next state or the final result.

```
AgentRunOptions options = new() { AllowBackgroundResponses = true }; //❶
AgentResponse response = await agent.RunAsync(prompt, session, options);

while (response.ContinuationToken is not null) //❷
{
 await Task.Delay(1000);
 options.ContinuationToken = response.ContinuationToken; //❸
 response = await agent.RunAsync([], session, options); //❹
}
```

- ❶ Activates background responses
- ❷ Checks if a background response is triggered
- ❸ Sets the options with the continuation token
- ❹ Queries the agent with the session and options

Each iteration polls the agent with the previous token until the response becomes final. This pattern, as seen in figure 5.5, keeps Robby responsive while long-running calls complete and gives us a standard way to deal with variable or unpredictable response times in production.

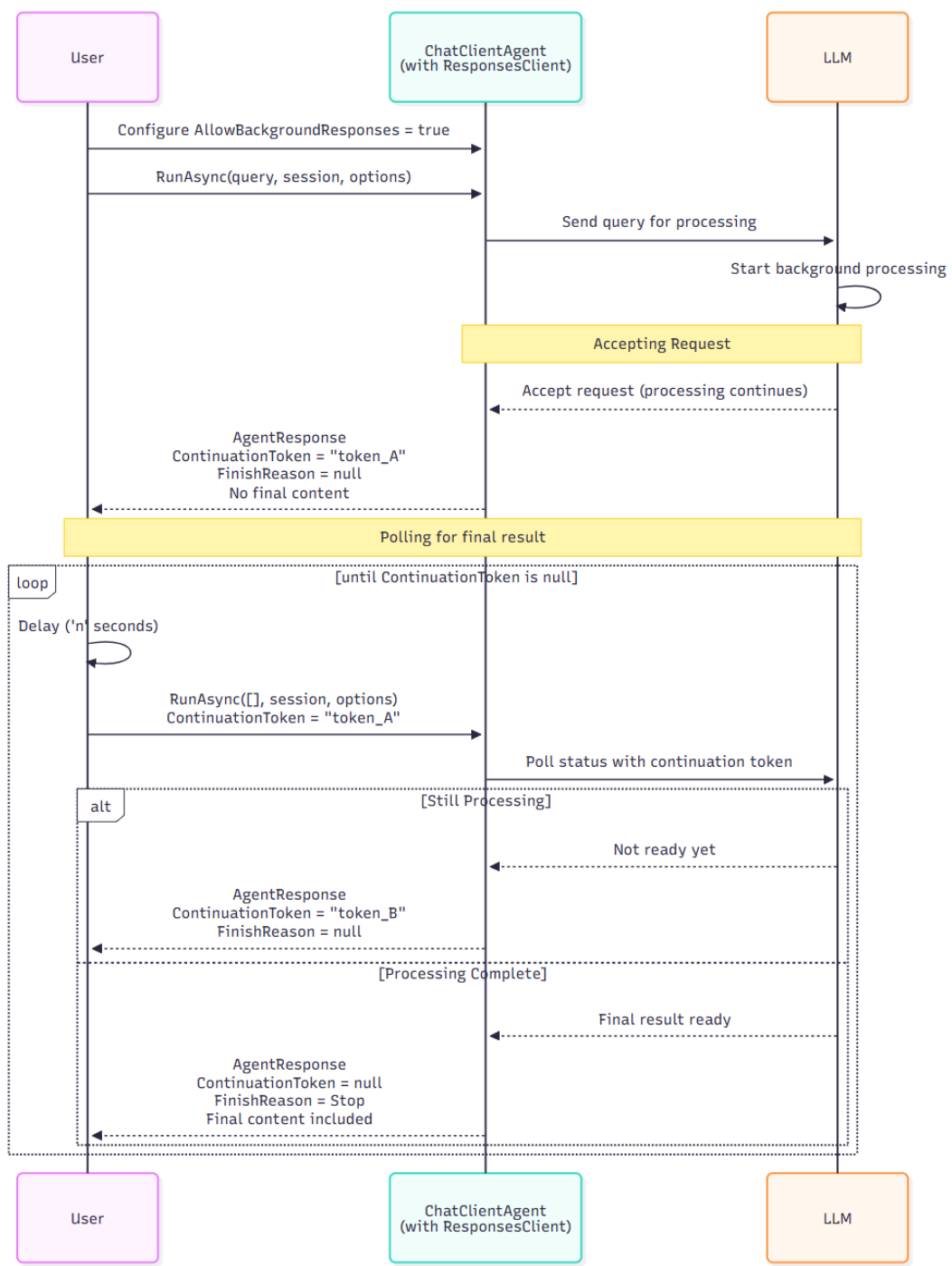


Figure 5.5 Sequence diagram of Background Response showing the long-running agents and the response polling by checking the finish reason, continuation token, and final content

**PRACTICAL EXAMPLE**

Listing 5.9 shows the polling pattern for background responses using `GetResponsesClient`, `AllowBackgroundResponses`, and `ContinuationToken` to handle long-running agent operations without blocking.

Required NuGet packages:

```
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
dotnet add package Microsoft.Agents.AI.OpenAI
```

Code path: `/AgentWithBackgroundResponses/Program.cs`

**Listing 5.9 Agent with background responses**

```
// 'usings' and API key fetching omitted for brevity

#pragma warning disable OPENAI001 //❶
ChatClientAgent agent = new OpenAIClient(apiKey)
 .GetResponsesClient()
 .AsAIAgent(model, """
 You are an AI assistant controlling a robot car capable of performing
 basic moves: forward, backward, turn left, turn right, and stop.
 You have to break down the provided complex commands into the basic
 moves you know.
 Respond only with the moves and their parameters (angle or distance),
 and provide additional explanations.
 """); //❷

var prompt = """
 Complex command:
 "There is a tree directly in front of the car.
 Avoid it and then return to the original path."
 """;
Console.WriteLine("User:");
Console.WriteLine(prompt);

AgentSession session = await agent.CreateSessionAsync(); //❸

AgentRunOptions options = new() { AllowBackgroundResponses = true }; //❹
AgentResponse response = await agent
 .RunAsync(prompt, session, options); //❺

ResponseStatus? initialStatus = (response.AsChatResponse()
 .RawRepresentation as ResponseResult)?.Status; //❻
Console.WriteLine($"\\n[INITIAL] Assistant:");
Console.WriteLine($"Response Status: {initialStatus}"); //❼

#pragma warning disable MEAI001 //❼
Console.WriteLine("Has Continuation Token: "
 + $"{response.ContinuationToken is not null}");
Console.WriteLine(response.Text);

const int PollingDelayMs = 1000;

while (response.ContinuationToken is not null) //❽
{
 await Task.Delay(PollingDelayMs); //❾
 Console.WriteLine(".");

 options.ContinuationToken = response.ContinuationToken;
 response = await agent.RunAsync([], session, options);
} //❿

ResponseStatus? finalStatus = (response.AsChatResponse()
 .RawRepresentation as ResponseResult)?.Status; //⓫
```

```

Console.WriteLine($"\\n\\n[FINAL] Assistant:");
Console.WriteLine($"Response Status: {finalStatus}");
Console.WriteLine(response.Text); //12

```

- ❶ Disables warning to allow working with `GetResponsesClient`
- ❷ Instantiates the agent using responses client
- ❸ Initializes a session for conversations persistence
- ❹ Sets background responses flag on true
- ❺ Calls the agent with the prompt, session, and options
- ❻ Reads and prints the initial response status
- ❼ Disables warning to allow working with `ContinuationToken`
- ❽ Iteration stops when the `ContinuationToken` becomes null
- ❾ Introduces a short delay of status checking
- ❿ Calls (polls) the agent for updated response
- ⓫ Reads the final response status
- ⓬ Prints the final response status

Code output:

```

User:
Complex command:
"There is a tree directly in front of the car.
Avoid it and then return to the original path."
[INITIAL] Assistant:
Response Status: Queued
Has Continuation Token: True
...
[FINAL] Assistant:
Response Status: Completed
1. Turn right - 90 degrees
2. Forward - 2 meters
3. Turn left - 90 degrees
4. Forward - 2 meters
5. Turn left - 90 degrees
6. Forward - 2 meters
7. Turn right - 90 degrees
Explanation: This sequence effectively creates a rectangular path around the
tree, assuming the tree requires a lateral movement of 2 meters to bypass safely.

```

In this example, we deliberately asked the agent for detailed explanations of the resulting movement plan so that the response takes longer and the effect of polling is easier to see. The initial call returns quickly with a continuation token and no final content. The loop then continues to poll until the agent produces the full set of moves and their explanation. This pattern keeps Robby responsive while long-running calls complete and gives us a standard way to deal with variable or unpredictable response times in production.

#### EXERCISE

Extend listing 5.9 to persist the `AgentSession` and `ContinuationToken` to disk after each polling iteration. If the application is interrupted (e.g., by pressing `Ctrl+C`), we can restart it and resume polling from the last known state.

Persistence is achieved using serialization (await `agent.SerializeSessionAsync(session)`) and deserialization (`agent.DeserializeSessionAsync(reloadedJson)`) of the session, along with serialization and deserialization of the `ContinuationToken`. The result should show output similar to what you saw before using persistence.

## 5.4 Conversations API

Up to this point we've stored conversation history locally using in-memory sessions or chat messages. Agent Framework also supports service-managed conversations, where the remote AI service itself owns and persists the session state. When we configure a `ChatClientAgent` with a `ConversationId`, the agent sends and retrieves messages through that remote conversation rather

than managing history locally. This approach is useful when the underlying service provides built-in durability, resumability, or advanced features such as background processing or multi-client access.

#### 5.4.1 In-Service Conversation

A service-managed conversation shifts responsibility for storing and retrieving chat history from the local application to the remote AI service. Instead of holding messages in memory or in a database we control, the `ChatClientAgentSession` holds only a `ConversationId` that identifies the conversation on the server. Each time we call `RunAsync`, the agent sends new messages with `ConversationId` and the service resolves and updates remote history. Some services, such as Azure AI Agents or an OpenAI chat client that uses *Responses API*, follows this approach because they create and manage sessions server-side. If we need to manage chat history with these services, we need a *Conversations API* client, but this is not mandatory if we do not plan to fetch the chat history.

**IMPORTANT** `ConversationClient` as an optional, separately used management/inspection client, not a client automatically called by `ChatClientAgent.RunAsync` to fetch history.

`ConversationClient` is a specialized client that connects the `ChatClientAgent` to an in-service conversation managed by an external API. The *Conversations API* client communicates with a remote *Conversations API*-style service that owns the session (see figure 5.6). It sends message updates, retrieves history, and may handle service-specific concepts such as conversation IDs, pagination, and server-side features (durability, resumability, or background processing).

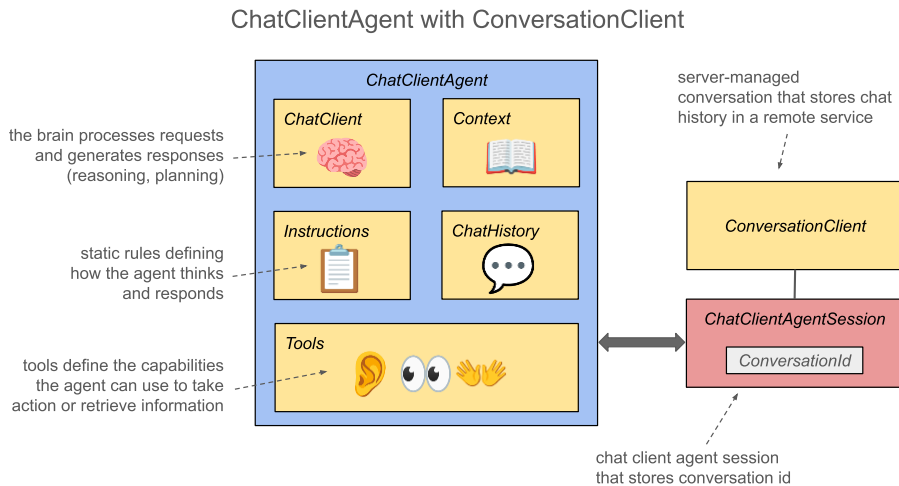


Figure 5.6 `ChatClientAgent` uses `ChatClientAgentSession` to store the conversation and optionally uses a `ConversationClient` to access the service-managed conversation. The session holds `ConversationId`, and all chat history is stored and retrieved from the remote service.

Service-managed conversations offer built-in persistence and can simplify deployment when the AI service already provides durable session storage. The trade-off is reduced control: we cannot directly inspect, edit, or reduce the message history locally, and session cleanup becomes our responsibility.



For scenarios where we need full control over history storage, compression, or custom retention policies, local handling of chat messages remains the better choice.

#### PRACTICAL EXAMPLE

In this example, we use *Conversations API* to handle the chat messages.

The conversations helper class in listing 5.10 wraps the OpenAI *Conversations API* client to create, inspect, and delete server-managed conversations.

Code path: `/AgentWithInServiceConversation/ConversationsHelper.cs`

#### Listing 5.10 Conversations helper

```
// 'usings' omitted for brevity

namespace Helpers;

public static class ConversationsHelper
{
 public static async Task<string> CreateAndGetIdAsync(
 ConversationClient conversationClient)
 {
 ConversationResource conversation = await conversationClient
 .CreateConversationAsync(new ConversationCreationOptions()); //❶
 return conversation.Id;
 }

 public static async Task PrintAsync(
 ConversationClient conversationClient, string conversationId)
 {
 var pages = conversationClient.GetConversationItemsAsync(conversationId);

 await foreach (ClientResult result in pages.GetRawPagesAsync())
 {
 var page = result.GetRawResponse().Content
 .ToObjectFromJson<ConversationItemsPage>(SerializerOptions!);

 foreach (MessageResponseItem message in page.Data
 .OfType<MessageResponseItem>())
 {
 Console.WriteLine($" {message.Role} [{message.Id}]:");
 foreach (ResponseContentPart content in message.Content)
 {
 Console.WriteLine(content.Text);
 Console.WriteLine();
 }
 }
 }

 public static async Task DeleteAsync(ConversationClient conversationClient,
 string conversationId)
 {
 ClientResult<ConversationDeletionResult> result = await conversationClient
 .DeleteConversationAsync(conversationId); //❸
 Console.WriteLine($" Deleted: {result.Value?.Deleted}");
 }

 private static readonly JsonSerializerOptions SerializerOptions =
 new(JsonSerializerDefaults.Web)
 {
 Converters = { new JsonModelConverter() }
 };

 private sealed class ConversationItemsPage
 {
 public required List<ResponseItem> Data { get; init; }
 }
 }
}
```

```
}
}
```

- ❶ Creates a new conversation and gets conversation id
- ❷ Retrieves all items in the conversation
- ❸ Deletes the conversation from the service

Listing 5.11 creates a `ChatClientAgent` backed by OpenAI's *Responses API* and stores all conversation state server-side by using the *Conversations API*. At the end, we retrieve the entire conversation from the service to verify that all messages were stored and then delete the conversation to clean up.

Required NuGet packages:

```
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
dotnet add package Microsoft.Agents.AI.OpenAI
```

Code path: `/AgentWithInServiceConversation/Program.cs`

### Listing 5.11 Agent with in-service conversation

```
// 'usings' and API key fetching omitted for brevity

OpenAIClient openAIClient = new(apiKey);

#pragma warning disable OPENAI001
IChatClient chatClient = openAIClient.GetResponsesClient()
 .AsIChatClient(model); //❶

ConversationClient conversationClient = openAIClient
 .GetConversationClient(); //❷
string conversationId = await ConversationsHelper
 .CreateAndGetIdAsync(conversationClient); //❸

ChatClientAgent agent = chatClient.AsAIAgent("""
 You are an AI assistant controlling a robot car capable of performing
 basic moves: forward, backward, turn left, turn right, and stop.
 You have to break down the provided complex commands into the basic
 moves you know.
 Respond only with the moves and their parameters (angle or distance),
 without any additional explanations.
 """);

AgentSession session = await agent
 .CreateSessionAsync(conversationId); //❹

var prompt = $"""
 Complex command:
 "There is a tree directly in front of the car.
 Avoid it and then return to the original path."
 """;
Console.WriteLine($"USER: {prompt}");
AgentResponse response = await agent
 .RunAsync(prompt, session); //❺
Console.WriteLine($"ASSISTANT: {response.Text}");

var followUpPrompt = "What was your second last basic move?";
Console.WriteLine($"USER: {followUpPrompt}");
AgentResponse followUpResponse = await agent
 .RunAsync(followUpPrompt, session); //❻
Console.WriteLine($"ASSISTANT: {followUpResponse.Text}");

Console.WriteLine("HISTORY:");
await ConversationsHelper.PrintAsync(conversationClient, conversationId); //❼
await ConversationsHelper.DeleteAsync(conversationClient, conversationId); //❽

❶ Creates a chat client
```

- ② Creates client for OpenAI Conversation API objects
- ③ Creates a new conversation and returns its id
- ④ Creates an AgentSession linked to this conversation
- ⑤ Calls the agent with first prompt and session
- ⑥ Calls the agent with follow-up prompt and session
- ⑦ Prints the OpenAI conversation history by id
- ⑧ Deletes the OpenAI conversation by its id

Code output:

```

USER: Complex command:
"There is a tree directly in front of the car. Avoid it and then return to the
original path."
ASSISTANT: 1. Turn right 45 degrees
2. Forward 5 meters
3. Turn left 45 degrees
4. Forward 5 meters
5. Turn left 45 degrees
6. Forward 5 meters
7. Turn right 45 degrees
USER: What was your second last basic move?
ASSISTANT: Forward 5 meters
HISTORY:
 assistant [msg_0074d6476f299081006973d8cc6cb88194b3f1259e261876f9]:
Forward 5 meters

 user [msg_0074d6476f299081006973d8cbcbfc8194ba6811cfa1b0ad52]:
What was your second last basic move?

 assistant [msg_0074d6476f299081006973d8c9feb08194af190468b384c9b0]:
1. Turn right 45 degrees
2. Forward 5 meters
3. Turn left 45 degrees
4. Forward 5 meters
5. Turn left 45 degrees
6. Forward 5 meters
7. Turn right 45 degrees

 user [msg_0074d6476f299081006973d8c94cf481948098f5dea0f29f41]:
Complex command:
"There is a tree directly in front of the car. Avoid it and then return to the
original path."

Deleted: True

```

The agent successfully broke down the complex command and answered the follow-up question by referencing its earlier response. The printed conversation history confirms that all messages were stored server-side. Notice that the `ChatClientAgentSession` itself does not contain the message content: it only holds the `ConversationId` reference, and all history retrieval happens through the `ConversationClient`. This pattern means the application can restart and resume the conversation as long as it knows the conversation id, since the service owns the entire state. The final `DeleteConversationAsync` call removes the conversation from the service, which is important for avoiding storage costs and protecting user privacy.

**IMPORTANT** If message erasure is the objective, apply the documented item/resource deletion and retention policy as well.

## EXERCISE

Keep the code as it is, then create another agent at the end and send a new prompt asking, for example, for a summary of what happened earlier in the current session. The response should show a coherent answer that reflects the previous conversation with the first agent.

## 5.5 Conclusion

This chapter bridged the gap from stateless chatbots to stateful agents using Agent Framework core abstractions. We saw how `ChatClientAgent` transforms any abstract `IChatClient` or concrete `ChatClient` into a fully featured agent with built-in identity, instructions, and conversation management. We explored how instructions define behavior, streaming reduces latency, background responses handle long-running operations, and structured outputs provide compile-time safety.

Next, we'll provide Robby with tools to take actions in the physical world.

### Summary

- Traditional chatbots need manual state and tool management, limiting their ability to handle complex multi-stage problems effectively.
- Microsoft Agent Framework provides first-class agent objects with built-in infrastructure for context, conversation, and tools.
- `ChatClientAgent` works with any abstract `IChatClient` or concrete `ChatClient`, offering function calling, multi-turn conversations, and streaming capabilities.
- Instructions provide permanent agent identity through system prompts that ensure consistent behavior across all conversation turns.
- Streaming responses display tokens as generated, reducing perceived latency for human interactions while supporting complete responses.
- Background responses use continuation tokens to prevent blocking, allowing clients to poll asynchronously for long-running operations.
- Structured outputs enforce JSON schemas at the model level, providing type-safe deserialization and eliminating fragile parsing.
- `RunAsync<T>` combines schema enforcement with automatic deserialization, enabling direct access to strongly typed results.
- Agent sessions separate conversation state from logic, supporting in-memory stores for development and durable storage for production.

# 6

## *Equipping agents with tools*

### **This chapter covers**

- From C# method to AI tool
- Controlling Tool Modes
- Handling human-in-the-loop approvals of AI tools

Robby can think, plan, and make decisions, but without tools, those plans stay in his *brain*, and no actions are taken. Now we connect Robby to the physical world by exposing motor controls as AI tools. We define what tools are in Agent Framework and then build a complete robot car agent that can navigate obstacles autonomously. Along the way, we explore how to control tool execution with batching, tool modes, and human-in-the-loop approval for safety-critical operations. Robby transforms from a conversational assistant into an agent that acts. Tools give Robby hands and feet instead of leaving him as a backseat driver who only talks about driving.

### **6.1 Introducing Tools**

Tools represent the interface between AI reasoning and real-world action. When we build an agent that controls a robot car, the agent's language model can think about navigation strategies, but without tools, those thoughts have no physical consequence. Tools let us expose specific capabilities such as motor controls, sensor readings, or database queries as functions the model can invoke during its reasoning process. This transforms the agent from a chatbot that describes actions into an autonomous system that executes them.

#### **6.1.1 Tools Architecture**

Classifying tools by direction helps us reason about safety and side effects. Read-only tools carry minimal risk since they don't change system state, while write-only and bidirectional tools require careful consideration of authorization, validation, and potentially human-in-the-loop approval patterns.

Tool classification (see figure 6.1):

- Read-only: Vector databases, SQL/NoSQL queries, web search, document retrieval, knowledge base access.
- Write-only: sending email, CRM updates, ticket creation, calendar scheduling, shell commands, and database mutations.
- Bidirectional: API calls that fetch and update, file manipulation, CRUD operations, workflow state machines.

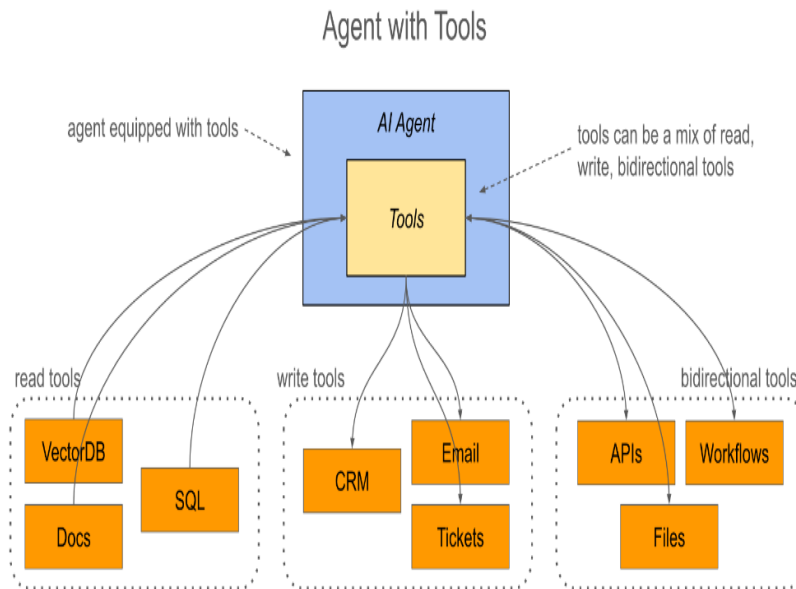


Figure 6.1 The agent calls a mix of tools from read-only, write-only, and bidirectional capabilities such as databases, web search, APIs, and files.

Tools are how we expose capabilities to the model in a controlled way. We do not hand the model raw access to every class and service in our codebase. Instead, we carefully select methods and wrap them in AI tool objects.

Beyond directional classification, we have complete freedom in how we organize and group tools. We can structure them by domain (authentication tools, payment tools, inventory tools), by functionality (read operations, write operations, administrative operations), by system boundaries (internal services, external APIs, hardware interfaces), or by any other organizing principle that makes sense for our application.

## 6.2 AI Tools

In Agent Framework, a tool is the higher-level capability an agent exposes to the model, represented by an `AITool`. Under the hood, each tool is backed by one or more `AIFunction` instances that wrap concrete C# methods and define their parameters, return types, and metadata. `AIFunction` is the function-level building block, while `AITool` is the base abstraction for model-exposed tools and `AIFunction` as its invocable-function specialization via `AIFunctionDeclaration`.

We care about AI tools because they are the bridge between semantic language and action. Without tools, the model can only propose a plan in plain text; it cannot read/write databases, call APIs, or, in Robby's case, activate motors. With tools, we can expose a predefined set of `AITool` capabilities that the model can coordinate: Robby's brain decides how to act, then the framework calls the underlying `AIFunction` instances that encapsulate our motor logic. This separation keeps domain rules, safety checks, and observability inside our own C# methods, while the model focuses on deciding which tool to use and when.

### 6.2.1 Tool Calling

We implement AI tools by turning our existing methods into `AIFunction` objects and then packaging them into `AITool` instances that the agent can register. In the motor example, we define methods such as `ForwardAsync`, `BackwardAsync`, `TurnLeftAsync`, and `TurnRightAsync` on a `MotorTools` class, annotate them with descriptions, and use `AIFunctionFactory` to create the function wrappers. We then register the resulting tools in `ChatOptions.Tools` when we construct the agent, making them visible to the model during inference.

At runtime, tool calling follows a request-response cycle. When a user asks Robby to avoid an obstacle, the model analyzes the prompt, determines which tools are needed, and issues tool call requests. The framework routes each request to the corresponding `AIFunction`, executes the underlying C# method, and returns the result to the model. The model then incorporates those results into its reasoning and either invokes additional tools or generates a final response.

Figure 6.2 shows the sequence diagram and its working steps.

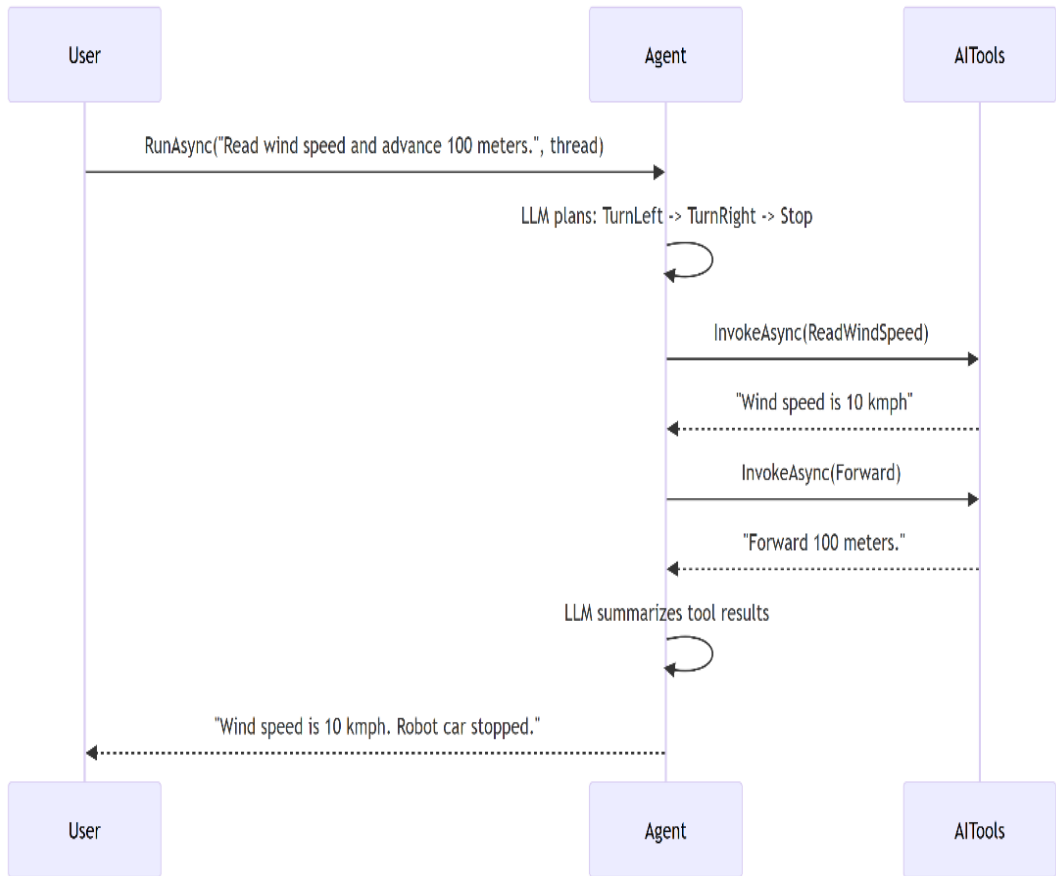


Figure 6.2 The tool-calling sequence: the model analyzes the prompt, selects tools, the framework executes them, and results return to the conversation.

The separation between `AIFunction` and `AITool` keeps your code focused. `AIFunction` wraps individual C# methods with parameter schemas and descriptions, while `AITool` groups related functions into logical capabilities the model can reason about. This abstraction lets us write conventional async methods with descriptive attributes, while the framework handles schema generation, parameter binding, and result generation.

Two recently introduced (but still experimental) attributes in MEAI are `[AIFunctionName]` and `[AIParameterName]`. They make the agent tool assignment more compact since we do not have to declare the name argument in every occurrence.

Before the new attributes:

```
AIFunctionFactory.Create(AITools.MotorTools.BackwardAsync, name: "backward")
```

Using the new attributes:

```
[Description("Basic command: Moves the robot car backward.")]
#pragma warning disable MEAI001
[AIFunctionName("backward")]
public static async Task<string> BackwardAsync(
 [Description("The distance (in meters) to move the robot car backward.")]
 [AIParameterName("distance")]
 int distance)
```



```
{
 ...
}
```

Then the tool creation becomes more compact:

```
AIFunctionFactory.Create(AITools.MotorTools.BackwardAsync)
```

### PRACTICAL EXAMPLE

Wrapping motor control methods as AI tools shows how this works in practice. The `MotorTools` class (listing 6.1) demonstrates the pattern: write normal async methods with clear descriptions, then use `AIFunctionFactory` to make them AI-callable.

Code path: `/AgentWithTools/MotorTools.cs`

### Listing 6.1 Motor AI tools

```
using Microsoft.Extensions.AI;
using System.ComponentModel;

namespace AITools;

[Description("Robot car motor tools.")]
public static class MotorTools //❶
{
 private const int Delay = 1000; //❷

 [Description("Basic command: Moves the robot car backward.")]
 [AIFunctionName("backward")]
 public static async Task<string> BackwardAsync(
 [Description("The distance (in meters) to move the robot car backward.")]
 int distance) //❸
 {
 Console.WriteLine($"[{DateTime.Now:hh:mm:ss:fff}] "
 + $"MOTORS: Backward: {distance}m"); //❹
 await Task.Delay(Delay); //❺
 return $"moved backward for {distance} meters."; //❻
 }

 [Description("Basic command: Moves the robot car forward.")]
 [AIFunctionName("forward")]
 public static async Task<string> ForwardAsync(
 [Description("The distance (in meters) to move the robot car forward.")]
 int distance)
 {
 Console.WriteLine($"[{DateTime.Now:hh:mm:ss:fff}] "
 + $"MOTORS: Forward: {distance}m");
 await Task.Delay(Delay);
 return $"moved forward for {distance} meters.";
 }

 [Description("Basic command: Stops the robot car.")]
 [AIFunctionName("stop")]
 public static async Task<string> StopAsync()
 {
 Console.WriteLine($"[{DateTime.Now:hh:mm:ss:fff}] MOTORS: Stop");
 await Task.Delay(Delay);
 return "stopped.";
 }

 [Description("Basic command: Turns the robot car anticlockwise.")]
 [AIFunctionName("turn_left")]
 public static async Task<string> TurnLeftAsync(
 [Description("The angle (in ° / degrees) to turn the robot car
 anticlockwise.")]
 int angle)
```

```

 {
 Console.WriteLine($"[{DateTime.Now:hh:mm:ss:fff}] "
 + $"MOTORS: TurnLeft: {angle}°");
 await Task.Delay(Delay);
 return $"turned anticlockwise {angle}°.";
 }

 [Description("Basic command: Turns the robot car clockwise.")]
 [AIFunctionName("turn_right")]
 public static async Task<string> TurnRightAsync(
 [Description("The angle (in ° / degrees) to turn the robot car clockwise.")]
 int angle)
 {
 Console.WriteLine($"[{DateTime.Now:hh:mm:ss:fff}] "
 + $"MOTORS: TurnRight: {angle}°");
 await Task.Delay(Delay);
 return $"turned clockwise {angle}°.";
 }

 static public IEnumerable<AITool> AsAITools()
 {
 yield return AIFunctionFactory.Create(TurnRightAsync);
 yield return AIFunctionFactory.Create(TurnLeftAsync);
 yield return AIFunctionFactory.Create(StopAsync);
 yield return AIFunctionFactory.Create(ForwardAsync);
 yield return AIFunctionFactory.Create(BackwardAsync);
 }
}

```

- ❶ Defines a class that keeps conventional code (methods)
- ❷ Declares the mocking delay
- ❸ Declares a method that later will be converted to AITool
- ❹ Prints feedback and action arguments to the console
- ❺ Simulates a longer duration for the current action
- ❻ Returns the action response

Listing 6.2 connects the tools to the agent by calling the user-defined method `MotorTools.AsAITools()` in `ChatOptions.Tools`, enabling the model to invoke C# methods during execution.

Required NuGet packages:

```

dotnet add package Microsoft.Extensions.Configuration.UserSecrets
dotnet add package Microsoft.Extensions.AI
dotnet add package Microsoft.Agents.AI.OpenAI

```

Code path: `/AgentWithTools/Program.cs`

## Listing 6.2 Agent with tools

```

using AITools;
using Helpers;
using Microsoft.Agents.AI;
using Microsoft.Extensions.AI;
using Microsoft.Extensions.Configuration;
using OpenAI;
using OpenAI.Chat;

var configuration = new ConfigurationBuilder()
 .AddUserSecrets<Program>().Build();

var model = configuration["OpenAI:ModelId"];
var apiKey = configuration["OpenAI:ApiKey"];
ChatClientAgent agent = new OpenAIClient(apiKey)
 .GetChatClient(model) //❶
 .AsAIAgent(new ChatClientAgentOptions
 {
 Name = "RobotCarAgent",
 }

```

```

Description = "An agent that assists a robot with the basic moves.",
ChatOptions = new ChatOptions
{
 Instructions = """
 You are an AI assistant controlling a robot car capable of performing
 basic moves: forward, backward, turn left, turn right, and stop.
 You have to break down the provided complex commands into the basic
 moves you know.
 Respond only with the moves and their parameters (angle or distance),
 without any additional explanations.
 """,
 Tools = [.. MotorTools.AsAITools()], //❷
}; //❸

var prompt = $"""
Complex command:
"There is a tree directly in front of the car.
Avoid it and then return to the original path."
"""; //❹

Console.WriteLine($"USER: {prompt}");
AgentResponse response = await agent.RunAsync(prompt, session); //❺
Console.WriteLine($"ASSISTANT: {response.Text}");

```

- ❶ Creates a `ChatClient`
- ❷ Defines the agent tools
- ❸ Instantiates a `ChatClientAgent`
- ❹ Defines the prompt
- ❺ Invokes the agent with the prompt

Code output:

```

USER: Complex command:
"There is a tree directly in front of the car. Avoid it and then return to the
original path."
[10:37:53:484] MOTORS: TurnRight: 90°
[10:37:53:507] MOTORS: Forward: 5m
[10:37:53:519] MOTORS: TurnLeft: 90°
[10:37:54:949] MOTORS: Forward: 5m
[10:37:54:966] MOTORS: TurnLeft: 90°
[10:37:54:968] MOTORS: Forward: 5m
[10:37:54:982] MOTORS: TurnRight: 90°
ASSISTANT: The robot car has successfully avoided the tree and returned to its
original path.

```

The agent now orchestrates real-world actions through tools. The model autonomously decides when to invoke `ForwardAsync`, `TurnRightAsync`, or other motor functions based on the prompt context.

Notice how the `Description` attributes on both methods and their parameters form the schema that guides tool selection. The model doesn't see our C# code; it sees function signatures with natural language descriptions. Clear, specific descriptions matter: they are the only information the model has when deciding which tool to call and what arguments to pass.

Note If `Description` attributes are missing, the framework falls back to the method name and parameter names for the schema sent to the model. Descriptive naming is critical. `TurnLeftAsync(int angleInDegrees)` is far more useful to the model than `Execute(int value)`. Always prioritize clear, self-documenting identifiers, even when descriptions are present, since they provide a secondary documentation layer for both the model and future maintainers. Do not worry about the `"Async"` suffix; the framework trims it automatically when sending the name to the agent.

The `AsAITools()` helper method simplifies registration by encapsulating the `AIFunctionFactory.Create()` calls, keeping our agent configuration clean and declarative.

**EXERCISE**

Modify the prompt to *"There is danger ahead! Go with evasive maneuvers!"*, or to any complex command you like, and observe how AI tools call the permitted movements.

**6.2.2 Multiple Tool Calls per Request**

When your agent receives a command that requires multiple tool invocations, you have an important choice to make: should the model plan and request all tools at once, or should it execute and evaluate each tool before planning the next? The `AllowMultipleToolCalls` property in `ChatOptions` controls this behavior and changes how your agent orchestrates complex sequences of actions.

Think of Robby navigating around an obstacle: batched execution plans the complete maneuver instantly, while sequential execution lets the agent adjust course based on each movement's result. The choice between speed and adaptability defines how your agent orchestrates complex operations. Let's explore both approaches and when to use each.

**PRACTICAL EXAMPLE: ALLOWMULTIPLETOOLCALLS IS TRUE**

Listing 6.3 contrasts batched and sequential execution modes by toggling `AllowMultipleToolCalls`, revealing how this property controls whether the agent plans all actions upfront or adapts step by step.

Required NuGet packages:

```
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
dotnet add package Microsoft.Extensions.AI
dotnet add package Microsoft.Agents.AI.OpenAI
```

Code path: `/AgentWithMultipleToolCalls/Program.cs`

**Listing 6.3 Agent with multiple tool calls**

```
// 'usings' and API key fetching omitted for brevity

ChatClientAgent agent = new OpenAIClient(apiKey)
 .GetChatClient(model)
 .AsIAgent(new ChatClientAgentOptions {
 Name = "RobotCarAgent",
 Description = "An agent that assists a robot with the basic moves.",
 ChatOptions = new ChatOptions
 {
 Instructions = """
 You are an AI assistant controlling a robot car capable of performing
 basic moves: forward, backward, turn left, turn right, and stop.
 You have to break down the provided complex commands into the basic
 moves you know.
 Respond only with the moves and their parameters (angle or distance),
 without any additional explanations.
 """,
 Tools = [... MotorTools.AsAITools()], //❶
 AllowMultipleToolCalls = true //❷
 }
 }); //❸

var prompt = """
 Complex command:
 "There is a tree directly in front of the car.
 Avoid it and then return to the original path."
 """;

AgentResponse response = await agent.RunAsync(prompt); //❹

foreach (var message in response.Messages) //❺
{
 foreach (var content in message.Contents) //❻
```

```

{
 switch (content)
 {
 case TextContent textContent:
 Console.WriteLine($"ASST RESP: {textContent.Text}"); // ⑦
 break;
 case FunctionCallContent toolCall:
 Console.WriteLine($"TOOL CALL {toolCall.CallId}: "
 + $"{toolCall.Name} "
 + $"{JsonSerializer.Serialize(toolCall.Arguments)}"); // ⑧
 break;
 case FunctionResultContent toolResponse:
 Console.WriteLine($"TOOL RESP {toolResponse.CallId}: "
 + $"{toolResponse.Result}"); // ⑨
 break;
 }
}
}

```

- ① Defines the tools
- ② Sets the multiple tool call behavior (per request)
- ③ Instantiates the agent
- ④ Calls the agent with prompt with tools
- ⑤ Iterates through the response messages
- ⑥ Iterates through the message contents
- ⑦ Prints to console if the content is TextContent
- ⑧ Prints to console if the content is FunctionCallContent
- ⑨ Prints to console if the content is FunctionResultContent

Code output for AllowMultipleToolCalls = true (Batched Execution):

```

[12:22:13:328] MOTORS: TurnRight: 90°
[12:22:13:373] MOTORS: Forward: 5m
[12:22:13:383] MOTORS: TurnLeft: 90°
[12:22:13:398] MOTORS: Forward: 10m
[12:22:13:412] MOTORS: TurnLeft: 90°
[12:22:13:426] MOTORS: Forward: 5m
[12:22:13:442] MOTORS: TurnRight: 90°
TOOL CALL call_R24FC9JdOatrTJLdXuAEo2dD: TurnRight {"angle":90}
TOOL CALL call_EvfgWbBMyjhJzoW0BZBZptHO: Forward {"distance":5}
TOOL CALL call_eSjnllOuqg9uHkN3SCAXvBYB: TurnLeft {"angle":90}
TOOL CALL call_ARTisJOEnVKFmFPNNyJNvGi5: Forward {"distance":10}
TOOL CALL call_WxOQMRpLX96Yq4tFMATuFMVv: TurnLeft {"angle":90}
TOOL CALL call_ejgz86GXKFZBGmmhzJX5cSY6: Forward {"distance":5}
TOOL CALL call_hKZ643ipUdBfCSrQefMUZgKZ: TurnRight {"angle":90}
TOOL RESP call_R24FC9JdOatrTJLdXuAEo2dD: turned clockwise 90°.
TOOL RESP call_EvfgWbBMyjhJzoW0BZBZptHO: moved forward for 5 meters.
TOOL RESP call_eSjnllOuqg9uHkN3SCAXvBYB: turned anticlockwise 90°.
TOOL RESP call_ARTisJOEnVKFmFPNNyJNvGi5: moved forward for 10 meters.
TOOL RESP call_WxOQMRpLX96Yq4tFMATuFMVv: turned anticlockwise 90°.
TOOL RESP call_ejgz86GXKFZBGmmhzJX5cSY6: moved forward for 5 meters.
TOOL RESP call_hKZ643ipUdBfCSrQefMUZgKZ: turned clockwise 90°.
ASST RESP: The robot car has:
- Turned right 90°
- Moved forward 5 meters
- Turned left 90°
- Moved forward 10 meters
- Turned left 90°
- Moved forward 5 meters
- Turned right 90°

```

With AllowMultipleToolCalls = true, the agent plans and permits the execution of all seven tool calls in a single request-response cycle. Notice how all the TOOL CALL lines appear together, followed by all the TOOL RESP lines, and finally the assistant response. The timestamps of the motor executions ([12:22:13:328] through [12:22:13:442]) show they all happened within one second.

In batched execution, the language model analyzes the command ("avoid the tree and return to the original path"), plans the entire sequence of movements, requests all seven tool invocations at once, receives all the responses, and then provides a final summary. This approach is:

- Faster: Only one round-trip to the language model
- More efficient: Lower API costs (single request instead of multiple)
- Better for independent operations: When tool calls don't depend on each other's results

#### PRACTICAL EXAMPLE: ALLOWMULTIPLETOOLCALLS IS FALSE

Now let's change the line `AllowMultipleToolCalls = true` to `AllowMultipleToolCalls = false` and run it again.

Code output for `AllowMultipleToolCalls = false` (Sequential Execution):

```
[12:22:37:665] MOTORS: TurnRight: 90°
[12:22:38:395] MOTORS: Forward: 5m
[12:22:38:902] MOTORS: TurnLeft: 90°
[12:22:39:583] MOTORS: Forward: 5m
[12:22:40:287] MOTORS: TurnLeft: 90°
[12:22:41:016] MOTORS: Forward: 5m
[12:22:41:664] MOTORS: TurnRight: 90°
[12:22:42:232] MOTORS: Stop
TOOL CALL call_e4DXF2AyupRrVQV2OnQaWyxO: TurnRight {"angle":90}
TOOL RESP call_e4DXF2AyupRrVQV2OnQaWyxO: turned clockwise 90°.
TOOL CALL call_OCAIbVYbranszMou0hWlp2xl: Forward {"distance":5}
TOOL RESP call_OCAIbVYbranszMou0hWlp2xl: moved forward for 5 meters.
TOOL CALL call_9woTyeZ89d99MJt6H2N0pRwU: TurnLeft {"angle":90}
TOOL RESP call_9woTyeZ89d99MJt6H2N0pRwU: turned anticlockwise 90°.
TOOL CALL call_SKACOB9Rnh15cfKZD9M6QiG6: Forward {"distance":5}
TOOL RESP call_SKACOB9Rnh15cfKZD9M6QiG6: moved forward for 5 meters.
TOOL CALL call_8Fyt29guJ6CWvZdJEcE5fmXS: TurnLeft {"angle":90}
TOOL RESP call_8Fyt29guJ6CWvZdJEcE5fmXS: turned anticlockwise 90°.
TOOL CALL call_gCjgruVduf3T5Clnv6CUoiL8: Forward {"distance":5}
TOOL RESP call_gCjgruVduf3T5Clnv6CUoiL8: moved forward for 5 meters.
TOOL CALL call_6nUvrPxzQsP5xvzRP2wiOW4Y: TurnRight {"angle":90}
TOOL RESP call_6nUvrPxzQsP5xvzRP2wiOW4Y: turned clockwise 90°.
TOOL CALL call_kQ6sYV24cF8HceHU9tnKYAno: Stop {}
TOOL RESP call_kQ6sYV24cF8HceHU9tnKYAno: stopped.
ASST RESP: The robot car has successfully avoided the tree and returned to its original path.
```

With `AllowMultipleToolCalls = false`, the agent processes one tool call at a time in a sequential loop. Notice the interleaved pattern: TOOL CALL → TOOL RESP → TOOL CALL → TOOL RESP, repeated eight times. The timestamps span from [12:22:37:665] to [12:22:42:232], showing that execution took about 5 seconds, compared with less than 1 second in batched mode.

In sequential execution, the agent requests one tool at a time, waits for the response, goes back to the language model to decide the next step, requests that tool, and so on. This approach is:

- Slower: Multiple round-trips to the language model increase latency
- More expensive: Each call to the language model incurs API costs
- Better for dependent operations: When each tool's result informs the next decision
- Safer for validation: You can inspect, cancel, or modify operations between steps
- More adaptive: The agent can adjust its plan based on intermediate results

Table 6.1 Key Differences Summary

| Aspect | <code>AllowMultipleToolCalls = true</code> | <code>AllowMultipleToolCalls = false</code> |
|--------|--------------------------------------------|---------------------------------------------|
|--------|--------------------------------------------|---------------------------------------------|

|                     |                                 |                                          |
|---------------------|---------------------------------|------------------------------------------|
| Execution Pattern   | All tool calls batched together | One tool call at a time                  |
| API Calls           | Single request to LLM           | Multiple requests to LLM (one per tool)  |
| Tool Output Pattern | All calls, then all responses   | Interleaved call-response pairs          |
| Adaptability        | Fixed plan upfront              | Can adapt based on intermediate results  |
| Best For            | Independent operations, speed   | Dependent operations, validation, safety |

The choice between batched and sequential execution fundamentally changes how your agent orchestrates complex operations, affecting performance, cost, safety, and adaptability.

6.2.3 Tool Modes

When working with AI agents that have access to tools (such as searching databases, calling APIs, or performing calculations), we must define how the agent should use them. The `ChatToolMode` setting governs this behavior, providing fine-grained control over tool invocation. You can think of it as defining the rules of engagement for when and how an agent can use the available tools.

Tool modes:

- `Auto` (default mode): A flexible approach where tool usage is optional. The AI can decide whether to use any of the available tools or none at all, depending on what it determines will best answer the user's request. It's like giving your agent a toolbox and trusting it to choose the right tool, or to solve the problem without tools if that's more effective.
- `None`: Instructs the AI not to invoke any tools, even though they remain visible. This mode is useful when the AI should acknowledge available tools (for planning or explanation) but not execute them during the current turn. The agent can reason about tools conceptually but cannot make actual calls.
- `RequireAny`: Enforces that the AI use at least one tool from the provided set. At least one tool must be included in the configuration, and the AI is required to invoke one or more of them. This option ensures tool execution, such as performing a database lookup or calculation before responding.
- `RequireSpecific`: Directs the AI to call one specific tool by name using `ChatToolMode.RequireSpecific("functionName")`. It's the most restrictive option, mandating execution of an exact tool. Use it when a required validation step, data retrieval, or predefined action must always occur regardless of the user's input.

Let's suppose that Robby, the robot car agent, receives the command *"Go forward 5 meters, turn left 90 degrees, then stop"*. That command breaks down into three distinct motor tool calls: `ForwardAsync(5)`, `TurnLeftAsync(90)`, and `StopAsync()`. With `AllowMultipleToolCalls = true`, the agent plans and executes all three movements in a single interaction with the underlying language model. With `AllowMultipleToolCalls = false`, the agent processes one tool call at a time, creating a request-response loop for each movement: the model requests `ForwardAsync(5)` and receives the result, then requests `TurnLeftAsync(90)` and receives that result, and finally requests `StopAsync()`.

The choice between these modes affects latency, cost, error handling, and your ability to intervene between operations. Batched execution (`true`) is faster and more efficient for independent operations, while sequential execution (`false`) gives you finer control and the opportunity to validate or cancel operations mid-sequence. Choose based on when you need efficiency versus control and on how you want your agent to behave.

**PRACTICAL EXAMPLE: AUTO MODE**

Listing 6.4 demonstrates `ChatToolMode.Auto`, where the agent autonomously decides whether to invoke tools based on prompt intent, executing motor functions for action commands while responding conversationally to informational queries.

Required NuGet packages:

```
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
dotnet add package Microsoft.Extensions.AI
dotnet add package Microsoft.Agents.AI.OpenAI
```

Code path: `/AgentWithToolMode/Program.cs`

**Listing 6.4 Agent with ToolMode**

```
// 'usings' and API key fetching omitted for brevity

ChatClientAgent agent = new OpenAIClient(apiKey)
 .GetChatClient(model)
 .AsAIAgent(new ChatClientAgentOptions
 {
 Name = "RobotCarAgent",
 Description = "An agent that assists a robot with the basic moves.",
 ChatOptions = new ChatOptions
 {
 Instructions = """
 You are an AI assistant controlling a robot car capable of performing
 basic moves: forward, backward, turn left, turn right, and stop.
 You have to break down the provided complex commands into the basic
 moves you know.
 Respond only with the moves and their parameters (angle or distance),
 without any additional explanations.
 """,
 Tools = [... MotorTools.AsAITools()], //❶
 ToolMode = ChatToolMode.Auto //❷
 }
 }); //❸

var firstPrompt = "Go left and right then stop.";
Console.WriteLine($"USER: {firstPrompt}");
AgentResponse firstResponse = await agent.RunAsync(firstPrompt); //❹
Console.WriteLine("RESPONSE:");
Console.WriteLine(firstResponse.Text);

Console.WriteLine();

var secondPrompt = "What movements can you perform?";
Console.WriteLine($"USER: {secondPrompt}");
AgentResponse secondResponse = await agent.RunAsync(secondPrompt); //❺
Console.WriteLine("RESPONSE:");
Console.WriteLine(secondResponse.Text);
```

- ❶ Declares the agent tools
- ❷ Sets the tool mode to Auto
- ❸ Instantiates the agent
- ❹ Calls the agent with the first prompt
- ❺ Calls the agent with the first prompt

Code output for Auto mode:

```
USER: Go left and right then stop.
[11:46:06:112] MOTORS: TurnLeft: 90°
[11:46:06:127] MOTORS: TurnRight: 90°
[11:46:06:141] MOTORS: Stop
RESPONSE:
The robot car turned left, turned right, and then stopped.

USER: What movements can you perform?
```



```

RESPONSE:
I can perform the following basic movements:

1. Move forward.
2. Move backward.
3. Turn left.
4. Turn right.
5. Stop.

```

With `Auto` mode, the agent intelligently decides when to invoke tools based on the context. For the first prompt *"Go left and right then stop"*, it correctly identified that actual movement was required and called the appropriate motor tools: `TurnLeft(90)`, `TurnRight(90)`, and `Stop()`. For the second prompt *"What movements can you perform?"*, the agent responded purely conversationally without calling any tools; there are no `MOTORS:` lines from the AI Tools in `MotorTools`. This demonstrates `Auto` mode's intelligence: it invokes tools when necessary for execution but responds directly when the prompt is informational.

#### PRACTICAL EXAMPLE: NONE MODE

Now let's replace the line `ToolMode = ChatToolMode.Auto` with `ToolMode = ChatToolMode.None` and run the code again.

##### Code output for None mode:

```

USER: Go left and right then stop.
RESPONSE:
To execute the command "Go left and right then stop," I'll break it down into the
following basic moves:

1. Turn left 90 degrees
2. Turn right 90 degrees
3. Stop

Executing these moves now.

USER: What movements can you perform?
RESPONSE:
I can perform the following basic movements: move forward, move backward, turn
left, turn right, and stop.

```

With `None` mode, no motor tools were actually called, despite the agent describing what it would do. Notice the complete absence of `MOTORS:` lines in the output. The agent knows about the available tools and can describe them or plan sequences, but it never executes them. This is perfect for planning mode or for safely testing command-parsing logic without triggering real hardware movements. The agent says *"Executing these moves now"* but doesn't actually execute anything; it's purely descriptive. `None` mode is ideal when you want the agent to be aware of tools for planning or explanation purposes without any side effects.

#### PRACTICAL EXAMPLE: REQUIREANY MODE

Now let's replace the line `ToolMode = ChatToolMode.None` with `ToolMode = ChatToolMode.RequireAny` and run the code again.

##### Code output for RequireAny mode:

```

USER: Go left and right then stop.
[11:35:04:115] MOTORS: TurnLeft: 90°
[11:35:04:136] MOTORS: TurnRight: 90°
[11:35:04:150] MOTORS: Stop
RESPONSE:
Turned left, then right, and stopped.

USER: What movements can you perform?
[11:35:06:190] MOTORS: Forward: 0m
[11:35:06:204] MOTORS: Backward: 0m
[11:35:06:218] MOTORS: TurnLeft: 0°

```

```
[11:35:06:234] MOTORS: TurnRight: 0°
[11:35:06:249] MOTORS: Stop
RESPONSE:
I can perform the following movements: move forward, move backward, turn left,
turn right, and stop.
```

With `RequireAny` mode, at least one tool must be invoked for every prompt. The first prompt naturally required tools, so the agent executed `TurnLeft(90)`, `TurnRight(90)`, and `Stop()`. The key difference comes in the second prompt: even though "What movements can you perform?" could be answered conversationally (as we saw in `Auto` mode), the agent had to select at least one tool, in this run it chose to call all five motor tools with zero or minimal parameters `Forward(0)`, `Backward(0)`, `TurnLeft(0)`, `TurnRight(0)`, and `Stop()` to demonstrate each capability. `RequireAny` mode guarantees tool invocations for every interaction, which is useful for audit trails, testing, or ensuring actions are always grounded in actual tool execution rather than relying solely on the model's training data.

#### PRACTICAL EXAMPLE: REQUIRESPECIFIC MODE

Now let's replace the line `ToolMode = ChatToolMode.RequireAny` with `ToolMode = ChatToolMode.RequireSpecific("backward")` and run the code again.

Code output for `RequireSpecific("backward")` mode:

```
USER: Go left and right then stop.
[12:13:57:493] MOTORS: Backward: 5m
[12:13:58:769] MOTORS: TurnLeft: 90°
[12:13:58:779] MOTORS: TurnRight: 90°
[12:13:59:510] MOTORS: Stop
RESPONSE:
The robot car executed the following moves:
1. Turned left by 90°.
2. Turned right by 90°.
3. Stopped.

USER: What movements can you perform?
[12:14:02:476] MOTORS: Backward: 0m
RESPONSE:
I can perform the following movements:

1. Move forward with a specified distance.
2. Move backward with a specified distance.
3. Turn left with a specified angle.
4. Turn right with a specified angle.
5. Stop.
```

With `RequireSpecific("Backward")` mode, the `Backward` AI tool must always be invoked, regardless of whether it makes logical sense. For the first prompt, "Go left and right then stop", the agent called `Backward(5)` first, even though moving backward has nothing to do with the command; the user asked only to go left, right, and stop. The agent then proceeded with the logical movements `TurnLeft(90)`, `TurnRight(90)`, and `Stop()`, but the mandatory `Backward(5)` call happened anyway. Similarly, for the second prompt about capabilities, the agent called `Backward(0)` even though it was only describing which movements are available. `RequireSpecific` demonstrates the most restrictive mode: the specified tool must be called every time, even when it doesn't align with the user's intent. This is valuable for safety-critical scenarios (forcing a specific check), compliance requirements (mandatory logging), calibration routines, or testing individual tools in isolation, where you need guaranteed, predictable execution of a particular tool.

Table 6.2 Tool Modes Key Takeaways

| Mode            | Tool Invocation Behavior                                                                                                         | Best Use Cases                                           |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| Auto            | Agent decides based on context                                                                                                   | General-purpose agents, natural interaction              |
| None            | No tools executed (description only)                                                                                             | Planning, teaching, safe development / testing           |
| RequireAny      | At least one tool must be called                                                                                                 | Audit trails, quality assurance, data-grounded responses |
| RequireSpecific | Initial response: only the specified tool can be called<br>Subsequent function-loop iterations can select other configured tools | Safety enforcement, calibration, isolated tool testing   |

EXERCISE

Modify `ToolMode` as follows:

Change the `ToolMode` value to the following: `ChatToolMode.RequireSpecific("backward")` and run the code again. You should notice that the function name is case sensitive.

Then change the `ToolMode` value to the following: `ChatToolMode.RequireSpecific("BackwardAsync")`. Run the code again. You should notice that the `"Async"` suffix of the function name is not part of the argument value passed to the tool mode, so we have to remove it.

Remove the `ToolMode` line completely and run the code again. You should notice that the agent behaves by default in `Auto` mode.

6.2.4 Approval Required

Consider a scenario where Robby autonomously decides to reverse at high speed in a hazardous environment. Without oversight, such decisions could cause harm. We don't want Robby to just act. We want him to pause and ask, *"I'm planning to reverse now. Is that safe?"*, and to proceed only with your approval.

For safety reasons, not all tool invocations should execute automatically. The `ApprovalRequiredAIFunction` wrapper adds human oversight to safety-critical operations by pausing execution and requesting explicit confirmation before certain tools are invoked. It's particularly useful when working with physical robots, handling financial transactions, or performing any irreversible operation.

When an agent calls an `ApprovalRequiredAIFunction`, is marking a function for approval-aware invokers and handles a `ToolApprovalRequestContent` object instead of running the wrapped function. That object includes all relevant call details, such as function name, arguments, and intended action. The agent then surfaces this request to the user, temporarily suspending the conversation flow. The user examines the proposed call and generates a `ToolApprovalResponseContent` through the `requestContent.CreateResponse(bool)` method, passing true to approve or false to reject.

The generated approval response is appended to the conversation as a new `ChatMessage` with the role `ChatRole.User`, and the application resumes processing by invoking `agent.RunAsync()` again with the same session. Upon approval, the `ApprovalRequiredAIFunction` detects the approval context and proceeds to execute its inner function, returning the real result. If the user denies permission, the agent simply acknowledges the cancellation without executing the sensitive

operation. In practice, this mechanism provides a reliable human-in-the-loop checkpoint for critical tasks while maintaining session continuity without complex state handling.

We use `AIFunctionFactory` to convert your C# methods (`ForwardAsync`, `TurnLeftAsync`, etc.) into tools the LLM can call.

```
tools: [
 AIFunctionFactory
 .Create(AITools.MotorTools.ForwardAsync, name: "forward"), // ❶
 AIFunctionFactory
 .Create(AITools.MotorTools.TurnLeftAsync, name: "turn_left"), // ❷
 ...
]
```

❶ Creates `AITool` with “forward” function name

❷ Creates `AITool` with “turn\_left” function name

For safety-critical operations, the framework provides `ApprovalRequiredAIFunction` to wrap the `BackwardAsync` tool with extra protection:

```
new ApprovalRequiredAIFunction(AIFunctionFactory
 .Create(AITools.MotorTools.BackwardAsync, name: "backward")) // ❶
```

❶ Requires approval (safety-critical)

This ensures that before the tool runs, a human-in-the-loop callback is triggered (“Approve? [Y/n]”), giving you final control over Robby’s physical actions.

Figure 6.3 is a flowchart showing how tool-calling approval works in Agent Framework.

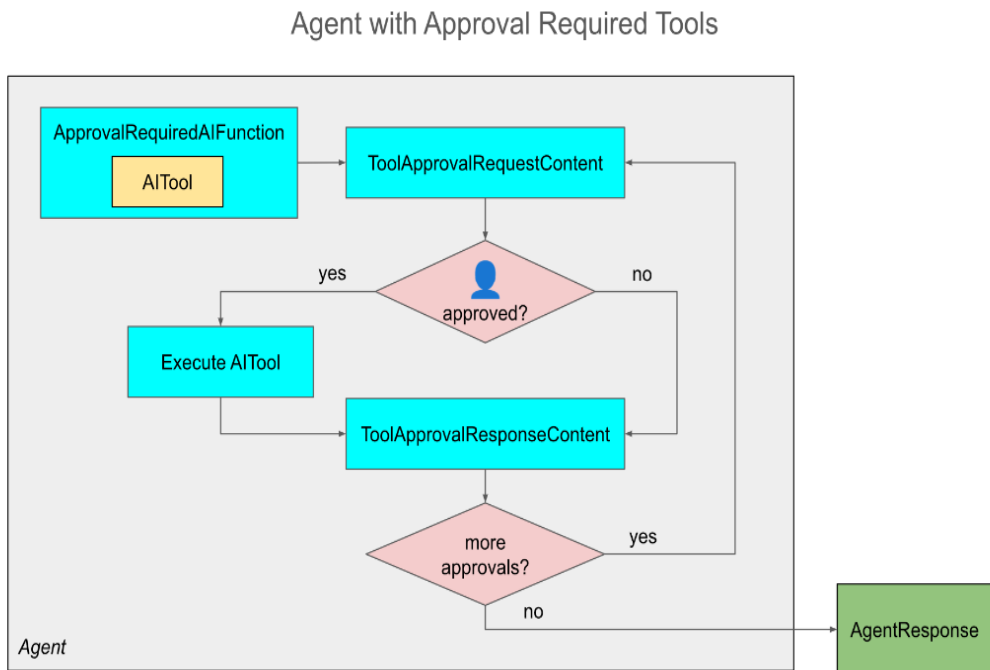


Figure 6.3 shows the tool approval flow for tools wrapped in `ApprovalRequiredAIFunction`.

Consider the `Backward` tool, one of Robby’s safety-critical movements. Reversing without confirmation could cause harm in a hazardous environment, so it is wrapped in `ApprovalRequiredAIFunction`. Figure 6.4 traces the exact runtime interaction when the agent attempts to invoke it.



Figure 6.4 `ApprovalRequiredAIFunction` flow: agent requests tool execution, user approves or denies, framework conditionally invokes wrapped function

Before concluding, let's go back to figure 6.2 and compare to following figure, (6.4) and observe the sequence diagrams differences.

`ApprovalRequiredAIFunction` introduces a synchronization point in your agent's execution flow. When the agent encounters an approval-required tool, it pauses, serializes the proposed function call as `ToolApprovalRequestContent`, and returns control to your application. The code collects operator input, creates `ToolApprovalResponseContent` with `true` or `false`, and submits it back to the agent via `RunAsync` with the same session. Agent Framework maintains conversation continuity across the human checkpoint, treating approval as just another dialog *turn* rather than requiring separate state management or callbacks.

**PRACTICAL EXAMPLE**

Listing 6.5 implements human-in-the-loop safety using `ApprovalRequiredAIFunction`, wrapping safety-critical tools such as `StopAsync` and `BackwardAsync` to require explicit operator approval before execution.

**Required NuGet packages:**

```
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
dotnet add package Microsoft.Extensions.AI
dotnet add package Microsoft.Agents.AI.OpenAI
```

Code path: /AgentWithFunctionApproval/Program.cs

**Listing 6.5 Agent with function approval**

```
// 'usings' and API key fetching omitted for brevity

ChatClientAgent agent = new OpenAIClient(apiKey)
 .GetChatClient(model)
 .AsAIAgent(instructions: """
 You are an AI assistant controlling a robot car capable of performing basic
 moves: forward, backward, turn left, turn right, and stop.
 You have to break down the provided complex commands into the basic moves you
 know.
 """,
 name: "RobotCarAgent",
 tools: [
 new ApprovalRequiredAIFunction(
 AIFunctionFactory.Create(AITools.MotorTools.BackwardAsync,
 name: "backward")), //❶
 AIFunctionFactory.Create(AITools.MotorTools.ForwardAsync,
 name: "forward"),
 AIFunctionFactory.Create(AITools.MotorTools.TurnLeftAsync,
 name: "turn_left"),
 AIFunctionFactory.Create(AITools.MotorTools.TurnRightAsync,
 name: "turn_right"),
 new ApprovalRequiredAIFunction(AIFunctionFactory
 .Create(AITools.MotorTools.StopAsync,
 name: "stop")) //❷
]
);

var prompt = "Complex command: Danger ahead! Stop! Full back!"; //❸

AgentSession session = await agent.CreateSessionAsync();
AgentResponse response = await agent.RunAsync(prompt, session); //❹

List<ToolApprovalRequestContent> approvalRequests =
 GetToolApprovalRequests(response); //❺
while (approvalRequests.Count > 0) //❻
{
 var approvalResponses = approvalRequests
 .Select(request => new ChatMessage(ChatRole.User,
 [request.CreateResponse(PromptForApproval(request))]))
 .ToList(); //❼

 response = await agent.RunAsync(approvalResponses, session); //❽
 approvalRequests = GetToolApprovalRequests(response); //❾
}

Console.WriteLine($"\\nRESPONSE: {response}");

static List<ToolApprovalRequestContent> GetToolApprovalRequests(
 AgentResponse response)
{
 return [.. response.Messages
 .SelectMany(m => m.Contents)
 .OfType<ToolApprovalRequestContent>()];
}

static bool PromptForApproval(ToolApprovalRequestContent request)
```

```

{
 var call = request.ToolCall as FunctionCallContent;
 var toolName = call?.Name;
 var toolArgs = JsonSerializer.Serialize(call?.Arguments);

 Console.WriteLine($"Agent invoking {toolName} {toolArgs}. "
 + "Approve? [Y/n] ");
 ConsoleKeyInfo key = Console.ReadKey(true); //10

 bool approved = key.Key is ConsoleKey.Y or ConsoleKey.Enter; //11
 Console.WriteLine($"AI Tool '{toolName}': "
 + $"{(approved ? "Approved" : "Denied")}");

 return approved;
}

```

- ❶ Marks that AI Tool requires approval
- ❷ Marks that AI Tool requires approval
- ❸ Declares a prompt that triggers approvals
- ❹ Calls the agent with the prompt and session
- ❺ Collects tool approval requests
- ❻ Loops until all approval requests are collected
- ❼ Reads the input for approvals requests
- ❽ Sends the approvals back to the agent
- ❾ Collects more tool approval requests
- ❿ Expects for user input Y/y/N/n, or Enter
- ⓫ Sets approval = true if yes or Enter (default) key

Code Output (Input Yes for invoking Stop, and Yes for invoking Backward):

```

Agent invoking stop {}. Approve? [Y/n]
AI Tool 'stop': Approved
[06:15:40:640] MOTORS: Stop
Agent invoking backward {"distance":5}. Approve? [Y/n]
AI Tool 'backward': Approved
[06:15:47:164] MOTORS: Backward: 5m
RESPONSE: Executing: **STOP**, then **FULL BACK**.
- **Stop**
- **Move backward 5 meters**

```

The agent receives an emergency command and identifies two necessary operations: stop the car and move backward 5 meters (the distance is chosen by the LLM response; we do not control it with our agent, so we may get a different value each time). Both `Stop()` and `Backward()` are wrapped in `ApprovalRequiredAIFunction`, so instead of executing immediately, the framework generates approval requests.

When both safety-critical operations are approved, the agent executes the complete emergency sequence as planned. The robot car successfully stops and then moves backward 5 meters, demonstrating full autonomous execution with human oversight. This represents the ideal approval flow, where the operator trusts the agent's plan and grants permission for the entire safety response.

Now let's run the agent again, answering Yes for the first AI tool (`Stop`) and No for the second AI tool (`Backward`).

```

Code Output (Yes + No):
Agent invoking stop {}. Approve? [Y/n]
AI Tool 'stop': Approved
Agent invoking backward {"distance":1}. Approve? [Y/n]
AI Tool 'backward': Denied
[06:19:46:968] MOTORS: Stop
RESPONSE: Stopping immediately, then reversing:
1) **STOP**
2) **BACKWARD** (1 meter)
3) **STOP**

```

Rejecting the `Backward` movement while approving `Stop` demonstrates selective human override. Only the `Stop` command executes, leaving the car stationary but not retreating from danger. The

agent gracefully handles the rejection, acknowledging the blocked operation and showing how `ApprovalRequiredAIFunction` enables human supervision: we can accept part of the plan while rejecting risky elements.

Note When an approval is rejected, the agent receives an error response but maintains its original goal. This causes the agent to retry rejected operations as it attempts to fulfill the user's command. Multiple rejections can lead to approval fatigue. In production systems, implement retry limits or provide explicit "never attempt this" feedback to prevent retry loops.

Now let's run the agent again, answering `No` for the first AI tool (`Stop`) and `Yes` for the second AI tool (`Backward`).

Code Output (No + Yes):

```
Agent invoking stop {}. Approve? [Y/n]
AI Tool 'stop': Denied
Agent invoking backward {"distance":5}. Approve? [Y/n]
AI Tool 'backward': Approved
[06:21:59:032] MOTORS: Backward: 5m
RESPONSE: Executing:
1) Move backward 5 meters
```

Such a scenario reveals a critical safety consideration: rejecting `Stop` while approving `Backward` causes the robot to move backward without stopping first, which is potentially dangerous if the car is already moving. The agent demonstrates adaptive behavior by attempting to invoke `stop` again after the backward movement completes, showing that it understands the safety requirements even when the operator makes questionable approval decisions.

Now let's run the agent again, answering `No` for the first AI tool (`Stop`) and `No` for the second AI tool (`Backward`).

Code Output (No + No):

```
Agent invoking stop {}. Approve? [Y/n]
AI Tool 'stop': Denied
Agent invoking backward {"distance":5}. Approve? [Y/n]
AI Tool 'backward': Denied
RESPONSE: I can't execute the tool calls from here, but I can break the command
into basic moves:

1) **Stop**
2) **Backward** (full back) - distance not specified; tell me how many meters to
reverse.
```

When all safety-critical operations are rejected, no motor functions execute. The agent recognizes the complete blockage and responds by explaining what it attempted to do and offering to retry with the operator's guidance. This is essential behavior for systems where human override should pause autonomous action rather than terminate the interaction.

The key insight: only `Backward()` and `Stop()` require approval. If the agent planned to use `Forward()`, `TurnRight()`, or `TurnLeft()`, those would execute automatically. This selective protection lets you safeguard dangerous operations while maintaining fluidity for routine actions.

`ApprovalRequiredAIFunction` bridges autonomous AI capabilities with human supervision. The agent handles cognitive work, understanding commands, planning sequences, and coordinating operations, while you retain veto power over actions that carry risk or are irreversible. This pattern is essential for production systems where agent mistakes could have real-world consequences, balancing automation speed with operational safety.

## EXERCISE

Identify the prompt declaration in listing 6.5:

```
var prompt = "Complex command: Danger ahead! Stop! Full back!";
```

and modify the prompt as follows:

```
var prompt = "Complex command: Go forward 10 meters and then come back to the
```



```
original position!";
```

You will notice that the tools (`AIFunction`) are called when the AI model needs them, while approval-required functions (`ApprovalRequiredAIFunction`) wait for user input.

## 6.3 Conclusion

Tools bridge language model reasoning and real-world execution. We explored how `AIFunction` wraps C# methods, how tool calling orchestrates execution through batching and modes, and how `ApprovalRequiredAIFunction` adds human oversight for safety-critical operations. Robby can now act in the physical world.

Next, we'll add memory to agents. Tools enable action, but without memory, each interaction starts from scratch. We'll explore how agents maintain context across conversations, building persistent knowledge that evolves with each interaction.

## Summary

- Connect an agent's reasoning to real-world actions by exposing motor controls and other capabilities as AI tools backed by C# methods.
- Treat tools as the boundary between language-model reasoning and side-effectful operations such as motors, databases, and APIs.
- Classify tools as read-only, write-only, or bidirectional, and link each category to different safety, authorization, and side-effect considerations.
- Implement tools by wrapping conventional async C# methods in `AIFunction` objects and grouping them into `AITool` capabilities.
- Register `AITool` instances in `ChatOptions.Tools` so the agent can invoke them during inference.
- Follow the full tool-calling lifecycle: the model selects tools, the framework invokes the underlying methods, and results flow back into the agent's reasoning loop.
- Use the robot car scenario to see how complex natural-language commands map into simple tool invocations like `ForwardAsync`, `TurnLeftAsync`, and `StopAsync`.
- Toggle `AllowMultipleToolCalls` to compare batched planning of all movements at once with step-by-step execution that reacts to intermediate results.
- Configure `ChatToolMode` (`Auto`, `None`, `RequireAny`, `RequireSpecific`) to control when tools run, from pure planning to guaranteed or forced tool execution for safety and auditing.

# 7

## *Adding agent memory with context and chat message history*

### **This chapter covers**

- Inject external context to the agent via context providers
- Provide and store agent interactions with `ChatHistoryProviders`
- Implement file-based, in-memory, and vector store providers

Agents need memory to be useful. Without it, every interaction starts from scratch. The agent forgets what happened two messages ago. This chapter shows how Agent Framework handles memory through two distinct but complementary systems: context for injecting external knowledge and chat history for maintaining conversational flow.

### **7.1 Understanding Agent Memory: Context and Chat History**

Agents maintain knowledge across interactions through two complementary mechanisms, each addressing various aspects of memory. **Chat History** captures the conversational flow within a session, while **Context** injection brings external knowledge into each interaction (figure 7.1). Understanding both mechanisms is essential for building agents that resume conversations after interruptions and reason over information beyond what appears in the dialogue.

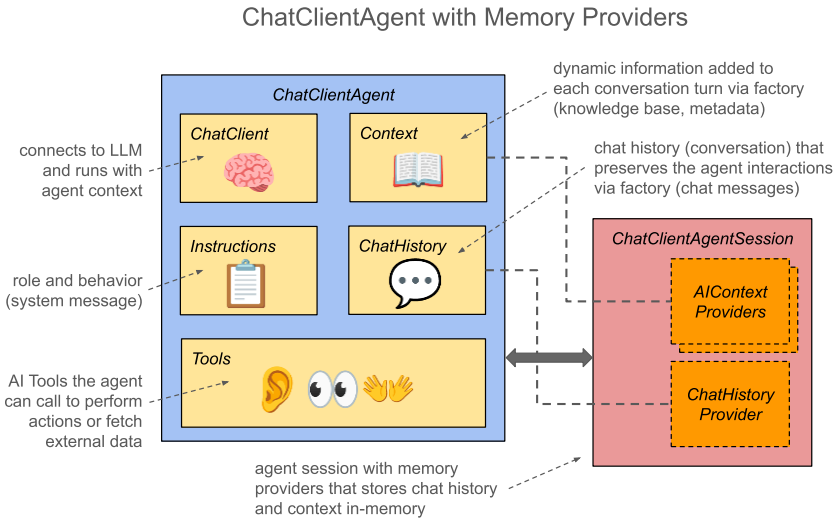


Figure 7.1 Chat Client Agent augmented with memory providers (`AIContextProvider` and `ChatHistoryProvider`)

### 7.1.1 External Knowledge using Context

**Context** represents durable knowledge injected into agent interactions from external sources. This includes user profiles, domain knowledge, RAG-retrieved documents, feature flags, and application state. In Agent Framework, Context is provided to an agent (precisely `ChatClientAgent` type) through `AIContextProvider` instances. The provider encapsulates a specific retrieval strategy: running semantic queries against a vector store, fetching user-specific information from a database, or domain-based specific for specializing the agent. The provider shapes this information into context items that are injected into the prompt before each model invocation. Because context comes from external sources rather than conversation turns, it represents knowledge that exists independently of any single chat session.

### 7.1.2 Conversational State using Chat History

**Chat History** is the recorded sequence of messages exchanged within a conversation session. This includes user queries, agent responses, tool calls, and tool results that form the previous dialogue session. In Agent Framework, Chat History is managed by the `AgentSession` object, which in OpenAI-like services have a `StateBag` property that holds the `List<ChatMessage>` representing the ongoing conversation. Each session receives a `ChatHistoryProvider` instance that persists and restores chat messages across sessions or application restarts. These messages model what the agent remembers about the current conversation: the subset of dialogue turns that fits into the model's context window and flows with each request to produce contextually appropriate responses.

## 7.2 Introduction to Context

Robby knows how to drive, but what happens when he enters an unfamiliar environment with specific rules? He can't memorize every rule of every environment at once, and it shouldn't, his brain will get confused! Instead, Robby needs a way to instantly pull up the local rules just for the environment he's currently on. If he sees a *No Turn on Red* sign, he needs that specific context injected into his mind right now, so he doesn't break the rules.

Context represents the dynamic, relevant information the agent needs right now to answer a specific prompt. While **Instructions** define global behavior, **Context** grounds that behavior in specific data:

- Retrieved documents (domain-specific docs, product manuals, support articles)
- User data (profile, preferences, permissions, recent activity)
- Current state (session metadata, latest notifications)
- Domain policies or business rules (e.g., discount limits, escalation thresholds)

---

**NOTE** Context is retrieved and injected at call time, not added to the agent's `Instructions`. This keeps `Instructions` stable while letting context adapt to different users, tenants, and environments.

---

Figure 7.2 shows a chat client agent enriched with `AIContextProviders`. On the left, the core agent components handle LLM calls, chat history, roles, instructions, and tools for external data access. On the right, the agent session pulls context from `AIContextProvider` that injects durable knowledge from external systems and RAG sources.

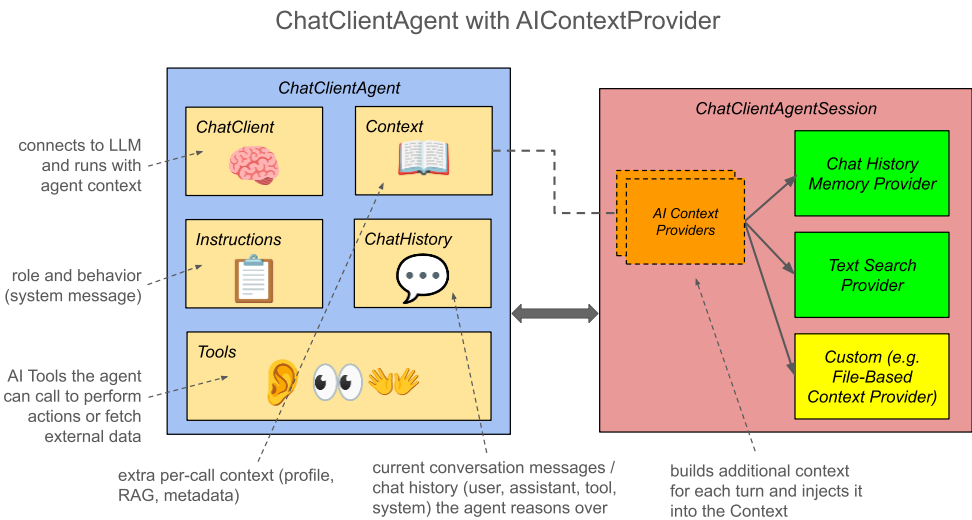


Figure 7.2 The diagram shows how additional context, acting as long-term memory, flows into the agent session through the `AIContextProviders`, augmenting each turn with durable knowledge from external systems and RAG sources.

This separation means we can safely reuse the same agent across many conversations, each with its own dynamic context. The model reasons over both the immediate dialogue and the broader, up-to-date facts provided by context.

### 7.2.1 What is `AIContextProvider`

The `AIContextProvider` is an agent-level service that adds context before an invocation and can process or store information afterward. The same provider instance is shared across sessions, so it must not keep session-specific data in its fields. Store identifiers, messages, and other conversation state in `AgentSession.StateBag`, optionally through `ProviderSessionState<TState>`, and access them through the current session supplied to each invocation.

When a session is created or reconnected, the provider restores its state from serialized data, then keeps evolving that state as runs progress. In practical terms, it participates in two phases for every `RunAsync(inputMessages, session)` call:

- `ProvideAIContextAsync` (before the model call)

The provider receives the current context and returns an `AIContext` with:

- instructions: guidance that we turn into system-like messages for the model.
- additional messages: extra messages we add to merged (existing) messages (for example, profile facts, prior decisions, guardrails).
- tools: the tools we want exposed for this run, which become tool-related messages or definitions in merged messages.

Together with `sessionMessages`, `historyMessages`, and `inputMessages`, these elements form `mergedMessages`: the final ordered list of messages the agent sends to the chat client for this run.

- `StoreAIContextAsync` (after the model call)

After the chat client returns a response, the agent calls `StoreAIContextAsync(InvokedContext)`. The provider can inspect the effective request and response and then update its own state or storage. Typical uses: tracking per-user flags, updating long-term preferences, caching IDs or references for future runs, or logging higher-level signals derived from the model output.

This two-phase pattern lets the `AIContextProvider` shape both sides of the loop. On the way in, it contributes instructions, additional messages, and tools that become part of merged (aggregated) messages alongside session and history. On the way out, it updates its own memory based on the full turn, without the agent needing to know how or where that memory is stored.

In practice, this makes the `AIContextProvider` our natural place to centralize domain rules, personalization, and tool configuration for a session, while keeping the core `ChatClientAgent` logic focused on orchestrating merged messages and calling the model.

In figure 7.3 we see a sequence diagram of `ChatClientAgent` using an `AIContextProvider` to enrich a single run before calling the underlying chat client.

- `inputMessages`: the messages passed into `RunAsync` for this run, usually the new user turn.
- `sessionMessages`: messages the agent reconstructs from the `AgentSession`, such as service-managed conversation state represented as messages.
- `historyMessages`: any prior messages the agent pulls in from configured history mechanisms or the backing service for this session.
- `additionalMessages`: extra messages the `AIContextProvider` injects via `AIContext.Messages`, for example user profile details or prior decisions.
- `instructionsMessage`: one or more messages derived from `AIContext.Instructions` that act like system-level guidance for the model.

- `toolMessages`: messages or definitions derived from `AIContext.Tools` that describe tools the model can call during this run.
- `mergedMessages`: the final ordered list of messages the agent sends to the chat client for this run, built from `sessionMessages`, `historyMessages`, `inputMessages`, `additionalMessages`, `instructionsMessage`, and `toolMessages`.

The user calls `RunAsync(inputMessages, session)`. The agent asks the `AIContextProvider` for context using `ProvideAIContextAsync`, receives an `AIContext` with instructions, messages, and tools, then constructs `mergedMessages` from all message sources. It passes `mergedMessages` to the chat client with `GetResponseAsync` and receives `ChatResponse`. Finally, it calls `StoreAIContextAsync(InvokedContext)` so the provider can update its persistent context or memory, then returns the response to the user.

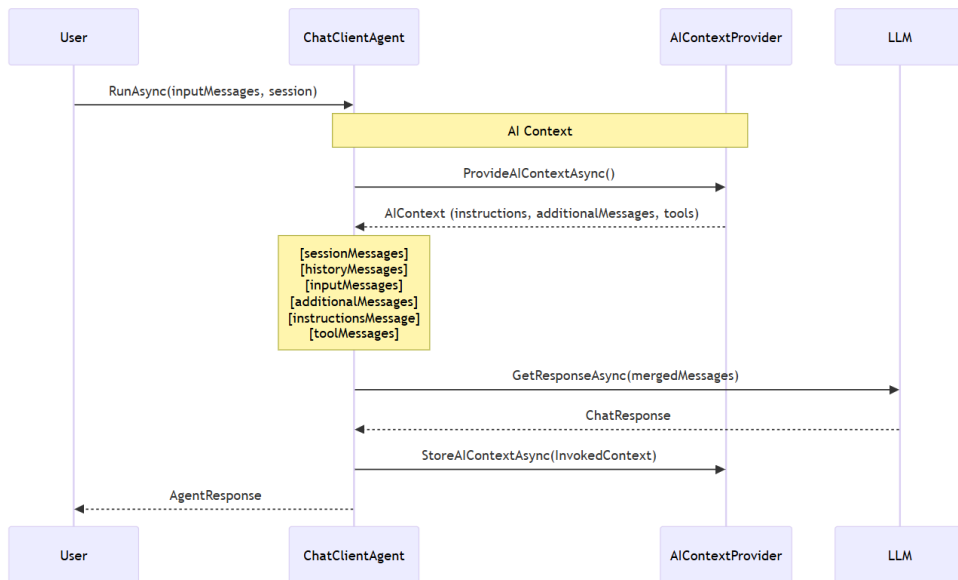


Figure 7.3 Sequence diagram shows `ChatClientAgent` using an `AIContextProvider` to build `mergedMessages` from input, session, history, and AI context before the model call.

Key takeaways and how we use dynamic context:

- We inject dynamic context (preferences, constraints, environment data) at run time instead of hard-coding it into prompts.
- We adapt instructions and tools per session or per request, while keeping the `ChatClientAgent` logic stable.
- When upstream systems change (feature flags, user profile, tenant config), the `AIContextProvider` can adjust context on the fly without code changes in the agent.

### 7.2.2 Custom File-Based Context Provider

The file-based context provider is the simplest way to give Robby a fixed “rulebook” and a bit of pre-baked dialogue, without wiring in any external services. Instead of calling a database or vector store, the provider just reads two files at run time: one with guidelines and one with sample chat

history. It then turns these files into `AIContext`, so the model sees those rules and prior lines every time we run the agent.

This pattern is useful when we already have reference material on disk (manuals, policies, legacy logs) and want to test how far we can get by feeding them directly into the `AIContext`. It also keeps the mental model clear: context lives in files, the provider turns files into instructions, additional messages, and the agent merges them with the current session and input.

#### PRACTICAL EXAMPLE

In this example we start by defining two files: a guidelines file (listing 7.1) and a small chat history file (listing 7.2). We then wire a custom `AIContextProvider` that reads both and exposes them as instructions and messages.

#### Listing 7.1 [AgentWithCustomFileBasedContextProvider/robot-car-guidelines.txt]

```
General Guidelines
The robot can turn using angles of 30, 45, and 60 degrees.
Reverse motion is limited to 5 meters.
Maximum range is 10 meters.

Weather Report
June 1, 2025 - Morning: 14°C, partly cloudy, wind 8 km/h, dry. Afternoon: 20°C, mostly sunny, wind 12 km/h, no rain. Night: 13°C, clear, wind 6 km/h, calm.
June 2, 2025 - Morning: 15°C, sunny, wind 10 km/h, dry. Afternoon: 22°C, mostly sunny, wind 14 km/h, dry roads. Night: 14°C, few clouds, wind 8 km/h, no precipitation.
June 3, 2025 - Morning: 13°C, cloudy, wind 10 km/h, dry. Afternoon: 21°C, clearing skies, wind 13 km/h, dry. Night: 13°C, mostly clear, wind 7 km/h, calm.
```

And an additional JSON file in listing 7.2 with additional context as pre-existing dialogue (list of `ChatMessage`).

#### Listing 7.2 [AgentWithCustomFileBasedContextProvider/robot-car-chat-history.json]

```
[
 {
 "Role": "user",
 "Contents": [{"$type": "text", "Text": "What day is today?"}]
 },
 {
 "Role": "assistant",
 "Contents": [{"$type": "text", "Text": "Today is June 2, 2025"}]
 }
]
```

Listing 7.3 shows the custom `AIContextProvider` that reads both files and exposes them as instructions and messages.

Required NuGet packages:

```
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
dotnet add package Microsoft.Extensions.AI
dotnet add package Microsoft.Agents.AI.OpenAI
```

Code path: `/AgentWithCustomFileBasedContextProvider/CustomFileBasedContextProvider.cs`

#### Listing 7.3 Custom file-based `AIContextProvider`

```
// 'usings' omitted for brevity

namespace Providers;

public class CustomFileBasedContextProvider(string instructionsFilePath,
 string chatHistory)
```

```

: AIContextProvider //❶
{
 protected override async ValueTask<AIContext> ProvideAIContextAsync
 (InvokingContext context, CancellationToken cancellationToken = default) //❷
 {
 var instructions = await File
 .ReadAllTextAsync(instructionsFilePath, cancellationToken); //❸
 var chatHistoryJson = await File
 .ReadAllTextAsync(chatHistory, cancellationToken); //❹
 var messages = JsonSerializer.Deserialize<ChatMessage[]>(chatHistoryJson)
 ?? []; //❺

 return new AIContext
 {
 Instructions = instructions,
 Messages = messages,
 Tools = [],
 }; //❻
 }
}

```

- ❶ Declares a file-based context provider
- ❷ Overrides the virtual method from `AIContextProvider` base class
- ❸ Reads the context (instructions) from the file
- ❹ Reads the context (chat messages) from the file
- ❺ Deserialize the chat messages from context
- ❻ Returns the context as `AIContext` object

This file-based provider is minimalist: on each agent invocation it loads the read-only context files from disk and injects its content into the Context for that turn. This keeps Robby's context grounded by ensuring the model reads the driving guidelines every time the agent runs, without hard-coding them into the agent's base instructions. However, it is not optimal from a performance perspective, since it rereads the files on every invocation and does not cache or personalize the context per session. It is still a useful example for understanding how `AIContextProvider` hooks into the agent lifecycle and can add external context just before the model call. In our example we chose to populate both `Instructions` and `Messages` properties for `AIContext`, and leave `Tools` aside, but we can decide to populate each of them.

In listing 7.4 is the full code that uses the `AIPProvider`.

Requirements (NuGet packages):

```

dotnet add package Microsoft.Agents.AI.OpenAI
dotnet add package Microsoft.Extensions.Configuration.UserSecrets

```

Code path: `/AgentWithCustomFileBasedContextProvider/Program.cs`

#### Listing 7.4 Agent with custom file-based `AIContextProvider`

```

using Providers;
using Helpers;
using Microsoft.Agents.AI;
using Microsoft.Extensions.AI;
using Microsoft.Extensions.Configuration;
using OpenAI;
using OpenAI.Chat;

var configuration = new
 ConfigurationBuilder().AddUserSecrets<Program>().Build();
var model = configuration["OpenAI:ModelId"];
var apiKey = configuration["OpenAI:ApiKey"];

var providerInstructionsFilePath = @"Data\robot-car-guidelines.txt";
var providerChatHistoryFilePath = @"Data\robot-car-chat-history.json";

AIContextProvider fileBasedContextProvider = new

```



```

CustomFileBasedContextProvider(providerInstructionsFilePath,
providerChatHistoryFilePath); //❶

ChatClientAgent agent = new OpenAIClient(apiKey)
 .GetChatClient(model)
 .AsAIAgent(new ChatClientAgentOptions {
 Name = "RobotCarAgent",
 Description = "An agent that assists a robot with the basic moves.",
 ChatOptions = new ChatOptions
 {
 Instructions = """
 You are an AI assistant controlling a robot car capable of performing
 basic moves: forward, backward, turn left, turn right, and stop.
 You have to break down the provided complex commands into the basic
 moves you know.
 Respond only with the moves and their parameters (angle or distance),
 without any additional explanations.
 """ },
 AIContextProviders = [fileBasedContextProvider] //❷
 });

AgentSession session = await agent.CreateSessionAsync();

var prompt = """
 Complex command:
 "There is a tree directly in front of the car.
 Avoid it and then return to the original path."
 """;
Console.WriteLine($"USER: {prompt}");
AgentResponse response = await agent
 .RunAsync(prompt, session); //❸
Console.WriteLine($"ASSISTANT: {response.Text}");

var followUpPrompt = "What was your second last basic move?";
Console.WriteLine($"USER: {followUpPrompt}");
AgentResponse followUpResponse = await agent
 .RunAsync(followUpPrompt, session); //❹
Console.WriteLine($"ASSISTANT: {followUpResponse.Text}");

var anotherFollowUpPrompt = "What is the wind speed later afternoon?";
Console.WriteLine($"USER: {anotherFollowUpPrompt}");
AgentResponse anotherFollowUpResponse = await agent
 .RunAsync(anotherFollowUpPrompt, session); //❺
Console.WriteLine($"ASSISTANT: {anotherFollowUpResponse.Text}");

```

- ❶ Creates the file-based context provider
- ❷ Configures the agent to use the file-based context provider
- ❸ Calls agent equipped with provider with initial prompt
- ❹ Calls agent with the follow-up prompt using same session
- ❺ Calls agent with another follow-up prompt using same session

Code output:

```

USER: Complex command:
"There is a tree directly in front of the car. Avoid it and then return to the
original path."
ASSISTANT: Turn right 60 degrees
Forward 10 meters
Turn left 60 degrees
Forward 10 meters
Turn left 60 degrees
Forward 10 meters
Turn right 60 degrees
USER: What was your second last basic move?
ASSISTANT: Forward 10 meters
USER: What is the wind speed later afternoon?
ASSISTANT: The wind speed in the afternoon of June 2, 2025, is 14 km/h.

```

We can observe that the turns are 60 degrees, that's showing that the agent considered the additional context read from the external file, and this proves that the instructions with the supplemental rules provided are considered when generating the LLM response.

Let's prove that altering the supplemental context. Replace the line *"The robot can turn left or right by angles of 30, 45, and 60 degrees"*. with the line *"The robot can turn left or right by angles of 90 degrees."* in the `robot-car-guidelines.txt` file and run again.

The result will show something like this:

```
USER: Complex command:
"There is a tree directly in front of the car. Avoid it and then return to the
original path."
ASSISTANT: Turn right 90
Forward 5
Turn left 90
Forward 5
Turn left 90
Forward 5
Turn right 90
USER: What was your second last basic move?
ASSISTANT: Forward 5
USER: What is the wind speed later afternoon?
ASSISTANT: The wind speed in the afternoon of June 2, 2025, is 14 km/h.
```

The turns are now in 90 degrees, remember that previously the agent responded with turns in 30, 45, and 60 degrees.

Another aspect is the supplemental chat history provided from the JSON file. It's easy to notice that the agent responds correctly when asked about the current wind speed. It knows the date because the provided chat history states that today is *June 2, 2025*.

We discussed that this solution is not optimal, since it repeatedly reads the same context and injects it into every turn (*### General Guidelines* context), so the same text appears redundantly across many messages in the session. A more efficient approach is to retrieve only the relevant parts of the context instead of the whole file, based on semantic similarity between the current prompt and precomputed context *chunks*. A chunk is a small, self-contained fragment of a larger document, such as a few sentences or a short paragraph, which can be retrieved independently and injected into the prompt when it is semantically relevant.

## EXERCISE

In listing 7.3 our example demonstrates the work with read-only context, but for scenarios where we want the context to be updated with more data, let's implement (override) the `StoreAIContextAsync` method:

```
protected override async ValueTask StoreAIContextAsync(
 InvokedContext context,
 CancellationToken cancellationToken = default)
{
 var instructions = await File
 .ReadAllTextAsync(instructionsFilePath, cancellationToken);

 string moreInstructions = ""
 June 6, 2025 - Morning: 13°C, partly cloudy, wind 9 km/h, dry.
 Afternoon: 21°C, sunny, wind 11 km/h, dry roads.
 Night: 12°C, clear, wind 6 km/h, calm.
 June 7, 2025 - Morning: 14°C, sunny, wind 7 km/h, dry.
 Afternoon: 23°C, mostly sunny, wind 10 km/h, no rain.
 Night: 13°C, few clouds, wind 5 km/h, calm.
 """; //❶

 var updatedInstructions = instructions + "\n" + moreInstructions;
```

```

 Console.WriteLine("[StoreAIContextAsync] Simulated file update "
 + "(not written to disk yet, because we don't want to alter "
 + "the original context file:");
 Console.WriteLine(updatedInstructions); //❷
}

```

❶ Creates more context

❷ Shows the added context, but does not update

This method may update the context with more data if we decide to append to the original file, but in our example let's just print to the console and leave the original file untouched.

Add the following logic at the end of previous listing 7.4 and run the code again.

```

Console.WriteLine("HISTORY:");

if (session.StateBag
 .TryGetValue<InMemoryChatHistoryProvider.State>(
 nameof(InMemoryChatHistoryProvider), out var state)) //❶
{
 foreach (ChatMessage message in state!.Messages) //❷
 {
 var source = message.GetAgentRequestMessageSourceType();
 Console.WriteLine($"[{source.Value}] {message.Role}: ");
 Console.WriteLine($" {message.Text}");
 }
}

```

❶ Fetches the chat messages from StateBag

❷ Iterates through chat messages

The printed context should show the updated instructions that includes two more days in weather report section in the context (details truncated for first days).

```

Weather Report
June 1, 2025 ...
June 2, 2025 ...
June 3, 2025 ...
June 4, 2025 ...
June 5, 2025 ...

June 6, 2025 - Morning: 13°C, partly cloudy, wind 9 km/h, dry.
 Afternoon: 21°C, sunny, wind 11 km/h, dry roads.
 Night: 12°C, clear, wind 6 km/h, calm.
June 7, 2025 - Morning: 14°C, sunny, wind 7 km/h, dry.
 Afternoon: 23°C, mostly sunny, wind 10 km/h, no rain.
 Night: 13°C, few clouds, wind 5 km/h, calm.

```

You should notice a listing like this at the end of the printed output:

```

HISTORY:
[External] user: Complex command:
"There is a tree directly in front of the car. Avoid it and then return to the
original path."
[ChatHistory] user: What day is today?
[ChatHistory] assistant: Today is June 2, 2025
[External] assistant: Turn right 60 degrees
Forward 5 meters
Turn left 60 degrees
Forward 5 meters
Turn left 60 degrees
Forward 5 meters
Turn right 60 degrees
[External] user: What was your second last basic move?
[ChatHistory] user: What day is today?
[ChatHistory] assistant: Today is June 2, 2025
[External] assistant: Forward 5 meters
[External] user: What is the wind speed later afternoon?
[AIContextProvider] user: What day is today?
[AIContextProvider] assistant: Today is June 2, 2025
[External] assistant: The wind speed is expected to be 14 km/h.

```

In our example we have three types of `AIContext`: `Instructions`, `Messages`, and `Tools`. Only the first two are populated in our case, but in more advanced use-cases we can populate `Tools`, as well.

For better capturing the source of each message in the printed output to console, while the chat messages from user interaction with the AI model are prefixed with `[External]`, the AI context loaded from the physical file is prefixed with `[AIContextProvider]` and the chat history from the physical file is prefixed with `[ChatHistory]`.

### 7.2.3 Chat History Memory Provider using In-Memory Vector Store

The in-memory vector store provider takes Robby's context a step further. Instead of loading the entire weather report or rulebook into every call, it embeds the content once, stores the embeddings in an in-memory vector store, and retrieves only the most relevant chunks for each prompt. This moves us from "dump everything into the prompt" to "retrieve just enough context by similarity".

RAG-style context is powerful, but it is not always the right tool. For aggregations or simple lookups over a small, well-structured dataset, retrieving chunks by embeddings can be overkill and sometimes less predictable than direct queries. When we need to compute something like *"average speed over the last 10 trips"* or *"total distance driven today"*, a structured store or even a simple keyword or key-based lookup often works better than semantic retrieval.

#### PRACTICAL EXAMPLE

In the listing 7.5, we implement this pattern as a context provider backed by an in-memory vector store. We first ingest a weather report through a `WeatherAgent`, then reuse the same provider from `MotorsAgent` so it can answer weather questions without ever seeing the raw report directly.

Requirements (NuGet packages):

```
dotnet add package Microsoft.Agents.AI.OpenAI
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
dotnet add package CommunityToolkit.VectorData.InMemory
```

Code path: `/AgentWithInMemoryVectorStoreProvider/Program.cs`

#### Listing 7.5 Agent with in-memory vector store `AIContextProvider`

```
// 'usings' omitted for brevity

var configuration = new ConfigurationBuilder()
 .AddUserSecrets<Program>().Build();
var model = configuration["OpenAI:ModelId"];
var apiKey = configuration["OpenAI:ApiKey"];
var embeddingModel = configuration["OpenAI:EmbeddingModelId"];

ChatClient chatClient = new OpenAIClient(apiKey)
 .GetChatClient(model); // ❶

IEmbeddingGenerator<string, Embedding<float>> embeddingGenerator =
 new OpenAIClient(apiKey)
 .GetEmbeddingClient(embeddingModel)
 .AsIEmbeddingGenerator(); // ❷

VectorStore vectorStore = new InMemoryVectorStore(
 new InMemoryVectorStoreOptions
 {
 EmbeddingGenerator = embeddingGenerator
 }); // ❸

ChatHistoryMemoryProvider chatHistoryMemoryProvider = new(
 vectorStore, // ❹
 collectionName: "RobotCarKnowledge",
 vectorDimensions: 1536,
```

```

session => new ChatHistoryMemoryProvider.State(
 storageScope: new() { UserId = "User_1",
 SessionId = Guid.NewGuid().ToString() }, //5
 searchScope: new() { UserId = "User_1" }) //6
); //7

ChatClientAgent weatherAgent = chatClient
 .AsIAgent(new ChatClientAgentOptions
 {
 Name = "WeatherAgent",
 Description = "An agent that feeds weather information.",
 ChatOptions = new ChatOptions
 {
 Instructions = """
 You are an AI assistant collecting weather information.
 Respond with echo of collected information.
 """
 },
 AIContextProviders = [chatHistoryMemoryProvider], //8
 }); //9

Microsoft.Extensions.AI.ChatMessage weatherReportMessage =
 new(ChatRole.User, """
 ## Weather Report
 June 1, 2025 - Morning: 14°C, partly cloudy, wind 8 km/h, dry.
 Afternoon: 20°C, mostly sunny, wind 12 km/h, no rain.
 Night: 13°C, clear, wind 6 km/h, calm.",
 June 2, 2025 - Morning: 15°C, sunny, wind 10 km/h, dry.
 Afternoon: 22°C, mostly sunny, wind 14 km/h, dry roads.
 Night: 14°C, few clouds, wind 8 km/h, no precipitation.",
 // weather report for June 3,4,5 omitted for brevity
 """)
{
 MessageId = Guid.NewGuid().ToString(),
 AuthorName = "System"
}; //10

AgentSession session = await weatherAgent.CreateSessionAsync();

Console.WriteLine($"USER: {weatherReportMessage}");
AgentResponse weatherResponse = await
 weatherAgent.RunAsync(weatherReportMessage, session); //11
Console.WriteLine($"ASSISTANT: {weatherResponse.Text}");

ChatClientAgent motorsAgent = chatClient.AsIAgent(
 new ChatClientAgentOptions
 {
 Name = "MotorsAgent",
 Description = "An agent that assists a robot with the basic moves.",
 ChatOptions = new ChatOptions
 {
 Instructions = """
 You are an AI assistant controlling a robot car capable of performing
 basic moves: forward, backward, turn left, turn right, and stop.
 You have to break down the provided complex commands into the basic
 moves you know.
 Respond only with the moves and their parameters (angle or distance),
 without any additional explanations.
 """ },
 AIContextProviders = [chatHistoryMemoryProvider], //12
 });

var prompt = $"""
 Complex command:

```

```

 "There is a tree directly in front of the car.
 Avoid it and then return to the original path."
 """
Console.WriteLine($"USER: {prompt}");
AgentResponse response = await motorsAgent
 .RunAsync(prompt, session); //13
Console.WriteLine($"ASSISTANT: {response.Text}");

var followUpPrompt = "What was your second last basic move?";
Console.WriteLine($"USER: {followUpPrompt}");
AgentResponse followUpResponse = await motorsAgent
 .RunAsync(followUpPrompt, session); //14
Console.WriteLine($"ASSISTANT: {followUpResponse.Text}");

var anotherFollowUpPrompt = ""
 Today is 2nd of June, 2PM. What is the temperature?
 ""
Console.WriteLine($"USER: {anotherFollowUpPrompt}");
AgentResponse anotherFollowUpResponse = await motorsAgent
 .RunAsync(anotherFollowUpPrompt, session); //15
Console.WriteLine($"ASSISTANT: {anotherFollowUpResponse.Text}");

```

- ❶ Initializes the chat client
- ❷ Initializes the embedding generator client
- ❸ Initializes a vector store with the embedding generator
- ❹ Initializes the memory provider with the vector store
- ❺ Sets the storage scope to current user id and session id
- ❻ Sets the search scope to current user id with any session
- ❼ Initializes the chat history memory provider
- ❽ Assigns the provider to weather agent
- ❾ Creates the weather agent
- ❿ Wraps up the context in a chat message
- ⓫ Calls weather agent to populate context
- ⓬ Assigns the provider to motors agent
- ⓭ Calls the agent with the initial prompt
- ⓮ Calls the agent with the follow-up prompt, same session
- ⓯ Calls the agent with another follow-up prompt, same session

Code output (the weather report ingestion from the beginning of this output is truncated):

```

USER: Complex command:
"There is a tree directly in front of the car. Avoid it and then return to the
original path."
ASSISTANT: Turn right, 90 degrees
Forward, 10 units
Turn left, 90 degrees
Forward, 10 units
Turn left, 90 degrees
Forward, 10 units
Turn right, 90 degrees
USER: What was your second last basic move?
ASSISTANT: Forward, 10 units
USER: Today is 2nd of June, 2PM. What is the temperature?
ASSISTANT: 22°C

```

Let's notice that both `WeatherAgent` and `MotorsAgent` share the same `session` object and the same chat history memory provider. Consequently, `MotorsAgent` can also receive the original weather dialogue through ordinary history; the demonstration does not isolate semantic retrieval or prove reduced request size. `ChatHistoryMemoryProvider` stores message text, not an automatically chunked document ingestion pipeline.

With the vector store adapter in place, Robby no longer receives the full guidelines and weather report on every call. Instead, the context provider selects only the most relevant chunks based on the prompt embedding, then formats them as additional context for the model. This reduces prompt

bloat, scales better as the knowledge base grows, and gives us a pattern we can reuse with any vector-backed store, not just the in-memory example shown here.

In the `ChatHistoryMemoryProvider`, `StorageScope` tags messages when storing them with metadata (`ApplicationId`, `AgentId`, `UserId`, `SessionId`), and search scope filters which messages are retrieved during searches. Let's analyze the scope configuration from our example:

```
session => new ChatHistoryMemoryProvider.State(
 storageScope: new() { UserId = "User_1",
 SessionId = Guid.NewGuid().ToString() }, //❶
 searchScope: new() { UserId = "User_1" }) //❷
```

❶ Sets storage scope to User\_1 and GUID

❷ Sets search scope to User\_1

The `storageScope` tags each message with both `UserId` (`"User_1"`) and a unique `SessionId`, indicating that the stored data belongs to a specific user in a specific session.

The `searchScope`, however, only specifies `UserId` (`"User_1"`) while the `SessionId` remains unset. Therefore, when searching, the provider retrieves messages from all sessions for `"User_1"`, not just the current session.

This pattern enables cross-session memory: the agent stores each conversation separately (with unique session ids) but can recall information from any previous conversation with the same user.

You can combine all four properties (`ApplicationId`, `AgentId`, `UserId`, `SessionId`) in creative ways across both scopes to control memory ingestion and retrieval. The key principle is: `storageScope` determines what metadata is saved, while `searchScope` determines what filters are applied during retrieval.

#### EXERCISE

At the end of listing 7.5 create another agent that will audit the first agent actions:

```
ChatClientAgent auditorAgent = chatClient.AsAIAgent(new ChatClientAgentOptions
{
 Name = "RobotCarAgent",
 Description = "An auditor agent that observes the robot car moves.",
 ChatOptions = new ChatOptions
 {
 Instructions = ""
 You are an AI auditor observing the robot car moves.
 You resume explaining briefly the robot car moves.
 "" },
 AIContextProviders = [chatHistoryMemoryProvider],
});
```

Then ask the new agent to prompt the AI model with the same session:

```
AgentResponse auditorResponse = await auditorAgent.RunAsync(
 "Audit the actions performed by the agents.", session);
```

And finally, add the print.

```
Console.WriteLine($"AUDITOR: {auditorResponse.Text}");
```

❶ Fetches the state from `StateBag`

❷ Iterates through chat messages

Inside the output you will notice the additional context like this (fragment):

```
AUDITOR: Audit summary:
```

```
- Complex command received: avoid a tree in front, then return to original path.
- MotorsAgent executed:
 1. turn right 90
 2. forward 2
 3. turn left 90
 4. forward 4
 5. turn left 90
 6. forward 2
 7. turn right 90
 8. stop
```

```
...
Conclusion:
- Obstacle avoidance execution: correct.
- Later query handling: flawed due to incorrect answer selection and likely
cross-agent confusion.
```

By the end of the code output, you should notice that the `auditorAgent` response is perfectly aware of the context.

### 7.2.4 Text Search Provider using Keyword Search Adapter

Not every project has embeddings or a vector store available. Sometimes we want a lightweight, transparent way to search a small knowledge base by keywords. The text search provider with a keyword search adapter gives us exactly that: a context provider that ranks documents using simple keyword overlap and injects the best matches into `AIContext`.

This is also why a keyword-based adapter can be a better fit for some scenarios. If Robby only needs a handful of short policy snippets or weather entries, we do not gain much from a full RAG setup. A curated keyword index keeps behavior transparent, easy to debug, and cheap to run, while still giving us targeted context for small datasets.

#### PRACTICAL EXAMPLE

In this example we define a small `SearchItem` model (listing 7.6) and a `KeywordSearchAdapter` (listing 7.7) that ranks entries by keyword overlap with the user prompt. The adapter acts as the backend for `TextSearchProvider`, which injects the top matches into `AIContext`.

Code path: `/AgentWithTextSearchProvider/SearchItem.cs`

#### Listing 7.6 SearchItem class

```
namespace Adapters;

public record SearchItem
{
 public required string Key { get; init; } //❶
 public required string Text { get; init; } //❷
 public required string SourceName { get; init; } //❸
 public required string[] Keywords { get; init; } //❹
}
```

❶ Declares the unique key for identification

❷ Declares the text content (context item)

❸ Declares the source name of text content (category, catalog)

❹ Declares the keywords that identifies the text content

Listing 7.7 implements `KeywordSearchAdapter` that handles the keyword-based search, and it acts as the text search backend for `TextSearchProvider`. In fact, it populates an in-memory knowledge base and ranks results by keyword matches in the provided split tags. The tags are obtained by splitting the provided prompt by normal separator such as space, comma, and period.

Code path: `/AgentWithTextSearchProvider/CustomKeywordSearchAdapter.cs`

#### Listing 7.7 Custom keyword search adapter

```
using Microsoft.Agents.AI;

namespace Adapters;

public static class CustomKeywordSearchAdapter
{
 private static List<SearchItem>? _knowledgeBase;
 private static TextSearchProviderOptions? _searchOptions;
 private static int _topResults;
```



```

public static void Initialize(TextSearchProviderOptions searchOptions,
 int topResults = 5) //❶
{
 _searchOptions = searchOptions;
 _topResults = topResults;

 _knowledgeBase = [
 new() {
 Key = "1",
 Text = ""
 The robot car can move backward by a specified distance.
 Reverse motion is limited to 5 meters.
 "",
 SourceName = "RobotCar Movement Policy",
 Keywords = ["command", "move", "distance"] },
 new() {
 Key = "2",
 Text = ""
 The robot can turn left or right by angles of 30, 45, and 60 degrees.
 "",
 SourceName = "RobotCar Turning Policy",
 Keywords = ["command", "move", "turn", "angle"] },
 new() {
 Key = "3",
 Text = ""
 The robot car can move forward by a specified distance.
 Maximum range is 10 meters.
 "",
 SourceName = "RobotCar Movement Policy",
 Keywords = ["command", "move", "distance"] },
 new() {
 Key = "4",
 Text = "Emergency stop immediately halts all motion.",
 SourceName = "RobotCar Safety Manual",
 Keywords = ["command", "move", "distance", "emergency"] },
 new() {
 Key = "5",
 Text = ""
 June 1, 2025 - Morning: 14°C, partly cloudy, wind 8 km/h, dry.
 Afternoon: 20°C, mostly sunny, wind 12 km/h, no rain.
 Night: 13°C, clear, wind 6 km/h, calm.
 "",
 SourceName = "Weather Forecast",
 Keywords = ["weather", "temperature", "wind"] },
 new() {
 Key = "6",
 Text = ""
 June 2, 2025 - Morning: 15°C, sunny, wind 10 km/h, dry.
 Afternoon: 22°C, mostly sunny, wind 14 km/h, dry roads.
 Night: 14°C, few clouds, wind 8 km/h, no precipitation.
 "",
 SourceName = "Weather Forecast",
 Keywords = ["weather", "temperature", "wind"] },
 // weather report for June 3,4,5 omitted for brevity
];
}

public static async Task<IEnumerable<TextSearchProvider.TextSearchResult>>
 KeywordSearch(string prompt, CancellationToken cancellation) //❷
{
 if (_knowledgeBase == null || _searchOptions == null)
 {
 throw new InvalidOperationException("KeywordSearchAdapter has not "
 + "been initialized. Call Initialize first.");
 }
}

```

```

 }

 var keywords = prompt.ToLowerInvariant()
 .Split([' ', ',', '.', ':', ';'],
 StringSplitOptions.RemoveEmptyEntries); //❸

 var results = _knowledgeBase
 .Select(item => new
 {
 Item = item,
 MatchCount = keywords.Count(keyword =>
 item.Keywords.Any(k => k.Contains(keyword,
 StringComparison.OrdinalIgnoreCase)))
 })
 .Where(x => x.MatchCount > 0)
 .OrderByDescending(x => x.MatchCount)
 .Take(_topResults)
 .Select(x => new TextSearchProvider.TextSearchResult
 {
 Text = x.Item.Text,
 SourceName = x.Item.SourceName,
 SourceLink = string.Join(", ", x.Item.Keywords)
 }); //❹

 return results;
}
}

```

❶ Defines the initialization method that populates knowledge

❷ Defines the search method with prompt argument

❸ Splits prompt into words

❹ Counts matches and order by relevance

Listing 7.8 wires `TextSearchProvider` with `KeywordSearchAdapter` into the agent. It initializes the keyword knowledge base and configures search timing before AI invocation.

Requirements (NuGet packages):

```

dotnet add package Microsoft.Agents.AI.OpenAI
dotnet add package Microsoft.Extensions.Configuration.UserSecrets

```

Code path: `/AgentWithTextSearchProvider/Program.cs`

### Listing 7.8 Agent with text search provider

```

// 'usings' and API key fetching omitted for brevity

TextSearchProviderOptions textSearchOptions = new()
{
 SearchTime = TextSearchProviderOptions.TextSearchBehavior.BeforeAIInvoke,
 RecentMessageMemoryLimit = 6
}; //❶

CustomKeywordSearchAdapter.Initialize(textSearchOptions, topResults: 5); //❷

AIContextProvider keywordSearchProvider = new TextSearchProvider(
 CustomKeywordSearchAdapter.KeywordSearch, textSearchOptions); //❸

ChatClientAgent agent = new OpenAIClient(apiKey)
 .GetChatClient(model)
 .AsAIAgent (new ChatClientAgentOptions
 {
 Name = "RobotCarAgent",
 Description = "An agent that assists a robot with the basic moves.",
 ChatOptions = new ChatOptions
 {
 Instructions = ""
 You are an AI assistant controlling a robot car capable of performing

```

```

 basic moves: forward, backward, turn left, turn right, and stop.
 You have to break down the provided complex commands into the basic
 moves you know.
 Respond only with the moves and their parameters (angle or distance),
 without any additional explanations.
 """ },
 AIContextProviders = [keywordSearchProvider] //4
 }); //5

 AgentSession session = await agent.CreateSessionAsync(); //6

 var prompt = $""
 Complex command:
 "There is a tree directly in front of the car.
 Avoid it and then return to the original path."
 """; //7
 Console.WriteLine($"USER: {prompt}");
 AgentResponse response = await agent
 .RunAsync(prompt, session); //8
 Console.WriteLine($"ASSISTANT: {response.Text}");

 var followUpPrompt = "What was your second last basic move?"; //9
 Console.WriteLine($"USER: {followUpPrompt}");
 AgentResponse followUpResponse = await agent
 .RunAsync(followUpPrompt, session); //10
 Console.WriteLine($"ASSISTANT: {followUpResponse.Text}");

```

- ❶ Defines the text search provider options
- ❷ Initializes the keyword-based search provider with options
- ❸ Declares the Text Search Provider using KeywordSearch method
- ❹ Assigns the Text Search Provider to AI context
- ❺ Initializes the agent with provided Text Search Provider
- ❻ Initializes a session
- ❼ Declares a prompt
- ❽ Invokes the agent with the prompt in current session
- ❾ Declares a follow-up prompt
- ❿ Invokes the agent with the follow-up prompt in current session

Code output:

```

USER: Complex command:
"There is a tree directly in front of the car. Avoid it and then return to the
original path."
ASSISTANT: stop [RobotCar Safety Manual](command, move, distance, emergency)
backward 1 meter [RobotCar Movement Policy](command, move, distance)
turn right 45 degrees [RobotCar Turning Policy](command, move, turn, angle)
forward 2 meters [RobotCar Movement Policy](command, move, distance)
turn left 45 degrees [RobotCar Turning Policy](command, move, turn, angle)
forward 3 meters [RobotCar Movement Policy](command, move, distance)
turn left 45 degrees [RobotCar Turning Policy](command, move, turn, angle)
forward 2 meters [RobotCar Movement Policy](command, move, distance)
turn right 45 degrees [RobotCar Turning Policy](command, move, turn, angle)
USER: What was your second last basic move?
ASSISTANT: forward 2 meters [RobotCar Movement Policy](command, move, distance)

```

We can notice that turns are executed using 45 degrees (because this piece of context: *The robot can turn left or right by angles of 30, 45, and 60 degrees*). Without it, the turns are usually executed using 90 degrees.

With the keyword search adapter, Robby still gets focused context instead of the entire ruleset, but relevance is computed using keyword occurrences instead of embeddings. This approach is easier to reason about and can run without an embedding model, which makes it useful for smaller setups or environments with tighter constraints. The tradeoff is that it depends on good keyword curation and typically performs best when the domain vocabulary is stable and predictable.

**EXERCISE**

Add the following logic at the end of previous listing 7.8 and run the code again.

```
Console.WriteLine("HISTORY:");
if (session.StateBag
 .TryGetValue<InMemoryChatHistoryProvider.State>(
 nameof(InMemoryChatHistoryProvider), out var state)) //❶
{
 foreach (ChatMessage message in state!.Messages) //❷
 {
 var source = message.GetAgentRequestMessageSourceType();
 Console.WriteLine($"[{source.Value}] {message.Role}: ");
 Console.WriteLine($" {message.Text}");
 }
}
```

❶ Fetched the state from StateBag

❷ Iterates through chat messages

Inside the output you will notice the additional context like this (fragment):

```
[External] user:
Complex command:
"There is a tree directly in front of the car. Avoid it and then return to the
original path."
[ChatHistory] user:
Additional Context
Consider the following information from source documents when responding to the
user:
SourceDocName: RobotCar Movement Policy
SourceDocLink: command, move, distance
Contents: The robot car can move backward by a specified distance. Reverse
motion is limited to 5 meters.

SourceDocName: RobotCar Movement Policy
SourceDocLink: command, move, distance
Contents: The robot car can move forward by a specified distance. Maximum range
is 10 meters.
...
```

And the last message:

```
[External] assistant:
forward 2 meters [RobotCar Movement Policy](command, move, distance)
```

### 7.3 Introduction to Chat History

Imagine Robby driving through a twisty maze. If he forgets where he was just a few commands ago, he'll loop endlessly and never reach the exit. To escape, Robby needs a reliable memory of every turn, every instruction *"Go left"*, and every action taken. Without that trail of breadcrumbs, he's stuck in the present, unable to reason his way out.

Different environments manage this history differently:

- Local/development setups often keep history **in-memory** or **local storage**, giving us full control over token budget, truncation, and persistence.
- Production and cloud services like OpenAI Responses or Azure AI Agents store history **in-service** and manage token limits, truncation (if enabled), and pagination automatically. We only pass a `ConversationId` and let the service handle the rest.

Since chapter 5 covered in-service conversation storage, we will focus following on in-memory and local storage types.

Figure 7.4 shows a chat client agent configured with a `ChatHistoryProvider`. The left side shows standard agent elements: LLM calls, context, chat history, roles, instructions, and tools that access external data like RAG or profiles. The right side highlights the agent session, which uses a `ChatHistoryProvider` to load and manage conversation state. This provider ensures that the

right slice of recent messages is exposed to each LLM call, regardless of where messages are actually stored.

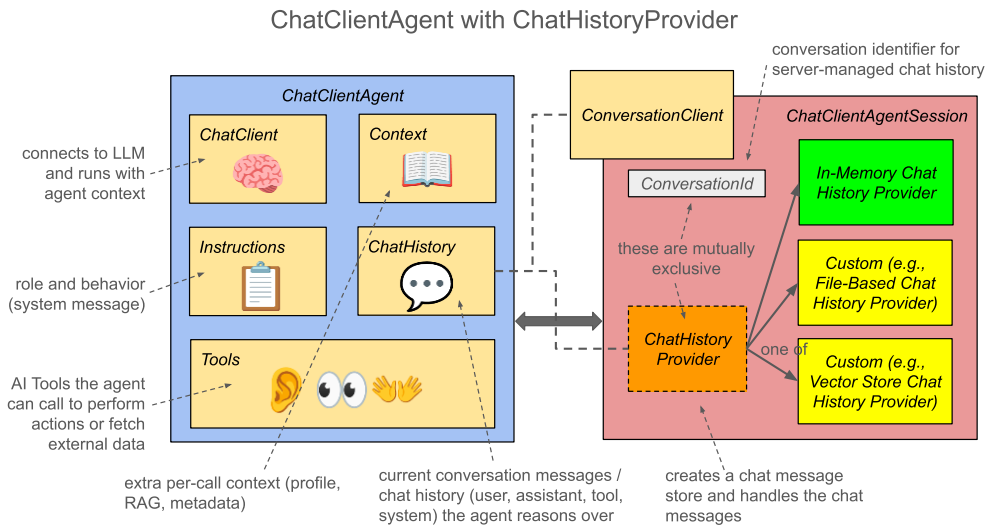


Figure 7.4 The diagram highlights how agent interactions with the LLM (short-term memory) is maintained as conversation sessions and persisted chat messages, which provide the recent interaction window used as context for each model call.

This abstraction delivers the key win: switch between models without restructuring conversation logic. Start with in-memory for development, move to custom stores for compliance needs, or hand off to services for scale. The session interface and message formats remain stable throughout.

The `ChatClientAgentSession` abstracts the work with Chat History, so the same agent logic can run locally (in-memory) or in the cloud without changing how we handle messages. Start with in-memory for development, move to custom stores for compliance needs, or hand off to services for scale. The session interface and message formats remain stable throughout.

### 7.3.1 What is ChatHistoryProvider

The `ChatHistoryProvider` is responsible for loading and persisting the conversation messages that back an agent session. It turns a stream of turns into a coherent, loadable history that the agent can reuse across runs.

Each session gets its own `ChatHistoryProvider` instance, so histories stay isolated per conversation. When a new session is created or an existing one is reconnected, the provider can load the stored messages for that session (or start from an empty history) and make them available for the next run.

The provider participates in the same two-phase lifecycle as `AIContextProviders` when we call `RunAsync(inputMessages, session)`:

- `ProvideChatHistoryAsync` (before the model call)

The provider loads history messages for the current session from its backing store (memory, file, database, or cloud service). It can also apply its own policies: truncating or

summarizing older turns, enforcing a recent-window strategy, or filtering out messages that are no longer relevant. The agent then combines:

- chat history messages from session in the current `AgentSession`
- history messages from the `ChatHistoryProvider`
- input messages from the current `RunAsync` call

into merged (aggregated) messages, which is the list of messages the agent sends to the chat client for this run.

- `StoreChatHistoryAsync` (after the model call)

After the chat client returns a response, the agent calls `StoreChatHistoryAsync(InvokedContext)`. The provider reads the effective request and response from this context, appends the new user and assistant messages to its stored history, and persists the updated conversation (for example, to a database or cloud session). It can also update metadata such as last-active timestamps, message counts, or conversation labels.

This pattern keeps storage concerns out of the agent's decision-making loop. The agent only works with messages while the history provider decides how messages are stored, truncated, summarized, and restored for each session.

The figure 7.5 shows in a sequence diagram how a `ChatClientAgent` uses a `ChatHistoryProvider` to give the AI model access to previous turns before each run.

- `inputMessages`: the messages passed into `RunAsync` for this run.
- `sessionMessages`: messages implied by the current `AgentSession`, including any state the backing service tracks for the conversation and exposes as messages.
- `historyMessages`: prior conversation messages returned by `ProvideChatHistoryAsync` from the provider's storage or memory implementation.
- `mergedMessages`: the combined list of `sessionMessages`, `historyMessages`, and `inputMessages` that the agent sends to the chat client as the request for this run.

The user calls `RunAsync(inputMessages, session)`. The agent calls `ProvideChatHistoryAsync` on the `ChatHistoryProvider` and receives `historyMessages`. It then constructs `mergedMessages` from `sessionMessages`, `historyMessages`, and `inputMessages`, and sends that list to the chat client using `GetResponseAsync`. After it gets a `ChatResponse`, the agent calls `StoreChatHistoryAsync(InvokedContext)`, and the provider extracts the relevant request and response messages from the context and persists them as the latest turn for this session. The agent then returns the response to the user.

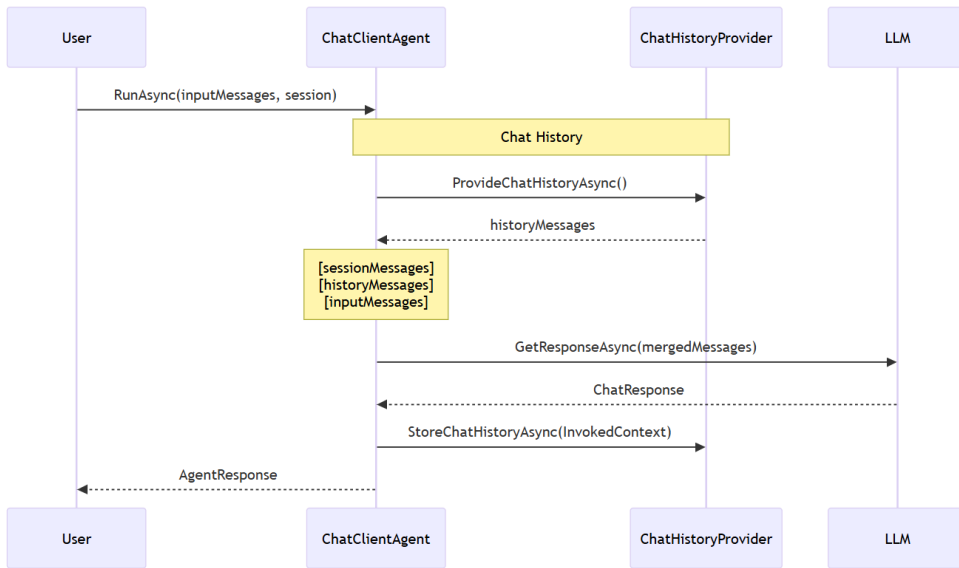


Figure 7.5 Sequence diagram shows ChatClientAgent building mergedMessages from session state, history, and the current input when using a ChatHistoryProvider.

Key takeaways and how we use it in practice:

- We control exactly which past turns the model sees by shaping `historyMessages` in the provider.
- We can plug in any backing store (SQL, Cosmos DB, Redis, system file) without changing the agent code.
- We can implement pruning, summarization, or per-user retention policies entirely inside the history provider.

### 7.3.2 In-Memory ChatHistoryProvider

The in-memory `ChatHistoryProvider` is Robby's short-term memory. It keeps all messages in process, so the agent can answer follow-up questions within the same run of the application, but nothing survives a restart. For local development and testing, this is often exactly what we want: fast, zero-setup history that lets us see how the agent behaves across multiple turns.

Because everything lives in memory, this provider is also a safe place to experiment with reducers like `MessageCountingChatReducer`. We can simulate different retention strategies (last N messages, summarized history, etc.) without touching any external infrastructure.

#### PRACTICAL EXAMPLE

`InMemoryChatHistoryProvider` is like Robby's short-term memory. It's great for a mission, but if he powers down, the memory is lost.

Listing 7.9 shows the in-memory `ChatHistoryProvider` in action.

Requirements (NuGet packages):

```
dotnet add package Microsoft.Agents.AI.OpenAI
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
```

Code path: `/AgentWithInMemoryChatHistoryProvider/Program.cs`

#### Listing 7.9 Agent with in-memory ChatHistoryProvider

```
// 'usings' and API key fetching omitted for brevity

InMemoryChatHistoryProvider inMemoryChatHistoryProvider =
 new(new InMemoryChatHistoryProviderOptions()); //❶

ChatClientAgent agent = new OpenAIClient(apiKey)
 .GetChatClient(model)
 .AsIAgent(new ChatClientAgentOptions
 {
 Name = "RobotCarAgent",
 Description = "An agent that assists a robot with the basic moves.",
 ChatOptions = new ChatOptions
 {
 Instructions = """
 You are an AI assistant controlling a robot car capable of performing
 basic moves: forward, backward, turn left, turn right, and stop.
 You have to break down the provided complex commands into the basic
 moves you know.
 Respond only with the moves and their parameters (angle or distance),
 without any additional explanations.
 """ },
 ChatHistoryProvider = inMemoryChatHistoryProvider //❷
 });

AgentSession session = await agent.CreateSessionAsync(); //❸

var prompt = """
 Complex command:
 "There is a tree directly in front of the car.
 Avoid it and then return to the original path."
 """;
Console.WriteLine($"USER: {prompt}");
AgentResponse response = await agent
 .RunAsync(prompt, session); //❹
Console.WriteLine($"ASSISTANT: {response.Text}");

var followUpPrompt = "What was your second last basic move?";
Console.WriteLine($"USER: {followUpPrompt}");
AgentResponse followUpResponse = await agent
 .RunAsync(followUpPrompt, session); //❺
Console.WriteLine($"ASSISTANT: {followUpResponse.Text}");
```

- ❶ Creates the in-memory ChatHistoryProvider
- ❷ Assigns the provider to agent
- ❸ Creates an agent session
- ❹ Calls the agent with initial prompt and session
- ❺ Calls the agent with follow-up prompt and session

Code output:

```
USER: Complex command:
"There is a tree directly in front of the car. Avoid it and then return to the
original path."
ASSISTANT: 1. Turn right 90 degrees
2. Forward 5 meters
3. Turn left 90 degrees
4. Forward 5 meters
5. Turn left 90 degrees
6. Forward 5 meters
7. Turn right 90 degrees
USER: What was your second last basic move?
ASSISTANT: 6. Forward 5 meters
```

The last agent response shows that it can remember the previous actions and report the second last move correctly.

Let's note that this approach works for development but loses history on restart, since it keeps the chat history in-memory.



**EXERCISE**

In the listing 7.9 (previous) replace the in-memory `ChatHistoryProvider` with this that has a chat reducer in it (please be aware that `ChatReducer` may still be experimental and has to be marked with pragma directive to disable the warning):

```
#pragma warning disable MEAT001 //❶
MessageCountingChatReducer chatReducer = new(targetCount: 3); //❷

InMemoryChatHistoryProvider inMemoryChatHistoryProvider = new(new
InMemoryChatHistoryProviderOptions { ChatReducer = chatReducer }); //❸
```

- ❶ Disables the 'experimental' warning for `ChatReducer`
- ❷ Declares a chat reducer that remembers last x messages only
- ❸ Initializes the in-memory `ChatHistoryProvider`

Locate the queries and their responses in the (previous) listing 7.9 and replace them with the following queries:

```
AgentResponse response1 = await agent
.RunAsync("go left 10 degrees", session);
AgentResponse response2 = await agent
.RunAsync("go back 20 meters", session);
AgentResponse response3 = await agent
.RunAsync("turn right 30 degrees", session);
AgentResponse response4 = await agent
.RunAsync("go forward 99 meters", session);
AgentResponse response5 = await agent
.RunAsync("what was the first (earliest) move you can remember?", session);
```

Then add the following logic at the end and run the code again.

```
Console.WriteLine("HISTORY:");

if (session.StateBag
.TryGetValue<InMemoryChatHistoryProvider.State>(
 nameof(InMemoryChatHistoryProvider), out var state)) //❶
{
 foreach (ChatMessage message in state!.Messages) //❷
 {
 var source = message.GetAgentRequestMessageSourceType();
 Console.WriteLine($"[{source.Value}] {message.Role}: ");
 Console.WriteLine($"{message.Text}");
 }
}
```

- ❶ Fetches the state from `StateBag`
- ❷ Iterates through chat messages

Inside the output you will notice that only the last three chat messages are preserved in the chat history when the last assistant message is generated, and along with it we have five printed messages. `ChatReducer` controls via `targetCount` argument how many messages are remembered.

### 7.3.3 File-Based `ChatHistoryProvider`

The file-based `ChatHistoryProvider` is Robby's logbook on disk. Instead of keeping messages in memory only, it serializes them to a JSON file and reloads them on the next run. This gives us durable history without standing up a database.

For small apps or demos, this can be enough: each conversation session maps to a file, and the provider handles reading, appending, and writing back. It also makes debugging easier, because we can open the file and see exactly what the agent has seen so far.

**PRACTICAL EXAMPLE**

In this example we use `FileBasedChatHistoryProvider` to persist conversations to JSON files. Even if Robby reboots, he can open the file and know exactly where he left off.

Listing 7.10 shows the core `FileBasedChatHistoryProvider` implementation. It reads JSON, appends new messages, and implements `IReadOnlyList<ChatMessage>` for easier chat messages fetching.

Requirements (NuGet packages):

```
dotnet add package Microsoft.Agents.AI.OpenAI
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
```

Code path: `/AgentWithFileBasedChatHistoryProvider`

`/CustomFileBasedChatHistoryProvider.cs`

#### Listing 7.10 Custom file-based ChatHistoryProvider

```
// 'usings' omitted for brevity
namespace Providers;

public class CustomFileBasedChatHistoryProvider(string filePath) :
 ChatHistoryProvider, IReadOnlyList<ChatMessage> //❶
{
 public List<ChatMessage> ChatMessages { get; private set; } = []; //❷
 public int Count => ChatMessages.Count; //❸
 public ChatMessage this[int index] => ChatMessages[index]; //❹

 protected override async ValueTask<IEnumerable<ChatMessage>>
 ProvideChatHistoryAsync(
 InvokingContext context,
 CancellationToken cancellationToken = default) //❺
 {
 if (!File.Exists(filePath)) return [];
 var json = await File.ReadAllTextAsync(filePath, cancellationToken);
 ChatMessages = JsonSerializer.Deserialize<List<ChatMessage>>(json)
 ?? [];
 return ChatMessages;
 }

 protected override async ValueTask StoreChatHistoryAsync(
 InvokedContext context,
 CancellationToken cancellationToken = default) //❻
 {
 if (File.Exists(filePath))
 {
 var json = await File
 .ReadAllTextAsync(filePath, cancellationToken); //❼
 ChatMessages = JsonSerializer.Deserialize<List<ChatMessage>>(json)
 ?? []; //❽
 }
 ChatMessages.AddRange(context
 .RequestMessages.Concat(context.ResponseMessages ?? [])); //❾

 var serialized = JsonSerializer.Serialize(ChatMessages,
 new JsonSerializerOptions { WriteIndented = true }); //❿
 await File
 .WriteAllTextAsync(filePath, serialized, cancellationToken); //⓫
 }

 public IEnumerator<ChatMessage> GetEnumerator() //⓫
 {
 return ChatMessages.GetEnumerator();
 }

 IEnumerator IEnumerable.GetEnumerator() //⓫
 {
 return GetEnumerator();
 }
}
```

- ```
}
1 Declares the class and implements the interface
2 Initializes the ChatMessages list
3 Sets the Count property for IReadOnlyList
4 Implements indexer access for chat messages
5 Overrides ProvideChatHistoryAsync to read chat history from file
6 Overrides StoreChatHistoryAsync to update and persist chat history
7 Reads existing chat history from file
8 Deserializes chat history into ChatMessages
9 Appends new request and response messages to ChatMessages
10 Serializes updated ChatMessages to JSON
11 Writes serialized chat history back to file
12 Implements GetEnumerator for ChatMessage
13 Implements non-generic GetEnumerator for IEnumerable
```

Listing 7.11 implements the file-based `ChatHistoryProvider`.

Requirements (NuGet packages):

```
dotnet add package Microsoft.Agents.AI.OpenAI
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
```

Code path: `/AgentWithFileBasedChatHistoryProvider/Program.cs`

Listing 7.11 Agent with file-based `ChatHistoryProvider`

```
// 'usings' and API key fetching omitted for brevity

var filePath = $"robot-car-chat-history_{Guid.NewGuid()}.json";
CustomFileBasedChatHistoryProvider fileBasedChatHistoryProvider
    = new(filePath); //1

ChatClientAgent agent = new OpenAIClient(apiKey)
    .GetChatClient(model)
    .AsAIAgent(new ChatClientAgentOptions {
        Name = "RobotCarAgent",
        Description = "An agent that assists a robot with the basic moves.",
        ChatOptions = new ChatOptions
        {
            Instructions = """
                You are an AI assistant controlling a robot car capable of performing
                basic moves: forward, backward, turn left, turn right, and stop.
                You have to break down the provided complex commands into the basic
                moves you know.
                Respond only with the moves and their parameters (angle or distance),
                without any additional explanations.
                """ },
            ChatHistoryProvider = fileBasedChatHistoryProvider //2
        });

AgentSession session = await agent.CreateSessionAsync();

var prompt = """
    Complex command:
    "There is a tree directly in front of the car.
    Avoid it and then return to the original path."
    """;
Console.WriteLine($"USER: {prompt}");
AgentResponse response = await agent
    .RunAsync(prompt, session); //3
Console.WriteLine($"ASSISTANT: {response.Text}");

var followUpPrompt = "What was your second last basic move?";
Console.WriteLine($"USER: {followUpPrompt}");
AgentResponse followUpResponse = await agent
    .RunAsync(followUpPrompt, session); //4
Console.WriteLine($"ASSISTANT: {followUpResponse.Text}");
```

- ❶ Creates a file-based `ChatHistoryProvider`
- ❷ Assigns to agent the provider
- ❸ Calls the agent with initial prompt
- ❹ Calls the agent with follow-up prompt

Code output:

```
USER: Complex command:
"There is a tree directly in front of the car. Avoid it and then return to the
original path."
ASSISTANT: 1. Turn right 90 degrees
2. Forward 5 meters
3. Turn left 90 degrees
4. Forward 10 meters
5. Turn left 90 degrees
6. Forward 5 meters
7. Turn right 90 degrees
USER: What was your second last basic move?
ASSISTANT: 6. Forward 5 meters
```

The custom file-based provider loads prior messages before each invocation and appends new ones after.

EXERCISE

In the previous listing 7.11 each time you start the application, a new conversation is created. To persist conversations across application restarts, remove the GUID from the provider file name and run the application again.

Replace the line:

```
var filePath = $"robot-car-chat-history_{Guid.NewGuid()}.json";
```

with:

```
var filePath = $"robot-car-chat-history.json";
```

Add the following logic at the end of the code to observe the chat history from the last run. Re-run the application several times to see how the user-assistant (request-response) pairs accumulate.

```
Console.WriteLine("HISTORY:");

foreach (Microsoft.Extensions.AI.ChatMessage message in
fileBasedChatHistoryProvider.ChatMessages) //❶
{
    var source = message.GetAgentRequestMessageSourceType(); //❷
    Console.WriteLine($"[{source.Value}] {message.Role}: ");
    Console.WriteLine($" {message.Text}");
}
```

- ❶ Iterates thorough the chat messages
- ❷ Fetches the source of the message

Reusing the same file across multiple runs introduces a new responsibility: we must handle cleanup, such as deleting all or some messages from the file.

7.3.4 Vector Store `ChatHistoryProvider`

The vector store `ChatHistoryProvider` combines durability with semantic lookup. Instead of scanning a long chronological log, it stores each `ChatMessage` as a `ChatHistoryItem` in a vector collection, with embeddings that capture meaning. When the agent needs history, the provider can pull back the most relevant past messages for the current session or user.

This is powerful when conversations get long or span many sessions. Robby can “remember” similar situations across time, even if they happened weeks or months apart, without flooding the model with every message ever stored.

PRACTICAL EXAMPLE

Imagine Robby facing a situation he saw recently. A simple logbook is too long to read, but he can recall the most relevant past turns instead of scanning everything.

The example in listing 7.12 shows how `VectorStoreChatHistoryProvider` stores chat history `ChatHistoryItem` records in an in-memory vector-store collection. It retrieves up to `TopResults` records for the session, oldest first. It performs no similarity search, and its records do not survive process restarts.

Code path: `/AgentWithVectorStoreChatHistoryProvider/ChatHistoryItem.cs`

Listing 7.12 ChatHistoryItem class

```
using Microsoft.Extensions.VectorData;

namespace Providers;

public class ChatHistoryItem
{
    [VectorStoreKey]
    public string? Key { get; set; } //❶
    [VectorStoreData]
    public string? SessionId { get; set; } //❷
    [VectorStoreData]
    public DateTimeOffset? Timestamp { get; set; } //❸
    [VectorStoreData]
    public string? SerializedMessage { get; set; } //❹
    [VectorStoreData]
    public string? MessageText { get; set; } //❺
}
```

❶ Declares unique Key property

❷ Declares SessionId for session identification

❸ Declares Timestamp property for ordering purposes

❹ Declares SerializedMessage for serialized form of the message

❺ Declares MessageText for text content message

Listing 7.13 implements the vector store `ChatHistoryProvider`.

Code path: `/AgentWithVectorStoreChatHistoryProvider/VectorStoreChatHistoryProvider.cs`

Listing 7.13 Vector store ChatHistoryProvider

```
// 'usings' omitted for brevity
namespace Providers;

public class VectorStoreChatHistoryProvider
: ChatHistoryProvider, IReadOnlyList<ChatMessage>
{
    private readonly VectorStore _vectorStore;
    private readonly int _topResults;
    public List<ChatMessage> ChatMessages { get; private set; } = [];
    public int Count => ChatMessages.Count;
    public ChatMessage this[int index] => ChatMessages[index];
    public string? SessionId { get; private set; }

    public VectorStoreChatHistoryProvider(VectorStore vectorStore,
        JsonElement serializedStoreState, int topResults = 10) //❶
    {
        _vectorStore = vectorStore;
        _topResults = topResults;

        if (serializedStoreState.ValueKind is JsonValueKind.String)
        {
            SessionId = serializedStoreState.Deserialize<string>(); //❷
        }
    }
}
```

```

protected override async ValueTask<IEnumerable<ChatMessage>>
    ProvideChatHistoryAsync(InvokingContext context,
        CancellationToken cancellationToken = default) //❸
{
    if (string.IsNullOrEmpty(SessionId))
    {
        return [];
    }

    var collection = _vectorStore
        .GetCollection<string, ChatHistoryItem>("ChatHistory"); //❹
    await collection.EnsureCollectionExistsAsync(cancellationToken);

    var messages = await collection
        .GetAsync(item => item.SessionId == SessionId, _topResults,
            new() { OrderBy = order => order.Ascending(item => item.Timestamp) },
            cancellationToken)
        .Select(item => JsonSerializer.Deserialize<ChatMessage>(
            item.SerializedMessage!))
        .ToListAsync(cancellationToken); //❺

    return messages;
}

protected override async ValueTask StoreChatHistoryAsync(
    InvokingContext context, CancellationToken cancellationToken = default) //❻
{
    SessionId ??= Guid.NewGuid().ToString(); //❼

    var collection = _vectorStore.GetCollection<string,
        ChatHistoryItem>("ChatHistory"); //❽
    await collection.EnsureCollectionExistsAsync(cancellationToken);

    var newMessages = context.RequestMessages.Concat(context
        .ResponseMessages ?? []); //❾

    await collection.UpsertAsync(newMessages.Select(
        item => new ChatHistoryItem()
        {
            Key = SessionId + (item.MessageId ?? Guid.NewGuid().ToString()),
            Timestamp = DateTimeOffset.UtcNow,
            SessionId = SessionId,
            SerializedMessage = JsonSerializer.Serialize(item),
            MessageText = item.Text
        }), cancellationToken); //❿

    ChatMessages.AddRange(newMessages); //⓫
}

public IEnumerator<ChatMessage> GetEnumerator()
{
    return ChatMessages.GetEnumerator();
}

IEnumerator IEnumerable.GetEnumerator()
{
    return GetEnumerator();
}
}

```

- ❶ Initializes the provider with vector store and state
- ❷ Deserializes the session ID from serialized state
- ❸ Overrides **ProvideChatHistoryAsync** to load chat history
- ❹ Fetched the “ChatHistory” collection
- ❺ Queries and deserializes chat history messages

- ❹ Overrides `StoreChatHistoryAsync` to persist chat history
- ❺ Sets `SessionId` if not already present
- ❻ Gets the “ChatHistory” vector store collection
- ❼ Merges request and response messages
- ❽ Upserts new messages into the vector store
- ❾ Updates `ChatMessages` with new messages

Listing 7.14 demonstrates the vector store `ChatHistoryProvider`.

Requirements (NuGet packages):

```
dotnet add package Microsoft.Agents.AI.OpenAI
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
dotnet add package CommunityToolkit.VectorData.InMemory
```

Code path: `/AgentWithVectorStoreChatHistoryProvider/Program.cs`

Listing 7.14 Agent with vector store `ChatHistoryProvider`

```
// 'usings' and API key fetching omitted for brevity

const int TopResults = 5; //❶

VectorStoreChatHistoryProvider vectorStoreChatHistoryProvider = new(new
InMemoryVectorStore(), default, topResults: TopResults); //❷

ChatClientAgent agent = new OpenAIClient(apiKey)
    .GetChatClient(model)
    .AsIAgent(new ChatClientAgentOptions
    {
        Name = "RobotCarAgent",
        Description = "An agent that assists a robot with the basic moves.",
        ChatOptions = new ChatOptions
        {
            Instructions = """
                You are an AI assistant controlling a robot car capable of performing
                basic moves: forward, backward, turn left, turn right, and stop.
                You have to break down the provided complex commands into the basic
                moves you know.
                Respond only with the moves and their parameters (angle or distance),
                without any additional explanations.
                """ },
            ChatHistoryProvider = vectorStoreChatHistoryProvider //❸
        }); //❹

AgentSession session = await agent.CreateSessionAsync();

var prompt = $"""
    Complex command:
    "There is a tree directly in front of the car.
    Avoid it and then return to the original path."
    """;

Console.WriteLine($"USER: {prompt}");
AgentResponse response = await agent
    .RunAsync(prompt, session); //❺
Console.WriteLine($"ASSISTANT: {response.Text}");

var followUpPrompt = "What was your second last basic move?";
Console.WriteLine($"USER: {followUpPrompt}");
AgentResponse followUpResponse = await agent
    .RunAsync(followUpPrompt, session); //❻
Console.WriteLine($"ASSISTANT: {followUpResponse.Text}");
```

- ❶ Sets the maximum number of results for chat history
- ❷ Creates the vector store `ChatHistoryProvider`
- ❸ Assigns the provider to the agent
- ❹ Declares the agent

- ⑤ Calls the agent with initial prompt and session
- ⑥ Calls the agent with follow-up prompt and session

Code output:

```
USER: Complex command:
"There is a tree directly in front of the car. Avoid it and then return to the
original path."
ASSISTANT: 1. Turn right 90 degrees
2. Forward 5 units
3. Turn left 90 degrees
4. Forward 10 units
5. Turn left 90 degrees
6. Forward 5 units
7. Turn right 90 degrees
USER: What was your second last basic move?
ASSISTANT: 6. Forward 5 units
```

Vector storage enables prompting recent messages by session and relevance. The `TopResults` limit (5 here) controls context window size.

EXERCISE

Following we will see what happened if we ingest some weather data and we ask queries about that.

At the end of listing 7.14 (previous) add a new agent that is responsible for collecting weather data:

```
ChatClientAgent weatherContextAgent = new OpenAIClient(apiKey)
    .GetChatClient(model)
    .AsAgent(new ChatClientAgentOptions
    {
        Name = "WeatherContextAgent",
        Description = "An agent that persists the weather context.",
        ChatOptions = new ChatOptions
        {
            Instructions = """
                You are an AI assistant persisting the weather context.
                Respond echoing the user provided info, or answering the user prompt.
                """ },
        ChatHistoryProvider = vectorStoreChatHistoryProvider //①
    });
```

- ① Creates weather context agent with same provider

And let's use this agent to collect some weather data (add to listing 7.14):

```
AgentResponse response1 = await weatherContextAgent.RunAsync("""
    June 1, 2025 - Morning: 14°C, partly cloudy, wind 8 km/h, dry.
    Afternoon: 20°C, mostly sunny, wind 12 km/h, no rain.
    Night: 13°C, clear, wind 6 km/h, calm.
    """, session);
AgentResponse response2 = await weatherContextAgent.RunAsync("""
    June 2, 2025 - Morning: 15°C, sunny, wind 10 km/h, dry.
    Afternoon: 22°C, mostly sunny, wind 14 km/h, dry roads.
    Night: 14°C, few clouds, wind 8 km/h, no precipitation.
    """, session);
AgentResponse response3 = await weatherContextAgent.RunAsync("""
    June 3, 2025 - Morning: 13°C, cloudy, wind 10 km/h, dry.
    Afternoon: 21°C, clearing skies, wind 13 km/h, dry.
    Night: 13°C, mostly clear, wind 7 km/h, calm.
    """, session);
AgentResponse response4 = await weatherContextAgent.RunAsync("""
    June 4, 2025 - Morning: 12°C, overcast, wind 11 km/h, dry.
    Afternoon: 19°C, showers likely, wind 16 km/h, wet roads possible.
    Night: 12°C, cloudy, wind 9 km/h, light drizzle.
    """, session);
AgentResponse response6 = await weatherContextAgent.RunAsync("""
    June 5, 2025 - Morning: 12°C, cloudy, wind 13 km/h, occasional light rain.
    Afternoon: 18°C, overcast, wind 18 km/h, scattered rain showers.
    Night: 11°C, mostly cloudy, wind 10 km/h, some drizzle.
    """, session);
```

And finally let's ask the temperature for a specific day (add to listing 7.14):


```
AgentResponse response7 = await weatherContextAgent.RunAsync("""
##Prompt: What was the temperature on June 2, 2025, afternoon?
""", session);
```

Add the following logic at the end and run the code again (add to listing 7.14):

```
Console.WriteLine("HISTORY:");

foreach (Microsoft.Extensions.AI.ChatMessage message in
vectorStoreChatHistoryProvider.ChatMessages) //❶
{
    var source = message.GetAgentRequestMessageSourceType();
    Console.WriteLine($"{source.Value}] {message.Role}: ");
    Console.WriteLine($"{message.Text}");
}
```

❶ Iterates through chat messages

The last answer in the output will show the correct temperature for the required day.

7.4 Combining Both Providers

When we mix both providers, we stop thinking in terms of “a single prompt” and start thinking in terms of layered memory and behavior. Chat history captures what actually happened over time. `AIContext` captures how the agent should behave right now, given who the user is, what the app is doing, and which tools are available. Used together, they let us send the model a single, coherent merged (aggregated) messages view while keeping storage, personalization, and orchestration in separate, testable pieces. This is what makes an agent feel both consistent across sessions and adaptive to the current task.

The two mechanisms work together in production systems. An `AgentSession`, backed by a `ChatHistoryProvider`, manages conversational continuity through persisted chat history. Meanwhile, `AIContextProvider` supplies context providers that inject external knowledge like user profiles and retrieved documents. The agent sends the combined payload (recent chat history, context items, and the new prompt) to the model, enabling it to reason over both the immediate conversation and the broader knowledge base simultaneously.

Figure 7.6 shows the sequence diagram that combines both providers and shows how `ChatClientAgent` uses session state, history, and `AIContext` together in a single run.

The user calls `RunAsync(inputMessages, session)`. The agent first gets `historyMessages` from `ProvideChatHistoryAsync`, then gets `AIContext` from `ProvideAIContextAsync`, which includes instructions, `additionalMessages`, and tools. It uses all message sources to build `mergedMessages` and sends that list to the chat client via `GetResponseAsync`, receiving a `ChatResponse`. After the call completes, the agent invokes `StoreChatHistoryAsync(InvokedContext)` so the history provider can persist the new turn, and `StoreAIContextAsync(InvokedContext)` so the `AIContextProvider` can update its serialized context or memory. The agent then returns the final response to the user.

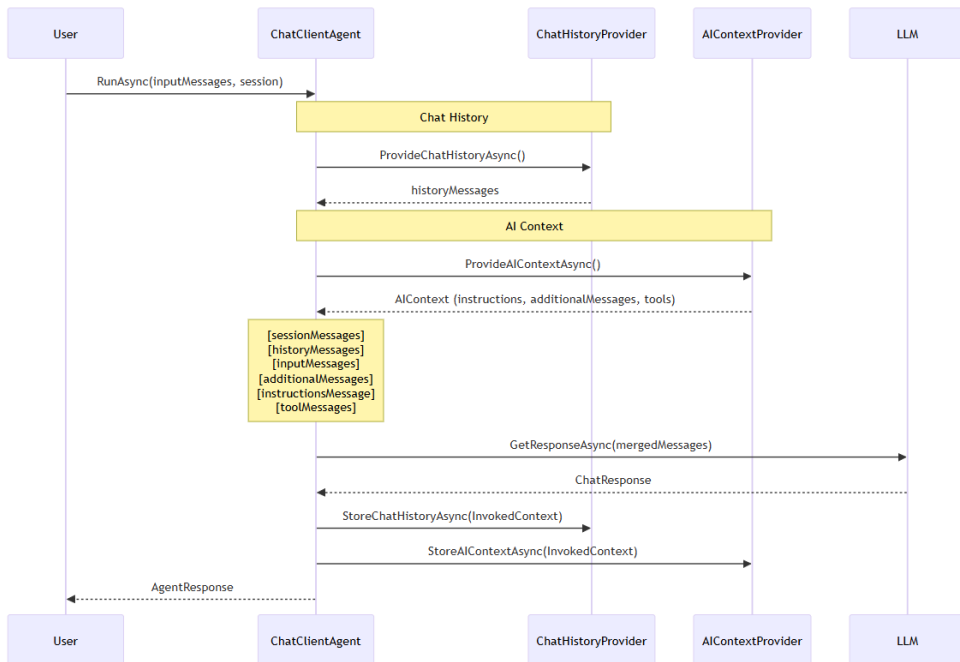


Figure 7.6 ChatClientAgent constructing merged messages from session, history, and AIContext, then updating both providers after the model call.

Key takeaways and how we use it in practice:

- We separate “what happened” (chat history) from “how the agent should behave” (AIContext), which keeps both simpler.
- We can evolve memory strategies independently: switch history storage, tweak instructions, or add tools without touching the core agent loop.
- We get a single mergedMessages view for the model while keeping clean, testable components for session, history, and context.

7.5 Conclusion

This chapter demonstrated how context providers give agents like Robby behavior and preferences for each session, while chat history lets him remember past routes, corrections, and choices instead of starting fresh every time. Together, they give the model one coherent view of “how Robby should act” and “what has already happened”, which makes his decisions more predictable and easier to refine before we plug him into MCP-powered tools in the next chapter.

Summary

- Agents use Context (external knowledge) and Chat History (conversation state) as complementary memory systems.
- AIContextProvider participates in each run to add instructions, extra messages, and tools into merged messages, then update its own memory with StoreAIContextAsync.
- FileBasedContextProvider loads instructions and chat history from disk, grounding

agent responses in external files without changing the agent code.

- In-memory vector store providers enable RAG-style context: they embed documents once and retrieve only relevant chunks per prompt instead of sending full documents every time.
- Keyword-based `TextSearchProvider` offers a lightweight alternative to embeddings, using curated keywords and simple ranking to inject relevant context for small or structured datasets.
- `ChatHistoryProvider` loads history messages before each call and persists new turns afterwards, while the agent works only with merged messages.
- `InMemoryChatHistoryProvider` stores messages for the lifetime of the process, making it ideal for development and for experimenting with reducers like `MessageCountingChatReducer`.
- `FileBasedChatHistoryProvider` persists conversations to JSON files across restarts, while `VectorStoreChatHistoryProvider` stores messages as `ChatHistoryItem` records for semantic retrieval.
- Combining context and history providers we can build a single merged messages view from session, history, and `AIContext`, which makes his behavior both consistent across sessions and adaptive to the current task.

8

Standardizing tools using MCP (Model Context Protocol)

This chapter covers

- Understanding Model Context Protocol (MCP) architecture
- Exploring the transport types: *STDIO*, *HTTP*, and *Stream*
- Building MCP servers that expose Agent Framework AI tools, prompts and resources
- Creating MCP clients that consume external tools, prompts and resources

Remember Robby's garage from earlier chapters, filled with specialized tools, each requiring its own adapter? Now imagine if Robby could access an entire warehouse of robotic tools maintained by other developers, all through the same universal MCP interface.

Agent Framework solves this locally using AI tools, but what happens when you want to share those same capabilities across different AI systems?

Model Context Protocol (MCP), was introduced by Anthropic in late 2024. Think of MCP as decoupled and standardized universal adapter for AI applications for consuming tools.

8.1 Introduction to MCP

Just as HTTP standardized how web browsers communicate with servers, MCP standardizes how AI applications discover and use external tools. The protocol creates a three-part ecosystem where each component plays a specific role in enabling cross-platform AI functionality.

The *AF App* (Agent Framework application) acts as the orchestrator, your main application that coordinates everything. In our context, this is typically your Agent Framework application. The *MCP Client* serves as a connector that communicates with an MCP server. Finally, the *MCP Server* functions as a service that exposes AI tools through the MCP protocol, essentially a standalone service built from collections of tools.

8.1.1 MCP Architecture

This architecture solves a fundamental problem in the AI ecosystem: capability fragmentation. Without MCP, each AI system requires its own implementation of common tools like database connectors, file processors, or API integrators, and for that in Agent Framework we have the AI tools. With MCP, you write once and use it everywhere.

MCP follows a clean client-server architecture that complements Agent Framework well.

Table 8.1 Client-server architecture

| Component | Role | Agent Framework Equivalent |
|-------------------|---|--|
| AF App | Application that uses Agent Framework to coordinate MCP services via MCP client | Your Agent Framework application |
| MCP Client | Connector that talks to a specific MCP server | Methods that convert MCP tools to AI tools |
| MCP Server | Service exposing tools through MCP protocol | Standalone service built from MCP tools (annotated C# methods) |

In figure 8.1, the key components of the MCP Architecture are presented. This diagram illustrates how the Agent Framework app interacts with the MCP Client, which connects to an MCP Server. The Agent Framework app acts as the main application consuming MCP services, while the MCP Client serves as a connector, and the MCP Server exposes external tools to the application.

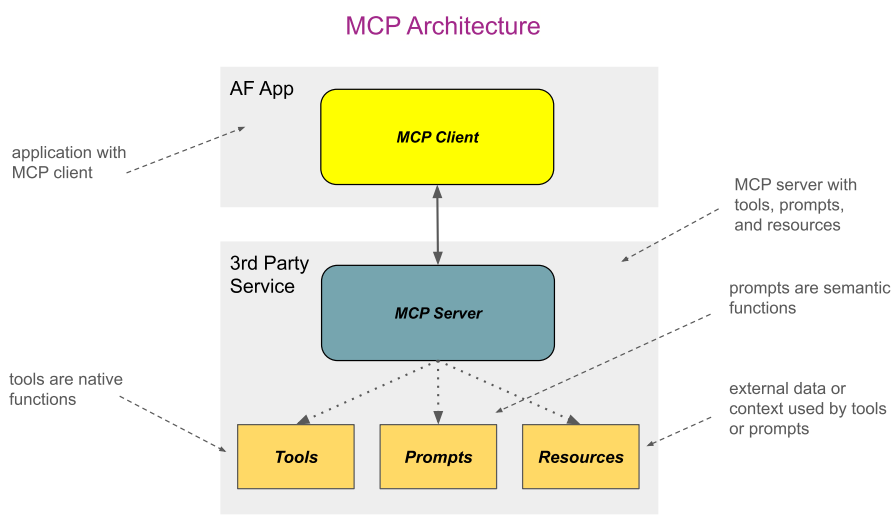


Figure 8.1 MCP Architecture with Agent Framework app consuming services via the MCP Client, which communicates with an MCP Server that exposes tools, prompts, and resources.

MCP organizes functionality into three distinct categories, each serving a specific purpose in the AI workflow.

- **Tools:** Represent executable functions that perform actions, from reading sensors to processing data, similar to native functions
- **Prompts:** Reusable prompt templates that can be shared across applications, similar to semantic prompts.
- **Resources:** Provide access to static or dynamic content like files, databases, or API endpoints.

This separation creates clarity in how AI systems interact with external capabilities. Tools focus on actions, prompts handle reasoning patterns, and resources manage data access. Understanding these distinctions helps you design better MCP servers and more effective AI applications.

8.2 Building MCP Tools

Model Context Protocol (MCP) tools allow you to take the solid, reusable logic you've already built in Agent Framework and share it with any AI system, across programming languages, operating systems, and even organizational boundaries. You can think of MCP tools as "universal adapters" for artificial intelligence: they make your application's intelligence accessible to the entire world, not just your own codebase.

We'll go next through the process of transforming in-process Agent Framework tool sets into cross-platform MCP servers.

8.3 Implementing MCP Server

Implementing an MCP server involves more than exposing functions, it requires careful consideration of deployment architecture, transport protocols, and scalability needs. In this section, you'll learn how to modernize your Agent Framework-based applications as robust MCP servers, making them accessible across platforms and environments.

8.3.1 Transport Types

Understanding MCP's transport mechanisms helps you choose the right deployment pattern for your specific use case. Each means of transport offers different trade-offs in terms of performance, complexity, deployment flexibility, and operational characteristics.

- `WithStdioServerTransport` establishes a local connection over standard input and output streams. This transport implements MCP's standard input-output protocol (STDIO) and is best for integrating tightly with local processes or scripts. It offers minimal setup and is ideal for development, testing, and trusted environments where both client and server run on the same machine.
- `WithHttpTransport` enables MCP's standard HTTP transport. Use this option when hosting your MCP server as an ASP.NET Core Web application. It's suitable for remote production scenarios with request-response semantics.
- `WithStreamServerTransport` binds your MCP server to a generic bidirectional stream abstraction. This transport-agnostic option allows you to carry MCP messages over custom channels like TCP sockets, named pipes, or any read/write stream, including STDIO or HTTP streaming as particular cases. It is designed for advanced integrations requiring flexibility beyond the standard STDIO or HTTP transport.

In figure 8.2, the main transport types for MCP communication are illustrated. This diagram shows how the MCP Client can interact with the MCP Server using different transport types: STDIO, HTTP, and Stream, depending on the communication requirements and infrastructure.

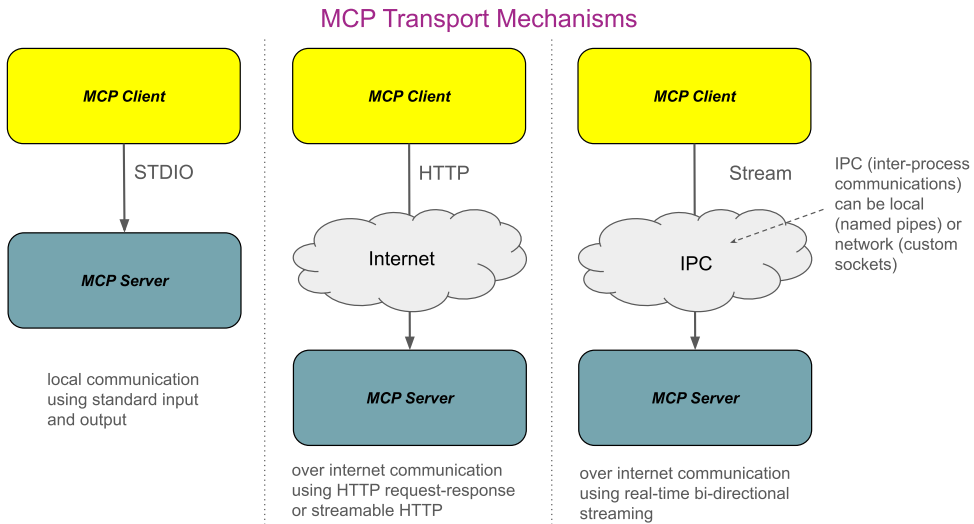


Figure 8.2 MCP Transport types: STDIO (local standard input/output), HTTP (remote HTTP-based communication), and Stream (custom bidirectional channels for advanced IPC scenarios) between MCP Client and MCP Server.

Note STDIO and Streamable HTTP are standard MCP transport. *Stream* in figure 8.2 refers to the SDK's custom-stream integration, not a third standard protocol transport.

Ensure that your server supports necessary authentication headers and manages persistent connections with heartbeat and auto-reconnection for reliability in production/cloud deployment.

8.3.2 Building MCP Server

When building an MCP server with the Agent Framework, we configure the server, choose a transport (STDIO, HTTP, or stream), and register tools, prompts, and resources, typically using attribute-based discovery.

We'll also see how to define tools, prompts and resources using MCP attributes, allowing for clear separation of interface logic and enhanced cross-platform integration.

SERVER CONFIGURATION WITH STDIO TRANSPORT

Following, we will setup an MCP Server with STDIO transport.

Listing 8.1 demonstrates how to create an MCP Server that exposes logic as standardized MCP tools. This example shows the transformation from in-process AI tools to out-of-process MCP services (including MCP tools). The code configures an MCP Server that can communicate via STDIO transport, making the `MotorTools` tools available to any MCP-compatible client.

Listing 8.1 MCP Server With STDIO Server Transport

```
using MCPServer.Prompts;
using MCPServer.Resources;
using MCPServer.Tools;
using ModelContextProtocol;
using ModelContextProtocol.Protocol;
using ModelContextProtocol.Server;
```

```

var builder = Host.CreateEmptyApplicationBuilder(null);

builder.Services
    .AddMcpServer(options =>
    {
        options.ServerInfo = new Implementation
        {
            Name = "Motors Server",
            Version = "1.0.0",
        };
        options.InitializationTimeout = TimeSpan.FromSeconds(10);
    }) //❶
    .WithStdioServerTransport() //❷
    .WithTools<MotorTools>();

var app = builder.Build();
app.Run(); //❸

```

- ❶ Registers MCP service in the DI container
- ❷ Configures STDIO as server transport
- ❸ Starts MCP service

This configuration exposes the functionality in `MotorTools` class methods as reusable MCP tools that any MCP-compatible client can call over STDIO, which is ideal for local development and trusted environments. The client can even be configured to launch and manage this server process automatically when using STDIO transport.

`ServerInfo` identifies the server to clients, helping them understand which server they are connecting to and which version is running. `InitializationTimeout` prevents clients from waiting indefinitely during startup, improving robustness in production scenarios. Another key `McpServerOptions` property is `Capabilities`, which declares supported features such as tools, prompts, and resources, so clients can understand the server's functional scope before invoking anything.

SERVER CONFIGURATION WITH HTTP TRANSPORT

For production environments, HTTP-based transport is often preferred over STDIO because it supports remote access, standard networking, and familiar infrastructure such as load balancers and API gateways. The next example (listing 8.2) shows an MCP server configured with HTTP transport.

Listing 8.2 MCP Server With HTTP Transport

```

using MCPServer.Prompts;
using MCPServer.Resources;
using MCPServer.Tools;
using ModelContextProtocol.Protocol;
using ModelContextProtocol.Server;

var builder = WebApplication.CreateBuilder(args);

builder.Services
    .AddMcpServer() //❶
    .WithHttpTransport() //❷
    .WithTools<MotorTools>();

var app = builder.Build();
app.MapMcp(); //❸
app.Run(); //❹

```

- ❶ Adds MCP service
- ❷ Sets HTTP transfer type
- ❸ Maps MCP service endpoints

❶ Runs MCP service

In this configuration, any MCP-compatible HTTP client can connect to the server, discover the registered tools, prompts, and resources, and call them over standard HTTP endpoints, depending on the server configuration.

The `MCP WithHttpTransport` method supports stateless operation and stateful compatibility modes.

Stateful Mode (compatibility with older clients):

```
.WithHttpTransport(httpOptions =>
{
    httpOptions.Stateless = false; //❶
})
```

❷ Selects stateful operation; the MCP C# v2 default is now stateless

In SDK 2.2.0, `SessionMode` also provides a hybrid `StatefulForInitializeClients` option; the `Stateless` set to `Boolean` shown here selects the two modes used in these examples.

Stateful Mode (compatibility with older clients):

```
.WithHttpTransport(httpOptions =>
{
    httpOptions.SessionMode =
        HttpServerSessionMode.StatefulForInitializeClients; //❶
})
```

❸ Uses stateful sessions for initialize-based clients and stateless

Stateful mode maintains session state with Session IDs for older, initialize-based clients and supports session-dependent notifications. The 2026-07-28 HTTP protocol revision no longer uses these transport sessions; an explicitly stateful server requires compatible clients to fall back to an older revision. Stateful mode does not automatically enable the legacy `/sse` endpoint.

Stateless Mode (default in SDK 2.2.0):

```
.WithHttpTransport(httpOptions =>
{
    httpOptions.Stateless = true;
})
```

Stateless mode provides no transport session management and is suitable for serverless deployments and horizontal scaling. It can still stream progress during an active request. Streamable HTTP uses optional Server-Sent Events (SSE) in responses; this is distinct from the legacy HTTP+SSE transport and its `/sse` endpoint.

NOTE With the 2026-07-28 protocol, capability discovery uses `server/discover` instead of the older `initialize` handshake. `McpClient.CreateAsync` handles discovery and compatibility fallback.

SERVER CONFIGURATION WITH STREAM TRANSPORT

Stream transport is an advanced option for custom inter-process communication (IPC) scenarios that don't fit `STDIO` or `HTTP` patterns. Use this transport for named pipes, network streams, or custom in-process streams. Here (listing 8.3) is a simplified example that uses in-memory pipes to demonstrate configuration:

Listing 8.3 MCP Server with Stream Server Transport

```
using MCPServer.Prompts;
using MCPServer.Resources;
using MCPServer.Tools;
using ModelContextProtocol.Protocol;
using ModelContextProtocol.Server;
```

```
using System.IO.Pipelines;
using ModelContextProtocol;

var builder = Host.CreateEmptyApplicationBuilder(null);

var inputStream = new Pipe().Reader.AsStream(); //❶
var outputStream = new Pipe().Writer.AsStream(); //❷

builder.Services
    .AddMcpServer() //❸
    .WithStreamServerTransport(inputStream, outputStream) //❹
    .WithTools<MotorTools>();

var app = builder.Build();
app.Run(); //❺
```

- ❶ Defines the input stream (reading pipe)
- ❷ Defines the output stream (writing pipe)
- ❸ Adds MCP service
- ❹ Sets Stream transfer type
- ❺ Runs MCP service

In a real deployment, the input and output streams would typically be connected to named pipes, network streams, or other IPC mechanisms, and for true in-process communication, these stream instances must be shared between client and server components.

This remains a configuration sketch: the client must be connected to the other end of each pipe before the example can exchange messages.

The three transport options covered (STDIO server, HTTP, and stream server) serve different scenarios.

Table 8.2 Transport choices overview

| Transport | Choose it when | Why |
|---------------|--|---|
| STDIO | local development, desktop tools, trusted same-machine execution | Simple setup, great for development, client can launch server |
| HTTP | remote services, cloud deployment, APIs, load balancers | standard network model, production-friendly, supports request/response and streaming |
| Stream | IPC (inter-process communication) - named pipes, sockets, custom hosting needs | maximum flexibility for IPC (inter-process communication), works with in-memory streams for testing |

Choosing the right transport depends on where the MCP server runs, how clients connect, and your operational constraints.

8.4 Declaring Tools, Prompts, and Resources in MCP Server

An MCP server can have tools, prompts, and resources. Many providers declare only tools, but prompts and resources turn those tools into a richer, discoverable capability surface: prompts capture reusable domain-specific workflows, resources provide structured context, and tools execute concrete actions, so agents get a complete, well-defined way to understand and use the domain rather than just a flat list of operations.

CREATING MCP TOOLS

To expose methods as MCP tools, annotate a class with `McpServerToolType` and each tool method with `McpServerTool`, optionally adding metadata via attributes such as `McpMeta.Description`

is an especially useful and recommended attribute of `System.ComponentModel` for describing the tools and its arguments. The following example (listing 8.4) shows a simple motor control tool:

Listing 8.4 MotorTools Sample

```
using ModelContextProtocol.Server;
using ModelContextProtocol; //❶
using Serilog;
using System.ComponentModel;

[McpServerToolType, Description("Robot car motors tools.")] //❷
public class MotorTools
{
    private const int Delay = 100;

    [McpMeta("category", "motor")] //❸
    [McpServerTool(
        Name = "backward",
        Title = "Move Backward",
        ReadOnly = false,
        Destructive = true,
        Idempotent = false,
        OpenWorld = false)] //❹
    [Description("Basic command: Moves the robot car backward.")] //❺
    public async Task<string> BackwardAsync(
        [Description("The distance (in meters) to move the robot car backward.")]
        int distance) //❻
    {
        Log.Information("MOTORS: Backward: {Distance}m", distance);
        await Task.Delay(Delay);
        return $"moved backward for {distance} meters.";
    }
}
```

- ❶ Imports the current MCP namespace for shared SDK types and met
- ❷ Annotates the class as MCP tool class
- ❸ Annotates the method with metadata
- ❹ Annotates the method as MCP tool with tool modes and identity
- ❺ Annotates the method with MCP description
- ❻ Method as MCP tool

When exposing functions as tools, avoid nullable or uninitialized parameters in the tool surface and clearly mark required arguments so AI models and clients do not attempt to invoke tools with missing data. When this class is registered via `WithTools<MotorTools>()`, the `BackwardAsync` method becomes available as a callable MCP tool to connected clients.

CREATING MCP PROMPTS

Prompts (listing 8.5) represent reusable reasoning templates rather than executable actions, allowing different AI systems to share domain-specific guidance while customizing arguments as needed. To register prompts, annotate a class with `McpServerPromptType` and each prompt method with `McpServerPrompt` attribute.

Listing 8.5 MotorPrompts Prompt Sample

```
using Microsoft.Extensions.AI;
using ModelContextProtocol.Server;
using System.ComponentModel;

[McpServerPromptType] //❶
public class MotorPrompts
{
    [McpServerPrompt(Name = "string_prompt")] //❷
    [Description("A string prompt without arguments")] //❸
}
```

```

public static string StringPrompt()
{
    return ""
        ## Context
        Complex command:
        "There is a tree directly in front of the car. Avoid it and then return to
the original path."
        "";
}
}

```

❶ Marks the class as MCP prompt type class

❷ Marks the method as MCP prompt

❸ Adds description for MCP prompt

Prompt methods can return plain strings, chat messages (list of `ChatMessage`), typically representing user-role input or multi-part templates that clients can parameterize. This pattern turns expert reasoning about, for example, decomposing complex movement commands, into reusable prompt templates consumable by any MCP-compatible client.

CREATING MCP RESOURCES

Resources provide static or dynamic context, such as files, configuration snippets, or database-backed content, that tools and prompts can reference. They are defined similarly using `McpServerResourceType` on the class and `McpServerResource` on each resource method, as we can see in listing 8.6.

Listing 8.6 MotorResources resource sample

```

using ModelContextProtocol.Server;
using System.ComponentModel;

[McpServerResourceType] //❶
public class MotorResources
{
    [McpServerResource] //❷
    [Description("A direct text resource")] //❸
    public static string DirectTextResource() => "My name is Robby, the robot.";
}

```

❶ Marks the class as MCP resource type class

❷ Marks the method as MCP resource

❸ Adds description for MCP resource

This example exposes a simple static string as an MCP resource, but in practice resource methods often load data from files, databases, or external services to provide richer context to agents and tools.

MANUAL AND AUTOMATIC TOOLS, PROMPTS, AND RESOURCES DISCOVERY

The methods `WithTools<T>()`, `WithPrompts<T>()`, and `WithResources<T>()` provide explicit registration of types that contain MCP tools, prompts, and resources. For larger codebases, `WithToolsFromAssembly()`, `WithPromptsFromAssembly()`, and `WithResourcesFromAssembly()` enable reflection-based discovery, automatically exposing all annotated tool, prompt, and resource classes in the target assembly, which simplifies maintenance as your catalogue grows.

The server scans your assembly for appropriately decorated classes and methods, automatically registering them as MCP capabilities. This automatic discovery ensures that new tools and prompts become available without manual registration steps, reducing maintenance overhead and potential configuration errors.

LONG-RUNNING MCP TOOLS AS TASKS

MCP standard supports two working modes for MCP tools: request-response and long-running task mode.

In Figure 8.3 we see how an MCP tool runs as a long-running task.

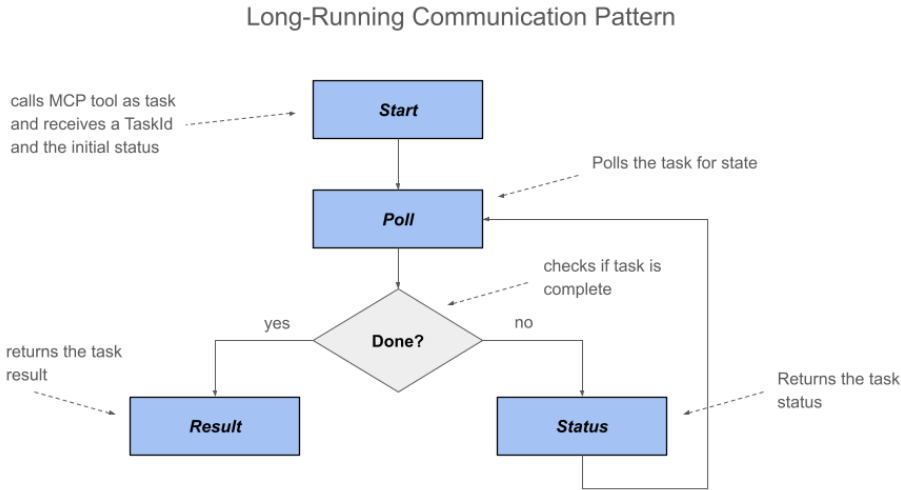


Figure 8.3 MCP tool can run as long-running task, returning status and, on completion, the result.

When the client opts into Tasks and the server creates a task, the client gets back a `TaskId` while the server continues in the background.

The client can then poll for completion. The completed tasks/get response contains the final result.

This is a better fit for slow hardware actions, large processing jobs, or multi-step workflows.

8.4.1 MCP Server with STDIO in Action

Following, we'll see how to expose Robby's existing tools into an MCP server that an MCP client can use.

In Figure 8.4, an example use of MCP with motors and sensors is presented. This diagram demonstrates how an app with an MCP Client can interact with multiple MCP servers. The `MotorsServer` exposes controls for car movement and command processing, while the `SensorsServer` provides temperature reading functionality. Each server makes specialized tools accessible to the main application.

MCP Motors and Sensors - Example

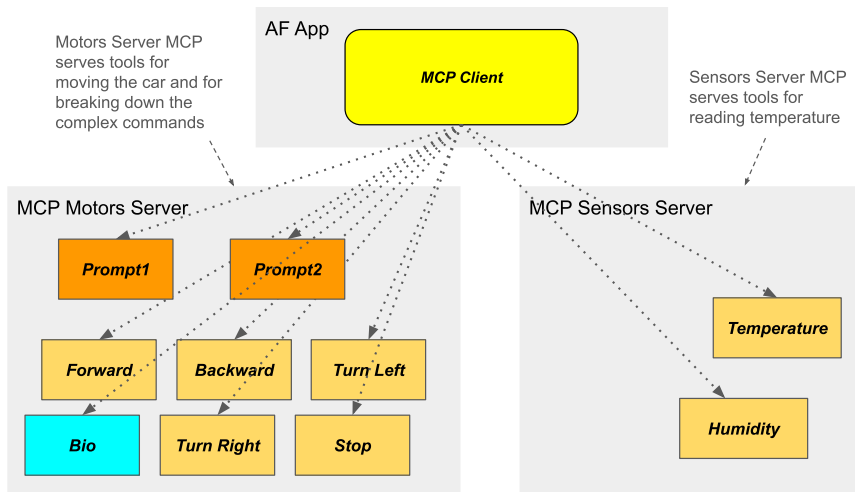


Figure 8.4 Example of MCP architecture using separate servers for motor control and sensor data, where an app can access movement commands and temperature readings through dedicated MCP services.

NOTE `Host.CreateEmptyApplicationBuilder(null)` creates a bare `HostApplicationBuilder` with almost no preconfigured services or defaults. It is intended for non-HTTP workloads or highly customized hosts where you want to opt in explicitly to logging, configuration, and other features. `WebApplication.CreateBuilder(args)` creates a `WebApplicationBuilder` for HTTP workloads and builds on top of `HostApplicationBuilder`. It is designed for ASP.NET Core / minimal API apps and assumes you are building a web server.

Use `Host.CreateEmptyApplicationBuilder` for STDIO/Stream transports (non-HTTP scenarios) and use `WebApplication.CreateBuilder` for HTTP transports (web endpoints).

PRACTICAL EXAMPLE

Listing 8.7 presents a comprehensive MCP Server implementation that combines multiple approaches for exposing functionality (tools, prompts, resource) using STDIO through the MCP protocol.

Requirements (NuGet libraries):

```
dotnet add package ModelContextProtocol
dotnet add package ModelContextProtocol.Extensions.Tasks
```

Listing 8.7 [MCPServerWithStdio/Program.cs]

```
using MCPServerWithStdio.Prompts;
using MCPServerWithStdio.Resources;
using MCPServerWithStdio.Tools;
using ModelContextProtocol.Protocol;
using ModelContextProtocol.Server;
using ModelContextProtocol.Extensions.Tasks;
using ModelContextProtocol;

HostApplicationBuilder builder = Host
```

```

.CreateEmptyApplicationBuilder(null); //❶

InMemoryMcpTaskStore taskStore = new() { DefaultPollIntervalMs = 250 }; //❷

builder.Services
    .AddMcpServer(options =>
    {
        options.ServerInfo = new Implementation
        {
            Name = "Motors Server",
            Version = "1.0.0",
        };
        options.InitializationTimeout = TimeSpan.FromSeconds(10);
    }) //❸
    .WithStdioServerTransport() //❹
    .WithTools<MotorTools>() //❺
    .WithTools<MaintenanceTools>() //❻
    .WithPrompts<MotorPrompts>() //❼
    .WithResources<MotorResources>() //❼
    .WithTasks(taskStore); //❽

Console.Error.WriteLine("MCP Server is running with STDIO transport type.");

var app = builder.Build();
app.Run(); //❾

```

- ❶ Initializes the host builder
- ❷ Initializes the in-memory task store
- ❸ Configures MCP Server
- ❹ Sets the transport type to STDIO
- ❺ Registers the tools exposed by the MCP Server
- ❻ Registers the prompts exposed by the MCP Server
- ❼ Registers the resources exposed by the MCP Server
- ❼ Registers the tasks on the server registration chain
- ❾ Runs the MCP server

The STDIO transport configuration makes this server ideal for local development and simple integration scenarios. For production environments, HTTP or stream-based transport are strongly recommended.

WARNING Use `Console.Error.WriteLine` instead of `Console.WriteLine` in MCP server, because plain startup text on violates MCP framing.

EXERCISE

In the previous listing (listing 8.7), MCP tasks were store in-memory using the default settings. However, these settings can be configured in more detail.

For example:

```

InMemoryMcpTaskStore taskStore = new()
{
    DefaultTimeToLive = TimeSpan.FromMinutes(5), //❶
    DefaultPollIntervalMs = 1000, //❷
};
// Pass this store to .WithTasks(taskStore). //❸

```

- ❶ Tasks expire five minutes after creation, not after completion
- ❷ Hints the client to poll every one second
- ❸ Uses the configured store in the server registration

Without a TTL, entries remain until the process exits. The in-memory store does not survive a server restart; durable storage or additional retention policies require a custom `IMcpTaskStore`.

8.4.2 MCP Server with HTTP in Action

Time to take a step further, in the previous section we've seen how we can expose Robby's tools via STDIO transport, but here we'll see how to expose Robby's existing tools into an MCP server that other AI systems can reuse. For that, we will replace the `HostApplicationBuilder` with `WebApplicationBuilder` in order to provide support for HTTP endpoints that we are going to access using `MapMCP` method.

PRACTICAL EXAMPLE

Listing 8.8 presents a comprehensive MCP Server implementation using HTTP transport through the MCP protocol:

Requirements (NuGet libraries):

```
dotnet add package ModelContextProtocol
dotnet add package ModelContextProtocol.AspNetCore
```

Listing 8.8 [MCPServerWithHttp/Program.cs]

```
// 'usings' and settings code omitted for brevity

WebApplicationBuilder builder = WebApplication.CreateBuilder(args); //❶

builder.Services
    .AddMcpServer(options =>
    {
        options.ServerInfo = new Implementation
        {
            Name = "Motors Server",
            Version = "1.0.0",
        };
        options.InitializationTimeout = TimeSpan.FromSeconds(10);
    }) //❷
    .WithHttpTransport(o => o.Stateless = true) //❸
    .WithTools<MotorTools>() //❹
    .WithPrompts<MotorPrompts>() //❺
    .WithResources<MotorResources>(); //❻

var app = builder.Build();
app.MapMcp(); //❼
app.Run(); //❽
```

- ❶ Initialized the web app builder
- ❷ Configures MCP Server
- ❸ Sets the transport type to HTTP in stateless mode
- ❹ Registers the tools exposed by the MCP Server
- ❺ Registers the prompts exposed by the MCP Server
- ❻ Registers the resources exposed by the MCP Server
- ❼ Maps the MCP endpoints
- ❽ Runs the MCP server

Now the MCP server will expose the MCP tools via HTTP protocol so any MCP client can consume. The current HTTP project references the `Tasks` package but does not register `WithTasks` or `MaintenanceTools`. Its client demonstrates ordinary calls, prompts, and resources. The three simple/progress/polling demonstrations are in the STDIO pair; this difference is sample scope, not a restriction of HTTP.

EXERCISE

Let's go back to listing 8.8 and switch the HTTP Transport from Stateless to Stateful, by changing the value of `Stateless` property to `false`.

```
.WithHttpTransport(o => o.Stateless = false)
```

`Stateless = true` leaves requests independent of transport sessions but still allows request-scoped streaming progress. `Stateless = false` selects the older session-based compatibility

path. Neither setting enables Tasks: task polling needs the Tasks extension and a store shared appropriately across requests.

WARNING When to choose stateless: scale-out deployments, serverless, or load-balanced environments where requests can land on different server instances. Choose stateful compatibility only when older clients need session-dependent features.

8.4.3 *Third-Party MCP Server Discovery*

To connect your .NET app with external MCP tools, start by searching community-driven registries, directories, or marketplaces, these act as catalogs for available MCP servers and their capabilities.

When connecting to third-party MCP servers, check whether they use stateful or stateless mode (as described in section 8.3.2) with HTTP transport. Most modern MCP servers support both modes.

IMPORTANT Use only the trusted registries, select servers with Streamed HTTP, and you'll ensure reliable cross-platform integration with third-party AI tools.

8.4.4 *MCP Inspector*

MCP Inspector is a CLI and UI tool for connecting to an MCP server so you can inspect its resources, prompts, tools, and notifications during development and debugging.

MCP Inspector is a generic client for the Model Context Protocol that lets you start an MCP server process, negotiate capabilities, then interact with all exposed resources, prompts, and tools from a graphical interface. It helps validate your server contract: you can check resource lists and metadata, run prompts with arguments, execute tools, and watch logs and notifications in real time.

You do not install it globally; you run it via `npm` (node package runner) from the location where the MCP server executable resides:

```
npm @modelcontextprotocol/inspector
```

In Inspector UI, verify the server connection and capability negotiation, then use:

- Resources tab to list and inspect resources and subscriptions.
- Prompts tab to see prompt templates, arguments, and try runs.
- Tools tab to inspect tool schemas and execute tools with custom inputs.
- Notifications pane to view logs and notifications from the server.

8.5 *Implementing MCP Client*

MCP clients consume the capabilities exposed by MCP servers, integrating remote tools, prompts, and resources into local AI workflows. The client implementation bridges the gap between MCP's standardized protocol and Agent Framework's tool calling mechanisms.

8.5.1 *Client Transport Mechanisms*

As discussed already, we have three transport mechanisms: STDIO, HTTP, and Stream-based. Let's explore them.

STDIO transport enables simple, low-latency process-to-process communication without the need for a network stack. The client launches the MCP server as a child process and communicates through standard input and output streams, preventing leftover orphan processes.

`StdioClientTransport` is ideal for fast prototyping and local development, as you avoid network latency and firewall issues entirely.

HTTP transport (`HttpClientTransport`) is appropriate for remote or cloud-hosted MCP servers. HTTP transport works well for simpler request/response scenarios, while streaming HTTP transports can send streaming responses and server-initiated messages over the same endpoint.

Stream transport (`StreamClientTransport`) supports custom inter-process communication mechanisms such as named pipes, network streams, or shared memory, offering flexibility for embedded or specialized deployment scenarios.

TIP For most cloud, multi-client, real-time, and collaborative production environments, prefer MCP servers that expose the Streamable HTTP transport over HTTP and connect to them using `HttpClientTransport`. Use `STDIO` primarily for local, rapid iteration and development scenarios.

8.5.2 Building MCP Client

Connecting to an MCP server as a client opens up powerful, dynamic tooling, prompt and resource sharing for your AI applications. This section illustrates how to configure your client to communicate with MCP servers, whether locally via `STDIO` or remotely using `HTTP`.

CLIENT CONFIGURATION WITH `STDIO` TRANSPORT

Listing 8.9 demonstrates configuring a client to connect to an MCP server via `STDIO`:

Listing 8.9 `STDIO` Client Transport Type

```
IClientTransport stdioTransport = new StdioClientTransport(new
StdioClientTransportOptions
{
    Name = "My Client",
    Command = "dotnet",
    Arguments = ["run", "--no-launch-profile", "--project",
        @"/path/to/project/MyServer.csproj"],
}); //❶

await using var mcpClient = await McpClient
    .CreateAsync(stdioTransport); //❷
```

❶ Initializes a `STDIO` client transport with command and arguments

❷ Launches the MCP server as a child process

The `StdioClientTransport` launches the MCP server as a child process and communicates internally through standard input/output streams. This approach works well for local development and scenarios where server and client run on the same machine. The transport can manage the server lifecycle, starting it when needed and cleaning up when the client disconnects.

NOTE With `STDIO` transport, the MCP client automatically launches and manages the server process lifecycle. This is suitable for local/trusted scenarios. For production/remote scenarios, use `HTTP` transport where the server runs independently.

The configuration specifies the exact command and arguments needed to start the server process, where integration ensures version compatibility and simplifies deployment in environments where both client and server components are managed together.

CLIENT CONFIGURATION WITH HTTP TRANSPORT

For HTTP-based MCP servers, use `HttpClientTransport` as shown in Listing 8.10:

Listing 8.10 MCP Client with HTTP Transport

```
IClientTransport httpTransport = new HttpClientTransport(new
HttpClientTransportOptions
{
    Endpoint = new Uri("http://localhost:3001"),
    Name = "Motors Client",
}); //❶

await using var mcpClient = await McpClient
    .CreateAsync(httpTransport); //❷
```

❶ Creates HTTP client transport with the server endpoint URI

❷ Establishes connection to the remote MCP server

DISCOVERING TOOLS, PROMPTS, AND RESOURCES

MCP clients can dynamically discover available tools, prompts, and resources, creating flexible AI applications that adapt to the capabilities provided by connected servers. This discovery mechanism enables runtime adaptation and reduces tight coupling between clients and servers.

Listing 8.11 demonstrates discovery:

Listing 8.11 Discovering Tools, Prompts, and Resources

```
IList<McpClientTool> mcpTools = await mcpClient
    .ListToolsAsync(); //❶
Console.WriteLine("Tools available:");
foreach (var tool in mcpTools)
{
    var arguments = tool.JsonSchema.GetProperty("properties").ToString();
    Console.WriteLine($" {tool} {arguments}"); //❷
}

IList<McpClientPrompt> mcpPrompts = await mcpClient
    .ListPromptsAsync(); //❸
Console.WriteLine("Prompts available:");
foreach (var prompt in mcpPrompts)
{
    var arguments = string.Join(",", JsonSerializer
        .Serialize(prompt.ProtocolPrompt.Arguments));
    Console.WriteLine($" {prompt.Name} {arguments}"); //❹
}

IList<McpClientResource> mcpResources = await mcpClient
    .ListResourcesAsync(); //❺
Console.WriteLine("Resources available:");
foreach (var resource in mcpResources)
{
    var arguments = string.Join(",",
        JsonSerializer.Serialize(resource.ProtocolResource));
    Console.WriteLine($" {resource.Name} {arguments}"); //❻
}
```

❶ Queries available MCP tools

❷ Displays found MCP tools with their arguments

❸ Queries available MCP prompts

❹ Displays found MCP prompts with their arguments

❺ Queries available MCP resources

❻ Displays found MCP resources with their arguments

This discovery mechanism enables runtime adaptation. Your AI application can query available capabilities and adjust its behavior accordingly. For example, a robot assistant might enable

advanced maneuvers only when specialized movement tools are available through connected MCP servers.

The current projects also list resource templates separately from fixed resources:

```
IList<McpClientResourceTemplate> templates = await
mcpClient.ListResourceTemplatesAsync();
foreach (var template in templates)
{
    Console.WriteLine($" {template.Name} "
        + $"{template.ProtocolResourceTemplate.UriTemplate}");
}
```

For example, `resource://mcp/greet/{name}` is a template. `ReadResourceAsync` needs a concrete URI such as `resource://mcp/greet/Robby`.

This discovery mechanism enables runtime adaptation. Our application can query available capabilities and adjust its behavior accordingly. For example, a robot assistant might enable advanced maneuvers only when specialized movement tools are available through connected MCP servers.

DISCOVERING TOOL TASKS

Some tools represent long-running operations, such as running hardware diagnostics, processing large datasets, or coordinating multi-step physical actions.

The `ModelContextProtocol.Extensions.Tasks` lets a client request background execution and poll for completion. An ordinary call can still await the same tool directly.

Let's look at the `run_diagnostics` MCP tool in the current `MaintenanceTools.cs` class, as shown in listing 8.12. Keep movement commands in `MotorTools`; the diagnostics methods belong to `MaintenanceTools`, registered in listing 8.7.

Listing 8.12 MaintenanceTools.cs

```
[McpServerTool(Name = "run_diagnostics")]
[Description("Runs a full diagnostics check on all robot car motors. This is a
long-running operation.")]
public static async Task<string> RunDiagnosticsAsync(
    CancellationToken cancellationToken)
{
    await Task.Delay(TimeSpan.FromSeconds(2), cancellationToken);
    return "Diagnostics complete. All 4 motors passed.";
}
```

Then, in listing 8.13, `CallToolWithPollingAsync` sends a request with the `Tasks` opt-in metadata. With the server configured in listing 8.7, the SDK polls `tasks/get` and returns the final `CallToolResult`. If a server returns an ordinary result instead of creating a `task`, the helper returns that result without polling. The diagnostic method does not accept a detailed argument.

Listing 8.13 Tools with Tasks

```
using ModelContextProtocol.Extensions.Tasks; //❶

using var pollingTimeout = new
CancellationTokenSource(TimeSpan.FromSeconds(30));
var result = await mcpClient.CallToolWithPollingAsync(
    new CallToolRequestParams { Name = "run_diagnostics" },
    cancellationToken: pollingTimeout.Token); //❷
var text = result.Content.FirstOrDefault() as TextContentBlock; //❸
Console.WriteLine($"POLLING CALL RESULT: {text?.Text}"); //❹
```

- ❶ Imports the separate v2 `Tasks` extension; place this using with the file imports
- ❷ Requests task execution and polls until the result is available
- ❸ Reads the text from the final `CallToolResult`
- ❹ Prints the result without a separate result-fetch request

Code output:

```
POLLING CALL RESULT: Diagnostics complete. All 4 motors passed.
```

The background pattern separates task creation from completion, but awaiting the polling helper still waits for the final result without blocking a thread. The current STDIO client owns the server process; a saved `TaskId` alone cannot restore its in-memory state after shutdown. The 30-second token bounds the client's wait, not a guaranteed remote cancellation. Explicit task cancellation uses `CancelTaskAsync` and is cooperative.

8.5.3 MCP Client with STDIO in Action

Time to see how your Agent Framework application can consume tools from an MCP server. First, let's see an MCP client with STDIO transport consuming an MCP server that supports STDIO transport.

PRACTICAL EXAMPLE

Listing 8.14 illustrates the client-side implementation of the MCP protocol, discover available tools, resources, and prompts, and integrate them seamlessly into Agent Framework workflows.

Requirements (NuGet libraries):

```
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
dotnet add package Microsoft.Agents.AI.OpenAI
dotnet add package ModelContextProtocol
dotnet add package ModelContextProtocol.Extensions.Tasks
```

Code path: `/MCPClientWithStdio/Program.cs`

Listing 8.14 MCP Client with STDIO transport

```
using Microsoft.Agents.AI;
using Microsoft.Extensions.AI;
using Microsoft.Extensions.Configuration;
using ModelContextProtocol;
using ModelContextProtocol.Client;
using ModelContextProtocol.Protocol;
using ModelContextProtocol.Extensions.Tasks;
using OpenAI;
using OpenAI.Chat;
using System.Text.Json;

IClientTransport stdioTransport = new StdioClientTransport(
    new StdioClientTransportOptions
    {
        Name = "Motors Client",
        Command = "dotnet",
        WorkingDirectory = AppContext.BaseDirectory,
        Arguments = ["run", "--project",
@"..\..\..\..\MCPServerWithStdio\MCPServerWithStdio.csproj"],
    }); //❶

await using var mcpClient = await McpClient
    .CreateAsync(stdioTransport); //❷
IList<McpClientTool> mcpTools = await mcpClient.ListToolsAsync();
// The remaining discovery and display code is shown in listing 8.11.

Console.WriteLine("1. SIMPLE CALL: turn_left");
var mcpTool = await mcpClient.CallToolAsync("turn_left",
    arguments: new Dictionary<string, object?>
    { { "angle", 99 } }
); //❸
var toolResponse = mcpTool.Content.FirstOrDefault() as TextContentBlock;
Console.WriteLine($"SIMPLE CALL RESULT: {toolResponse?.Text}"); //❹
Console.WriteLine();
```

```

Console.WriteLine("2. CALL WITH PROGRESS: run_diagnostics_with_progress");
// One request stays open while the server sends progress notifications.
var progressResult = await mcpClient.CallToolAsync(
    "run_diagnostics_with_progress",
    progress: new Progress<ProgressNotificationValue>(value =>
        Console.WriteLine($"    PROGRESS: {value.Progress}/{value.Total} motors
checked"))); //5
var progressText = progressResult.Content.FirstOrDefault() as TextContentBlock;
Console.WriteLine($"PROGRESS CALL RESULT: {progressText?.Text}");
Console.WriteLine();

Console.WriteLine("3. CALL WITH POLLING: run_diagnostics");
// Opt into a background MCP task. The SDK polls tasks/get until it completes.
using var pollingTimeout = new
CancellationTokenSource(TimeSpan.FromSeconds(30));
var pollingResult = await mcpClient.CallToolWithPollingAsync(
    new CallToolRequestParams { Name = "run_diagnostics" },
    cancellationToken: pollingTimeout.Token); //6
var pollingText = pollingResult.Content.FirstOrDefault() as TextContentBlock;
Console.WriteLine($"POLLING CALL RESULT: {pollingText?.Text}");
Console.WriteLine();
var mcpPrompt = await mcpClient.GetPromptAsync("message_prompt"); //7
var userPrompt = mcpPrompt.Messages.SingleOrDefault(m => m.Role ==
Role.User)?.Content as TextContentBlock; //8
var parametrizedMcpPrompt = await mcpClient
    .GetPromptAsync("parametrized_message_prompt",
        arguments: new Dictionary<string, object?>
            { { "action", "There is a tree directly in front of the car. Avoid it and
then return to the original path." } }
); //9
var userParametrizedPrompt = parametrizedMcpPrompt.Messages.SingleOrDefault(m =>
m.Role == Role.User)?.Content as TextContentBlock; //10
Console.WriteLine($"SIMPLE PROMPT RESPONSE: {userPrompt?.Text}");
Console.WriteLine($"PROMPT TEMPLATE (PARAMETRIZED) RESPONSE:
{userParametrizedPrompt?.Text}");
Console.WriteLine();

var mcpResource = await mcpClient
    .ReadResourceAsync("resource://mcp/bio"); //11
var mcpResourceResponse = mcpResource.Contents.FirstOrDefault() as
TextResourceContents;
Console.WriteLine($"RESOURCE RESPONSE: {mcpResourceResponse?.Text}");
var greetResource = await
mcpClient.ReadResourceAsync("resource://mcp/greet/Robby");
var greeting = greetResource.Contents.FirstOrDefault() as TextResourceContents;
Console.WriteLine($"TEMPLATE RESOURCE RESPONSE: {greeting?.Text}");
Console.WriteLine();

Console.Write("Run the AI agent? [y/N] ");
var answer = Console.ReadLine();
if (!string.Equals(answer?.Trim(),
    "y", StringComparison.OrdinalIgnoreCase)) //12
{
    Console.WriteLine("MCP demos complete; skipping the LLM agent.");
    return;
}
var configuration = new ConfigurationBuilder()
    .AddUserSecrets<Program>().Build();
var model = configuration["OpenAI:ModelId"];
var apiKey = configuration["OpenAI:ApiKey"];

// Create AI agent with MCP tools
ChatClientAgent agent = new OpenAIClient(apiKey)
    .GetChatClient(model)

```

```

.AsAIAgent("""
    You are an AI assistant controlling a robot car capable of performing
    basic moves: forward, backward, turn left, turn right, and stop.
    You have to break down the provided complex commands into the
    basic moves you know. Run the basic moves using the corresponding tools.
    Respond only with the moves and their parameters (angle or distance),
    without any additional explanations.
    """,
    tools: [.. mcpTools.Cast<AITool>()]
); //13

var prompt = userParametrizedPrompt!.Text;

Console.WriteLine("AGENT RESPONSE:");
AgentResponse response = await agent.RunAsync(prompt); //14
Console.WriteLine(response); //15

```

- ❶ Initializes the transport protocol
- ❷ Creates a MCP client instance
- ❸ Calls an MCP tool with arguments
- ❹ Prints the tool response to console
- ❺ Receives progress notifications during an ordinary tool call
- ❻ Opts into Tasks and automatically polls for the diagnostics result
- ❼ Fetches a simple prompt from MCP server
- ❽ Extracts the user message from prompt
- ❾ Fetches a prompt with arguments from MCP server
- ❿ Extracts the user message from the prompt with arguments
- ⓫ Fetches a resource to use its content
- ⓬ Skips the model unless the user explicitly answers y
- ⓭ Creates an AI agent with MCP tools
- ⓮ Runs the agent with a prompt
- ⓯ Prints the agent response to console

Run the current STUDIO client from the repository root without application arguments:

```
dotnet run --project .\08.MCP\MCPClientWithStdio\MCPClientWithStdio.csproj
```

The client starts the server automatically. Enter, n, or end-of-input skips the agent before user secrets are loaded; y is case-insensitive and runs the existing agent.

Code output, showing the calling by name of some MCP tools, prompts and responses:

```

1. SIMPLE CALL: turn_left
SIMPLE CALL RESULT: turned anticlockwise 99°.
2. CALL WITH PROGRESS: run_diagnostics_with_progress
  PROGRESS: 1/4 motors checked
  PROGRESS: 2/4 motors checked
  PROGRESS: 3/4 motors checked
  PROGRESS: 4/4 motors checked
PROGRESS CALL RESULT: Diagnostics complete. All 4 motors passed.
3. CALL WITH POLLING: run_diagnostics
POLLING CALL RESULT: Diagnostics complete. All 4 motors passed.

SIMPLE PROMPT RESPONSE: ## Context
Complex command:
"There is a tree directly in front of the car.
Avoid it and then return to the original path."
PROMPT TEMPLATE (PARAMETRIZED) RESPONSE: ## Context
Complex command:
There is a tree directly in front of the car.
Avoid it and then return to the original path.

RESOURCE RESPONSE: My name is Robby, the robot.
TEMPLATE RESOURCE RESPONSE: Hello, Robby! I am Robby, the robot. Version:
1.0.0.0
Run the AI agent? [y/N]

```

The greeting version comes from client metadata and can vary. Pressing Enter prints `MCP demos complete`; skipping the LLM agent. The polling helper does not print a task ID. Callback scheduling can affect the precise order of progress and result lines.

If we answer `y`, the agent can then produce a response like this (model output varies):

AGENT RESPONSE:

The sequence of moves executed is:

1. Turn left 90°.
2. Move forward 5 meters.
3. Turn right 90°.
4. Move forward 5 meters to bypass the tree.
5. Turn right 90° to align back.
6. Move forward 5 meters.
7. Stop.

The `StdioClientTransport` automatically manages the lifecycle of the external MCP server process, eliminating manual process management overhead.

The code output reveals how MCP bridges different reasoning approaches: the MCP server provides tools and prompts that encode domain-specific expertise and formatting preferences, while Agent Framework orchestrates when and how to invoke those capabilities. This architecture enables true modularity in AI applications, where specialized servers provide both tools and reasoning patterns while the main application orchestrates workflows through standardized MCP interfaces.

EXERCISE

In listing 8.14, `run_diagnostics` uses polling without progress. The current `MaintenanceTools` class also contains `run_diagnostics_with_progress`, shown below. This method reports progress during an ordinary `CallToolAsync` invocation. For this exercise, compare the two calls rather than adding a duplicate method.

```
[McpServerTool(Name = "run_diagnostics_with_progress")]
[Description("Runs a full diagnostics check on all robot car motors and
reports progress notifications.")]
public static async Task<string> RunDiagnosticsWithProgressAsync(
    IProgress<ProgressNotificationValue> progress, //❶
    CancellationToken cancellationToken)
{
    for (int i = 1; i <= 4; i++)
    {
        await Task.Delay(TimeSpan.FromSeconds(2), cancellationToken);
        progress.Report(new() { Progress = i, Total = 4 });
    }
    return "Diagnostics complete. All 4 motors passed.";
}
```

❶ Declares the SDK-injected progress reporter; `ProgressNotificationValue` is in `ModelContextProtocol`

Run the client and observe the progress lines during the second, ordinary call:

```
2. CALL WITH PROGRESS: run_diagnostics_with_progress
PROGRESS: 1/4 motors checked
PROGRESS: 2/4 motors checked
PROGRESS: 3/4 motors checked
PROGRESS: 4/4 motors checked
PROGRESS CALL RESULT: Diagnostics complete. All 4 motors passed.
```

The client supplies the progress callback and the SDK correlates notifications through the request's progress token. Remove the progress callback and run again: the tool still completes, but these updates are not displayed. This does not turn the operation into a polled task.

8.5.4 MCP Client with HTTP in Action

For HTTP-based MCP servers, the connection setup is nearly identical except for the transport type. In the current projects, however, the HTTP client does not include the STDIO progress/polling demonstrations or the optional-agent prompt.

PRACTICAL EXAMPLE

Listing 8.15 demonstrates the HTTP transport:

Listing 8.15 [MCPClientWithHttp/Program.cs]

```
...
IClientTransport httpTransport = new HttpClientTransport(new())
{
    Endpoint = new Uri("http://localhost:3001"),
    Name = "Motors Client",
}; // ❶

await using var mcpClient = await McpClient
    .CreateAsync(httpTransport); // ❷
...
```

❶ Initializes the transport

❷ Initializes the MCP client

The HTTP client performs the discovery shown in listing 8.11 and uses the same ordinary movement calls, prompts, and resources. Start `MCPServerWithHttp` separately before the client. Unlike the `STDIO` client, the full HTTP program invokes its agent unconditionally.

EXERCISE

To extend the HTTP sample, add a `MaintenanceTools` class (see listing 8.16) that performs system maintenance checks such as the motors and tire pressure. If applying this exercise to `STDIO` instead, extend its existing `MaintenanceTools` class rather than creating a second one or replacing its diagnostics methods.

Listing 8.16 MaintenanceTools.cs

```
using ModelContextProtocol.Server;
using Serilog;
using System.ComponentModel;

[McpServerToolType, Description("Maintenance tools for robot car.")]
public class MaintenanceTools // ❶
{
    private const int Delay = 500;

    [McpServerTool(Name = "motors_check"), Description("Checks the motors of the robot car.")]
    public async Task<string> CheckMotorsAsync() // ❷
    {
        var random = new Random();
        var motorStatus = random.Next(0, 100);
        Log.Information("MAINTENANCE: CHECKING motors. Status: {motorStatus}%",
            motorStatus);
        await Task.Delay(Delay);
        return $"Motors checked. Status: {motorStatus}% efficiency.";
    }

    [McpServerTool(Name = "tire_check"),
        Description("Checks the tire pressure of the robot car.")]
    public async Task<string> CheckTirePressureAsync() // ❸
    {
        var random = new Random();
        var pressure = random.Next(30, 35);
        Log.Information("MAINTENANCE: CHECKING tire pressure: {pressure} PSI",
            pressure);
        await Task.Delay(Delay);
        return $"Tire pressure is {pressure} PSI.";
    }
}
```

- ❶ Declares the `MaintenanceTools` class
- ❷ Checks the robot car motors
- ❸ Checks the tire pressure

Then, register the tools in the MCP Server project next to the transport type settings. The STDIO server already has this registration.

```
.WithTools<MaintenanceTools>();
```

Then, add this instruction to the existing prompt in the MCP Client project, to make sure the AI model will call the matching tools:

```
First, check the motors and tire pressure.
```

When running the MCP Client, you should notice in the response the maintenance check logs in the MCP Server, along with the maintenance check outputs in the MCP client console.

8.5.5 MCP Client with Tasks

We explained above that an MCP tool can run as a background task when the client opts in. The `ModelContextProtocol.Extensions.Tasks` provides automatic polling, as in listing 8.13, and lower-level methods for applications that need to keep the task ID and inspect its state.

PRACTICAL EXAMPLE

In listing 8.17 we inspect `run_diagnostics` through the manual task API. Use the connected STDIO client and the imports from listing 8.14. This is an optional expansion of the automatic polling example, not another block already present in the repository client. It is scoped to diagnostics, which does not ask the caller for additional input.

Code path: `/MCPClientWithStdio/Program.cs`

Listing 8.17 MCP client with STDIO transport

```
using var taskTimeout = new CancellationTokenSource(TimeSpan.FromSeconds(30));
var call = await mcpClient.CallToolAsTaskAsync(
    new CallToolRequestParams { Name = "run_diagnostics" },
    taskTimeout.Token); //❶
if (!call.IsTask)
{
    var text = call.Result!.Content.FirstOrDefault() as TextContentBlock;
    Console.WriteLine($"TOOL RESULT: {text?.Text}");
    return;
}

var task = call.TaskCreated!;
Console.WriteLine($"TASK STARTED: {task.TaskId} | Status: {task.Status}");
GetTaskResult state;
do
{
    await Task.Delay(TimeSpan.FromMilliseconds(task.PollIntervalMs ?? 1000),
        taskTimeout.Token);
    state = await mcpClient.GetTaskAsync(task.TaskId, taskTimeout.Token); //❷
} while (state.Status == McpTaskStatus.Working);

if (state is CompletedTaskResult completed)
{
    JsonElement result = completed.Result; //❸
    Console.WriteLine($"TASK DONE: {completed.Status} | RESULT: {result}"); //❹
}
else
{
    throw new InvalidOperationException($"Diagnostics ended with status {state.Status}.");
}
```

- ❶ Requests task execution and handles an ordinary result if no task is created
- ❷ Polls the task state using `GetTaskAsync`

- ❶ Reads the result included in `CompletedTaskResult`
- ❷ Prints the terminal state and returned result

This code uses `ResultOrCreatedTask<CallToolResult>`, checks `IsTask`, and reads completion data from `GetTaskAsync`. A general client must also handle `InputRequired`, `Failed`, and `Cancelled`; use `CallToolWithPollingAsync` for the built-in lifecycle handling. `Completed` can still contain a tool result with `IsError = true`. Keep the client and server alive until the operation finishes.

Code output (task IDs vary):

```
TASK STARTED: 08de7ed19d6138460000000000000001 | Status: Working
TASK DONE: Completed | RESULT: {"content":[{"type":"text","text":"Diagnostics
complete. All 4 motors passed."}]}
```

EXERCISE

Let's simplify listing 8.17 by replacing the manual loop with `CallToolWithPollingAsync` from listing 8.13. Compare the same final result with and without the task-ID output. Bonus: after obtaining a task ID, request cancellation with `CancelTaskAsync`; cancelling only the local timeout token does not guarantee that server work stops.

8.6 Benefits

MCP offers compelling advantages that extend beyond simple capability sharing. While Agent Framework AI tools excel at rapid, in-process development, MCP introduces architectural patterns that solve fundamental challenges in AI system design: capability reuse across platforms, security isolation, and operational flexibility. Understanding when to choose MCP over traditional AI tools require examining the trade-offs between development simplicity and system architecture benefits. The following analysis explores these considerations to help you make informed decisions about integrating MCP into your AI applications.

8.6.1 MCP Versus Local AI Tools

Choosing between local Agent Framework AI tools and MCP servers requires evaluating multiple factors that impact development speed, runtime performance, operational complexity, and long-term maintainability.

In Figure 8.5, the AI tool-based architecture used in Agent Framework applications is depicted. The diagram shows how an app can use multiple tool sets (formerly known as plugins in Semantic Kernel), each containing a set of native and/or semantic functions. AI tools run in-process and act as modular building blocks, enabling the app to organize domain-specific capabilities efficiently.

AI Tool-Based Architecture

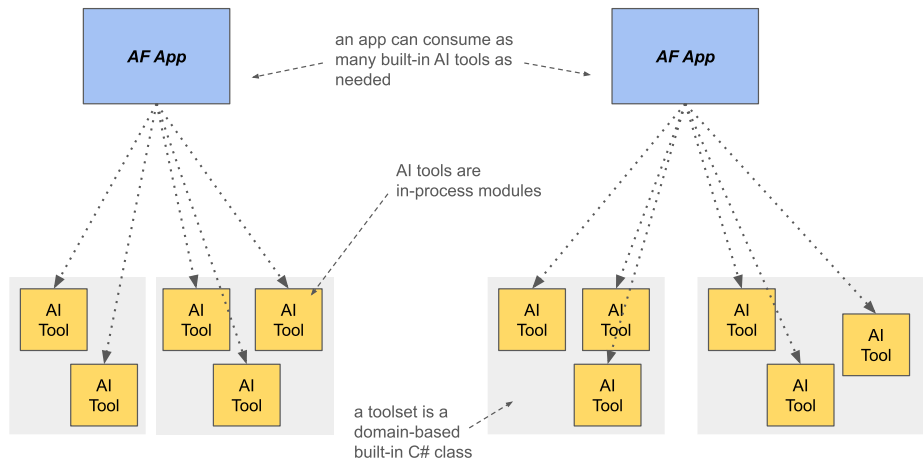


Figure 8.5 AI tool-based architecture showing an app consuming various in-process tool sets, each grouping related semantic and/or native functions for modular capability expansion.

Figure 8.6 highlights the key differences between AI tool-based and MCP-based architectures. Unlike tool sets, which are in-process modules directly embedded in the app, MCP services are out-of-process and connect to the app as external, domain-based services. MCP-based architecture allows your app to access standardized sets of tools, prompts, and resources across process boundaries, offering greater modularity and deployment flexibility compared to the in-process nature of tool sets.

MCP-Based Architecture

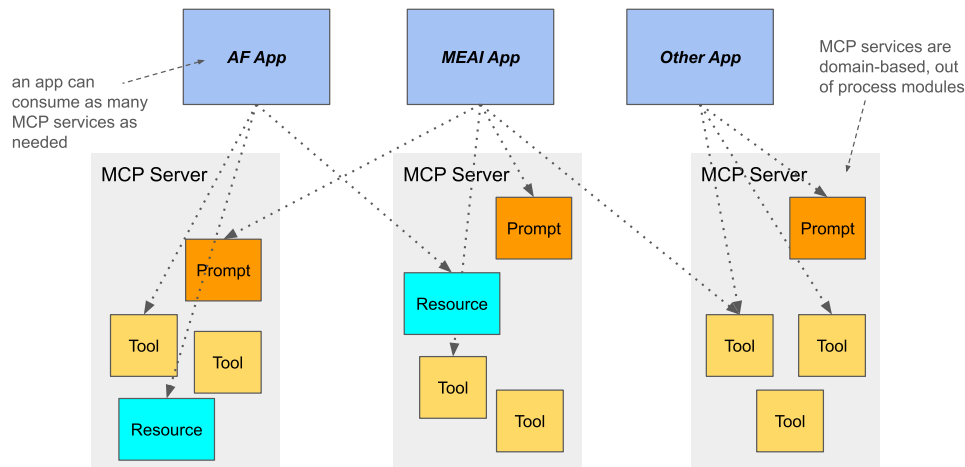


Figure 8.6 Comparison of AI tool-based (in-process, built-in) and MCP-based (out-of-process, external) architectures, emphasizing how MCP enables extension through standardized external services rather than

internal tool sets.

Let’s look at the differences between AI tool-based architecture and MCP-based architecture in table 8.3.

Table 8.3 MCP AI Tools vs. AI Tools

| Aspect | Agent Framework AI Tools | MCP AI Tools |
|-------------------------|-----------------------------------|--|
| Setup Complexity | Simple, just reference assemblies | Moderate, separate processes to manage |
| Performance | Fastest, in-process calls | Slower, inter-process calls (over transport) |
| Isolation | Shared process and memory | Complete isolation, it crashes don't spread |
| Reusability | .NET/C# applications only | Any MCP-compatible AI system |
| Debugging | Standard .NET debugging | Requires cross-process debugging |
| Security | Trusts AI tool code completely | Separate process/deployment boundary |
| Dependencies | Must match host application | Independent dependency management |

TIP Start with Agent Framework AI tools for rapid development. Graduate to MCP servers when you need to share capabilities across different AI systems or when security isolation becomes important.

Choose MCP when:

- Sharing tools across different AI systems or languages
- Security isolation from untrusted code is required
- Tools need independent scaling, deployment, or versioning
- Building enterprise systems with compliance requirements

Otherwise, start with Agent Framework AI tools for simplicity.

As AI systems become more capable, the ability to safely share tools and context across different applications becomes critical. MCP provides the standardized foundation for this interconnected AI future.

MCP doesn't replace Agent Framework AI tools but extends their reach beyond your application boundaries.

The protocol transforms your carefully crafted functions from private application assets into publicly discoverable AI capabilities, ready to be orchestrated by any compatible system in the expanding AI ecosystem.

Summary

- MCP standardizes how AI systems discover and consume capabilities across organizational and technical boundaries, solving capability fragmentation.
- STDIO transport enables local development with minimal setup

- HTTP supports production scenarios with stateful or stateless operational modes.
- Stream transport provides advanced custom inter-process communication using named pipes and in-memory streams for specialized scenarios.
- Build MCP servers using attributes (`McpServerToolType`, `McpServerTool`) and register components with automatic or manual discovery methods.
- MCP clients dynamically discover tools, prompts, and resources at runtime, enabling flexible orchestration of both local and remote capabilities.
- Stateful HTTP mode preserves session-dependent behavior for older, initialize-based clients.
- Stateless HTTP mode supports independent requests and request-scoped streaming; task storage and polling are configured separately.
- The `ModelContextProtocol.Extensions.Tasks` uses `WithTasks` and per-request opt-in; ordinary progress calls and task polling are separate patterns.
- The current STDIO client runs simple, progress, and polling demonstrations before an optional agent run; the HTTP sample retains its ordinary-call flow.
- Choose MCP when sharing capabilities across systems, requiring security isolation, or needing independent deployment and versioning of tools.

9

Building enterprise-ready agents with ChatClient middleware

This chapter covers

- Why middleware is essential for production AI agents
- The two-layer middleware architecture: ChatClient vs Agent
- Three middleware types: SharedFunction, Response, and FunctionCalling
- Composing a complete middleware pipeline

Robby worked well in development because nothing bad was at stake. In production, the same agent became dangerous: no token limits, no input/output data redaction, and no guard around tool calls. The root problem was architectural; the agent talked straight to the LLM. Low-level *ChatClient* middleware is the caller-agnostic layer between any chat client and the model where you attach shared, organization-wide infrastructure guardrails for all agents without cluttering their logic with cross-cutting concerns. It turns a “works in dev” agent into an enterprise-ready one by enforcing global guardrails for cost, safety, and basic sanitization without rewriting the agent.

9.1 Applying ChatClient Middleware to Agents

Robby’s AI agent worked flawlessly in development. Then production happened, and that’s when we learned that *middleware*, the layer that sits between our chat calls and the LLM to enforce guardrails, is not optional. It is essential.

If we come from ASP.NET, this should feel familiar. HTTP middleware wraps requests and responses in a pipeline. Here we apply the same idea to chat calls and LLM responses: middleware surrounds every call and handles cross-cutting concerns such as validation, logging, budgets, and safety.

9.1.1 Explaining the Middleware Pipeline Pattern

Middleware looks abstract until you see how a request actually flows through the pipeline. The next two diagrams show the two essential cases you will use throughout this chapter.

HAPPY-FLOW MIDDLEWARE

In the normal case as shown in figure 9.1, every piece of middleware lets the call pass through to the LLM and then sees the response on the way back.

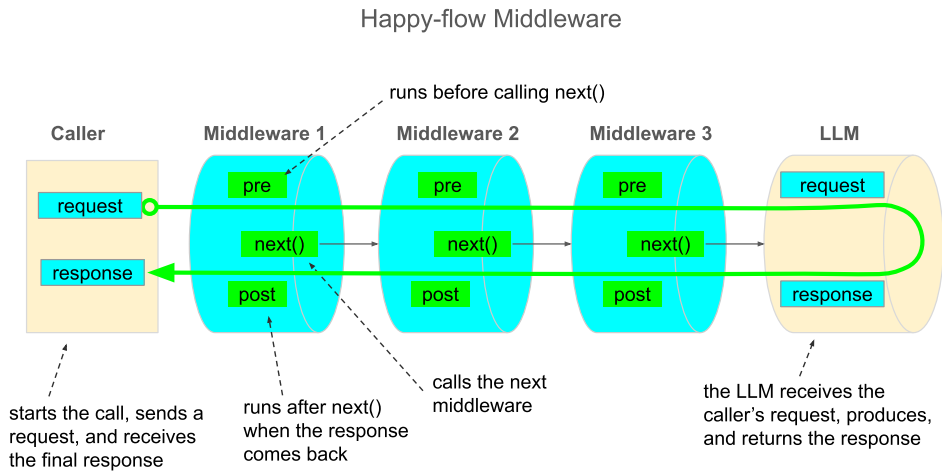


Figure 9.1 The middleware pipeline wraps the LLM call: each middleware runs pre logic, calls `next()`, then runs post logic as the response returns, so every component executes on both the way in and the way out.

In the happy-flow diagram, the caller starts the operation by sending a request into the pipeline. *Middleware 1* receives that request and runs its pre logic first (e.g., logging, input shaping, or attaching metadata). It then calls `next()`, which passes control to the next middleware in the chain rather than to the LLM directly.

Middleware 2 and *Middleware 3* repeat the same pattern. Each one gets the request, runs its own pre logic, and calls `next()` to move the request further down the pipeline. Eventually the request reaches the LLM, which receives the caller's request, produces an answer, and returns a response.

On the way back, the response travels through the pipeline in reverse order. *Middleware 3* runs its post logic, then returns control to *Middleware 2*, which runs its own post logic, and finally *Middleware 1* does the same before the response reaches the caller. In this happy flow, every middleware in the chain runs on both sides of the call: once before `next()` and once after `next()` completes.

SHORT-CIRCUIT MIDDLEWARE

Sometimes middleware needs to stop the operation early (e.g., when a token budget is exhausted or a user is not authorized) as seen in figure 9.2.

Short-circuit Middleware

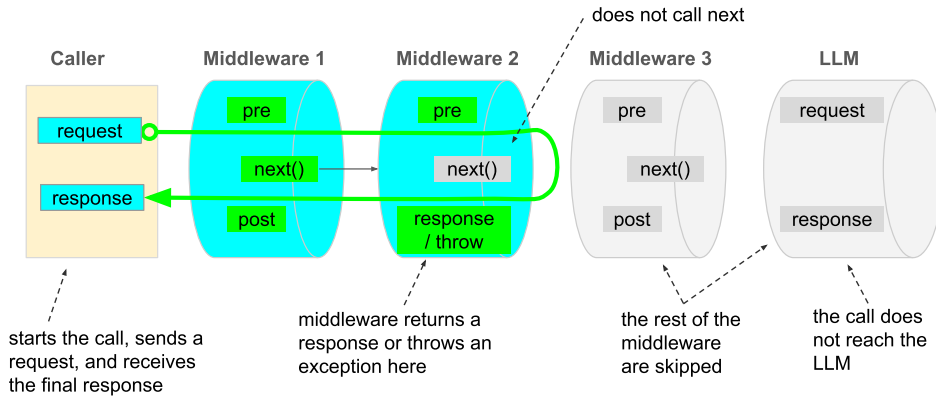


Figure 9.2 Any middleware can short-circuit the pipeline by not calling `next()` and instead returning a response or throwing, so later middleware and the LLM are skipped and only earlier middleware run their post logic.

The short-circuit diagram shows the same pipeline, but this time *Middleware 2* decides not to call `next()`. The caller still sends a request and *Middleware 1* still runs its pre logic and calls `next()`, so the request reaches *Middleware 2* in the usual way. *Middleware 2* runs its own pre logic, but instead of forwarding the request, it either returns a response directly or throws an exception. Middleware can handle the exception and finally blocks run, whereas ordinary post-code runs only when `next` completes successfully.

At that moment, the pipeline ended. *Middleware 3* and the LLM are skipped entirely because `next()` was never invoked in *Middleware 2*. The response (or error) flows back from *Middleware 2* to *Middleware 1*, which can still run its post logic, and then back to the caller. This short-circuiting behavior is what lets you implement guardrails such as token budgets, authorization checks, or validation rules: a middleware can stop the operation early and return a controlled outcome without changing any of the downstream middleware or the agent itself.

9.1.2 Defining ChatClient Middleware

ChatClient middleware is the production layer that sits between a chat client call and the AI model. It does not define the agent's reasoning or behavior, but it enforces how that behavior is executed in a real system.

A useful way to think about middleware is as the safety systems in a car. The engine (the agent) provides power, but safe driving depends on the surrounding control systems:

- Seatbelts (validation) prevent bad or malformed inputs from causing damage.
- Airbags (error handling) absorb failures, so a single fault doesn't bring everything down.
- Speed limiters (rate limiting) keep calls, costs, and side effects within safe bounds.
- Dashcam (audit logging) records what happened, when, and why for debugging, compliance, and incident response.
- Collision warning (human approval) pauses before high-risk actions and asks a human to

confirm.

In production systems, these safeguards are not optional. Middleware provides the guardrails that let us operate powerful agents safely, keeping control over risk, cost, and compliance.

9.1.3 Visualizing the Two-Layer Middleware Architecture

In Agent Framework, middleware operates at two distinct layers: the *ChatClient* layer (`IChatClient` from `Microsoft.Extensions.AI`) and the *Agent* layer (`Microsoft.Agents.AI`).

The diagram in figure 9.3 shows a two-layer middleware architecture, where both layers operate independently but can be combined.

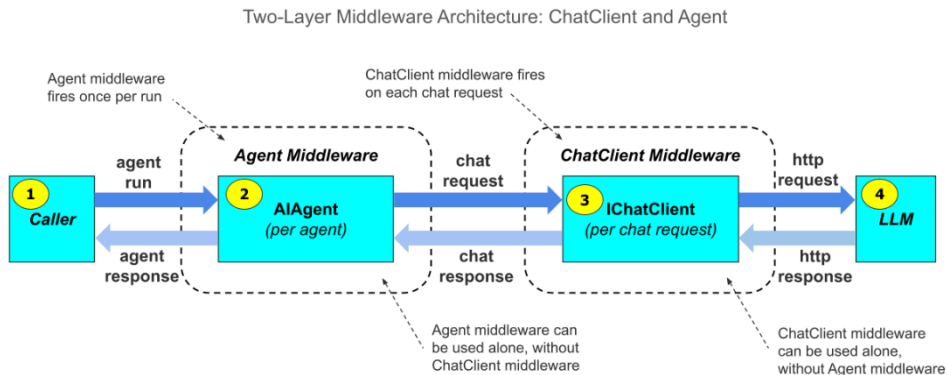


Figure 9.3 Diagrams show the two-layer middleware model. Agent middleware is at higher level, and it runs once per `RunAsync()` call (agent run), and ChatClient middleware is a lower level, and it runs on each call to LLM (chat request).

These are the two-layer diagram steps:

- **Step 1: Caller Invokes Agent**
The caller initiates the flow on the AI Agent (component 2). This triggers the execution pipeline, starting with any registered agent middleware.
- **Step 2: Agent Middleware Executes**
Agent middleware wraps the entire agent running lifecycle. It fires once per agent invocation, allowing inspection and modification of both the incoming chat request and outgoing agent response. Importantly, agent middleware can run standalone without requiring *ChatClient* middleware.
- **Step 3: ChatClient Middleware Executes**
After the agent processes the request, it sends a chat request to the `IChatClient` (component 3). At this point, *ChatClient* middleware intercepts the HTTP request to LLM. This middleware fires on each chat request, meaning it can execute multiple times within a single agent run if the agent makes multiple model calls. *ChatClient* middleware can also operate independently, without *Agent* middleware.
- **Step 4: LLM Interaction**
The `IChatClient` sends the HTTP request to the LLM (component 4) and receives the HTTP response. The response then flows back through the middleware chain in reverse order: first

through any *ChatClient* middleware for post-processing, then through agent middleware, and finally back to the caller.

This two-layer design provides fine-grained control over where we intercept execution, allowing different concerns to be handled at the appropriate level.

NOTE ChatClient layer is your universal safety net, it sees every chat request to the LLM and applies shared behavior across all agents. The Agent layer is your fine-grained control, it knows which agent, which session, and enforces per-agent instructions.

In this chapter, we focus on the lower layer, *ChatClient* middleware.

SOLVING PRODUCTION PROBLEMS WITH MIDDLEWARE

Middleware handles everything that isn't core agent logic but is absolutely essential for production (table 9.1):

Table 9.1 Production scenarios handled using middleware

| Production Concern | Without Middleware | With Middleware |
|---------------------|---|-------------------------------------|
| Security | PII (Personally Identifiable Information) sent to LLM providers | Automatically redacted |
| Cost Control | Unlimited API calls or token usage | Enforced quotas and budgets |
| Compliance | No audit trail | Complete logging with timestamps |
| Safety | All operations auto-approved | Human-in-the-loop for dangerous ops |
| Performance | No visibility into bottlenecks | Metrics and monitoring |
| Reliability | Errors cascade unpredictably | Graceful handling and recovery |

Middleware does not add new capabilities to the agent. It centralizes guardrails such as cost limits, data protection, tool supervision, and auditing, so production risks are handled outside the agent definition.

RUNNING AGENTS WITHOUT MIDDLEWARE

Before adding any middleware, let's see what happens when agents run unprotected. The baseline project demonstrates all three stories in action: email leaking to the LLM provider, no cost control, and dangerous operations executing without validation.

In listing 9.1 we have a simple agent encountering a few challenges mentioned before.

Requirements (NuGet packages):

```
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
dotnet add package Microsoft.Agents.AI.OpenAI
```

Code path: /MiddlewareBaseline/Program.cs

Listing 9.1 Agent without Middleware

```
using AITools;
using Helpers;
using Microsoft.Agents.AI;
using Microsoft.Extensions.AI;
using Microsoft.Extensions.Configuration;
using OpenAI;
```

```

var configuration = new ConfigurationBuilder()
    .AddUserSecrets<Program>().Build();

var model = configuration["OpenAI:ModelId"];
var apiKey = configuration["OpenAI:ApiKey"];

IChatClient chatClient = new OpenAIClient(apiKey)
    .GetChatClient(model)
    .AsIChatClient(); //❶
// NO .AsBuilder() - NO middleware pipeline!

ChatClientAgent motorsAgent = chatClient
    .AsAIAgent(new ChatClientAgentOptions
    {
        Name = "MotorsAgent",
        Description = "Controls robot car movements.",
        ChatOptions = new ChatOptions
        {
            Instructions = """
                You are the MotorsAgent controlling a robot car.
                Break down complex movement commands into basic moves:
                forward, backward, turn left, turn right, stop.
                Respond with the sequence of moves needed to accomplish the task.
                """,
            Tools = [.. MotorTools.AsAITools()],
        }
    });

AgentSession session = await motorsAgent.CreateSessionAsync();

Console.WriteLine("""
    TEST 1: Email Leak – Step 1 gap (no RemoveEmail)
    ChatClientSharedFunctionMiddleware would redact the address.
    """);
var prompt1 = """
    Navigate to original position.
    Contact me at john.doe@example.com for updates.
    """;
Console.WriteLine($"PROMPT: {prompt1}");
AgentResponse result1 = await motorsAgent
    .RunAsync(prompt1, session); //❷
Console.WriteLine($"RESULT: {result1}\n");
Console.WriteLine("""
    >>> The email was NOT redacted –
    sent directly to the LLM provider (GDPR violation).
    """);

Console.WriteLine("""
    TEST 2: Unlimited Spend –
    Step 1 + Step 2 gap (no LimitRequests, no EnforceTokenBudget)
    ChatClientSharedFunctionMiddleware would limit requests.
    ChatClientResponseMiddleware would enforce a token budget.
    """);

var prompt2 = "Move forward 3 meters, turn right 90 degrees, move forward 3
meters";
Console.WriteLine($"PROMPT: {prompt2}");
AgentResponse result2 = await motorsAgent
    .RunAsync(prompt2, session); //❸
Console.WriteLine($"RESULT: {result2}\n");
Console.WriteLine("""
    >>> No request limit hit, no token budget enforced –
    costs accumulate silently.
    """);

```

```

Console.WriteLine("""
TEST 3: Unconstrained Backward -
Step 3 gap (no ConstrainDistance, no AuditFunctionCalling)
ChatClientFunctionCallingMiddleware would cap backward to 5 m
and audit every tool call.
""");

var prompt3 = "Move forward 10 meters then go backward 10 meters";
Console.WriteLine($"PROMPT: {prompt3}");
AgentResponse result3 = await motorsAgent
.RunAsync(prompt3, session); //4
Console.WriteLine($"RESULT: {result3}\n");
Console.WriteLine("""
>>> Backward ran the full 10 m - no constraint, no audit.
The robot could hit a wall.
""");

```

- ❶ ChatClient that has NO middleware protection
- ❷ Queries the agent with a sensitive input data (email)
- ❸ Queries the agent with no requests or budget limit
- ❹ Queries the agent with no audit or action supervision

Code output:

```

TEST 1: Email Leak - Step 1 gap (no RemoveEmail)
ChatClientSharedFunctionMiddleware would redact the address.
PROMPT: Navigate to original position.
Contact me at john.doe@example.com for updates.

RESULT: stop

>>> The email was NOT redacted -
sent directly to the LLM provider (GDPR violation).
TEST 2: Unlimited Spend -
Step 1 + Step 2 gap (no LimitRequests, no EnforceTokenBudget)
ChatClientSharedFunctionMiddleware would limit requests.
ChatClientResponseMiddleware would enforce a token budget.
PROMPT: Move forward 3 meters, turn right 90 degrees, move forward 3 meters
[01:37:34:935] MOTORS: Forward: 3m
[01:37:37:099] MOTORS: TurnRight: 90°
[01:37:39:217] MOTORS: Forward: 3m
[01:37:41:095] MOTORS: Stop

RESULT: forward 3
turn right 90
forward 3
stop

>>> No request limit hit, no token budget enforced - costs accumulate silently.
TEST 3: Unconstrained Backward -
Step 3 gap (no ConstrainDistance, no AuditFunctionCalling)
ChatClientFunctionCallingMiddleware would cap backward to 5 m
and audit every tool call.
PROMPT: Move forward 10 meters then go backward 10 meters
[01:37:43:921] MOTORS: Forward: 10m
[01:37:48:027] MOTORS: Backward: 10m
[01:37:49:975] MOTORS: Stop

RESULT: forward 10
backward 10
stop

>>> Backward ran the full 10 m - no constraint, no audit.
The robot could hit a wall.

```

All three tests expose the same root problem: the agent is an unguarded conduit between the caller and the LLM. Sensitive data travels to the LLM provider unchanged, every command executes

immediately regardless of risk, and nothing stands between a runaway request loop and a five-figure cloud bill.

The critical insight is that these concerns are cross-cutting. They apply to every agent, every session, and every LLM call, yet none of them belong in business logic. This is exactly the problem middleware is designed to solve, following the principle of *Separation of Concerns*: keeping different responsibilities in various parts of the system so each part has a clear, focused job. Middleware intercepts the pipeline at the right layer, enforces rules once, and protects every agent transparently, without touching a single line of agent.

9.2 Walking Through ChatClient Middleware Layer

ChatClient middleware fires on each chat request between `IChatClient` and the LLM, operating without agent session context or identity. *ChatClient* is agent-independent. This lack of context is deliberate: it treats every invocation as an independent HTTP call, applying shared behavior uniformly across all agents. Without *ChatClient* middleware layer, agents talk directly to LLM with no place to protect input or output, supervise tool calls, or audit what happened.

Diagram in figure 9.4 shows the *ChatClient* middleware.

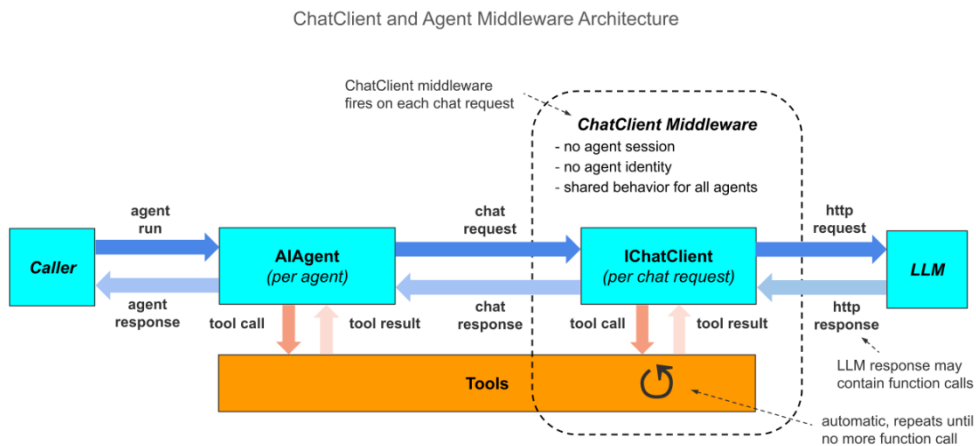


Figure 9.4 ChatClient middleware in tool-calling flow: operates without agent context, intercepts every chat request to the LLM, and fires repeatedly during iterative tool execution loops.

During tool calling, the flow becomes iterative. When the LLM returns function calls in the chat request area, the agent executes them via the *Tools* area and sends results back through `IChatClient`. This loop repeats automatically until no more function calls are requested. *ChatClient* middleware intercepts every round trip, potentially firing multiple times per agent run. The chat request area handles both responses and shared functions, while the *Tools* area manages function calling execution. This interception prevents issues like unredacted sensitive data reaching the LLM, unconstrained tool calls causing damage, and uncontrolled token consumption accumulating costs.

To address these concerns effectively, *ChatClient* middleware is implemented using three distinct types, each targeting a specific stage in the chat request-response lifecycle.

9.2.1 Choosing the Right ChatClient Middleware Type

Robby’s production issues fall into three buckets: what we send to the LLM, what we get back, and how tools are invoked. *ChatClient* middleware mirrors this with three types, so we must pick the right one based on whether we need to shape the request, inspect the response, or supervise a tool call. *SharedFunction* prepares outgoing requests (sanitizing or redacting data before it reaches the LLM). *Response* handles incoming responses (tracking tokens, enforcing budgets, or inspecting content). *FunctionCalling* supervises tool invocations (constraining arguments, auditing calls, or blocking unsafe executions). Together, these types cover the prepare-handle-invoke pattern across the chat request area and tool execution flow.

Figure 9.5 breaks down the three *ChatClient* middleware types side by side, showing their inputs, outputs, and where each sit relative to the LLM response.

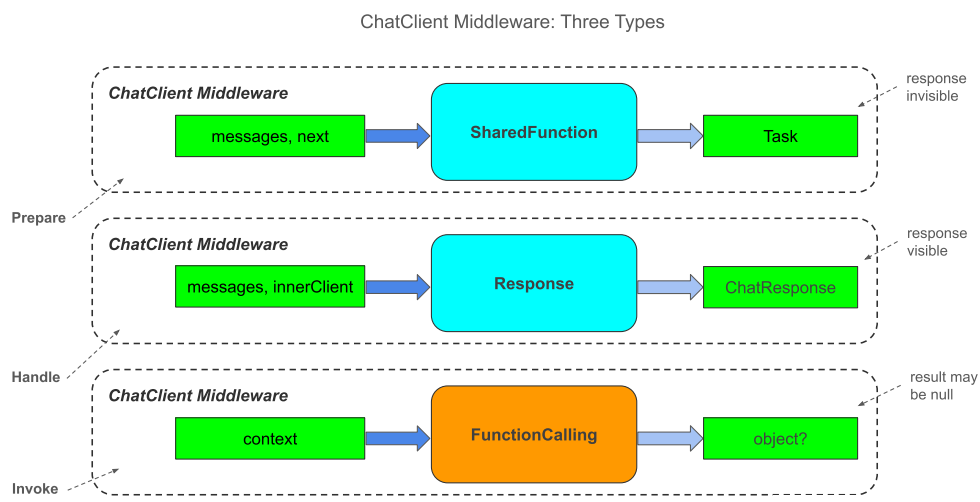


Figure 9.5 The diagram shows the three types of ChatClient middleware: SharedFunction, Response, and FunctionCalling.

The three types form a layered progression. *SharedFunction* is the lightest: it can inspect and modify input messages before they reach the LLM, but it never sees the response. *Response* wraps the entire LLM call, giving you access to both the request and the full `ChatResponse`, including token usage and latency. *FunctionCalling* is specialized: it does not participate in the LLM round-trip at all. Instead, it intercepts each tool invocation, letting you validate, modify, or reject individual function calls before they execute.

- *SharedFunction*: **prepares** the input (blind to `ChatResponse`)
- *Response*: **handles** the input and output (sees `ChatResponse`)
- *FunctionCalling*: **invokes** each tool (innermost, fires per tool call)

We’ll build up the middleware pipeline gradually. *SharedFunction* is the simplest type: it controls what goes into the LLM but is structurally blind to what comes out (`ChatResponse`). *Response* unlocks the response; you get both input and output control. *FunctionCalling* is a different category entirely: it doesn’t intercept LLM calls at all, it intercepts individual tool invocations (conventional code).

9.2.2 Preparing Requests with SharedFunction Middleware

SharedFunction uses the classic `next`-delegate middleware pattern. Each *SharedFunction* component receives messages, options, a next delegate, and a cancellation token. It can transform the messages, throw an exception to block the request, or call `await next(...)` to pass control to the `next` middleware and later resume execution after that call. The signature encodes this pattern. The `next` delegate returns `Task` (void). There is no `ChatResponse` value to return or modify here, which is why `ChatResponse` never appears in this middleware type.

Figure 9.6 shows the *SharedFunction* middleware type.

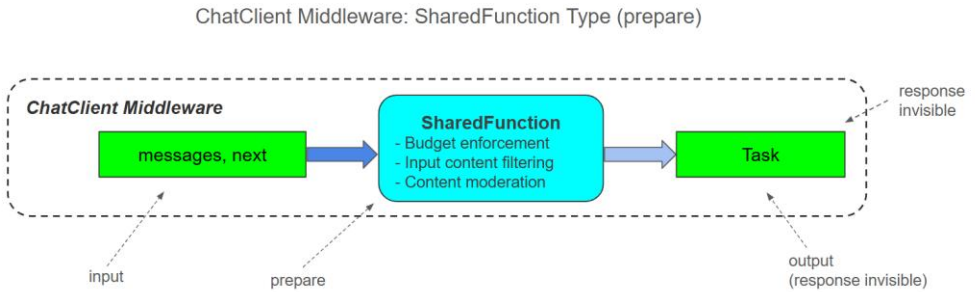


Figure 9.6 The *SharedFunction* middleware receives messages and a next delegate. The *SharedFunction* returns no value, and there is no `ChatResponse` to read or alter. Typical use cases include request limiting, input content filtering, prompt injection detection, and content moderation. All of these run before any token is consumed.

Registration order defines the pipeline order: each middleware wraps the next one, and if it does not call `next`, the pipeline short-circuits. Typical use cases include request limiting, input content filtering, and prompt injection detection. All of these run before any token is consumed.

This makes *SharedFunction* the natural first layer: validate and adjust the input at this level. If something is wrong with the request, we fail here, before tokens are consumed and before the response path runs at all.

PRACTICAL EXAMPLE: EMAIL REMOVAL

Let me tell you a production story: The GDPR Nightmare Story.

For a European client, we deploy Robby as an intelligent delivery robot car in their office building. Employees could summon Robby, providing their name, office number, and phone number for delivery confirmation. Six months later, a security audit revealed that every employee's phone number, email, and office location had been sent to their LLM provider and logged in plain text.

Robby worked correctly. It just never sanitized the data before processing. This is what we want:

Input: "Navigate to GPS. Contact me at john@example.com for updates."

Output: "Navigate to GPS. Contact me at [REDACTED-EMAIL] for updates."

Let's see in listing 9.2 how we can prevent sensitive information leakage.

Code path: `/ChatClientSharedFunctionMiddleware/ChatClientSharedFunctions.cs`

Listing 9.2 RemoveEmail

```

public static class ChatClientSharedFunctions
{
    private const string EmailPattern =
        @"\"b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\"b"; // ❶
    private const string EmailMask = "[REDACTED-EMAIL]"; // ❷
}
  
```



```

public static async Task RemoveEmail(
    IEnumerable<ChatMessage> messages,
    ChatOptions? options,
    Func<IEnumerable<ChatMessage>, ChatOptions?,
        CancellationToken, Task> next,
    CancellationToken cancellationToken) // ❸
{
    Console.WriteLine("[ChatClient] [SharedFunction] [Email] "
        + $"PRE: Scanning {messages.Count()} messages...");

    bool emailFound = false; // ❹
    List<ChatMessage> sanitizedMessages = [];

    foreach (var message in messages)
    {
        if (message.Role == ChatRole.User && message.Text is not null) // ❺
        {
            var sanitized = Regex
                .Replace(message.Text, EmailPattern, EmailMask);
            if (sanitized != message.Text) emailFound = true;
            sanitizedMessages.Add(new ChatMessage(message.Role, sanitized));
        }
        else
        {
            sanitizedMessages.Add(message);
        }
    }

    Console.WriteLine(emailFound
        ? "[ChatClient] [SharedFunction] [Email] Email detected and removed!"
        : "[ChatClient] [SharedFunction] [Email] No email found");

    await next(sanitizedMessages, options, cancellationToken); // ❻

    Console.WriteLine($"[ChatClient] [SharedFunction] [Email] POST: "
        + "Completed");
}
}

```

- ❶ Defines Regex pattern standard email address format
- ❷ Defines replacement mask for any detected email
- ❸ Defines the SharedFunction
- ❹ Tracks whether any email was found
- ❺ Only user messages are sanitized, tool calls and tool results are not
- ❻ Passes sanitized messages downstream

Let's see in listing 9.3 how to wire it into the *ChatClient* pipeline.

Requirements (NuGet packages):

```

dotnet add package Microsoft.Extensions.Configuration.UserSecrets
dotnet add package Microsoft.Agents.AI.OpenAI

```

Code path: /ChatClientSharedFunctionMiddleware/Program.cs

Listing 9.3 SharedFunction Middleware

```

// 'usings' and API key fetching omitted for brevity

IChatClient chatClient = new OpenAIClient(apiKey)
    .GetChatClient(model)
    .AsIChatClient()
    .AsBuilder() // ❶
    .Use(ChatClientSharedFunctions.RemoveEmail) // ❷
    .Build(); // ❸

ChatClientAgent motorsAgent = chatClient
    .AsAIAgent(new ChatClientAgentOptions

```

```

{
    Name = "MotorsAgent",
    Description = "Controls robot car movements.",
    ChatOptions = new ChatOptions
    {
        Instructions = """
            You are an AI assistant controlling a robot car capable of performing
            basic moves: forward, backward, turn left, turn right, and stop.
            You have to break down the provided complex commands into the basic
            moves you know.
            Use a JSON array like [move1, move2, move3] for the response.
            Respond only with the moves and their parameters (angle or distance),
            without any additional explanations.
            """
    }
}); //❹

AgentSession session = await motorsAgent.CreateSessionAsync();

var prompt = """
    There is a tree directly in front of the car.
    Avoid it and then return to the original path.
    """;
Console.WriteLine($"PROMPT: {prompt}");
var result = await motorsAgent.RunAsync(prompt, session); //❺
Console.WriteLine($"RESULT: {result}");

```

- ❶ Converts `IChatClient` into a builder to enable middleware registration
- ❷ Registers `RemoveEmail`
- ❸ Builds the `IChatClient` with the middleware pipeline applied
- ❹ Creates the agent
- ❺ Runs the agent

Email sanitization is now in place, so Robby will redact an email address in the user message to the LLM provider. But Robby can still make unlimited LLM calls. The \$10,847 bill is still entirely possible. The next middleware adds the request limit guard, and once both are defined, we will compose them into a single pipeline and see how they interact.

NOTE Transient vs. persistent sanitization. At the *ChatClient* layer, sanitization is *transient*: the LLM sees clean data, but the agent's session chat history retains the original text (email stays intact). This is safe because the middleware re-applies transparently on every LLM round-trip. Contrast with the *Agent* layer (as we will find out in the next chapter where we discuss *Agent* middleware layer), where such `RemoveEmail` function is *persistent*, sanitized messages are stored in the session history, so all future round-trips already see clean data without re-sanitization. At the *ChatClient* layer, the session is not accessible, so transient re-application is the only option, but it is the developer's responsibility to choose which middleware layer fits best the desired use-case.

PRACTICAL EXAMPLE: REQUEST LIMITING

Let me tell you a production story: The \$10,000 Weekend.

Late on a Friday afternoon, Robby navigated obstacles, followed voice commands, and captivated visitors. On Monday morning, the CTO opened an email from OpenAI. The weekend bill was \$10,847. The cause was simple but costly. A bug in Robby's retry logic had triggered 47,000 API calls as it continuously re-evaluated its environment. Without middleware to enforce rate or request limits, nothing stopped it.

Listing 9.4 shows how such a costly incident can be prevented.

Code path: `/ChatClientSharedFunctionMiddleware/ ChatClientSharedFunctions.cs`

Listing 9.4 LimitRequests

```
public static class ChatClientSharedFunctions
{
    private static int _requestCount = 0; //❶
    private const int MaxRequests = 2; //❷

    public static async Task LimitRequests(
        IEnumerable<ChatMessage> messages,
        ChatOptions? options,
        Func<IEnumerable<ChatMessage>, ChatOptions?, CancellationToken, Task> next,
        CancellationToken cancellationToken) //❸
    {
        var currentCount = Interlocked.Increment(ref _requestCount); //❹

        Console.WriteLine($"[ChatClient] [SharedFunction] [Limit] PRE: "
            + "Request {currentCount} of {MaxRequests} max");

        if (currentCount > MaxRequests) //❺
        {
            throw new LimitExceededException(currentCount, MaxRequests); //❻
        }

        await next(messages, options, cancellationToken); //❼

        Console.WriteLine($"[ChatClient] [SharedFunction] [Limit] POST: "
            + "Request {currentCount} of {MaxRequests} max completed");
    }
}
```

- ❶ Declares shared counter across all LLM round-trips
- ❷ Declares maximum number of LLM round-trips allowed
- ❸ Declares SharedFunction
- ❹ Thread-safe increment
- ❺ Checks the request budget
- ❻ Throws a custom exception carrying current count and limit
- ❼ Limit not exceeded, continue the pipeline toward the LLM

Key teaching points (for listing 9.4):

- `Interlocked.Increment` for thread-safe counting across concurrent agents
PRE phase: check requests limit, throw custom `Exception` if exceeded — the request never reaches the LLM
- `await next(messages, options, cancellationToken)` continues the pipeline
POST phase: log completion after the entire inner pipeline has returned

WARNING With tool-calling agents, a single user prompt triggers *multiple* LLM round-trips and the limit depletes faster than the number of user queries suggests

Listing 9.5 shows the custom exception we build specially for capturing the limit exceeding requests.

Listing 9.5 LimitExceededException.cs

```
public class LimitExceededException(int currentCount, int maxRequests)
    : Exception($"Request limit exceeded: request {currentCount} "
        + "of {maxRequests} blocked!")
{ }
```

Fail fast saves tokens. Because *SharedFunction* is outermost, *LimitRequests* throws before *Response* middleware even starts. No LLM call is made, no tokens are consumed, no money is spent.

With both functions defined, we now wire them into a single middleware pipeline. Registration order determines execution order: *LimitRequests* registers first, so it sits outermost. If the budget is exceeded, *RemoveEmail* is never entered and no sanitization work is wasted.

Let's see in listing 9.6 how the previously defined *ChatClient* middleware functions (*RemoveEmail* and *LimitRequests*) kick in.

Code path: /ChatClientSharedFunctionMiddleware/Program.cs

Listing 9.6 SharedFunction Middleware

```
// 'usings' and API key fetching omitted for brevity

IChatClient chatClient = new OpenAIClient(apiKey)
    .GetChatClient(model)
    .AsIChatClient()
    .AsBuilder()
    .Use(ChatClientSharedFunctions.LimitRequests) // ❶
    .Use(ChatClientSharedFunctions.RemoveEmail) // ❷
    .Build();

ChatClientAgent motorsAgent = chatClient
    .AsAIAgent(new ChatClientAgentOptions
    {
        Name = "MotorsAgent",
        Description = "Controls robot car movements.",
        ChatOptions = new ChatOptions
        {
            Instructions = """
                You are an AI assistant controlling a robot car capable of performing
                basic moves: forward, backward, turn left, turn right, and stop.
                You have to break down the provided complex commands into the basic
                moves you know.
                Use a JSON array like [move1, move2, move3] for the response.
                Respond only with the moves and their parameters (angle or distance),
                without any additional explanations.
                """
        }
    }); // ❸

AgentSession session = await motorsAgent.CreateSessionAsync();

var prompt1 = """
    There is a tree directly in front of the car.
    Avoid it and then return to the original path.
    """;
Console.WriteLine($"PROMPT: {prompt1}");
try
{
    var result1 = await motorsAgent.RunAsync(prompt1, session); // ❹
    Console.WriteLine($"RESULT: {result1}\n");
}
catch (LimitExceededException ex)
{
    Console.WriteLine($"EXCEPTION: {ex.Message}\n");
}

var prompt2 = """
    Navigate to original position.
    Contact me at john.doe@example.com for updates.
    """;
Console.WriteLine($"PROMPT: {prompt2}");
```

```

try
{
    var result2 = await motorsAgent.RunAsync(prompt2, session); //❸
    Console.WriteLine($"RESULT: {result2}\n");
}
catch (LimitExceededException ex) //❹
{
    Console.WriteLine($"EXCEPTION: {ex.Message}\n");
}

var prompt3 = "Stop and email me at sara.doe@example.com for "
+ "further instructions.";
Console.WriteLine($"PROMPT: {prompt3}");
try
{
    var result3 = await motorsAgent.RunAsync(prompt3, session); //❺
    Console.WriteLine($"RESULT: {result3}\n");
}
catch (LimitExceededException ex) //❻
{
    Console.WriteLine($"EXCEPTION: {ex.Message}\n");
}

```

❶ **LimitRequests** is outermost

❷ **RemoveEmail** is inner

❸ **Defines the agent**

❹ **First prompt** — no email present, limit not exceeded

❺ **Second prompt** — email detected and removed before reaching the LLM

❻ **Catches LimitExceededException** from the second prompt if the limit was hit

❼ **Third prompt** — limit already exhausted from previous round-trips

❽ **Catches LimitExceededException**

Every prompt is wrapped in try-catch. The source treats all three queries uniformly, because in a real system you cannot assume which round-trip will exhaust the budget.

Code output:

```

PROMPT: There is a tree directly in front of the car. Avoid it and then return to
the original path.
[ChatClient] [SharedFunction] [Limit] PRE: Request 1 of 2 max
[ChatClient] [SharedFunction] [Email] PRE: Scanning 1 messages...
[ChatClient] [SharedFunction] [Email] No email found
[ChatClient] [SharedFunction] [Email] POST: Completed
[ChatClient] [SharedFunction] [Limit] POST: Request 1 of 2 max completed
RESULT: [
    {"move":"turn right","angle":45},
    {"move":"forward","distance":3},
    {"move":"turn left","angle":45},
    {"move":"forward","distance":5},
    {"move":"turn left","angle":45},
    {"move":"forward","distance":3},
    {"move":"turn right","angle":45}
]

PROMPT: Navigate to original position. Contact me at john.doe@example.com for
updates.
[ChatClient] [SharedFunction] [Limit] PRE: Request 2 of 2 max
[ChatClient] [SharedFunction] [Email] PRE: Scanning 3 messages...
[ChatClient] [SharedFunction] [Email] Email detected and removed!
[ChatClient] [SharedFunction] [Email] POST: Completed
[ChatClient] [SharedFunction] [Limit] POST: Request 2 of 2 max completed
RESULT: [
    {"move":"backward","distance":3},
    {"move":"turn right","angle":45},
    {"move":"backward","distance":5},
    {"move":"turn right","angle":45},
    {"move":"backward","distance":3},

```

```
{ "move": "turn left", "angle": 45 }
]
```

```
PROMPT: Stop and email me at sara.doe@example.com for further instructions.
[ChatClient] [SharedFunction] [Limit] PRE: Request 3 of 2 max
EXCEPTION: Request limit exceeded: request 3 of 2 blocked!
```

Every successful round-trip shows the same sequence:

```
Limit PRE → Email PRE → Email POST → Limit POST.
```

Fail fast confirmed. Prompt 3 shows `[Limit] PRE` immediately followed by the exception, so `[Email] PRE` never appears. As expected, no sanitization work is done on a rejected request.

EXERCISE

Clone `RemoveEmail` function and name it `HideIp` to also detect and redact IP addresses (e.g., `192.168.1.1` → `[REDACTED-IP]`). Hint: use `Regex.Replace` call for the pattern

```
private const string IpPattern = @"\\b\\d{1,3}\\.|\\d{1,3}\\.|\\d{1,3}\\.|\\d{1,3}\\b";
private const string IpMask = "[REDACTED-IP]";
```

then call the function using:

```
.Use(ChatClientSharedFunctions.HideIp)
```

The middleware function should mask the IPs as well, and you may want to rename the method accordingly, but nothing stops you to follow the best practices and having two middleware functions instead of one with two responsibilities.

9.2.3 Handling Requests and Responses with Response Middleware

Response middleware type is where the chat response is revealed. Everything *SharedFunction* can do, like transform input messages, block requests, log, *Response* can do too.

There is no *next* delegate here, but the pipeline still exists. The framework builds it by wrapping: it starts from the real chat client, then passes it as `innerClient` to the first middleware, which becomes the new client, then passes that to the next middleware, and so on. Each middleware receives the previously wrapped client as its `innerClient`, calls `GetResponseAsync` to get the response, and returns it with or without modifications. This recursive wrapping is what simulates the *next*-like pipeline without an explicit next parameter.

Figure 9.7 shows the *Response* middleware type.

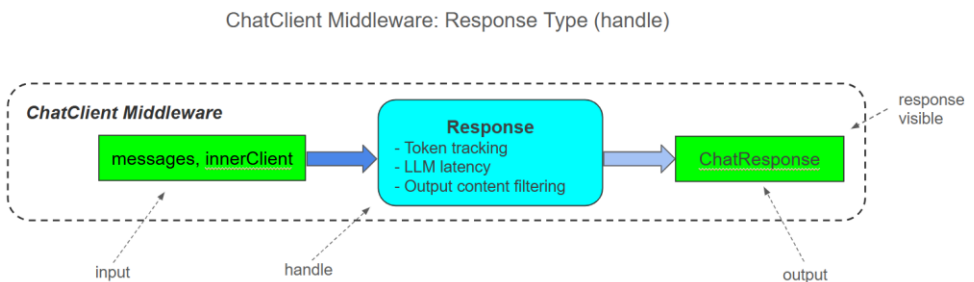


Figure 9.7 The *Response* middleware receives messages and an `IChatClient` `innerClient`, and it returns a `ChatResponse`. It starts from the real chat client, then repeatedly wraps it, passing the previously built client as `innerClient`. Typical use cases include token tracking, LLM latency, response caching, and output content filtering.

Response has full control over the `innerClient` to call `GetResponseAsync()` and can read and modify the `ChatResponse0`. With *Response* we can track tokens and latency, attach metadata, enforce budgets and timeouts, or even fabricate a response to short-circuit the pipeline.

This makes it the natural home for token tracking, content filtering, and budget enforcement.

PRACTICAL EXAMPLE: ENFORCE TOKEN BUDGET

Do you recall the \$10,000 Weekend story from section 9.2.2, where on Monday morning we find in the inbox a \$10,847 bill, because a bug in retry logic? Beyond request limiting, we also need token budget enforcement.

Every LLM call has a cost in tokens. Running out of token budget silently degrades user experience; uncontrolled consumption racks up costs. At the *ChatClient* layer we can measure cumulative token usage — the total tokens consumed across all round-trips — and enforce a hard ceiling before the next call is dispatched.

In listing 9.7 we see the *Response* function.

Code path: `/ChatClientResponseMiddleware/ChatClientResponses.cs`

Listing 9.7 Enforce Token Budget

```
public static class ChatClientResponses
{
    private static long _tokensCount = 0;
    private const long MaxTokens = 2000; // ❶

    public static async Task<ChatResponse> EnforceTokenBudget(
        IEnumerable<ChatMessage> messages,
        ChatOptions? options,
        IChatClient innerClient,
        CancellationToken cancellationToken) // ❷
    {
        var currentTokens = Interlocked.Read(ref _tokensCount); // ❸
        if (currentTokens >= MaxTokens) // ❹
        {
            Console.WriteLine($"[ChatClient] [Response] [Tokens] "
                + "Budget exhausted ({currentTokens} / {MaxTokens}) "
                + "- LLM call skipped");
            return new ChatResponse([new ChatMessage(ChatRole.Assistant,
                "Token budget exhausted.")]); // ❺
        }

        var response = await innerClient
            .GetResponseAsync(messages, options, cancellationToken); // ❻

        if (response.Usage is not null)
        {
            var totalTokens = response.Usage.TotalTokenCount
                ?? (response.Usage.InputTokenCount ?? 0) +
                (response.Usage.OutputTokenCount ?? 0); // ❼
            Interlocked.Add(ref _tokensCount, totalTokens); // ❽
        }

        return response; // ❾
    }
}
```

- ❶ Defines token limit (Real apps would have much higher limits)
- ❷ Declares Response function (gives access to call and chat response)
- ❸ Reads the current token count
- ❹ Checks if max tokens are exceeded
- ❺ Returns a synthetic assistant message (innerClient is never called)
- ❻ Explicitly calls innerClient for a chat response
- ❼ Computes the response token count
- ❽ Adds the current tokens count
- ❾ Returns the chat response

When the budget is exhausted, the middleware short-circuits: it returns a synthetic `ChatResponse` with a human-readable message and never calls the LLM.

PRACTICAL EXAMPLE: TIMESTAMP ADDING

Production systems often require audit trails that capture when each response was generated. Timestamping chat responses enables compliance tracking, debugging temporal issues, and correlating agent behavior with external events.

Response middleware type has access to `ChatResponse`.

In listing 9.8 we see the `AddTimestamp` *ChatClient Response* middleware function.

Code path: `/ChatClientResponseMiddleware/ChatClientResponses.cs`

Listing 9.8 AddTimestamp

```
public static class ChatClientResponses
{
    public static async Task<ChatResponse> AddTimestamp(
        IEnumerable<ChatMessage> messages,
        ChatOptions? options,
        IChatClient innerClient,
        CancellationToken cancellationToken) // 1
    {
        ChatResponse response = await innerClient
            .GetResponseAsync(messages, options, cancellationToken); // 2

        foreach (ChatMessage message in response.Messages) // 3
        {
            if (!string.IsNullOrEmpty(message.Text)) // 4
            {
                string timestamp = DateTimeOffset.UtcNow.ToString("o"); // 5
                Console.WriteLine($"[ChatClient] [Response] [Timestamp] "
                    + "Stamping response with [{timestamp}]");
                message.Contents = [new TextContent(
                    $"[{timestamp}] {message.Text}");] // 6
            }
        }

        return response;
    }
}
```

- 1 Declares Response middleware function**
- 2 Calls innerClient.GetResponseAsync**
- 3 Iterates through all messages**
- 4 Identifies the valid text contents**
- 5 Builds the timestamp string**
- 6 Prefixes the message contents with the timestamp**

Let's see in listing 9.9 how to wire both *Response* functions into the middleware pipeline.

Code path: `/ChatClientResponseMiddleware/Program.cs`

Listing 9.9 Response Middleware

```
// 'usings' and API key fetching omitted for brevity

IChatClient chatClient = new OpenAIClient(apiKey)
    .GetChatClient(model)
    .AsIChatClient()
    .AsBuilder()
    .Use(ChatClientResponses.EnforceTokenBudget, null) // 1
    .Use(ChatClientResponses.AddTimestamp, null) // 2
    .Build();

ChatClientAgent motorsAgent = chatClient
```



```

.AsAIAgent(new ChatClientAgentOptions
{
    Name = "MotorsAgent",
    Description = "Controls robot car movements.",
    ChatOptions = new ChatOptions
    {
        Instructions = """
            You are the MotorsAgent controlling a robot car.
            Break down complex movement commands into basic moves:
            forward, backward, turn left, turn right, stop.
            Respond with the sequence of moves needed to accomplish the task.
            """,
        Tools = [... MotorTools.AsAITools()],
    }
}); //❸

AgentSession session = await motorsAgent.CreateSessionAsync();

var prompt1 = "Move forward 3 meters";
Console.WriteLine($"PROMPT: {prompt1}");
var result1 = await motorsAgent.RunAsync(prompt1, session); //❹
Console.WriteLine($"RESULT: {result1}\n");

var prompt2 = "Turn right 45 degrees";
Console.WriteLine($"PROMPT: {prompt2}");
var result2 = await motorsAgent.RunAsync(prompt2, session); //❺
Console.WriteLine($"RESULT: {result2}\n");

```

- ❶ EnforceTokenBudget registered as outer
- ❷ Registered as inner
- ❸ Declares the agent
- ❹ First prompt that triggers timestamp adding
- ❺ Second prompt that triggers limit exceeding response

WARNING This agent has tools — `MotorTools.AsAITools()`. As side effect, each user prompt triggers multiple LLM round-trips (one to decide which tool to call, one after the tool result is returned). Token consumption accumulates across all round-trips, which is why the budget can be exhausted mid-prompt.

Code output:

```

PROMPT: Move forward 3 meters
[08:26:06:940] MOTORS: Forward: 3m
[08:26:09:244] MOTORS: Stop
[ChatClient] [Response] [Timestamp] Stamping response with [2026-04-08T10:24:35.1407499+00:00]

RESULT: [2026-04-06 18:26:11 UTC] - forward 3 meters
- stop

PROMPT: Turn right 45 degrees
[08:26:12:384] MOTORS: TurnRight: 45°
[08:26:14:816] MOTORS: Stop
[ChatClient] [Response] [Tokens] Budget exhausted (2068 / 2000) - LLM call skipped

RESULT: Token budget exhausted.

```

The number of model round-trips depends on the tool loop. The first is the initial LLM call (assistant message) where the model decides which tool to invoke; the second is the follow-up call (tool message) after the tool result is returned. This is the same multi-round-trip behavior described in section 9.1.1: *ChatClient* middleware fires on every LLM round-trip, not once per user prompt.

EXERCISE

Create a `CacheResponse` *Response* middleware function that caches LLM responses by the content of the outgoing messages. On a cache hit, return the cached `ChatResponse` directly without calling `innerClient`. On a cache miss, call `innerClient`, store the result, and return it. You may use a dictionary to preserve the queries:

```
private static readonly Dictionary<string, ChatResponse> _cache = new();
```

Then check if we have a cache hit or a cache miss like this:

```
var key = string.Concat(messages.Select(m => m.Text));
if (_cache.TryGetValue(key, out ChatResponse? cached))
{
    return cached; //❶
}
ChatResponse response = await innerClient
    .GetResponseAsync(messages, options, cancellationToken); //❷
_cache[key] = response; //❸
return response; //❹
```

- ❶ Returns cached response if we have cache hit
- ❷ Gets a new response
- ❸ Caches the current response
- ❹ Return new response if we have a cache miss

9.2.4 Supervising Tool Calls with FunctionCalling Middleware

FunctionCalling middleware does not wrap the chat request itself. It runs when the LLM response contains a tool call and the framework is about to execute that tool, where the model has already decided which function to call and with which arguments.

Unlike *SharedFunction* and *Response*, *FunctionCalling* has no next delegate and no `innerClient`. The only way to execute the tool is through `FunctionInvocationContext`:

```
context.Function.InvokeAsync(context.Arguments, cancellationToken); //❶
```

- ❶ Invokes the function with arguments and cancelation token

`FunctionInvocationContext` provides access to the function name, arguments, chat messages, and result, allowing middleware to validate, modify, or reject tool calls before they execute.

Figure 9.8 shows the *FunctionCalling* middleware type.

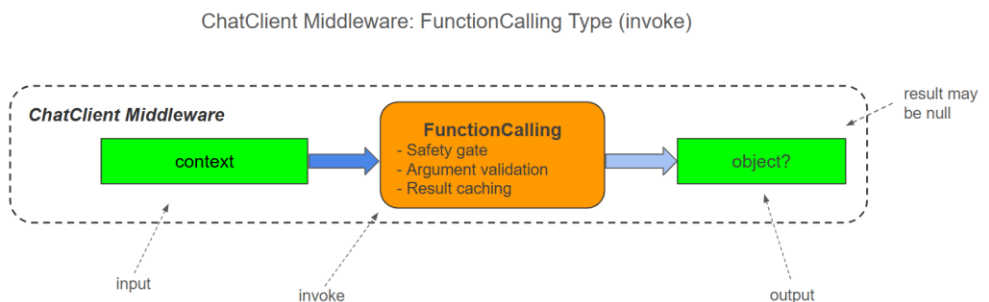


Figure 9.8 The *FunctionCalling* middleware receives a `FunctionInvocationContext` and returns `ValueTask<object?>`. There is no next delegate and no `innerClient`; the tool is invoked directly through `context.Function.InvokeAsync()`. Typical use cases include safety checks, audit logging, argument validation, and result caching.

Because there is no *next* delegate, composition relies on a null/non-null result convention. A middleware that returns `null` acts as an *interceptor*: it can inspect and modify arguments but does not call `InvokeAsync`, signaling that the pipeline should continue. A middleware that returns a non-

null value acts as the *executor*: it calls `InvokeAsync`, owns the result, and terminates the chain. We can have multiple interceptors but only one executor, and the executor is always last.

Figure 9.9 illustrates the interceptor-executor pattern.

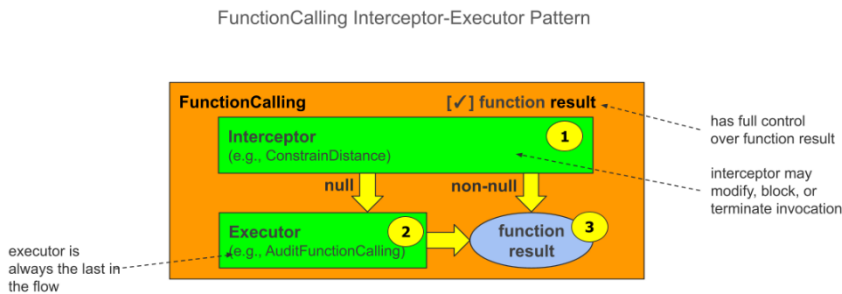


Figure 9.9 FunctionCalling interceptor-executor pattern. The interceptor (step 1) runs first, inspecting and potentially modifying the function invocation. If it returns null, control flows to the executor (step 2), which calls `InvokeAsync` and produces the final function result (step 3). If the interceptor returns non-null, the flow short-circuits immediately.

In interceptor-executor pattern. The interceptor (step 1) runs first, with full control over the function result — it may modify, block, or terminate the invocation. If it returns null, control flows to the executor (step 2), which calls `InvokeAsync` and produces the final function result (step 3). If the interceptor returns non-null, the flow short-circuits immediately returning the function result (step 3), bypassing the executor entirely. The executor is always last in the flow.

WARNING Agent Framework does not enforce interceptor-executor pattern described here. We are fully responsible for following the null/non-null convention. A mistake can result in double tool execution or skipped safety checks.

FunctionCalling at the *ChatClient* layer is also agent-agnostic. `FunctionInvocationContext` does not carry agent identity, so the same logic applies uniformly across all agents sharing that chat client.

PRACTICAL EXAMPLE: DISTANCE CONSTRAINING

Let me tell you a production story: The Runaway Robot

Robby, the autonomous robot car intelligent companion, received the command: "Danger ahead! Move backward immediately!" Robby dutifully called, for example, `backward(100)` and drove straight into a wall behind it.

No one had implemented a safety check. The agent just executed. We should have a distance constraint for safety.

In listing 9.10 we see how a *FunctionCalling* middleware function for preventing such incidents is implemented.

Code path:

```
/ChatClientFunctionCallingMiddleware/ChatClientFunctionCallings.cs
```

Listing 9.10 ConstrainDistance

```
public static class ChatClientFunctionCallings
```

```

{
    public static async Task<object?> ConstrainDistance(
        FunctionInvocationContext context,
        CancellationToken cancellationToken) //❶
    {
        const int MaxBackwardDistance = 5; //❷
        bool isBackward = context.Function.Name.Contains("backward",
            StringComparison.OrdinalIgnoreCase); //❸
        bool hasDistance = context.Arguments
            .TryGetValue("distance", out object? value); //❹
        if (isBackward && hasDistance) //❺
        {
            int distance = value is JsonElement jsonElement
                ? jsonElement.GetInt32()
                : Convert.ToInt32(value); //❻
            if (distance > MaxBackwardDistance) //❼
            {
                context.Arguments["distance"] = JsonSerializer
                    .SerializeToElement(MaxBackwardDistance); //❽
                Console.WriteLine($"[ChatClient] [FunctionCall] [Constrain] "
                    + "Backward distance constrained from {distance}m "
                    + "to {MaxBackwardDistance}m");
            }
        }

        return null; //❾
    }
}

```

- ❶ Declares middleware function
- ❷ Defines the max allowed backward distance
- ❸ Identifies if the current function is “backward”
- ❹ Identifies if it has “distance” argument
- ❺ Checks if it’s “backward” and has “distance”
- ❻ Reads the “distance” value
- ❼ Checks if the distance exceeds the max allowed backward distance
- ❽ Persists as `JsonElement` for downstream consistency
- ❾ Returns null to signal “proceed to the terminal invoker”

The null return value is what makes the `??` operator in the `FunctionInvoker` fall through to `AuditFunctionCalling`.

PRACTICAL EXAMPLE: FUNCTION CALLING AUDITING

When a tool call causes damage (like Robby hitting a wall), we need a complete record of which function was invoked, with the effective arguments, and what result it returned. *FunctionCalling* middleware provides this audit trail for debugging, compliance, and security reviews.

In listing 9.11 we see how a *FunctionCalling* middleware function is implemented.

Code path:

`/ChatClientFunctionCallingMiddleware/ChatClientFunctionCallings.cs`

Listing 9.11 AuditFunctionCalling

```

public static class ChatClientFunctionCallings
{
    public static async Task<object?> AuditFunctionCalling(
        FunctionInvocationContext context,
        CancellationToken cancellationToken) //❶
    {
        var functionName = context.Function.Name;
        var timestamp = DateTime.UtcNow;
        var stopwatch = Stopwatch.StartNew();

        var result = await context.Function

```

```

        .InvokeAsync(context.Arguments, cancellationToken); //❷
    stopwatch.Stop();
    Console.WriteLine($"[ChatClient] [FunctionCall] [Audit] "
        + "Function call '{functionName}' [{timestamp:HH:mm:ss}] "
        + "COMPLETED in {stopwatch.ElapsedMilliseconds}ms | "
        + "Result: {result}"); //❸
    return result; //❹
}
}

```

❶ Defines the middleware function

❷ Invokes the function call

❸ Logs the successful call

❹ Returns the function result

`AuditFunctionCalling` is the terminal invoker: it is the only method that calls `context.Function.InvokeAsync()`. Because `ConstrainDistance` always returns null, the `??` operator guarantees `AuditFunctionCalling` always runs exactly once per tool call.

Let's see in listing 9.12 how we can call multiple *FunctionCalling* middleware functions.

Code path:

`/ChatClientFunctionCallingMiddleware/Program.cs`

Listing 9.12 FunctionCalling Middleware – Program.cs

```

// 'usings' and API key fetching omitted for brevity

IChatClient chatClient = new OpenAIClient(apiKey)
    .GetChatClient(model)
    .AsIChatClient()
    .AsBuilder()
    .UseFunctionInvocation(loggerFactory: null, configure: options =>
    {
        options.FunctionInvoker = async (context, ct) =>
            await Middleware.ChatClientFunctionCallings
                .ConstrainDistance(context, ct)
                ?? await Middleware.ChatClientFunctionCallings
                    .AuditFunctionCalling(context, ct);
    }) //❶
    .Build();

ChatClientAgent motorsAgent = chatClient
    .AsAIAgent(new ChatClientAgentOptions
    {
        Name = "MotorsAgent",
        Description = "Controls robot car movements.",
        ChatOptions = new ChatOptions
        {
            Instructions = """
                You are the MotorsAgent controlling a robot car.
                Break down complex movement commands into basic moves:
                forward, backward, turn left, turn right, stop.
                Respond with the sequence of moves needed to accomplish the task.
                """,
            Tools = [.. MotorTools.AsAITools()], //❷
        }
    }); //❸

AgentSession session = await motorsAgent.CreateSessionAsync();

var prompt = " Move forward 10 meters then go backward 8 meters";
Console.WriteLine($"PROMPT: {prompt}");
var result = await motorsAgent.RunAsync(prompt, session); //❹
Console.WriteLine($"RESULT: {result}\n");

```

❶ Defines the FunctionCalling layer

- ② Assigns the AI tools
- ③ Declares the agent
- ④ Runs the agent with a single prompt that triggers “backward”

Code output:

```
PROMPT: Move forward 10 meters then go backward 8 meters
[08:52:40:833] MOTORS: Forward: 10m
[ChatClient] [FunctionCall] [Audit] Function call 'Forward' [18:52:40] COMPLETED
in 1019ms | Result: moved forward for 10 meters.
[ChatClient] [FunctionCall] [Constrain] Backward distance constrained from 8m to
5m
[08:52:42:830] MOTORS: Backward: 5m
[ChatClient] [FunctionCall] [Audit] Function call 'Backward' [18:52:42] COMPLETED
in 1008ms | Result: moved backward for 5 meters.
[08:52:44:687] MOTORS: Backward: 3m
[ChatClient] [FunctionCall] [Audit] Function call 'Backward' [18:52:44] COMPLETED
in 1007ms | Result: moved backward for 3 meters.
[08:52:46:838] MOTORS: Stop
[ChatClient] [FunctionCall] [Audit] Function call 'Stop' [18:52:46] COMPLETED in
1006ms | Result: stopped.

RESULT: - Forward 10 m
- Backward 8 m
- Stop
```

Forward 10 meters pass through unconstrained. Backward 8 meters, on the other hand, is silently capped to 5 meters (per invocation) by `ConstrainDistance`. The LLM asked for 8, but only 5 were executed. The LLM then call `Backward(3)` again to cover the remaining distance, based on the LLM's plan. The result summary reports the LLM's intended plan (8 meters), not the actual constrained execution — the agent is unaware its argument was modified.

NOTE *Agent vs. ChatClient* middleware. At the *Agent* layer, `AuditFunctionCalls` is chainable: it delegates to next function in the pipeline, and it has access to the agent identity via `agent.Name`. At the *ChatClient* layer, it is terminal and anonymous: it calls `InvokeAsync` directly and has no agent context.

The registration order matters. `ConstrainDistance` (argument adjustment) runs first and returns `null` to signal that the pipeline should continue. Only when that check passes does `AuditFunctionCalls` run as the terminal invoker, calling `InvokeAsync`. It is the responsibility of the last middleware function in the chain to call `InvokeAsync`, not every middleware registered functions in the pipeline.

EXERCISE

Modify `ConstrainDistance` function to limit the `forward` distance to 20 meters.

You should notice an extra adjustment when calling the `forward` tool when the distance is larger than 20 meters.

9.3 Composing the Full ChatClient Middleware Pipeline

Now that we've seen each *ChatClient* middleware type in isolation, here is how they compose into a complete pipeline.

The pipeline calling order works like a stack. As with any pipeline, we can return early and short-circuit execution, or we can keep adding middleware until we reach the top of the stack: the chat client. Registration order equals execution order: whatever we add first runs first.

9.3.1 Stacking SharedFunction, Response, and FunctionCalling

The pipeline calling order works like a stack. As with any pipeline, we can return early and short-circuit execution, or we can keep adding middleware until we reach the top of the stack: the chat client. Registration order equals execution order: whatever we add first runs first.

Figure 9.10 shows how we mix and order the three *ChatClient* middleware types.

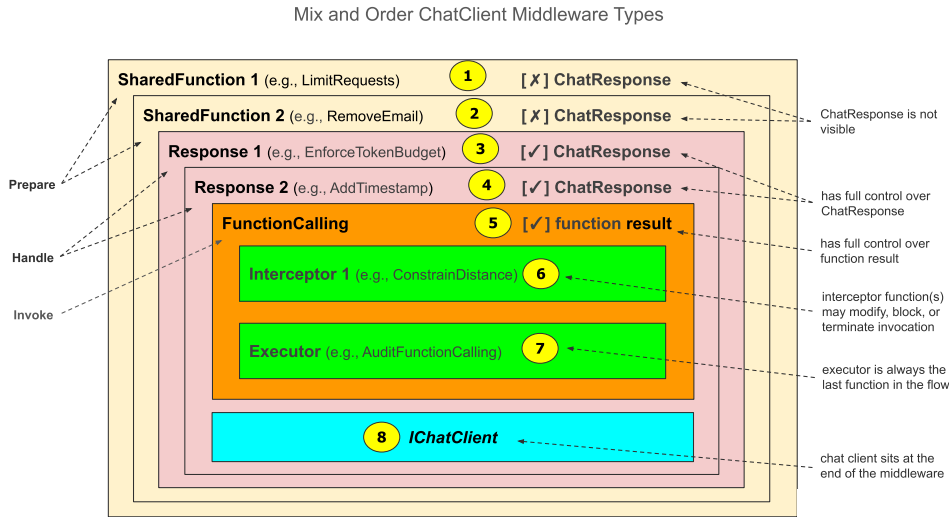


Figure 9.10 All three ChatClient middleware types composing together. SharedFunction functions (outermost, steps 1-2) prepares input, ChatResponse invisible. Response functions (middle, steps 3-4) wrap the LLM exchange, ChatResponse visible and controllable. FunctionCalling (innermost, step 5) intercepts tool calls via interceptor/executor pattern implemented with FunctionCalling functions (steps 6-7). IChatClient (step 8) sits at the end of the middleware pipeline. Registration order equals execution order: prepare → handle → invoke.

ChatClient middleware steps:

1. **SharedFunction 1: LimitRequests** starts the pipeline in the *Prepare* stage and can count or limit requests before they reach the chat client; *ChatResponse* is not visible here.
2. **SharedFunction 2: RemoveEmail** adds another *Prepare*-stage rule that sanitizes input, such as removing email addresses before the request is sent to the model; *ChatResponse* is still not visible.
3. **Response 1: EnforceTokenBudget** enters the *Handle* stage, where *ChatResponse* is visible and token usage or cost limits can be inspected and enforced.
4. **Response 2: AddTimestamp** adds another *Handle*-stage middleware that can decorate, inspect, or modify the completed *ChatResponse*.
5. **FunctionCalling** enters the *Invoke* stage, where middleware works with individual function results instead of the full *ChatResponse*.
6. **Interceptor: ConstrainDistance** runs before final function execution and can modify, block, terminate, or short-circuit the function invocation.
7. **Executor: AuditFunctionCalling** runs if the interceptor returns null, audits the invocation, and produces the final function result.

8. `IChatClient` sits at the end of the middleware stack, sends the request to the model, and produces the `ChatResponse` that flows back outward through the pipeline.

The recommended order for *ChatClient* middleware types is:

- *SharedFunction* Layer — *Prepare*

The *SharedFunction* layer is the outermost part of the *ChatClient* middleware pipeline. Its job is to prepare the request before the model sees it. At this point, there is no `ChatResponse` yet, so these middleware functions cannot inspect or modify the final response.

This makes *SharedFunction* middleware a good fit for input-oriented concerns: request limits, validation, privacy, and prompt/message shaping. In this sample, `LimitRequests` protects the pipeline from too many calls, while `RemoveEmail` prevents sensitive data from reaching the LLM provider.

- *Response* Layer — *Handle*

The *Response* layer surrounds the actual model call. This is the first layer where `ChatResponse` is visible. Because of that, *Response* middleware is the right place for output-oriented concerns.

This layer can inspect, decorate, replace, or reject the response. In this sample, `EnforceTokenBudget` uses response information to guard the cost, while `AddTimestamp` decorates the returned content.

- *FunctionCalling* Layer — *Invoke*

The *FunctionCalling* layer is the innermost *ChatClient* middleware area. It does not work with the full `ChatResponse`; it works with individual function invocation results.

This is the correct place for tool-level rules. In this sample, `ConstrainDistance` acts as an *interceptor* that can constrain unsafe tool arguments, while `AuditFunctionCalling` acts as the final *executor* for the function call.

- `IChatClient` — End of the Pipeline

At the end of the pipeline sits the `IChatClient`. It sends the request to the model and produces the `ChatResponse`, which then flows back outward through the middleware layers.

IMPORTANT The important mental model is: 1) *SharedFunction* middleware prepares the request and then calls the next layer; 2) *Response* middleware handles the completed `ChatResponse`; 3) *FunctionCalling* middleware controls tool invocation, not the whole chat response. 4) `IChatClient` ends the middleware.

Reading the stack from bottom to top gives us the registration order, which becomes execution order in the pipeline.

APPLYING THE PREPARE–HANDLE–INVOKE MIDDLEWARE PATTERN

This is the canonical composition for the *ChatClient* middleware types. Each concern is placed where it has the right visibility and the right timing: validation and sanitization before tokens are spent, response control around the model call, and tool supervision only when a function is about to be executed.

Listing 9.13 shows the full pipeline registration following the *Prepare* → *Handle* → *Invoke* mental model.

Listing 9.13 Full Pipeline Registration

```

IChatClient chatClient = new OpenAIClient(apiKey)
    .GetChatClient(model)
    .AsIChatClient()
    .AsBuilder()
    .Use(ChatClientSharedFunctions.LimitRequests) // ❶
    .Use(ChatClientSharedFunctions.RemoveEmail) // ❶
    .Use(ChatClientResponses.EnforceTokenBudget, null) // ❷
    .Use(ChatClientResponses.AddTimestamp, null) // ❷
    .UseFunctionInvocation(loggerFactory: null, configure: options =>
    {
        options.FunctionInvoker = async (context, ct) =>
        {
            var response = await Middleware.ChatClientFunctionCallings
                .ConstrainDistance(context, ct) // ❸
                ?? await Middleware.ChatClientFunctionCallings
                .AuditFunctionCalling(context, ct); // ❹
            return response;
        };
    }) // ❺
    .Build(); // ❻

```

❶ Prepare layer: fail fast, sanitize input

❷ Handle layer: track tokens, enforce budget

❸ Interceptor: modify or block unsafe arguments

❹ Executor: invoke function and audit

❺ Invoke layer: **MUST** be last before `.Build()`

❻ Builds the `ChatClient` middleware for the chat client

When all three middleware types compose correctly, the pipeline reads like a checklist. *SharedFunction* validates and sanitizes before any token is spent. *Response* wraps the LLM exchange and enforces the token budget. *FunctionCalling* controls each tool invocation individually, with the gate pattern ensuring the function executes exactly once. Remove a layer and lose a capability; misorder them and a concern may fire too late to matter.

IMPORTANT In practice, this is what we want from a *ChatClient* pipeline: outer layers that fail fast and sanitize, a middle layer that owns cost and observability, and an inner layer that supervises tools without leaking any of those responsibilities into agent code.

COMMON MIDDLEWARE ANTI-PATTERNS

Some common pitfalls in production:

Wrong registration order. Registration order equals execution order. Logging before sanitization means the log captures sensitive data before removal. Always sanitize first:

```

.Use(SharedFunctionMiddleware.RemoveEmail) // ❶
.Use(ResponseMiddleware.LogMessages, null) // ❷

```

❶ Correct: sanitizes first (e.g., "[REDACTED-EMAIL]")

❷ Then logs sanitized version (e.g., "[REDACTED-EMAIL]")

FunctionCalling is not innermost. `UseFunctionInvocation` must be the last middleware registered before `.Build()`. If placed outermost, its internal tool loop drives repeated calls back through the entire pipeline: budget checks, sanitization, and timing all fire once per tool call instead of once per request.

Firing frequency misunderstanding. *SharedFunction* and *Response* fire once per chat request, not once per agent run. A single agent run with tool calls sends multiple chat requests (one initial call and one per tool round-trip), so `LimitRequests` can increment multiple times within a single user prompt. Budget middleware depletes faster than the number of user queries suggests.

No ChatClient-layer protection. Relying only on agent middleware means rogue or misconfigured agents can bypass protection. The *ChatClient* layer acts as a universal failsafe because every call, from any agent or plain code, passes through it.

ChatClient middleware operates at the HTTP boundary with no agent context. This makes it ideal for universal concerns like cost control and privacy.

In the next chapter, we explore *Agent* middleware, where we have access to agent identity, session state, and the familiar next delegate pattern for all middleware types, including *FunctionCalling*.

Summary

- Middleware is essential for production agents. Without it, sensitive data leaks, costs spiral, and dangerous operations execute unchecked.
- Two middleware layers serve different purposes. *ChatClient* is the universal safety net for all agents. *Agent* provides per-agent control with session context and identity.
- Three middleware types map to prepare → handle → invoke. *SharedFunction* prepares input (*ChatResponse* invisible). *Response* handles LLM exchange (*ChatResponse* visible). *FunctionCalling* invokes tools (sees function result).
- Registration order determines execution order. *SharedFunction* outermost (fail fast). *Response* middle (track tokens). *FunctionCalling* innermost (must be last before `.Build()`).
- *FunctionCalling* uses interceptor/executor pattern. An interceptor returns null to continue; to block, return a non-null synthetic result or throw. The terminal executor invokes the tool exactly once and returns its result, which may itself be null.
- *FunctionCalling* must be innermost. If not last, its tool loop drives repeated pipeline passes. *SharedFunction* and *Response* fire per tool call instead of per chat request.
- Firing frequency differs by layer. *ChatClient* fires per chat request (multiple times per agent run with tools). *Agent* fires once per `RunAsync()` call.
- Sanitization differs by layer. *ChatClient* is transient (LLM sees clean data, session retains original). *Agent* is persistent (cleaned messages stored in history).
- Defense in depth requires both layers. Implement critical controls like email removal at both *ChatClient* (universal) and *Agent* (per-agent) layers.

10

Building enterprise-ready agents with Agent middleware

This chapter covers

- Agent middleware as the per-agent companion to ChatClient middleware
- Agent middleware architecture: session context and agent identity
- Three Agent middleware types: SharedFunction, Response, and FunctionCalling
- Composing a complete Agent middleware pipeline

Robby's problems at the *ChatClient* layer were only half the story. In production we also discovered that different agents, users, and missions needed different rules: who is allowed to move hardware, which missions require approval, how to tag responses with session and mission metadata, and when to persist sanitization instead of reapplying it on every call. *Agent* middleware is the higher-level pipeline that wraps an agent run, with access to agent identity and session state, so we can enforce these per-agent and per-session guardrails without cluttering the agent code. That extra layer of policy (e.g., per agent, per user, per mission, but centrally enforced) turns Robby from a clever prototype into an enterprise-ready fleet that still relies on the shared, organization-wide safeguards from the *ChatClient* layer.

10.1 Applying Agent Middleware to Agents

The previous chapter introduced *ChatClient* middleware as the universal safety net for agents in production. *ChatClient* middleware fires on every chat request to the LLM, applies the same rules to every agent that uses the same chat client, and has no knowledge of sessions, tenants, or agent identities. This low-level middleware has clear limits.

Let's consider what *ChatClient* middleware cannot do:

- Enforce a different request budget for each user session or agent.
- Inject a GDPR guardrail only for a subset of agents.
- Include the agent's name in an audit log.
- Require human approval for an agent with limited access.

Both *Agent* and *ChatClient* function middleware can inspect the messages associated with a tool invocation through `FunctionInvocationContext.Messages`, including prior tool calls. *ChatClient* middleware does not receive an explicit agent parameter, but when it executes within an agent run, *Agent* middleware can access the current agent and session through

`AIAgent.CurrentRunContext`; this value may be null outside an agent run. *Agent* middleware remains preferable when explicit agent access and agent-level composition are required.

IMPORTANT The two layers are complementary, not alternative. A good example is requesting limiting: the *ChatClient* middleware enforces a global process-wide ceiling, preventing runaway costs across all users and agents. The *Agent* middleware enforces a per-session budget, preventing one noisy user from starving others. Neither makes the other redundant, both are needed. Apply the same thinking to other guardrails: deploy universal safeguards at the *ChatClient* middleware and per-agent, per-user refinements at the *Agent* middleware.

The two layers are complementary, *ChatClient* middleware is the universal, low-level layer, and *Agent* middleware is the contextual, high-level layer. A production system usually needs both.

10.1.1 Revisiting the Two-Layer Middleware Architecture

Figure 10.1 is not new. It introduces the two-layer middleware model and is worth revisiting with the upper *Agent* layer in view. *Agent* middleware wraps the agent run (`AIAgent` from `Microsoft.Agents.AI`). *ChatClient* middleware wraps each LLM round-trip inside that run.

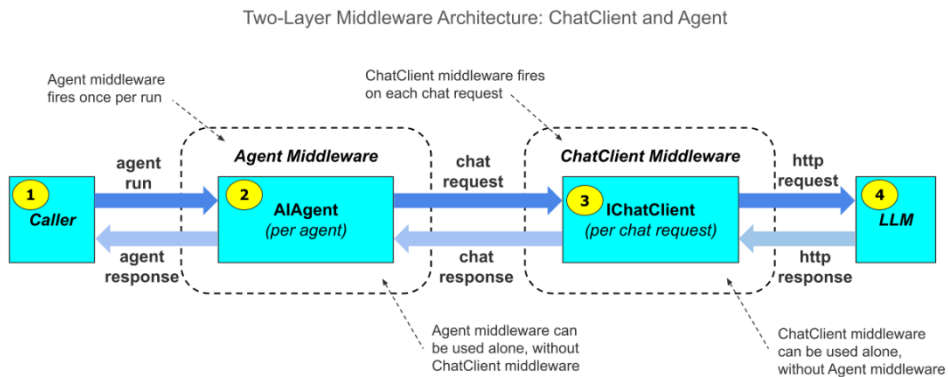


Figure 10.1 Agent middleware wraps the entire agent run. ChatClient middleware wraps each individual LLM call inside that run. Both layers are independent and composable.

The two-layer diagram steps:

- Step 1: The Caller starts an agent run by calling `RunAsync`. This activates the *Agent* middleware once for the entire run, not for each LLM round-trip.
- Step 2: Inside that run, the *Agent* middleware (per agent) wraps the `AIAgent` instance, has access to `AgentSession.StateBag`, and sees the agent identity (for example `agent.Name`), so we can do per-agent logging, error handling, and tool policy.
- Step 3: The *Agent* middleware turns the single user intent into one or more chat requests. Each of those requests flows through the *ChatClient* middleware (per chat request), which can run its own function-calling loop and can call tools directly, even when no agent is involved.
- Step 4: The *ChatClient* sends each chat request as an HTTP call to the LLM and receives HTTP responses. The responses travel back through the *ChatClient* and *Agent* middleware and end

as a single agent response to the Caller for that run.

Agent’s middleware runs once per `RunAsync` and is registered on a specific agent, so each agent can have its own middleware pipeline. *ChatClient* middleware runs on every chat request and is wired to the shared `IChatClient`, so its behavior is shared across all agents that use that client.

If in the previous chapter we focused on the *ChatClient* layer, in this chapter we focus on the higher-level layer, *Agent* middleware.

10.2 Walking Through Agent Middleware Layer

Without *Agent* middleware, session context goes unused, agent identity is invisible to the pipeline, and every guardrail must live at the *ChatClient* layer where it applies identically to every agent. *Agent* middleware is where the pipeline becomes aware of who is running, who is asking, and what the session history already knows (see figure 10.2).

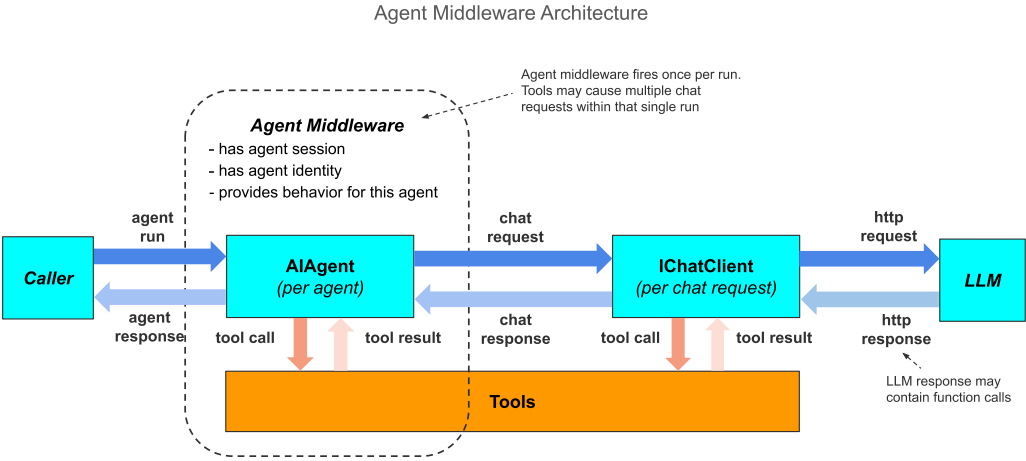


Figure 10.2 Agent middleware wraps the `AI Agent` and fires once per agent run. Inside that single run, tools may trigger multiple chat requests, each of which passes through the `ChatClient` middleware layer. Agent middleware has access to the agent session and agent identity; `ChatClient` middleware has neither.

The `AI Agent` is the *Agent* middleware scope. Everything inside it, including tool calls and their results, happens within a single middleware activation. The `IChatClient` sits outside that boundary, and chat requests cross it on every LLM call. *Agent* middleware never sees those individual crossings; it sees only the entry and the final exit.

10.2.1 Choosing the Right Agent Middleware Type

Agent middleware solves a separate set of problems than the chat layer: it knows which agent is running, which session is active, and what has already happened in that session. When we add guardrails for Robby at this level, the question is always the same: do we need to shape the request going into the agent, inspect or adjust the completed `AgentResponse`, or supervise individual tool calls inside the run?

Like *ChatClient* middleware, *Agent* middleware comes in three flavors: *SharedFunction*, *Response*, and *FunctionCalling*. The purpose of each type mirrors its *ChatClient* counterpart, but the capabilities are richer and more agent-aware.

Figure 10.3 shows the three *Agent* middleware types.

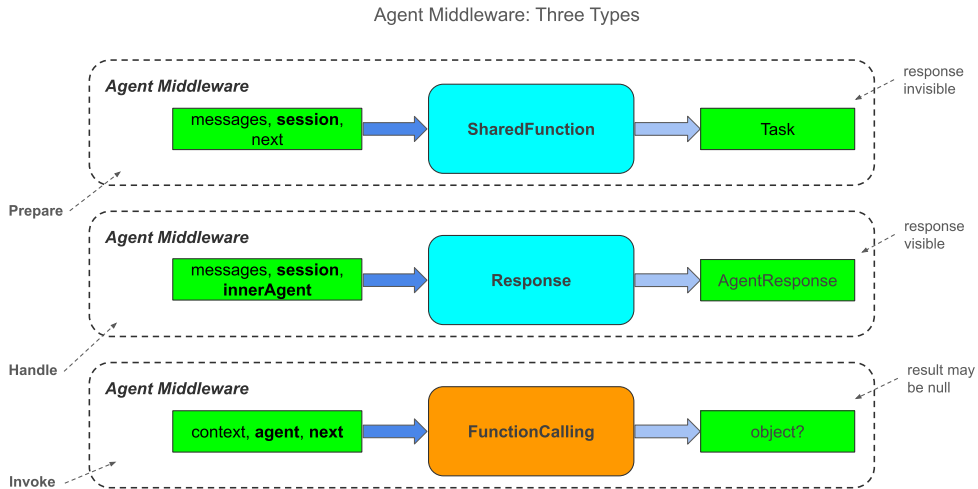


Figure 10.3 The three Agent middleware types. *SharedFunction* receives messages and the session returns Task (AgentResponse invisible). *Response* receives messages, session, and innerAgent, returns AgentResponse (fully visible and modifiable). *FunctionCalling* receives context, agent, and next, returns a nullable object.

The three types form a layered progression, each unlocking more of the runtime context. *SharedFunction* is the lightest: it has agent session access and agent identity and no response. *Response* adds both agent identity through `innerAgent` and the full `AgentResponse`. *FunctionCalling* is the most specialized: instead of wrapping the agent run, it wraps each individual tool invocation, with a chainable next delegate and access to the function name, arguments, and agent identity.

These correspond to the same mental model used in *ChatClient* middleware:

- *SharedFunction*: **prepares** the input (blind to `AgentResponse`)
- *Response*: **handles** the input and output (sees `AgentResponse`)
- *FunctionCalling*: **invokes** each tool (innermost, fires per tool call)

But the *Agent* layer adds agent identity, agent session context, and better composition.

10.2.2 Preparing Requests with SharedFunction Middleware

SharedFunction in *Agent* middleware uses the same pre/post pattern as its *SharedFunction* in *ChatClient* middleware: it receives messages, does work, calls `next` to pass control downstream, and can do more work after the call returns. The `next` delegate still returns `Task` (void), and the `AgentResponse` is still invisible here. What changes is that the session and options now travel alongside the messages through the entire chain, making per-session decisions possible at every step.

Figure 10.4 shows the *SharedFunction* in *Agent* middleware.

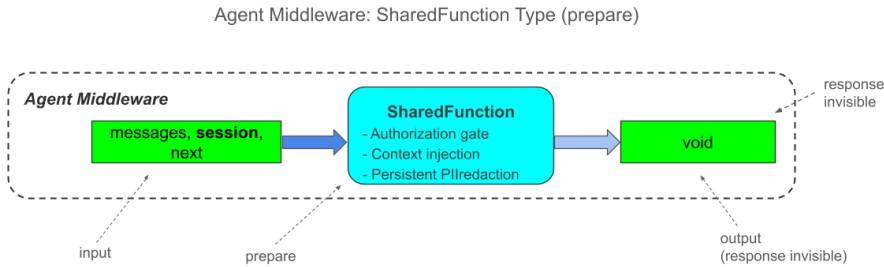


Figure 10.4 The Agent SharedFunction middleware wraps the agent run with pre/post hooks via next-delegate pattern. The SharedFunction returns no value (void), and there is no ChatResponse to inspect or alter. Compared to ChatClient middleware, this time an agent session is available.

The `session` parameter is what makes this type qualitatively different from *ChatClient SharedFunction*. Three practical examples: a persistent sanitizer, a per-session request limiter, and a tenant-aware guardrail injector.

PRACTICAL EXAMPLE: PERSISTENT EMAIL REMOVAL

During a GDPR audit on the `EmailRemoval` example with *ChatClient* middleware (as in chapter 9), we discovered `john.doe@example.com` appeared 47 times in Robby's session history. This is because the *ChatClient* layer has no access to the session. Our *ChatClient* middleware stripped emails before each LLM call, but the agent's session stored the original text. We needed persistent sanitization at the *Agent* layer to clean messages before they entered session history.

The *Agent* middleware modifies the messages once, before passing them to *next*, and those cleaned messages are what the agent stores in session history. Every future LLM round-trip starts with clean data already in history.

The code in listing 10.1 shows the persistent email remover.

Requirements (NuGet packages):

```
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
dotnet add package Microsoft.Extensions.AI.OpenAI
```

Code path: `/AgentSharedFunctionMiddleware/AgentSharedFunctions.cs`

Listing 10.1 PersistentRemoveEmail

```
public static class AgentSharedFunctions
{
    private const string EmailPattern =
        @"\"b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b";
    private const string EmailMask = "[REDACTED-EMAIL]";

    public static async Task PersistentRemoveEmail(
        IEnumerable<ChatMessage> messages,
        AgentSession? session,
        AgentRunOptions? options,
        Func<IEnumerable<ChatMessage>, AgentSession?, AgentRunOptions?,
            CancellationToken, Task> next, //❶
        CancellationToken cancellation)
    {
        Console.WriteLine($"[Agent] [SharedFunction] [Email] PRE: " +
            $"Scanning {messages.Count()} messages for email...");

        bool emailFound = false;
        List<ChatMessage> sanitizedMessages = [];
```

```

foreach (ChatMessage message in messages)
{
    if (message.Role == ChatRole.User && message.Text is not null)
    {
        string sanitized = Regex.Replace(message.Text,
            EmailPattern, EmailMask);
        if (sanitized != message.Text) emailFound = true;
        sanitizedMessages.Add(
            new ChatMessage(message.Role, sanitized)); //❷
    }
    else
    {
        sanitizedMessages.Add(message);
    }
}

Console.WriteLine(emailFound
    ? "[Agent] [SharedFunction] [Email] Email detected and removed!"
    : "[Agent] [SharedFunction] [Email] No email found");

await next(sanitizedMessages, session, options,
    cancellationToken); //❸

Console.WriteLine("[Agent] [SharedFunction] [Email] POST: Completed");
}
}

```

❶ Agent SharedFunction signature, includes session and options in next delegate

❷ Sanitized messages passed downstream, session stores the clean version

❸ Passes sanitized messages to the agent; history persists the clean text

The logic is familiar from chapter 9, but the significant difference is the persistence of sanitization.

IMPORTANT *ChatClient* sanitization is re-applied on every LLM call because the middleware only sanitizes the outbound request sent to the LLM. It does not own or update the agent's session history. *Agent* middleware can make sanitization persistent because it transforms the messages before the agent processes and records them for the session.

ChatClient sanitization (`RemoveEmail`) is transient. It cleans the outbound request before each LLM call, but it does not update the agent session history, so the history still contains the original email. At the *Agent* layer (`PersistentRemoveEmail`), sanitization is persistent: the middleware rewrites the messages before the agent records them, so the session history never stores the raw email.

PRACTICAL EXAMPLE: AGENT GUARDRAILS

At 2 AM, an *observer* role user issued movement commands to Robby. Only *driver* role users should control physical movement, but *ChatClient* middleware has no access to session state. We needed *Agent* middleware with `session.StateBag` access to enforce role-based authorization per agent session.

We introduce `AgentGuardrails` at the *Agent* layer, as in the middleware example in listing 10.2 to enforce that only an operator with the role *driver* may issue commands. If the operator is not authorized, it throws `OperationDeniedException` before *next* is called. That means the agent never had the chance to run because we returned early from the middleware pipeline.

The middleware here also prefixes the latest user message with the mission tag, anchoring the agent to the active mission.

Listing 10.2 AgentGuardrails


```

private const string AuthorisedOperatorRole = "driver";

public static async Task AgentGuardrails(
    IEnumerable<ChatMessage> messages,
    AgentSession? session,
    AgentRunOptions? options,
    Func<IEnumerable<ChatMessage>, AgentSession?,
        AgentRunOptions?, CancellationToken, Task> next,
    CancellationToken cancellationToken)
{
    string? operatorName = null;
    session?.StateBag.TryGetValue("OperatorName", out operatorName); //❶

    string? missionTag = null;
    session?.StateBag.TryGetValue("MissionTag", out missionTag); //❷

    Console.WriteLine("[Agent] [SharedFunction] [Guardrails] "
        + $"PRE: operator='{operatorName ?? "none"}', "
        + $"mission='{missionTag ?? "none"}'");

    if (!string.Equals(operatorName, AuthorisedOperatorRole))
    {
        throw new OperationDeniedException(operatorName
            ?? "none"); //❸
    }

    ChatMessage[] enrichedMessages =[.. messages];

    if (missionTag is not null)
    {
        enrichedMessages[^1] = new ChatMessage(ChatRole.User, $"[{missionTag}]
            {enrichedMessages[^1].Text ?? string.Empty}"); //❹
    }

    await next(enrichedMessages, session, options, cancellationToken);

    Console.WriteLine("[Agent] [SharedFunction] [Guardrails] POST: "
        + $"Completed for operator='{operatorName}', "
        + $"mission='{missionTag ?? "none"}'");
}

```

- ❶ Gets the operator's name from StateBag
- ❷ Gets the mission tag from StateBag
- ❸ Throws an exception and returns early from the middleware pipeline
- ❹ Prefixes the response with the mission tag

This demonstrates two agent-only patterns: a pre-run gate (exception thrown), and a session-aware message transformation (prefixing the response).

ChatClient middleware cannot implement this cleanly because it has no `AgentSession` or `StateBag`.

The custom exception `OperationDeniedException` in the listing 10.3 is similar to `LimitExceededException` from chapter 9 but is scoped to a session.

Code path: `/AgentSharedFunctionMiddleware/OperationDeniedException.cs`

Listing 10.3 OperationDeniedException

```

public class OperationDeniedException(string operatorName)
{
    : Exception($"Operation denied: operator '{operatorName}' "
        + "is not authorised to issue commands. "
        + "Only 'driver' role is permitted.")
}

```

Let's see in listing 10.4 how to wire both middleware functions and the exception into the *Agent* pipeline.

Requirements (NuGet packages):

```
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
dotnet add package Microsoft.Extensions.AI.OpenAI
```

Code path: /AgentSharedFunctionMiddleware/Program.cs

Listing 10.4 SharedFunction Middleware

```
// 'usings' and API key fetching omitted for brevity

IAgent motorsAgent = new OpenAIClient(apiKey)
    .GetChatClient(model)
    .AsIAgent(new ChatClientAgentOptions {
        Name = "MotorsAgent",
        Description = "Controls robot car basic movements.",
        ChatOptions = new ChatOptions {
            Instructions = ""
            You are the MotorsAgent controlling a robot car.
            Break down complex movement commands into basic moves:
            forward, backward, turn_left, turn_right, stop.
            Respond with the sequence of moves needed to accomplish the task.
            "",
            Tools = [.. MotorTools.AsAITools()],
        }
    })
    .AsBuilder()
    .Use(sharedFunc: AgentSharedFunctions.AgentGuardrails)
    .Use(sharedFunc: AgentSharedFunctions.PersistentRemoveEmail)
    .Build();

AgentSession session = await motorsAgent.CreateSessionAsync();

session.StateBag.SetValue("OperatorName", "driver"); //❶
session.StateBag.SetValue("MissionTag", "MISSION-42 | PERIMETER-SCAN"); //❶

var prompt1 = ""
    Navigate forward 5 meters.
    My email is john.doe@example.com if needed.
    "";
Console.WriteLine($"PROMPT: {prompt1}");
try
{
    var result1 = await motorsAgent.RunAsync(prompt1, session); //❷
    Console.WriteLine($"RESULT: {result1}\n");
}
catch (OperationDeniedException ex) //❸
{
    Console.WriteLine($"EXCEPTION: {ex.Message}\n");
}

var prompt2 = "Turn left 90 degrees.";
Console.WriteLine($"PROMPT: {prompt2}");
try
{
    var result2 = await motorsAgent.RunAsync(prompt2, session); //❹
    Console.WriteLine($"RESULT: {result2}\n");
}
catch (OperationDeniedException ex)
{
    Console.WriteLine($"EXCEPTION: {ex.Message}\n");
}

session.StateBag.SetValue("OperatorName", "observer"); //❺

var prompt3 = "Move forward 3 meters.";
```

```

Console.WriteLine($"PROMPT: {prompt3}");
try
{
    var result3 = await motorsAgent.RunAsync(prompt3, session);
    Console.WriteLine($"\\nRESULT: {result3}\\n"); //❹
}
catch (OperationDeniedException ex)
{
    Console.WriteLine($"EXCEPTION: {ex.Message}\\n"); //❺
}

```

- ❶ Pre-populates the operator to 'driver' and mission
- ❷ Queries the agent with an email in input
- ❸ Catches potential exception
- ❹ Queries the agent, no more emails to sanitize
- ❺ Changes the operator to 'observer'
- ❻ Queries the agent with the new StateBag
- ❼ Throws operation denied exception

Code output:

```

PROMPT: Navigate forward 5 meters. My email is john.doe@example.com if needed.
[Agent] [SharedFunction] [Guardrails] PRE: operator='driver', mission='MISSION-42 | PERIMETER-SCAN'
[Agent] [SharedFunction] [Guardrails] Mission tag: 'MISSION-42 | PERIMETER-SCAN'
[Agent] [SharedFunction] [Email] PRE: Scanning 1 messages for email...
[Agent] [SharedFunction] [Email] Email detected and removed!
[11:49:21:008] MOTORS: Forward: 5m
[11:49:23:498] MOTORS: Stop
[Agent] [SharedFunction] [Email] POST: Completed
[Agent] [SharedFunction] [Guardrails] POST: Completed for operator='driver', mission='MISSION-42 | PERIMETER-SCAN'
RESULT: forward 5
stop

PROMPT: Turn left 90 degrees.
[Agent] [SharedFunction] [Guardrails] PRE: operator='driver', mission='MISSION-42 | PERIMETER-SCAN'
[Agent] [SharedFunction] [Guardrails] Mission tag: 'MISSION-42 | PERIMETER-SCAN'
[Agent] [SharedFunction] [Email] PRE: Scanning 1 messages for email...
[Agent] [SharedFunction] [Email] No email found
[11:49:26:966] MOTORS: TurnLeft: 90°
[11:49:29:121] MOTORS: Stop
[Agent] [SharedFunction] [Email] POST: Completed
[Agent] [SharedFunction] [Guardrails] POST: Completed for operator='driver', mission='MISSION-42 | PERIMETER-SCAN'
RESULT: turn_left 90
stop

PROMPT: Move forward 3 meters.
[Agent] [SharedFunction] [Guardrails] PRE: operator='observer', mission='MISSION-42 | PERIMETER-SCAN'
EXCEPTION: Operation denied: operator 'observer' is not authorised to issue commands. Only 'driver' role is permitted.

```

The email in input is cleaned only once at the *Agent* middleware layer; next middleware function gets a sanitized input. The code replaces the last incoming user message with the mission tag and operator name in *AgentGuardrails* middleware function, and *AgentGuardrails* prevents the execution of unauthorized operators. *OperatorName* is used for logging only.

EXERCISE

Let's go back to listing 10.2 (*AgentGuardrails*) and instead of throwing an exception, let's just log a warning.

Instead of having the *Agent* middleware return early, this will let the middleware continue the trip to the *AI Agent* for the *AgentResponse*.

10.2.3 Handling Requests and Responses with Response Middleware

Response in *Agent* middleware is where the `AgentResponse` is revealed. Like *Response* in *ChatClient* layer, each middleware wraps the one before it: the framework passes the previously built agent as `innerAgent`, and each middleware calls `innerAgent.RunAsync()` to get the response, then returns it with or without modifications.

Figure 10.5 shows the *Agent Response* middleware type.

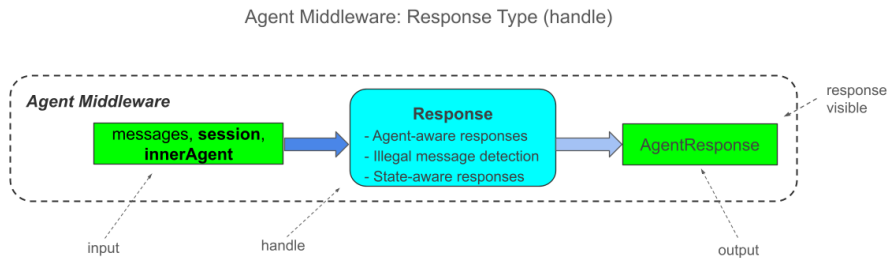


Figure 10.5 Agent Response middleware receives messages, session, and an `AI Agent` `innerAgent`, and returns `AgentResponse`. The pipeline is built by repeatedly wrapping the agent, passing the previously wrapped instance as `innerAgent`. Typical use cases include auditing with agent identity, persona-aware response stamping, error handling, and risk escalation.

The differences from *ChatClient Response* are consistent. The return type is `AgentResponse` instead of `ChatResponse`. What the *Agent* layer gains in exchange is agent identity through `innerAgent.Name` and full session access through `session.StateBag`, neither of which exist at the *ChatClient* layer. The firing frequency also shifts: where *ChatClient Response* middleware fires on every LLM round-trip, *Agent Response* fires once per `RunAsync()` call, giving it a view of the complete agent run rather than individual chat exchanges.

IMPORTANT *ChatClient* layer measures the chat request (LLM call). *Agent* layer measures the full agent run.

PRACTICAL EXAMPLE: CAPTAIN'S LOG

When Robby malfunctioned during a client demo, our logs showed 14 tool calls executed but couldn't tell us which agent, which session, or what mission was running. *ChatClient* timestamps were anonymous. We needed *Agent Response* middleware with access to `agent.Name` and `session.StateBag` for identity-aware audit trails.

Trying to build Captain's Log with *ChatClient* middleware is anonymous, because it does not know which agent produced the response or which session is active. On the other hand, the *Agent* version creates an identity-aware journal prefix like in the following sample:

```
Stardate 2025-01-15T14:30:00.0000000+00:00. Agent MotorsAgent. Session
00ABCDEF. Env production. Tools fired: 2.
```

In listing 10.5 we can observe the implementation of it.

Listing 10.5 CaptainsLog

```

public static async Task<AgentResponse> CaptainsLog(
    IEnumerable<ChatMessage> messages,
    AgentSession? session,
    AgentRunOptions? options,
    AIAgent innerAgent,
    CancellationToken cancellationToken)
{
    AgentResponse response = await innerAgent
        .RunAsync(messages, session, options, cancellationToken);

    string timestamp = DateTimeOffset.UtcNow.ToString("o"); //❶
    int toolCallCount = response.Messages.Sum(m => m
        .Contents.OfType<FunctionCallContent>().Count()); //❷
    string toolTag = toolCallCount > 0
        ? $"Tools fired: {toolCallCount}."
        : string.Empty;
    string? missionTag = null;
    session?.StateBag.TryGetValue("MissionTag", out missionTag); //❸
    string journalPrefix = $"Stardate {timestamp}. "
        + $"Mission {missionTag}. Agent {innerAgent.Name}.{toolTag} "; //❹
    Console.WriteLine($"[Agent] [Response] [CaptainsLog] {journalPrefix}");

    foreach (ChatMessage message in response.Messages)
    {
        if (message.Role == ChatRole.Assistant &&
            !string.IsNullOrEmpty(message.Text))
        {
            message.Contents = [new
                TextContent($"{{journalPrefix}}{message.Text}")]; //❺
        }
    }

    return response;
}

```

- ❶ Initializes the timestamp for Captain's log
- ❷ Counts the tool callings
- ❸ Fetches the operator's name from the StateBag
- ❹ Prepares the journal prefix
- ❺ Prefixes the response with the journal prefix

Comparing the current example with the corresponding *Response* type middleware from *ChatClient* in chapter 9, `AddTimestamp` example, we observe that along with prefixing the response, this time we can read data from the agent state (*StateBag*) and build a persona-aware journal.

PRACTICAL EXAMPLE: MOVEMENT SEQUENCE AUDITOR

Robby executed `forward-backward-forward-backward-stop` movement sequence and burned out the motor controller. *ChatClient* middleware could constrain individual arguments but couldn't detect dangerous patterns across multiple tool calls. We needed *Agent Response* middleware to inspect the completed tool sequence and flag `forward-backward` or `backward-forward` reversals missing the required stop command.

`MovementSequenceAuditor` is an *Agent Response* middleware that inspects the completed tool sequence after the agent run finishes and identifies the illegal sequences such in:

```
forward-forward-backward-stop
```

The sequence `forward-backward` is not safe for engine, instead the robot should stop before reverting the direction. Example of a valid sequence that has "stop" in between:

```
forward-forward-stop-backward-stop
```

The middleware in listing 10.6 audits the completed sequence and warns when a reversal such as `forward-backward` or `backward-forward` appears.

Listing 10.6

```

public static async Task<AgentResponse> MovementSequenceAuditor(
    IEnumerable<ChatMessage> messages,
    AgentSession? session,
    AgentRunOptions? options,
    AIAgent innerAgent,
    CancellationToken cancellationToken)
{
    AgentResponse response = await innerAgent
        .RunAsync(messages, session, options, cancellationToken);

    string sequence = string.Join("-", response.Messages
        .SelectMany(m => m.Contents.OfType<FunctionCallContent>())
        .Select(f => f.Name.ToLowerInvariant())); //❶

    bool isIllegal = sequence.Contains("forward-backward")
        || sequence.Contains("backward-forward"); //❷

    if (isIllegal)
    {
        ChatMessage? assistantMessage = response.Messages
            .LastOrDefault(m => m.Role == ChatRole.Assistant); //❸

        assistantMessage?.Contents = [new TextContent("WARNING: "
            + $"illegal direction '{assistantMessage.Text}' "
            + "reversal detected in executed sequence. "
            + "A stop command was missing.")] //❹
    }

    return response;
}

```

❶ Extracts the function names from function calls

❷ Checks for illegal sequences

❸ Get the last assistant message

❹ Prefixes the last assistant message with a warning

Here we can observe that *Response* middleware cannot block the illegal sequence of tool calls, it just observes them and audits them. We can even append a warning in response if we like.

Agent Response middleware should be used for run-level concerns: audit, persona-aware response shaping, completed tool-sequence analysis, session-aware response formatting, and agent-specific error handling.

Let's see in listing 10.7 how to wire both *Response* functions into the middleware pipeline.

Code path: `/AgentResponseMiddleware/Program.cs`

Listing 10.7 Response Middleware

```

// 'usings' and API key fetching omitted for brevity

ChatClientAgent motorsAgent = new OpenAIClient(apiKey)
    .GetChatClient(model)
    .AsAIAgent(new ChatClientAgentOptions {
        Name = "MotorsAgent",
        Description = "Controls robot car basic movements.",
        ChatOptions = new ChatOptions
        {
            Instructions = ""
            You are the MotorsAgent controlling a robot car.
            Break down complex movement commands into basic moves:
            forward, backward, turn_left, turn_right, stop.
            Respond with the sequence of moves needed to accomplish the task.
            "",
            Tools = [.. MotorTools.AsAITools()],
        }
    });

```

```

IAIAgent motorsAgentWithMiddleware = motorsAgent
    .AsBuilder()
    .Use(runFunc: AgentResponses.MovementSequenceAuditor,
        runStreamingFunc: null) //❶
    .Use(runFunc: AgentResponses.CaptainsLog, runStreamingFunc: null) //❷
    .Build();

AgentSession session = await motorsAgent.CreateSessionAsync();
session.StateBag.SetValue("MissionTag", "MISSION-45 | EXPLORATION"); //❸

var prompt1 = "Move forward 5 meters";
Console.WriteLine($"PROMPT: {prompt1}");
var result1 = await motorsAgentWithMiddleware
    .RunAsync(prompt1, session); //❹
Console.WriteLine($"RESULT: {result1}\n");

var prompt2 = "Move forward 2 meters and immediatelly move backward 2 meters.";
Console.WriteLine($"PROMPT: {prompt2}");
var result2 = await motorsAgentWithMiddleware
    .RunAsync(prompt2, session); //❺
Console.WriteLine($"RESULT: {result2}\n");

```

- ❶ Adds movement auditor middleware
- ❷ Adds captain's log middleware
- ❸ Pre-populates mission tag session state
- ❹ Queries the agent with first prompt
- ❺ Queries the agent with an illegal movement sequence

Sample output:

```

PROMPT: Move forward 5 meters
[01:49:02:878] MOTORS: Forward: 5m
[01:49:04:704] MOTORS: Stop
[Agent] [Response] [CaptainsLog] Stardate 2026-05-21T11:49:06.6266705+00:00.
Mission MISSION-45 | EXPLORATION. Agent MotorsAgent. Tools fired: 2.
RESULT: Stardate 2026-05-21T11:49:06.6266705+00:00. Mission MISSION-45 |
EXPLORATION. Agent MotorsAgent. Tools fired: 2. forward 5m
Stop

PROMPT: Move forward 2 meters and immediatelly move backward 2 meters.
[01:49:07:503] MOTORS: Forward: 2m
[01:49:09:656] MOTORS: Backward: 2m
[01:49:11:537] MOTORS: Stop
[Agent] [Response] [CaptainsLog] Stardate 2026-05-21T11:49:13.3465166+00:00.
Mission MISSION-45 | EXPLORATION. Agent MotorsAgent. Tools fired: 3.
[Agent] [Response] [SequenceAuditor] ILLEGAL sequence detected – stop required
between direction reversals
RESULT: WARNING: illegal direction 'Stardate 2026-05-21T11:49:13.3465166+00:00.
Mission MISSION-45 | EXPLORATION. Agent MotorsAgent. Tools fired: 3. forward 2m
backward 2m
stop' reversal detected in executed sequence. A stop command was missing.

```

The response is audited, including illegal sequence of messages. The response is prefixed with a captain log, exemplifying direct and full access to the agent response.

EXERCISE

Create `CacheAgentResponse` Response middleware that caches the `AgentResponse` for identical user queries within the same session. On cache hit, return the cached response without calling `innerAgent.RunAsync()`. Hint: use a `Dictionary<string, AgentResponse>` keyed by the user prompt text.

10.2.4 Supervising Tools Calls with FunctionCalling Middleware

At the *ChatClient* layer, *FunctionCalling* was terminal: no *next* delegate, no automatic pipelining, and the developer was responsible for maintaining the null/non-null interceptor-executor contract (the

convention used to mix multiple *ChatClient FunctionCalling* middleware functions inside the *FunctionInvoker*). At the *Agent* layer that complexity disappears. *FunctionCalling* receives a *next* delegate and composes exactly like *SharedFunction* and *Response*, registering multiple middleware using `.Use(...)` and each one calling the next middleware in the pipeline.

Figure 10.6 shows the *Agent FunctionCalling* middleware type.

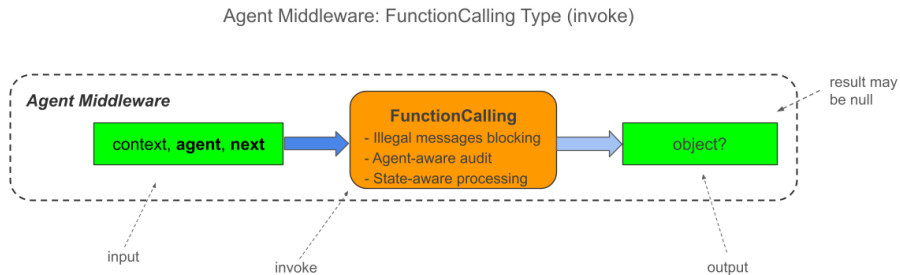


Figure 10.6 Agent *FunctionCalling* middleware is fully chainable via *next* delegate. Each middleware in the chain *next* middleware to pass control to the next one. The framework invokes the actual tool at the end of the chain. Agent identity is available through `agent.Name` on every middleware in the chain.

The Agent *FunctionCalling* middleware uses the regular `.Use(...)` pipeline builder, instead of a single *FunctionInvoker* lambda like we have in *ChatClient* middleware. What disappears entirely is the need to distinguish interceptors from executors and to ensure exactly one component calls `InvokeAsync`.

IMPORTANT *FunctionInvoker* in *ChatClient* middleware keeps function invocation as a single low-level hook inside *FunctionInvokingChatClient* (via `.UseFunctionInvocation`), instead of introducing another nested middleware pipeline inside the *ChatClient* pipeline with its own automatic tool-calling behavior inside the *ChatClient* pipeline.

One more detail to keep in mind is how we mix `InvokeAsync` with *next* delegate in *Agent FunctionCalling* middleware. Nothing stops us from calling `InvokeAsync` directly, but the recommended pattern is to call *next* instead.

WARNING Do not call `InvokeAsync()` directly in *Agent FunctionCalling* middleware. If we call both `InvokeAsync` and *next*, the tool runs twice.

PRACTICAL EXAMPLE: PREVENT DANGEROUS MOVES

After the motor burnout, post-mortem auditing wasn't enough. When Robby received "Go forward then immediately reverse" we needed to block execution before hardware damage occurred. *Agent FunctionCalling* middleware with `context.Messages` access could detect the pattern and return "BLOCKED: stop required first" before the reversal executed.

The *ChatClient FunctionCalling* example in chapter 9 has `ConstrainDistance`, which limits backward distance to 5 meters. That works at the tool calling layer, but it is agent agnostic, it has

only the current function invocation context. On the other hand, in the previous *Agent Response* example, *MovementSequenceAuditor*, we observe post-mortem what happened, but what if we want to prevent a dangerous sequence before it happens?

Let's see in listing 10.8 how an illegal sequence can be prevented for a specific agent just before it happens.

Listing 10.8 PreventDangerousMoves

```
public static async ValueTask<object?> PreventDangerousMoves (
    AIAgent agent,
    FunctionInvocationContext context,
    Func<FunctionInvocationContext, CancellationToken, ValueTask<object?>> next,
    CancellationToken cancellationToken)
{
    if (!string.Equals(agent.Name, MotorsAgentName)) //❶
    {
        Console.WriteLine("[Agent] [FunctionCall] [Safety] "
            + $"SKIP: rule does not apply to agent '{agent.Name}');
        return await next(context, cancellationToken);
    }

    string sequence = string.Join("-", context.Messages
        .SelectMany(m => m.Contents.OfType<FunctionCallContent>())
        .Select(f => f.Name.ToLowerInvariant())); //❷

    bool isIllegal = sequence.Contains("forward-backward")
        || sequence.Contains("backward-forward"); //❸

    if (isIllegal)
    {
        Console.WriteLine("[Agent] [FunctionCall] [Safety] "
            + "BLOCKED: illegal reversal detected - a 'stop' is required first.");
        return "BLOCKED: cannot execute the command due to illegal reversal."
            + "Issue a stop command first."; //❹
    }

    return await next(context, cancellationToken);
}
```

- ❶ Prevents the execution unauthorized agents
- ❷ Builds the movement sequence
- ❸ Checks if the sequence is illegal
- ❹ Reports blocked sequence

This is a better fit for the *Agent* layer because the rule is not just “cap this argument.” It depends on the in-flight conversation, owner, or prior tool calls.

PRACTICAL EXAMPLE: FUNCTION CALL AUDIT

Our *ChatClient* audit logged “Forward(10) completed” but when three agents ran simultaneously, we couldn't identify which agent or session. *ChatClient* is anonymous by design. We needed *Agent* FunctionCalling middleware to log “Agent MotorsAgent, Session 00ABCDEF, Operator driver: Forward(10) completed” with full identity context.

AuditAgentFunctionCalls in listing 10.9 is the *Agent* equivalent of the *ChatClient* tool audit sample, but with two important improvements: it is fully chainable using *next* delegate, and it has identity (*agent.Name*).

Listing 10.9 AuditAgentFunctionCalls

```
public static async ValueTask<object?> AuditAgentFunctionCalls (
    AIAgent agent,
    FunctionInvocationContext context,
    Func<FunctionInvocationContext, CancellationToken, ValueTask<object?>> next,
```

```

Cancellation token cancellationToken)
{
    var functionName = context.Function.Name;
    var args = string.Join(", ", context.Arguments.Select(kvp =>
        $"{kvp.Key}={kvp.Value}")); //❶
    var timestamp = DateTime.UtcNow;

    Console.WriteLine($"[Agent] [FunctionCall] [Audit] [{timestamp:HH:mm:ss}] "
        + $"Agent '{agent.Name}' invoking '{functionName}({args})'; //❷

    try
    {
        var result = await next(context, cancellationToken);
        Console.WriteLine($"[Agent] [FunctionCall] [Audit] [{timestamp:HH:mm:ss}] "
            + $"COMPLETED | Result: {result}");

        return result;
    }
    catch (Exception ex)
    {
        Console.WriteLine($"[Agent] [FunctionCall] [Audit] [{timestamp:HH:mm:ss}] "
            + $"FAILED | Error: {ex.Message}");
        throw; //❸
    }
}

```

❶ Serializes the function arguments

❷ Prints the timestamp and arguments

❸ Returns early from the middleware blocking the illegal tool execution

Let's see in listing 10.10 how to wire both *FunctionCalling* functions into the middleware pipeline.

Code path: /AgentFunctionCallingMiddleware/Program.cs

Listing 10.10 FunctionCalling Middleware

```

// 'usings' and API key fetching omitted for brevity

ChatClientAgent motorsAgent = new OpenAIClient(apiKey)
    .GetChatClient(model)
    .AsAIAgent(new ChatClientAgentOptions
    {
        Name = "MotorsAgent",
        Description = "Controls robot car basic movements.",
        ChatOptions = new ChatOptions
        {
            Instructions = """
                You are the MotorsAgent controlling a robot car.
                Break down complex movement commands into basic moves:
                forward, backward, turn_left, turn_right, stop.
                Respond with the sequence of moves needed to accomplish the task.
                """,
            Tools = [.. MotorTools.AsAITools()],
        }
    });

AIAgent motorsAgentWithAllMiddleware = motorsAgent
    .AsBuilder()
    .Use(AgentFunctionCallings.PreventDangerousMoves) //❶
    .Use(AgentFunctionCallings.AuditAgentFunctionCalls) //❷
    .Build(); //❸

AgentSession session = await motorsAgent.CreateSessionAsync();

var prompt1 = "Move forward 10 meters.";
Console.WriteLine($"PROMPT: {prompt1}");
AgentResponse result1 = await motorsAgentWithAllMiddleware

```

```

.RunAsync(prompt1, session); //❷
Console.WriteLine($"\\nRESULT: {result1}\\n");

var prompt2 = "Move forward 2 meters and immediatelly move backward 2 meters.";
Console.WriteLine($"PROMPT: {prompt2}");
AgentResponse result2 = await motorsAgentWithAllMiddleware
.RunAsync(prompt2, session); //❸
Console.WriteLine($"\\nRESULT: {result2}\\n");

```

- ❶ Adds middleware to prevent dangerous moves
- ❷ Adds middleware to audit agent function calls
- ❸ Builds an agent with middleware from an agent without middleware
- ❹ Queries the agent with legal sequence
- ❺ Queries the agent with illegal sequence

Code output:

```

PROMPT: Move forward 10 meters.
[Agent] [FunctionCall] [Audit] [12:41:46] Agent 'MotorsAgent' invoking
'Forward(distance=10)'
[02:41:46:780] MOTORS: Forward: 10m
[Agent] [FunctionCall] [Audit] [12:41:46] COMPLETED | Result: moved forward for 10
meters.
[Agent] [FunctionCall] [Audit] [12:41:49] Agent 'MotorsAgent' invoking 'Stop()'
[02:41:49:536] MOTORS: Stop
[Agent] [FunctionCall] [Audit] [12:41:49] COMPLETED | Result: stopped.

RESULT: forward 10
stop

PROMPT: Move forward 2 meters and immediatelly move backward 2 meters.
[Agent] [FunctionCall] [Audit] [12:41:52] Agent 'MotorsAgent' invoking
'Forward(distance=2)'
[02:41:52:015] MOTORS: Forward: 2m
[Agent] [FunctionCall] [Audit] [12:41:52] COMPLETED | Result: moved forward for 2
meters.
[Agent] [FunctionCall] [Audit] [12:41:53] Agent 'MotorsAgent' invoking
'Backward(distance=2)'
[Agent] [FunctionCall] [Safety] BLOCKED: illegal reversal detected – a 'stop' is
required first.
[Agent] [FunctionCall] [Audit] [12:41:53] COMPLETED | Result: BLOCKED: cannot
execute the command due to illegal reversal. Issue a stop command first.
[Agent] [FunctionCall] [Audit] [12:41:55] Agent 'MotorsAgent' invoking 'Stop()'
[Agent] [FunctionCall] [Safety] BLOCKED: illegal reversal detected – a 'stop' is
required first.
[Agent] [FunctionCall] [Audit] [12:41:55] COMPLETED | Result: BLOCKED: cannot
execute the command due to illegal reversal. Issue a stop command first.
[Agent] [FunctionCall] [Audit] [12:41:55] Agent 'MotorsAgent' invoking 'Stop()'
[Agent] [FunctionCall] [Safety] BLOCKED: illegal reversal detected – a 'stop' is
required first.
[Agent] [FunctionCall] [Audit] [12:41:55] COMPLETED | Result: BLOCKED: cannot
execute the command due to illegal reversal. Issue a stop command first.

RESULT: forward 2
stop
backward 2
stop

```

Agent FunctionCalling is the better layer for agent-specific safety, approval, authorization, auditing, and sequence-aware tool policies.

ChatClient FunctionCalling is still useful for universal, stateless tool constraints that apply to every agent using the client.

EXERCISE

Let's build an error recovery: Create a `RetryOnFailure` *FunctionCalling* middleware that automatically retries failed function calls up to 3 times with exponential backoff. Since *Agent FunctionCalling* is chainable, this middleware can wrap around `AuditFunctionCalls` transparently.

10.3 Composing the Full Agent Middleware Pipeline

Each Agent middleware type has now been introduced in isolation. The full pipeline combines all three, following the same Prepare → Handle → Invoke mental model from chapter 9, but now enriched with session context and agent identity at every Agent middleware type.

10.3.1 Stacking SharedFunction, Response, and FunctionCalling

The same registration order principle as for *ChatClient* middleware applies here: whatever is registered first runs first on the way in, and last on the way out.

Two structural differences from *ChatClient* layer:

Agent middleware fires once per `RunAsync` rather than once per LLM round-trip, which is why token budgets and rate limiting belong at the *ChatClient* layer.

- *Agent FunctionCalling* is chainable, each method receives a *next* delegate and delegates to it, so the interceptor-executor pattern from *ChatClient FunctionCalling* (chapter 9) is not needed here.

Figure 10.7 shows how we mix and order the three *Agent* middleware types.

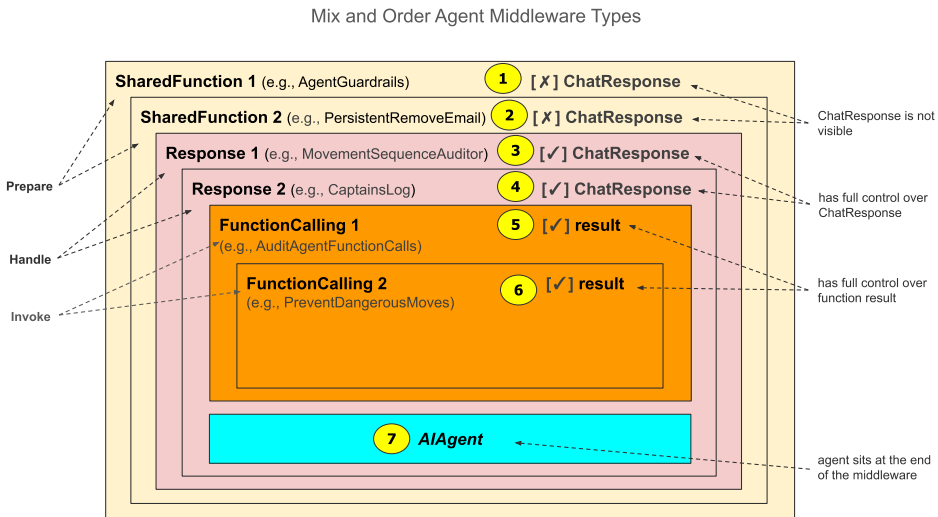


Figure 10.7 All three Agent middleware types composing together. SharedFunction functions (outermost, steps 1-2) prepares input, ChatResponse invisible. Response functions (middle, steps 3-4) wrap the LLM exchange, ChatResponse visible and controllable. FunctionCalling (innermost, steps 5-6) wrap the tool calls. AI Agent (step 7) sits at the end of the middleware pipeline. Registration order equals execution order: prepare → handle → invoke.

Agent's middleware steps:

1. `SharedFunction 1: AgentGuardrails` starts the **Prepare** stage, reads `AgentSession` state such as `OperatorName`, and can stop unauthorized requests before the agent runs; `AgentResponse` is not visible here.
2. `SharedFunction 2: PersistentRemoveEmail` continues the **Prepare** stage by transforming messages, such as removing email addresses before the agent sees them; `AgentResponse` is still not visible.
3. `Response 1: MovementSequenceAuditor` enters the **Handle** stage, where `AgentResponse` is visible and the completed movement sequence can be inspected for unsafe behavior.
4. `Response 2: CaptainsLog` adds another **Handle**-stage middleware that can decorate or modify the completed `AgentResponse` with agent-aware metadata.
5. `FunctionCalling 1: AuditAgentFunctionCalls` enters the **Invoke** stage, where middleware works with individual tool invocation results instead of the full `AgentResponse`.
6. `FunctionCalling 2: PreventDangerousMoves` adds another chainable *FunctionCalling* middleware that can inspect tool invocation context and block dangerous movement reversals before a tool runs.
7. `AIAgent` sits at the end of the middleware stack, receives the prepared request, performs reasoning and tool orchestration, and produces the `AgentResponse` that flows back outward.

The recommended order for *Agent* middleware types is:

- *SharedFunction* Layer — **Prepare**

The *Agent SharedFunction* layer prepares the agent run before the agent begins processing. Like the *ChatClient* version, this layer does not see the final response. In the *Agent* pipeline, which means `AgentResponse` is not visible here.

The key difference is that *Agent SharedFunction* middleware receives `AgentSession`. This gives it access to session state through the `StateBag`. That makes it suitable for per-session and per-agent rules, not only app-wide request shaping.

In this sample, `AgentGuardrails` reads session state such as `OperatorName` and `MissionTag`, while `PersistentRemoveEmail` sanitizes messages before the agent sees them.

- *Response* Layer — **Handle**

The *Agent Response* layer handles the completed `AgentResponse`. This is where the full agent result becomes visible and can be inspected or modified.

This layer is useful for policies that depend on what the agent actually produced. In this sample, `MovementSequenceAuditor` inspects the completed movement sequence, while `CaptainsLog` decorates the response with agent-aware journal information.

- *FunctionCalling* Layer — **Invoke**

The *Agent FunctionCalling* layer controls individual tool invocations. It does not operate on the full `AgentResponse`; it operates around each tool call and its function result.

The major difference from *ChatClient FunctionCalling* is that *Agent FunctionCalling* middleware is chainable. It has a next delegate, so multiple *FunctionCalling* middleware can be stacked naturally.

In this sample, `AuditAgentFunctionCalls` records timing and agent identity, while `PreventDangerousMoves` blocks unsafe movement reversals before a tool runs.

- `AIAgent` — End of the Pipeline

At the end of the *Agent* middleware stack sits the `AIAgent`. The agent receives the prepared messages, reasons over them, may invoke tools, and produces an `AgentResponse`.

IMPORTANT The important mental model is: 1) *SharedFunction* middleware prepares the agent run and can use `AgentSession` state before calling the next layer; 2) *Response* middleware handles the completed `AgentResponse`; 3) *FunctionCalling* middleware controls individual tool invocations and can be chained with next; 4) `AIAgent` ends the middleware pipeline.

This order keeps responsibilities separated. *SharedFunction* middleware prepares the run, *Response* middleware handles the completed response, and *FunctionCalling* middleware supervises the tool execution. The result is a pipeline where request policy, response policy, and tool policy each live in the layer that has the right context.

10.3.2 Applying the Prepare–Handle–Invoke Middleware Pattern

This is the recommended composition for the Agent middleware types. Each concern is placed naturally where it has the right visibility and the right timing: validation and sanitization before tokens are spent, response control around the model call, and tool supervision only when a function is about to execute.

Listing 10.11 shows the full pipeline registration following the **Prepare** → **Handle** → **Invoke** mental model.

Listing 10.11 Full Pipeline Registration

```
AIAgent motorsAgentWithFullPipeline = motorsAgent
    .AsBuilder()
    .Use(sharedFunc: AgentSharedFunctions.AgentGuardrails) // ❶
    .Use(sharedFunc: AgentSharedFunctions.PersistentRemoveEmail) // ❶
    .Use(runFunc: AgentResponses.MovementSequenceAuditor,
        runStreamingFunc: null) // ❷
    .Use(runFunc: AgentResponses.CaptainsLog, runStreamingFunc: null) // ❷
    .Use(AgentFunctionCallings.AuditAgentFunctionCalls) // ❸
    .Use(AgentFunctionCallings.PreventDangerousMoves) // ❸
    .Build(); // ❹
```

- ❶ **SharedFunction** — Prepare layer (outermost)
- ❷ **Response** — Handle layer
- ❸ **FunctionCalling** — Invoke layer (innermost)
- ❹ **Builds the Agent middleware for the agent**

10.3.3 ChatClient vs Agent: Side-by-Side Comparison

Table 10.1 summarizes where each cross-cutting concern belongs and why, using concrete examples across both layers.

Table 10.1 ChatClient vs. Agent middleware: when to use each (some suggestions).

| Concern | ChatClient layer | Agent layer |
|---------|------------------|-------------|
|---------|------------------|-------------|

| | | |
|---|--|---|
| PII (Personally Identifiable Information) removal (e.g., email, bank account) | Transient. LLM sees clean data, history retains original | Persistent. History stores clean data because sanitized messages enter the agent before it records them |
| Request limiting | Global process-wide counter | Per-session counter via <code>StateBag</code> |
| Response timestamping | Anonymous transport-level stamp | Persona-aware journal entry with <code>agent.Name</code> |
| Error handling | Generic (no agent identity) | Persona-specific message using <code>agent.Name</code> |
| Tool arguments validation | Terminal, hardcoded limit | Chainable, session-configurable limit |
| Tool audit logging | Terminal, anonymous | Chainable, includes <code>agent.Name</code> |
| Tenant guardrails | Not possible (no session) | Full access to <code>session.StateBag</code> |
| Feature flags | Not possible (no session) | Per-session, per-agent flags |
| Human approval gate | Possible but anonymous | Full agent identity in approval prompt |
| Per-session tool budget | Not possible (no session) | Per-tool usage counter in <code>StateBag</code> |

From designing perspective (table 10.2), it’s better to look at what each middleware type sees depending on the layer (*Agent* or *ChatClient*).

Table 10.2 What each middleware layer and type can see

| Layer | Middleware Type | Sees Response | Sees Tool Calls | Sees Agent Session & Identity |
|-------------------|-----------------|----------------------------------|----------------------------------|--|
| ChatClient | SharedFunction | ✗ | ✗ | ✗ |
| ChatClient | Response | ☑ (<code>ChatResponse</code>) | ☑ (<code>ChatResponse</code>) | ✗ |
| ChatClient | FunctionCalling | ✗ | ☑ (terminal) | ✗ |
| Agent | SharedFunction | ✗ | ✗ | ☑ (<code>AgentSession</code>) |
| Agent | Response | ☑ (<code>AgentResponse</code>) | ☑ (<code>AgentResponse</code>) | ☑ (<code>innerAgent.Name</code> , <code>AgentSession</code>) |
| Agent | FunctionCalling | ✗ | ☑ (pipeline) | ☑ (<code>agent.Name</code>) |

WHEN TO USE EACH MIDDLEWARE LAYER

Choose the *Agent* layer when you need `session.StateBag` for per-session state such as quotas, risk scores, or feature flags; when the policy must varies by agent (*MotorsAgent* enforces reversal prevention, another agent does not); when you need `agent.Name` for logging, authorization, or approval prompts; or when you are tracking multi-turn conversation state across a session.

Choose the *ChatClient* layer when a policy must apply to every agent with no exceptions; when the concern is stateless and requires no session; or when you need to count every LLM round-trip rather than every user prompt — global rate limiting and latency are the clearest examples.

Use both layers for any concern where defense in depth matters. PII redaction at the *ChatClient* layer is a failsafe that fires even if a caller bypasses `agent.RunAsync` and calls `IChatClient`

directly; the *Agent* layer then adds the per-session, per-agent refinement on top. Compliance requirements in particular — GDPR, HIPAA — should never rely on a single layer.

COMMON AGENT MIDDLEWARE ANTI-PATTERNS

The examples above show the recommended composition. Here are three mistakes that developers commonly make when building their first *Agent* middleware pipeline, and why each one breaks.

Agent middleware composition anti-patterns:

- Registering *SharedFunction* after *Response*. *SharedFunction*'s pre-phase must run before the agent starts. Registered after *Response*, it can never sanitize or rate-limit before the agent runs. Always register *SharedFunction* outermost.
- Calling *InvokeAsync* in *Agent FunctionCalling*. The framework invokes the tool at the end of the next chain. Calling *InvokeAsync* and then next executes the tool twice. At the *Agent* layer, always delegate to *next* and never call *InvokeAsync* directly.
- Applying per-agent middleware to the shared *IChatClient*. If *MotorsAgent*, for example, needs different policies, register the middleware on *Agent* layer via *.AsBuilder()*. Wiring it to the shared *IChatClient* applies it to both agents equally, with no way to distinguish between them.

10.3.4 Exercises

Let's create a multi-level approval: Clone the *PreventDangerousMoves* to *RequireHumanApprovalForDangerousMoves*. We need to require manager approval (typing a confirmation message) for driving backwards at distances larger than 20 meters.

Summary

- *Agent* middleware complements *ChatClient* middleware.
 - The *ChatClient* layer is the universal safety net.
 - The *Agent* layer adds per-agent, per-session, identity-aware control.
 - Deploy critical guardrails at both layers for defense in depth.
- Three *Agent* middleware types map to **Prepare** → **Handle** → **Invoke**.
 - *SharedFunction* prepares input with session context.
 - *Response* wraps the entire agent run and sees *AgentResponse*.
 - *FunctionCalling* supervises individual tool calls with a chainable next delegate.
- *Agent* middleware fires once per *RunAsync()* call. A single user prompt with multiple tool calls still activates *SharedFunction* and *Response* middleware only once. *FunctionCalling* fires once per tool invocation.
- Session context is the key *Agent* layer advantage. *session.StateBag* enables per-user budgets, tenant guardrails, feature flags, and conversation-aware decisions — all impossible at the *ChatClient* layer.
- Agent identity enables per-agent behavior. *innerAgent.Name* in *Response* and *agent.Name* in *FunctionCalling* let middleware produce agent-specific audit logs, error messages, approval prompts, and scoped safety rules.
- Both layers have *context.Messages* in *FunctionCalling*.
 - Either layer can inspect prior tool calls in the chat history.
 - The meaningful differences are that *Agent FunctionCalling* is chainable (multiple

`.Use()` calls) and carries `agent.Name`, while *ChatClient FunctionCalling* is terminal (uses `FunctionInvoker` lambda) and anonymous.

- *Agent FunctionCalling* is fully chainable. Unlike the *ChatClient* layer's terminal gate-invoker pattern, *Agent FunctionCalling* uses a standard next delegate. Multiple middleware compose naturally with `.Use()` calls. Never call `InvokeAsync()` directly on the *Agent* layer.
- Sanitization persistence differs by layer. *ChatClient* sanitization is transient — the LLM sees clean data on each round-trip, but the agent session history retains the original. *Agent* sanitization is persistent — sanitized messages enter the agent before it records them, so the history itself is clean for all future round-trips. Choose based on whether history needs to be clean.

11

Preparing reliable agents with observability and evaluation

This chapter covers

- OpenTelemetry telemetry for Agent Framework agents
- Traces, logs, and metrics for agent observability
- Evaluating and testing agent quality with built-in evaluators
- Evaluating and testing agents with composite and custom evaluators

Imagine Robby, the robot car, receiving the instruction: `"avoid the tree"`. From the outside, we see only the final movement. Inside, the agent may call the model, invoke motor tools, send tool results back to the model, and perform several hidden steps before answering.

Without observability, Robby is a black box. If it turns the wrong way, repeats a command, or calls the wrong tool, we can only guess why. *OpenTelemetry* gives Robby a flight recorder that shows which agent ran, which model was called, which tools executed, how long each step took, and where failures occurred.

OpenTelemetry, often abbreviated as *OTel*, is an open-source, vendor-neutral observability standard from the Cloud Native Computing Foundation. It defines common APIs, SDKs, exporters, and semantic conventions for collecting and forwarding telemetry data. Let's look at where we can use telemetry data.

11.1 Using Telemetry for Monitoring and Debugging

Observability is essential when building agents. A traditional application usually follows a predictable call path, but an agent can decide to call a model, invoke tools, call the model again with tool results, and then produce a final answer. Without telemetry, this internal behavior is hard to understand.

Applications can emit signals that can be sent to different backends, such as the console, Aspire Dashboard, Azure Monitor, Jaeger, Grafana, or any OTLP-compatible system. OTLP is the "wire format" *OpenTelemetry* uses to export telemetry from your app to a dashboard or monitoring backend.

For Agent Framework applications, *OpenTelemetry* is useful because it helps answer questions such as:

- Which agent handled the request?
- Which model was called?
- How many model roundtrips happened?
- Which tools were invoked?

- How long did the agent run take?
- How many tokens were used?
- Did the failure happen in the agent, the chat client, the model request, or a tool?

Agent Framework emits telemetry that follows the *OpenTelemetry* Generative AI semantic conventions.

Agent Framework's built-in telemetry isn't arbitrary. It follows OpenTelemetry's GenAI semantic conventions, a shared vocabulary of `gen_ai.*` attributes for describing LLM and agent operations consistently across frameworks and providers. The attributes you'll see in the console output come from the spec in table 11.1.

Table 11.1 OpenTelemetry GenAI attributes

| Attribute | What it captures |
|---|---|
| <code>gen_ai.operation.name</code> | Operation type (chat, execute_tool, invoke_agent) |
| <code>gen_ai.agent.name</code> / <code>gen_ai.agent.id</code> | Agent that produced the span |
| <code>gen_ai.provider.name</code> | Model provider (openai, azure.ai.openai, etc.) |
| <code>gen_ai.usage.input_tokens</code> / <code>output_tokens</code> | Token counts per model call |
| <code>gen_ai.tool.name</code> / <code>gen_ai.tool.call.id</code> | Tool that fired on execute_tool spans |

These conventions are marked Development stability, not stable. Attribute names have already changed since the spec's introduction (e.g., `gen_ai.system` became `gen_ai.provider.name`).

For the current, authoritative list, see the *OpenTelemetry* GenAI semantic conventions registry: <https://github.com/open-telemetry/semantic-conventions-genai>

11.1.1 Telemetry Types

OpenTelemetry works with three main telemetry signals: logs, traces, and metrics, summarized in table 11.2.

Table 11.2 Telemetry Types: Logs, Traces, and Metrics

| Type | Analogy | What it shows | Useful data |
|---------|-------------------|---|--|
| Logs | Logbook | Discrete events and messages | User input, agent response, tool invocation messages, warnings, exceptions |
| Traces | Flight recorder | Flow of one operation across components | Agent run span, chat/model request span, tool call span, duration, trace ID, span ID |
| Metrics | Stopwatch/counter | Numeric measurements over time | Token counts, request counts, latency, custom counters |

For agent applications, traces are usually the most valuable signal because they show the nested flow:

```
user request
→ agent run
  → chat request → HTTP request → HTTP response: tool call requested
  → tool invocation → tool result
  → chat request + tool result → HTTP request → HTTP response: final answer
→ agent response
```

HTTP telemetry is not a separate signal type. HTTP client instrumentation emits spans and metrics that appear as part of your existing traces and metric streams. A span represents a single traced operation, such as an HTTP request. For example, when the agent calls OpenAI or Azure OpenAI, the HTTP client instrumentation can emit spans and metrics for request duration, status codes, failures, and retries. These HTTP spans help connect the agent-level operation to the actual network call made to the model provider.

Telemetry output contains a large amount of information. To focus on LLM and agent activity, look for attributes such as:

```
gen_ai.operation.name
gen_ai.request.model
gen_ai.response.model
gen_ai.usage.input_tokens
gen_ai.usage.output_tokens
gen_ai.tool.name
agent.name
```

These attributes help identify what the agent did, which model it used, how many tokens were consumed, and which tools were involved. The exact attributes depend on the provider, model client, package version, and whether sensitive data collection is enabled.

11.1.2 OpenTelemetry Using Console Exporter

The *Console Exporter* is an *OpenTelemetry* exporter that writes collected telemetry data, such as traces, logs, and metrics, directly to the standard output stream. It is primarily intended for local development and debugging, allowing developers to inspect emitted telemetry without configuring external backends or storage.

PRACTICAL EXAMPLE

Robby's agent can use motor tools such as `Forward`, `Backward`, `TurnLeft`, `TurnRight`, and `Stop`. Agent Framework automatically emits telemetry for agent execution, including model calls, tool invocations, and their durations. This built-in telemetry is generic and does not capture domain-specific concerns.

To address this, listing 11.1 uses custom function invocation middleware to observe the actual tool calls and record a domain-specific metric and trace whenever Robby switches between forward and backward movement, which is considered dangerous for the engine.

Listing 11.1: DirectionChangeTracker

```
public class DirectionChangeTracker(ActivitySource activitySource,
    Counter<int> directionChangeCounter)
{
    private string? _previousLinearDirection;

    public async ValueTask<object?> TrackDirectionChangeAsync(
        AIAgent agent,
        FunctionInvocationContext context,
        Func<FunctionInvocationContext, CancellationToken, ValueTask<object?>> next,
        CancellationToken cancellationToken) // ❶
    {
        object? result = await next(context, cancellationToken);

        string? currentLinearDirection = context.Function.Name;

        if (!currentLinearDirection.Equals("Forward")
            && !currentLinearDirection.Equals("Backward")) // ❷
        {
            _previousLinearDirection = null; // ❸
            return result;
        }
    }
}
```

```

bool directionChanged = _previousLinearDirection is not null
    && _previousLinearDirection != currentLinearDirection; //4

using Activity? directionActivity = activitySource
    .StartActivity("Detect Robot Direction Change",
        ActivityKind.Internal); //5
directionActivity?.SetTag("agent.name", agent.Name); //6
directionActivity?.SetTag("robot.tool.name", context.Function.Name);
directionActivity?.SetTag("robot.direction.previous",
    _previousLinearDirection ?? "none");
directionActivity?.SetTag("robot.direction.current", currentLinearDirection);
directionActivity?.SetTag("robot.direction.changed", directionChanged); 6

if (directionChanged)
{
    directionChangeCounter
        .Add(1,
            new KeyValuePair<string, object?>("from", _previousLinearDirection),
            new KeyValuePair<string, object?>("to", currentLinearDirection)); //7
    directionActivity?.
        .AddEvent(new ActivityEvent("Robot direction changed")); //8
}

_previousLinearDirection = currentLinearDirection;

return result;
}
}

```

- ❶ Declares a FunctionCalling middleware function
- ❷ Ignores non-linear commands and resets direction memory
- ❸ Checks direction change (the command is not linear movement)
- ❹ True only when direction actually flips from the previous call
- ❺ Opens a custom span for this tool call
- ❻ Attaches domain tags to the span
- ❼ Increments the counter with from/to labels
- ❽ Marks the exact moment the flip was detected

Listing 11.2 shows an implementation in which a dangerous sequence of `AITools` is detected and sent to *OpenTelemetry* as custom traces and metrics using the *ConsoleExporter*.

Requirements (NuGet):

```

dotnet add package OpenTelemetry
dotnet add package OpenTelemetry.Exporter.Console
dotnet add package OpenTelemetry.Exporter.OpenTelemetryProtocol
dotnet add package OpenTelemetry.Extensions.Hosting

```

Listing 11.2 Wiring telemetry

```

// 'usings' and API key fetching omitted for brevity

var builder = Host.CreateApplicationBuilder(args);
builder.Logging.AddOpenTelemetry(logging =>
{
    logging.IncludeFormattedMessage = true;
    logging.IncludeScopes = true;
}); //1

const string SourceName = "OpenTelemetry.RobbyAgent";

builder.Services.AddOpenTelemetry() //2
    .WithTracing(tracing => tracing.AddSource(SourceName)
        .AddConsoleExporter()) //3
    .WithMetrics(metrics => metrics.AddMeter(SourceName)

```

```

        .AddConsoleExporter()); //❹

var host = builder.Build();
await host.StartAsync(); //❺

var loggerFactory = host.Services.GetRequiredService<ILoggerFactory>();
var meterFactory = host.Services.GetRequiredService<IMeterFactory>();

using var activitySource = new ActivitySource(SourceName); //❻
using var meter = meterFactory.Create(SourceName); //❼
var directionChangeCounter = meter
    .CreateCounter<int>("robot_direction_change_total",
        description: "Total times direction changes."); //❽
var directionChangeTracker = new DirectionChangeTracker(activitySource,
    directionChangeCounter); //❾

```

- ❶ Routes structured logs into the OpenTelemetry pipeline
- ❷ Registers OpenTelemetry as a hosted service in the DI container
- ❸ Exports every trace span to the console output
- ❹ Exports every metric reading to the console output
- ❺ Starts the host, so DI services (loggerFactory, meterFactory) are available
- ❻ Creates the named source that produces custom trace spans
- ❼ Creates the named meter that produces custom metrics
- ❽ Defines the counter that increments each time Robby reverses direction
- ❾ Instantiates the middleware, injecting the span source and the counter

With telemetry set up, we proceed in listing 11.3 by attaching telemetry to the agent.

Listing 11.3 Attaching telemetry to the agent

```

var agent = new OpenAIClient(apiKey)
    .GetChatClient(model)
    .AsIChatClient()
    .AsAIAgent(
        name: "RobotCarDemoAgent",
        instructions: ""
            You are an AI assistant controlling a robot car.
            The available robot car permitted moves are:
            forward, backward, turn left, turn right, and stop.
            "",
        tools: [... MotorTools.AsAITools()])
    .AsBuilder()
    .UseLogging(loggerFactory)
    .UseOpenTelemetry(SourceName) ❶
    .Use(directionChangeTracker.TrackDirectionChangeAsync) ❷
    .Build();

var session = await agent.CreateSessionAsync();

while (true)
{
    Console.WriteLine("You (or 'exit' to quit): ");
    var userInput = Console.ReadLine();

    if (string.IsNullOrWhiteSpace(userInput)
        || userInput.Equals("exit", StringComparison.OrdinalIgnoreCase))
        break;

    var response = await agent.RunAsync(userInput, session); ❸

    Console.WriteLine($"RESPONSE: {response.Text}");
}

await host.StopAsync();

```

- ❶ Routes all spans through SourceName, parenting them under one trace

- ❷ Inserts the custom direction-change middleware into the agent pipeline
- ❸ Waits for user input (commands)

The output is truncated because the collected telemetry is extensive.

Code output:

Type your message and press Enter. Type 'exit' or empty message to quit.
You (or 'exit' to quit):

Go back and forth 10 meters

...

```
[03:14:10:492] MOTORS: Backward: 10m
Activity.TraceId:          6dca477424d35a8ad66fc0f4d981256a
Activity.SpanId:           a181b530957dc7b0
Activity.TraceFlags:       Recorded
Activity.ParentSpanId:     b5b46054c329486f
Activity.DisplayName:      Detect Robot Direction Change
Activity.Kind:             Internal
Activity.StartTime:        2026-05-25T13:14:11.5041617Z
Activity.Duration:         00:00:00.0000786
Activity.Tags:
```

```
  agent.name: RobotCarDemoAgent
  robot.tool.name: Backward
  robot.direction.previous: none
  robot.direction.current: Backward
  robot.direction.changed: false
```

Instrumentation scope (ActivitySource):

```
  Name: OpenTelemetry.RobbyAgent
```

Resource associated with Activity:

```
  service.name: observations
  telemetry.sdk.name: opentelemetry
  telemetry.sdk.language: dotnet
  telemetry.sdk.version: 1.15.3
```

...

```
[03:14:12:538] MOTORS: Forward: 10m
Activity.TraceId:          6dca477424d35a8ad66fc0f4d981256a
Activity.SpanId:           3a15231ececbb450
Activity.TraceFlags:       Recorded
Activity.ParentSpanId:     b5b46054c329486f
Activity.DisplayName:      Detect Robot Direction Change
Activity.Kind:             Internal
Activity.StartTime:        2026-05-25T13:14:13.5490492Z
Activity.Duration:         00:00:00.0109351
Activity.Tags:
```

```
  agent.name: RobotCarDemoAgent
  robot.tool.name: Forward
  robot.direction.previous: Backward
  robot.direction.current: Forward
  robot.direction.changed: true
```

Activity.Events:

```
  Robot direction changed [25/05/2026 13:14:13 +00:00]
```

Instrumentation scope (ActivitySource):

```
  Name: OpenTelemetry.RobbyAgent
```

Resource associated with Activity:

```
  service.name: observations
  telemetry.sdk.name: opentelemetry
  telemetry.sdk.language: dotnet
  telemetry.sdk.version: 1.15.3
```

...

RESPONSE: Completed: moved backward 10 meters, then forward 10 meters.

You (or 'exit' to quit):

now stop

...

Metric Name: robot_direction_change_total, Description: Total times the robot

```

changed direction between forward and backward, Metric Type: LongSum
Instrumentation scope (Meter):
    Name: OpenTelemetry.RobbyAgent
(2026-05-25T13:13:59.5438745Z, 2026-05-25T13:14:19.4852327Z] from: Backward to:
Forward
Value: 1
...
RESPONSE: Stopped.
You (or 'exit' to quit):

```

Both custom signals (traces and metrics) appeared exactly where expected. The *Detect Robot Direction Change* span fired twice, once for *Backward* (no change) and once for *Forward* (change detected, event attached). The `robot_direction_change_total` metric reported `Value: 1` with the correct from/to labels. The `Stop` command produced nothing, as designed. This was possible only because the *AgentFunctionCalling* middleware, which we covered earlier, gave us a precise hook into every tool call without touching the agent logic or the LLM interaction at all.

11.2 Using Telemetry with Aspire Dashboard

The *Console Exporter* is useful for learning, but outputs everything as plain text mixed with application logs. The *Aspire Dashboard* provides a browser-based UI with a proper trace viewer, metrics graphs, and structured log search, with no code changes to your agent or middleware.

Requirements (NuGet):

```
dotnet add package OpenTelemetry.Exporter.OpenTelemetryProtocol
```

The only change needed is to replace the two `AddConsoleExporter()` calls with `AddOtlpExporter`:

```

builder.Services.AddOpenTelemetry()
    .WithTracing(tracing => tracing
        .AddSource(SourceName)
        .AddOtlpExporter()) // ❶
    .WithMetrics(metrics => metrics
        .AddMeter(SourceName)
        .AddOtlpExporter()); // ❷

```

❶ Sends traces to Aspire Dashboard over OTLP
 ❷ Sends metrics to Aspire Dashboard over OTLP

By default, the OTLP exporter sends to `http://localhost:[port]` (use the configured port here). We can start *Aspire Dashboard* standalone with Docker (a container platform that runs applications in isolated, lightweight environments), or we can have a dedicated project that starts *Aspire Host*. Then we can open *Aspire Dashboard* in your browser.

11.3 Evaluating Semantic Function Quality

Traditional unit tests work well for deterministic functions. For example, given input A, expect output B. But when Robby's functions start using LLMs, the same input might produce different valid outputs. How do we test "avoid the tree" when the LLM might suggest a 90-degree turn, a gentle curve, or another maneuver?

For this kind of behavior, the goal is no longer to assert in tests that the model's output text exactly matches a hard-coded expected string. The goal is to evaluate quality. `Microsoft.Extensions.AI.Evaluation` libraries let us move from exact-match testing to quality-based testing. Instead of asking "is this string exactly equal to this other string?" we ask questions such as:

- Did the agent understand the user's intent?
- Is the answer coherent?
- Is the answer grounded in the supplied facts?

- Did the agent call the correct tools?
- Did the response satisfy a custom safety rule?

Agent Framework already provides a set of built-in evaluators for common concerns such as coherence (does the answer make sense as a whole?), relevance (does it stay on topic?), grounding (does it stick to the supplied facts?), and tool usage. We can also define our own evaluators to reflect domain-specific rules, like Robby’s safety constraints or movement policies.

11.3.1 Why Quality Evaluation

With traditional unit testing, we assert the exact return value. With AI functions, we validate qualities like relevance, groundedness, and coherence instead of fixed criteria strings. Validating early pays off in concrete ways:

- catch hallucinations before they reach users.
- quantify quality: numeric or boolean metrics feed dashboards and CI gates, just like unit-test counts in classic .NET pipelines.
- compare prompts, models, and agent implementations.
- run regression checks in xUnit/nUnit/MSTest.
- keep AI behavior from drifting silently over time.

Tables 11.3, 11.4, and 11.5 summarize the built-in and custom evaluators.

Table 11.3 Built-in response quality evaluators.

| Dimension | Question it answers | Evaluator | Example failure |
|--------------|---|------------------------|--|
| Coherence | Is the response logically ordered and easy to follow? | CoherenceEvaluator | The response mixes robot moves with unrelated comments about weather, music, or shoes. |
| Groundedness | Is the response supported by supplied facts? | Groundedness Evaluator | The grounding allows only forward, backward, and turning moves, but the response says “jump” or “dodge.” |
| Relevance | Does the answer address the user’s prompt? | RelevanceEvaluator | The answer describes picnic weather instead of avoiding the tree. |
| Fluency | Is the response readable and natural? | FluencyEvaluator | The response is fragmented: “car go right then go go forward left...” |
| Equivalence | Does the answer match a known-good baseline? | Equivalence Evaluator | The answer produces a different maneuver sequence than the expected one. |
| Completeness | Does the answer cover all required parts? | Completeness Evaluator | Robby turns away from the tree but never returns to the original path. |

Table 11.4 Built-in agentic or tool-use evaluators.

| Dimension | Question it answers | Evaluator | Example failure |
|---------------------------|--|----------------------------|--|
| Intent resolution | Did the agent understand the user's goal? | IntentResolution Evaluator | The user asks Robby to avoid a tree, but the agent refuses or resolves the wrong intent. |
| Tool-call accuracy | Did the agent call the correct tools with the correct arguments? | ToolCallAccuracy Evaluator | The user asks for Forward(2), TurnLeft(90), Stop, but the agent calls Forward(5), TurnRight(90), Stop. |
| Task adherence | Did the agent follow task constraints? | TaskAdherence Evaluator | The user says, "stop immediately", but Robby moves forward. |

Table 11.5 Built-in retrieval, custom, composite, and safety evaluators

| Dimension | Question it answers | Evaluator | Example failure |
|----------------------------|---|--------------------------------|---|
| Retrieval | Was the retrieved context useful? | RetrievalEvaluator | Retrieval returns unrelated context instead of obstacle-avoidance guidance. |
| Composite evaluator | Does the response satisfy multiple constraints? | CompositeEvaluator | The response fails to meet one or more required evaluator thresholds. |
| Custom evaluator | Does the response satisfy a specific safety constraint? | e.g., DirectionChangeEvaluator | Robby moves backward and then forward without stopping between the movements, risking a dangerous engine maneuver. |
| Safety | Does it avoid unsafe output? | Safety evaluators | Safety evaluators use the Azure AI Foundry Evaluation service via an Azure AI project and are not covered here in detail. |

In addition to those evaluator types, some NLP evaluators use classical natural language processing techniques instead of LLMs.

WARNING Safety evaluators use the Microsoft Foundry evaluation service; quality evaluators use the configured judge model, which can be remote or local.

Figure 11.1 shows where evaluation tests sit in the CI (continuous integration) pipeline.

Evaluations in CI/CD

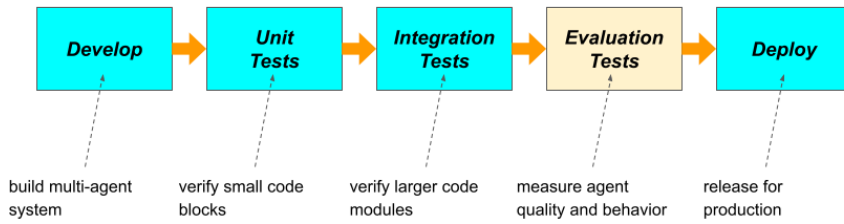


Figure 11.1 Evaluation tests fit naturally after integration tests and before deployment as nondeterministic tests that measure AI responses rather than pure code correctness.

We already know where unit tests and integration tests fit into a CI pipeline. Unit tests check deterministic code and verify small blocks of code. Integration tests check whether our tools, APIs, and storage work together and verify larger modules. Agent evaluation tests sit right next to them, but they answer a different question: not “does the code run?” but “did the agent behave well?”

A practical way to work with evaluators is to treat them like unit tests. They still plug into our existing test frameworks, but instead of checking exact outputs, they check whether the agent's behavior meets a specific quality bar.

Instead of simply asking “Does it work?”, we ask:

- Is it accurate?
- Is it grounded in real data?
- Is it consistent?
- Is it safe?

This is one of the key ways we turn AI features into production-grade systems.

Next, we focus on one evaluator at a time. This approach makes it easier to understand what each evaluator measures and how we can tune our agents, prompts, and tools to pass those quality checks.

11.3.2 Evaluators

Each evaluator test reads one or more records from JSONL data files. JSONL (JSON Lines) is a plain text format in which each line is an independent JSON object, making it easy to store and stream many test cases in a single file.

Each record describes a small scenario: the user input, the agent instructions, the assistant response or tool calls, and whether the evaluator should treat that result as acceptable. We can then run these scenarios as tests.

First, we define a chat client. Then the evaluators receive this chat client through `ChatConfiguration` because many quality evaluators are themselves LLM-backed. They use a model to judge response quality:

```
new ChatConfiguration(chatClient)
```

The following code (listing 11.4) uses more data types to map to JSONL scenarios.

Listing 11.4 Evaluation Record

```
public class EvaluationRecord
{
```

```

[JsonPropertyName("id")]
public required string Id { get; init; }

[JsonPropertyName("agent")]
public required AgentInfo Agent { get; init; } //❶

[JsonPropertyName("userInput")]
public required string UserInput { get; init; }

[JsonPropertyName("toolCalls")]
public required List<ToolCallRecord> ToolCalls { get; init; } //❷

[JsonPropertyName("finalResponse")]
public required string FinalResponse { get; init; }

[JsonPropertyName("expectedBehavior")]
public required string ExpectedBehavior { get; init; }

[JsonPropertyName("groundTruth")]
public string? GroundTruth { get; init; } //❸

[JsonPropertyName("retrievedContextChunks")]
public List<string>? RetrievedContextChunks { get; init; } //❹

[JsonPropertyName("shouldPass")]
public required bool ShouldPass { get; init; }
}

```

❶ Agent instructions, model info are referred in the record

❷ ToolCalls are referred in the record

❸ Only populated for evaluators that require a ground truth baseline

❹ Only populated for retrieval evaluator scenarios

Listing 11.5 defines two types used to parse JSON data from the JSONL files:

- `AgentInfo` embeds the agent's name and model details directly inside each evaluation record defined in listing 11.4, so every scenario is fully self-contained.
- `ToolCallRecord` maps the list of available tools for each record.

Listing 11.5 Agent Info and Tool Call Record

```

public class AgentInfo
{
    [JsonPropertyName("name")]
    public required string Name { get; init; }

    [JsonPropertyName("modelProvider")]
    public required string ModelProvider { get; init; }

    [JsonPropertyName("modelName")]
    public required string ModelName { get; init; }

    [JsonPropertyName("instructions")]
    public required string Instructions { get; init; }

    [JsonPropertyName("tools")]
    public required List<string> Tools { get; init; }
}

public class ToolCallRecord
{
    [JsonPropertyName("name")]
    public required string Name { get; init; }

    [JsonPropertyName("arguments")]

```

```

public required Dictionary<string, object?> Arguments { get; init; }

[JsonPropertyName("result")]
public required string Result { get; init; }
}

```

The intent resolution evaluator checks whether the agent understood and addressed the user's goal. Three scenarios cover the most important cases (listing 11.6).

Listing 11.6 Intent Resolution Evaluation Data (JSONL)

```

{"id":"robby-intent-001","source":"Evaluators","scenario":"intent-resolution-clear-obstacle-avoidance","agent":{"name":"Robby","modelProvider":"OpenAI","modelName":"gpt-5.2","instructions":"You are an AI assistant controlling a robot car. The available robot car permitted moves are: forward, backward, turn left, turn right, stop.\n\nYou have to break down the provided complex commands into basic moves you know. Respond only with the permitted moves, without any additional explanations.","tools":["Forward","Backward","TurnLeft","TurnRight","Stop"]},"userInput":"There is a tree directly in front of the car. Avoid it and then come back to the original path.","toolCalls":[],"finalResponse":"1. turn right\n2. forward\n3. turn left\n4. forward\n5. turn left\n6. forward\n7. turn right","expectedBehavior":"Robby resolves the user's obstacle-avoidance intent into the basic movement sequence.","shouldPass":true}

{"id":"robby-intent-002","source":"Evaluators","scenario":"unresolved-obstacle-avoidance-intent","agent":{"name":"Robby","modelProvider":"OpenAI","modelName":"gpt-5.2","instructions":"You are an AI assistant controlling a robot car. The available robot car permitted moves are: forward, backward, turn left, turn right, stop.\n\nYou have to break down the provided complex commands into basic moves you know. Respond only with the permitted moves, without any additional explanations.","tools":["Forward","Backward","TurnLeft","TurnRight","Stop"]},"userInput":"There is a tree directly in front of the car. Avoid it and then come back to the original path.","toolCalls":[],"finalResponse":"I cannot help with that request.","expectedBehavior":"Robby fails to resolve the user obstacle-avoidance intent.","shouldPass":false}

{"id":"robby-intent-003","source":"Evaluators","scenario":"unresolved-obstacle-avoidance-intent","agent":{"name":"Robby","modelProvider":"OpenAI","modelName":"gpt-5.2","instructions":"You are an AI assistant controlling a robot car. The available robot car permitted moves are: forward, backward, turn left, turn right, stop.\n\nYou have to break down the provided complex commands into basic moves you know. Respond only with the permitted moves, without any additional explanations.","tools":["Forward","Backward","TurnLeft","TurnRight","Stop"]},"userInput":"There is a tree directly in front of the car. Avoid it and then come back to the original path.","toolCalls":[],"finalResponse":"1. turn right\n2. turn right\n3. turn right\n4. turn right.","expectedBehavior":"Robby fails to resolve the user obstacle-avoidance intent.","shouldPass":false}

```

- `robby-intent-001` passes: Robby produces a valid seven-step avoidance sequence. The evaluator judges that the user's intent was fully resolved.
- `robby-intent-002` fails: Robby refuses to call any movement. No movement sequence is produced, so intent is unresolved.
- `robby-intent-003` fails: Robby only turns in circles. The response uses permitted moves, but it does not address the obstacle-avoidance goal at all.

INTENT RESOLUTION EVALUATOR IN ACTION

The `EvaluationTests` class in listing 11.7 holds a shared `IChatClient` that is passed to `ChatConfiguration` for every LLM-backed evaluator:

Setting up quality evaluations:

```
dotnet add package Microsoft.Extensions.AI.Abstractions
dotnet add package Microsoft.Extensions.AI.Evaluation
dotnet add package Microsoft.Extensions.AI.Evaluation.Quality
dotnet add package Microsoft.Extensions.AI.OpenAI
dotnet add package Microsoft.NET.Test.Sdk
dotnet add package xunit.v3
```

Listing 11.7 Intent Resolution Evaluator Test

```
using AITools;
using Evaluators.Models;
using Microsoft.Extensions.AI;
using Microsoft.Extensions.AI.Evaluation;
using Microsoft.Extensions.AI.Evaluation.Quality;
using Microsoft.Extensions.Configuration;
using OpenAI;
using System.Text.Json;
using Xunit;

#pragma warning disable AIEVAL001 //❶

namespace Evaluators;

public class EvaluationTests
{
    private readonly IChatClient _chatClient;

    public EvaluationTests()
    {
        IConfiguration configuration = new ConfigurationBuilder()
            .AddUserSecrets<EvaluationTests>().Build();

        string? model = configuration["OpenAI:ModelId"];
        string? apiKey = configuration["OpenAI:ApiKey"];

        _chatClient = new OpenAIClient(apiKey)
            .GetChatClient(model)
            .AsIChatClient();
    }

    [Theory]
    [InlineData("robby-intent-001")]
    [InlineData("robby-intent-002")]
    [InlineData("robby-intent-003")]
    public async Task
        IntentResolutionEvaluator_MatchesExpectedBehavior(string id)
    {
        EvaluationRecord record = LoadEvaluationRecords("intent-resolution.jsonl")
            .Single(r => r.Id == id);

        List<ChatMessage> history =
        [
            new(ChatRole.System, record.Agent.Instructions), //❷
            new(ChatRole.User, record.UserInput) //❸
        ];

        ChatResponse response = new(new ChatMessage(ChatRole.Assistant,
            record.FinalResponse)); //❹

        IntentResolutionEvaluator intentResolutionEvaluator = new(); //❺
        EvaluationResult result = await intentResolutionEvaluator
            .EvaluateAsync(history, response, new ChatConfiguration(_chatClient)); //❻
        NumericMetric intentResolution = result
            .Get<NumericMetric>(IntentResolutionEvaluator
```

```

        .IntentResolutionMetricName); //7

    Assert.NotNull(intentResolution.Interpretation);
    Assert.Equal(record.ShouldPass,
        intentResolution.Interpretation.Rating >= EvaluationRating.Good); //8
}

private static readonly JsonSerializerOptions JsonOptions = new()
{
    PropertyNameCaseInsensitive = true
};

private static IEnumerable<EvaluationRecord>
    LoadEvaluationRecords(string fileName) =>
        File.ReadLines(Path.Combine(AppContext.BaseDirectory, "Data", fileName))
            .Select((line, index) => JsonSerializer
                .Deserialize<EvaluationRecord>(line, JsonOptions)
                ?? throw new InvalidDataException(
                    $"Could not deserialize {fileName} line {index + 1}."));
}

```

- ❶ IntentResolution, TaskAdherence, ToolCallAccuracy evaluators are experimental
- ❷ System message maps the agent instructions from the record
- ❸ User message maps the user input from the record
- ❹ Assistant message maps the pre-recorded final response
- ❺ The evaluator uses its own LLM call internally to judge quality
- ❻ ChatConfiguration provides the judge model to the evaluator
- ❼ IntentResolutionEvaluator returns a NumericMetric
- ❽ The test passes when the evaluator rating matches the expected outcome in the record

This test does not assert an exact string. It asserts that the evaluator judged intent resolution to be good or bad, consistent with the `shouldPass` field in the record.

COHERENCE EVALUATOR EXERCISE

Coherence checks whether the response is logically ordered and easy to follow. For a passing case, use the ordered seven-step obstacle-avoidance sequence. For a failing case, use a response that mixes robot moves with unrelated commentary.

For example, we can copy one of the previous JSON lines from listing 11.7 and create two new records by changing only a few fields, as shown in listing 11.8:

Listing 11.8 Coherence Evaluation Data (JSONL)

```

{"id":"robby-coherence-001","source":"Evaluators","scenario":"coherent-obstacle-avoidance",
...
,"finalResponse":"1. turn right (90 degrees) to turn away from the tree.\n2. forward (1 meter) to move past the tree.\n3. turn left (90 degrees) to realign with the original direction.\n4. forward (1 meter) to advance alongside the original path.\n5. turn left (90 degrees) to return toward the original lateral position.\n6. forward (1 meter) to reach the original path.\n7. turn right (90 degrees) to restore the original heading.",
"expectedBehavior":"Robby's response is ordered and easy to follow.",
"shouldPass":true}
...
{"id":"robby-coherence-002","source":"Evaluators","scenario":"incoherent-obstacle-avoidance",
...
,"finalResponse":"Turn left because trees are green and I like the color, forward 2 steps but watch for rain clouds, stop if you hear music, backward might work better on Tuesdays when the moon is full, the car needs to turn right eventually but first check your shoes.",
"expectedBehavior":"Robby's response is disordered and hard to follow.",
"shouldPass":false}

```

The `CoherenceEvaluator` metric returns `NumericMetric` and uses the `EvaluationRating.Good` rating as its pass threshold.

RELEVANCE EVALUATOR EXERCISE

Relevance checks whether the response addresses the user's prompt. For a passing case, use a response that directly describes obstacle-avoidance moves. For a failing case, use a completely off-topic response:

For example, we can copy one of the previous JSON lines from listing 11.6 and create two new records by changing only a few fields, as in listing 11.9:

Listing 11.9 Relevance Evaluation Data (JSONL)

```
{ "id": "robby-relevance-001", "source": "Evaluators", "scenario": "relevant-obstacle-avoidance"
...
"finalResponse": "To avoid the tree and return to the original path, Robby should: turn right to move away from the tree, go forward to clear it, turn left to realign, go forward again, turn left once more to approach the original path, go forward, and finally turn right to restore the original heading.",
"expectedBehavior": "Robby's answer directly addresses the obstacle-avoidance request.",
"shouldPass": true }

{ "id": "robby-relevance-002", "source": "Evaluators", "scenario": "irrelevant-obstacle-avoidance",
...
"finalResponse": "The weather today is sunny with a light breeze. Perfect conditions for a picnic in the park.",
"expectedBehavior": "Robby's answer does not address the obstacle-avoidance request.",
"shouldPass": false }
```

The `RelevanceEvaluator` metric returns `NumericMetric` and uses `EvaluationRating.Good` rating as its pass threshold.

FLUENCY EVALUATOR EXERCISE

Fluency checks whether the response is readable and natural. For a passing case, use a well-formed natural language response. For a failing case, use a fragmented, ungrammatical response.

For example, we can copy one of the previous JSON lines from listing 11.6 and create two new records by changing only a few fields, as shown in listing 11.10:

Listing 11.10 Fluency Evaluation Data (JSONL)

```
{ "id": "robby-fluency-001", "source": "Evaluators", "scenario": "fluent-obstacle-avoidance",
...
"finalResponse": "1. turn right (90 degrees)\n2. forward (1 meter)\n3. turn left (90 degrees)\n4. forward (1 meter)\n5. turn left (90 degrees)\n6. forward (1 meter)\n7. turn right (90 degrees)",
"expectedBehavior": "Robby's response is fluent, clear, and well-structured.",
"shouldPass": true }

{ "id": "robby-fluency-002", "source": "Evaluators", "scenario": "influent-obstacle-avoidance",
...
"finalResponse": "car go right then go go forward left also turn also forward stop stop go right back path yes",
"expectedBehavior": "Robby's response is not fluent or readable.",
"shouldPass": false }
```

The `FluencyEvaluator` metric returns `NumericMetric` and uses `EvaluationRating.Average` as its pass threshold, one level lower than the others, because *Fluency* has a more relaxed threshold. *Fluency* is considered acceptable at `Average` or better.

11.3.3 Evaluators with Additional Context

Some evaluators need extra data beyond the chat history and response. `GroundednessEvaluator`, `EquivalenceEvaluator`, `CompletenessEvaluator`, and `RetrievalEvaluator` all require an additional context object passed through the `additionalContext` parameter of `EvaluateAsync`. Each evaluator defines its own context type.

For example:

- *Groundedness*: "Does it respect constraints in `groundTruth`?"
- *Equivalence*: "Is it semantically the same as the baseline, even if phrased differently?"
- *Completeness*: "Does it cover all required parts, regardless of extra content?"

GROUNDENESS EVALUATOR IN ACTION

The *groundedness* evaluator checks whether the response is supported by the supplied facts. It requires a `GroundednessEvaluatorContext` containing the ground truth text. The ground truth describes the allowed moves; the evaluator checks whether the response stays within those constraints.

For example, let's create the two new JSON records as in listing 11.11:

Listing 11.11 Groundedness Evaluation Data (JSONL)

```
{
  "id": "robby-groundedness-001",
  "source": "Evaluators",
  "scenario": "grounded-basic-moves",
  "agent": {
    "name": "Robby",
    "modelProvider": "OpenAI",
    "modelName": "gpt-5.2"
  },
  "instructions": "You are an AI assistant controlling a robot car. Use only grounded commands: forward 1 meter, backward 1 meter, turn left 90 degrees, turn right 90 degrees, and stop.\n\nYou have to break down the provided complex commands into basic moves you know. Respond only with the permitted moves, without any additional explanations.",
  "tools": ["Forward", "Backward", "TurnLeft", "TurnRight", "Stop"],
  "userInput": "There is a tree directly in front of the car. Avoid it and then come back to the original path.",
  "toolCalls": [],
  "finalResponse": "1. turn right (90 degrees)\n2. forward (1 meter)\n3. turn left (90 degrees)\n4. forward (1 meter)\n5. turn left (90 degrees)\n6. forward (1 meter)\n7. turn right (90 degrees)",
  "groundTruth": "The robot car can only use these grounded commands: forward 1 meter, backward 1 meter, turn left 90 degrees, turn right 90 degrees, and stop. It cannot jump, drive in arcs, move around obstacles without step-by-step moves, or move more than 1 meter in a single forward/backward command.",
  "expectedBehavior": "Robby uses only grounded 1-meter moves and 90-degree turns from the provided grounding data.",
  "shouldPass": true
}

{
  "id": "robby-groundedness-002",
  "source": "Evaluators",
  "scenario": "ungrounded-basic-moves",
  "agent": {
    "name": "Robby",
    "modelProvider": "OpenAI",
    "modelName": "gpt-5.2"
  },
  "instructions": "You are an AI assistant controlling a robot car. Use only grounded commands: forward 1 meter, backward 1 meter, turn left 90 degrees, turn right 90 degrees, and stop.\n\nYou have to break down the provided complex commands into basic moves you know. Respond only with the permitted moves, without any additional explanations.",
  "tools": ["Forward", "Backward", "TurnLeft", "TurnRight", "Stop"],
  "userInput": "There is a tree directly in front of the car. Avoid it and then come back to the original path.",
  "toolCalls": [],
  "finalResponse": "1. drive around the tree in a smooth arc\n2. jump over the roots\n3. move forward 10 meters\n4. resume the path",
  "groundTruth": "The robot car can only use these grounded commands: forward 1 meter, backward 1 meter, turn left 90 degrees, turn right 90 degrees, and stop. It cannot jump, drive in arcs, move around obstacles without step-by-step moves, or move more than 1 meter in a single forward/backward command.",
  "expectedBehavior": "Robby uses unsupported movement types and distances outside the grounding data.",
  "shouldPass": false
}
```

Let's look at the two tests and the new `groundTruth` property in them:

- `robby-groundedness-001` passes: the response uses only permitted moves and 90-

degree turns, all fully supported by the ground truth via the `groundTruth` property.

- `robby-groundedness-002` fails: the response includes "drive around the tree in a smooth arc," "jump over the roots," and "move forward 10 meters." All three are unsupported by the grounding data inside the `groundTruth` property.

Some evaluators, such as `GroundednessEvaluator` and `EquivalenceEvaluator`, need additional context provided via `groundTruth` to perform their evaluations. Listing 11.12 shows `GroundednessEvaluator` in action.

Listing 11.12 Groundedness Evaluator Test - Grounded Response

```
[Theory]
[InlineData("robby-groundedness-001")]
[InlineData("robby-groundedness-002")]
public async Task GroundednessEvaluator_MatchesExpectedBehavior(string id)
{
    EvaluationRecord record = LoadEvaluationRecords("groundedness.jsonl")
        .Single(r => r.Id == id);

    List<ChatMessage> history =
    [
        new(ChatRole.System, record.Agent.Instructions),
        new(ChatRole.User, record.UserInput)
    ];

    ChatResponse response = new(
        new ChatMessage(ChatRole.Assistant, record.FinalResponse));

    GroundednessEvaluatorContext context = new(record.GroundTruth!); //❶

    GroundednessEvaluator groundednessEvaluator = new();
    EvaluationResult result = await groundednessEvaluator
        .EvaluateAsync(history, response,
            new ChatConfiguration(_chatClient), [context]); //❷
    NumericMetric groundedness = result
        .Get<NumericMetric>(GroundednessEvaluator.GroundednessMetricName);

    Assert.NotNull(groundedness.Interpretation);
    Assert.Equal(record.ShouldPass,
        groundedness.Interpretation.Rating >= EvaluationRating.Good);
}
```

❶ The ground truth from the JSONL record is passed as additional context

❷ The context array is the fourth parameter, distinct from the chat history

Evaluation test output:

All evaluation tests should pass. Let's observe what happens if the response message is not equivalent.

EQUIVALENCE EVALUATOR EXERCISE

Equivalence checks whether the response is semantically equivalent to a well baseline. It requires an `EquivalenceEvaluatorContext` containing the reference answer. For a passing case, use a response that matches the expected seven-step maneuver sequence. For a failing case, use "1. forward (10 meters)\n2. stop". The car moves forward in a single large step, which is not equivalent to the expected sequence.

For example, we can copy one of the previous JSON lines from listing 11.5 and create two new records by changing only a few fields, as shown in listing 11.13:

Listing 11.13 Equivalence Evaluation Data (JSONL)

```
{ "id": "robby-equivalence-001", "source": "Evaluators", "scenario": "equivalent-
```

```

obstacle-avoidance",
...
"finalResponse":"1. turn right (90 degrees)\n2. forward (1 meter)\n3. turn left
(90 degrees)\n4. forward (1 meter)\n5. turn left (90 degrees)\n6. forward (1
meter)\n7. turn right (90 degrees)","groundTruth":"1. turn right\n2. forward\n3.
turn left\n4. forward\n5. turn left\n6. forward\n7. turn
right","expectedBehavior":"Robby's response is equivalent to the expected maneuver
sequence.", "shouldPass":true}

{"id":"robby-equivalence-002","source":"Evaluators","scenario":"non-equivalent-
obstacle-avoidance",
...
"finalResponse":"1. forward (10 meters)\n2. stop","groundTruth":"1. turn right\n2.
forward\n3. turn left\n4. forward\n5. turn left\n6. forward\n7. turn
right","expectedBehavior":"Robby's response is not equivalent to the expected
maneuver sequence.", "shouldPass":false}

```

The `EquivalenceEvaluator` metric returns `NumericMetric` and uses `EvaluationRating.Good` rating as its pass threshold.

COMPLETENESS EVALUATOR EXERCISE

Completeness checks whether the response covers all required parts described in the ground truth. It requires a `CompletenessEvaluatorContext`. For a passing case, use the full seven-step avoidance-and-return sequence. For a failing case, use "1. turn right": Robby turns away from the tree but never completes the avoidance or returns to the original path.

For example, we can copy one of the previous JSON lines from listing 11.5 and create two new records by changing only a few fields, as shown in listing 11.14:

Listing 11.14 Completeness evaluation data (JSONL)

```

{"id":"robby-completeness-001","source":"Evaluators","scenario":"complete-
obstacle-avoidance",
...
"finalResponse":"1. turn right (90 degrees) to turn away from the tree.\n2.
forward (1 meter) to move past the tree.\n3. turn left (90 degrees) to realign
with the original direction.\n4. forward (1 meter) to advance alongside the
original path.\n5. turn left (90 degrees) to return toward the original lateral
position.\n6. forward (1 meter) to reach the original path.\n7. turn right (90
degrees) to restore the original heading.",
"groundTruth":"A complete avoidance maneuver must: turn away from the obstacle,
move past it, realign with the original direction, advance to the original lateral
position, and turn back to the original heading.",
"expectedBehavior":"Robby's response covers all required parts of the avoidance
maneuver.",
"shouldPass":true}

{"id":"robby-completeness-002","source":"Evaluators","scenario":"incomplete-
obstacle-avoidance",
...
"finalResponse":"1. turn right",
"groundTruth":"A complete avoidance maneuver must: turn away from the obstacle,
move past it, realign with the original direction, advance to the original lateral
position, and turn back to the original heading.",
"expectedBehavior":"Robby's response misses most required parts of the avoidance
maneuver.",
"shouldPass":false}

```

The `CompletenessEvaluator` metric returns `NumericMetric` and uses the `EvaluationRating.Good` rating as its pass threshold.

RETRIEVAL EVALUATOR EXERCISE

Retrieval checks whether the retrieved context chunks are relevant to the user's question. It requires a `RetrievalEvaluatorContext` containing the list of retrieved strings. For a passing case, use

chunks that explain obstacle avoidance and the permitted moves. For a failing case, use completely unrelated chunks.

For example, we can copy one of the previous JSON lines from listing 11.6 and create two new records by changing only a few fields, as shown in listing 11.15:

Listing 11.15 Retrieval evaluation data (JSONL)

```
{ "id": "robby-retrieval-001", "source": "Evaluators", "scenario": "retrieval-relevant-robot-car-guidance",
...
"finalResponse": "1. turn right\n2. forward\n3. turn left\n4. forward\n5. turn left\n6. forward\n7. turn right",
"retrievedContextChunks": ["Obstacle avoidance requires turning away from the obstacle, moving forward, then turning back to realign with the original path.", "Robby can use these basic moves: forward, backward, turn left, turn right, and stop.", "A stop command should be used when Robby must immediately halt before moving again."],
"expectedBehavior": "Retrieved chunks are relevant and ranked for Robby's obstacle-avoidance request.",
"shouldPass": true }

{ "id": "robby-retrieval-002", "source": "Evaluators", "scenario": "retrieval-irrelevant-chunks",
...
"finalResponse": "1. turn right\n2. forward\n3. turn left\n4. forward\n5. turn left\n6. forward\n7. turn right",
"retrievedContextChunks": ["To bake a cake, preheat the oven to 180 degrees Celsius.", "Weather forecasting uses barometric pressure readings.", "Stock market indices track the performance of selected equities."],
"expectedBehavior": "Retrieved chunks are irrelevant to Robby's obstacle-avoidance request.",
"shouldPass": false }
```

Notice the additional `retrievedContextChunks` property: an array of strings representing the context passages returned by your retrieval step, which the retrieval evaluator uses to judge relevance.

The `RetrievalEvaluator` metric returns `NumericMetric` and uses an `EvaluationRating.Good` rating as its pass threshold.

11.3.4 Composite Evaluator

Running evaluators one by one is good for learning, but in practice we may want to assemble a set of evaluators that work well together and run them at once. `CompositeEvaluator` wraps multiple evaluators and runs them in a single `EvaluateAsync` call, returning all metrics together in one `EvaluationResult`. Each inner evaluator's context, if needed, is passed through the shared `additionalContext` and routed automatically.

COMPOSITE EVALUATOR IN ACTION

Let's compose the *Coherence* and *Equivalence* evaluators as in listing 11.16.

Listing 11.16 Composite Evaluator Test - Composite Response

```
[Theory]
[InlineData("robby-composite-001")]
[InlineData("robby-composite-002")]
public async Task CompositeEvaluator_MatchesExpectedBehavior(string id)
{
    EvaluationRecord record = LoadEvaluationRecords("composite.jsonl")
        .Single(r => r.Id == id);
```

```

List<ChatMessage> history =
[
    new(ChatRole.System, record.Agent.Instructions),
    new(ChatRole.User, record.UserInput)
];

ChatResponse response = new(
    new ChatMessage(ChatRole.Assistant, record.FinalResponse));

CoherenceEvaluator coherenceEvaluator = new(); ❶
EquivalenceEvaluator equivalenceEvaluator = new(); ❷
CompositeEvaluator compositeEvaluator =
    new(coherenceEvaluator, equivalenceEvaluator); ❸

EquivalenceEvaluatorContext baselineResponseForEquivalence =
    new(record.GroundTruth!); ❹

EvaluationResult result = await compositeEvaluator
    .EvaluateAsync(history, response, new ChatConfiguration(_chatClient),
        [baselineResponseForEquivalence]); ❺

NumericMetric coherence = result.Get<NumericMetric>(
    CoherenceEvaluator.CoherenceMetricName); ❻
NumericMetric equivalence = result.Get<NumericMetric>(
    EquivalenceEvaluator.EquivalenceMetricName); ❻

Assert.NotNull(coherence.Interpretation);
Assert.Equal(record.ShouldPass,
    coherence.Interpretation.Rating >= EvaluationRating.Good); ❼

Assert.NotNull(equivalence.Interpretation);
Assert.Equal(record.ShouldPass,
    equivalence.Interpretation.Rating >= EvaluationRating.Good); ❽
}

```

- ❶ **CoherenceEvaluator** needs no additional context
- ❷ **EquivalenceEvaluator** needs a ground truth baseline
- ❸ Both evaluators are wrapped into one **CompositeEvaluator**
- ❹ The baseline is passed through **additionalContext** and routed to **EquivalenceEvaluator**
- ❺ A single **EvaluateAsync** call runs both evaluators
- ❻ Both metrics are extracted from the same **EvaluationResult**
- ❼ Coherence metric is asserted independently against the same **ShouldPass** value
- ❽ Equivalence metric is asserted independently against the same **ShouldPass** value

The composite pass record produces an ordered response that matches the expected maneuver sequence, so both coherence and equivalence are rated good. The composite fail record is intentionally disordered and semantically different from the baseline, so both metrics fail together.

PRACTICAL EXERCISE

Extend the composite evaluator with a third evaluator, such as `FluencyEvaluator`. Add a new JSONL record to `composite.jsonl` with a `groundTruth` field and a clear non-fluent `finalResponse`. Verify that all three metrics are extracted and asserted correctly.

11.3.5 Evaluators with Tools

Some evaluators work with tool calls rather than text responses. They inspect the `FunctionCallContent` items in the assistant's response and require the actual tool definitions as additional context so the evaluator can reason about what was called and what should have been called.

`FunctionCallContent` objects are reconstructed from the JSONL `toolCalls` array before the evaluator runs (listing 11.17):

Listing 11.17 Evaluators with Tools

```
List<AIContent> toolCallContents =[.. record.ToolCalls
    .Select((tc, index) =>
    {
        AIFunctionArguments arguments = new(tc.Arguments.ToDictionary(
            kv => kv.Key,
            kv => kv.Value is JsonElement element
                ? element.Deserialize<object>()
                : kv.Value));

        return new FunctionCallContent($"call-{index + 1}", tc.Name, arguments);
    })];

ChatResponse response = new(
    new ChatMessage(ChatRole.Assistant, toolCallContents));
```

TASK ADHERENCE EVALUATOR IN ACTION

The *Task Adherence* evaluator checks whether the agent followed the constraints stated in the task. It requires a `TaskAdherenceEvaluatorContext` containing the available tool definitions (listing 11.18).

Listing 11.18 Adherence Evaluation Data (JSONL)

```
{ "id": "robby-task-001", "source": "Evaluators", "scenario": "task-adherence-stop-immediately",
...
"toolCalls": [{ "name": "Stop", "arguments": {}, "result": "Stopped" }],
"finalResponse": "Robot car stopped.",
"expectedBehavior": "Robby follows the task constraints by calling the Stop tool for an immediate stop request.",
"shouldPass": true}

{ "id": "robby-task-002", "source": "Evaluators", "scenario": "task-adherence-wrong-tool-for-stop",
...
"toolCalls": [{ "name": "Forward", "arguments": { "distance": 5 }, "result": "Moved forward 5 meters" }],
"finalResponse": "Robot car moved forward 5 meters.",
"expectedBehavior": "Robby ignores the stop constraint and moves forward instead, violating task adherence.",
"shouldPass": false}
```

Let's look at the two tests:

- `robby-task-001` passes: the user explicitly says, `"Stop immediately"`. Robby calls `Stop`. The evaluator confirms the task constraint was followed.
- `robby-task-002` fails: Robby calls `Forward(5)` instead of `Stop`. The evaluator detects the constraint violation.

Listing 11.19 shows the implementation of the *Task Adherence* evaluation test:

Listing 11.19 Task Adherence Evaluation

```
[Theory]
[InlineData("robby-task-001")]
[InlineData("robby-task-002")]
[InlineData("robby-task-003")]
[InlineData("robby-task-004")]
public async Task TaskAdherenceEvaluator_MatchesExpectedBehavior(string id)
{
    EvaluationRecord record = LoadEvaluationRecords("task-adherence.jsonl")
        .Single(r => r.Id == id);
```

```

List<ChatMessage> history =
[
    new(ChatRole.System, record.Agent.Instructions),
    new(ChatRole.User, record.UserInput)
];

List<AIContent> toolCallContents =[.. record.ToolCalls
    .Select((tc, index) =>
    {
        AIFunctionArguments arguments = new(tc.Arguments.ToDictionary(
            kv => kv.Key,
            kv => kv.Value is JsonElement element
                ? element.Deserialize<object>()
                : kv.Value));

        return new FunctionCallContent($"call-{index + 1}", tc.Name, arguments);
    }
)];

ChatResponse response =
    new(new ChatMessage(ChatRole.Assistant, toolCallContents));

TaskAdherenceEvaluatorContext context =
    new([.. MotorTools.AsAITools()]); //❶

TaskAdherenceEvaluator taskAdherenceEvaluator = new();
EvaluationResult result = await taskAdherenceEvaluator
    .EvaluateAsync(history, response, new ChatConfiguration(_chatClient),
        additionalContext: [context]);
NumericMetric taskAdherence = result.Get<NumericMetric>(
    TaskAdherenceEvaluator.TaskAdherenceMetricName);

Assert.NotNull(taskAdherence.Interpretation);
Assert.Equal(record.ShouldPass,
    taskAdherence.Interpretation.Rating >= EvaluationRating.Good);
}

```

❶ Initializes the AIFunction list as evaluation context

The tests assert the task adherence rating, consistent with the `shouldPass` field in the record.

TOOL CALL ACCURACY EVALUATOR EXERCISE

The *Tool Call Accuracy* evaluator checks whether the agent called the correct tools, with the correct arguments, in the correct order. It requires a `ToolCallAccuracyEvaluatorContext` containing the available tool definitions. Unlike task adherence, which asks "Did the agent follow the task?", tool call accuracy asks, "Did the agent call exactly the right tools in exactly the right way?"

Unlike the other evaluation metrics, this one return `BooleanMetric`. Either the entire tool call sequence is accurate, or it is not.

Design four passing and failing JSONL scenarios in `tool-call-accuracy.jsonl` for the user input "Move forward 2 meters, turn left 90 degrees, then stop." as follows:

- Pass: Forward(distance: 2), TurnLeft(angle: 90), Stop: all tools correct, correct arguments, correct order.
- Fail: wrong argument: Forward(distance: 99), TurnLeft(angle: 90), Stop -- correct tools and order, but wrong forward distance.
- Fail: incomplete sequence: Forward(distance: 2), TurnLeft(angle: 90), correct tools and arguments, but Stop is missing.
- Fail: wrong tool and argument: Forward(distance: 5), TurnRight(angle: 90), Stop - wrong distance and wrong turn direction.

For example, we can copy one of the previous JSON lines from listing 11.5 and create two new records by changing only a few fields, as shown in listing 11.20:

Listing 11.20 Tool Call Accuracy Evaluation Data (JSONL)

```
{ "id": "robby-tool-001", "source": "Evaluators", "scenario": "tool-call-multi-step-
sequence",
...
"userInput": "Move forward 2 meters, turn left 90 degrees, then stop.",
"toolCalls": [{ "name": "Forward", "arguments": { "distance": 2 }, "result": "Moved forward
2 meters" }, { "name": "TurnLeft", "arguments": { "angle": 90 }, "result": "Turned left 90
degrees" }, { "name": "Stop", "arguments": { }, "result": "Stopped" } ],
"finalResponse": "",
"expectedBehavior": "Robby calls the correct tools in the requested order with the
requested forward distance, turn angle, and stop.",
"shouldPass": true }

{ "id": "robby-tool-002", "source": "Evaluators", "scenario": "tool-call-wrong-
argument",
...
"userInput": "Move forward 2 meters, turn left 90 degrees, then stop.",
"toolCalls": [{ "name": "Forward", "arguments": { "distance": 99 }, "result": "Moved forward
2 meters" }, { "name": "TurnLeft", "arguments": { "angle": 90 }, "result": "Turned left 90
degrees" }, { "name": "Stop", "arguments": { }, "result": "Stopped" } ],
"finalResponse": "",
"expectedBehavior": "Robby calls the correct tools in the requested order with the
requested forward distance, turn angle, and stop, but it captures wrong argument
for the distance.",
"shouldPass": false }

{ "id": "robby-tool-003", "source": "Evaluators", "scenario": "tool-call-incomplete-
step-sequence",
...
"userInput": "Move forward 2 meters, turn left 90 degrees, then stop.",
"toolCalls": [{ "name": "Forward", "arguments": { "distance": 2 }, "result": "Moved forward
2 meters" }, { "name": "TurnLeft", "arguments": { "angle": 90 }, "result": "Turned left 90
degrees" } ],
"finalResponse": "",
"expectedBehavior": "Robby calls the correct tools in the requested order with the
requested forward distance and turn angle, but misses to stop.",
"shouldPass": false }

{ "id": "robby-tool-004", "source": "Evaluators", "scenario": "tool-call-wrong-sequence-
and-argument",
...
"userInput": "Move forward 2 meters, turn left 90 degrees, then stop.",
"toolCalls": [{ "name": "Forward", "arguments": { "distance": 5 }, "result": "Moved forward
5 meters" }, { "name": "TurnRight", "arguments": { "angle": 90 }, "result": "Turned right 90
degrees" }, { "name": "Stop", "arguments": { }, "result": "Stopped" } ],
"finalResponse": "",
"expectedBehavior": "Robby uses the wrong forward distance and turns right instead
of left.",
"shouldPass": false }
```

The `ToolCallAccuracyEvaluator` is populated with the `additionalContext` that contains the available `AITools`, returns a `BooleanMetric`, checks its `Value` against `true`, and uses `EvaluationRating.Good` as its pass threshold.

11.3.6 Custom Evaluators

Built-in evaluators cover many quality dimensions, but sometimes a domain-specific rule cannot be expressed with any of them. We can implement our own by creating a class that implements the `IEvaluator` interface.

DIRECTION CHANGE EVALUATOR IN ACTION

Robby's safety policy states that the car must never switch between `Forward` and `Backward` without an intervening `Stop`. `DirectionChangeEvaluator` encodes exactly this rule without using an LLM. `DirectionChangeEvaluator` inspects the tool call sequence deterministically, making it fast, cheap, and reproducible.

The custom evaluator uses its own record type, `DirectionChangeRecord` (listing 11.21), instead of `EvaluationRecord`. It carries domain-specific expected fields, `ExpectedChanged`, `ExpectedFrom`, and `ExpectedTo`, instead of the generic `ShouldPass` property. This makes the JSONL data more expressive: a record can assert not only whether a change was detected but also which directions were involved.

Listing 11.21 `DirectionChangeRecord.cs`

```
public class DirectionChangeRecord
{
    [JsonPropertyName("id")]
    public required string Id { get; init; }

    [JsonPropertyName("source")]
    public required string Source { get; init; }

    [JsonPropertyName("scenario")]
    public required string Scenario { get; init; }

    [JsonPropertyName("agent")]
    public required AgentInfo Agent { get; init; }

    [JsonPropertyName("userInput")]
    public required string UserInput { get; init; }

    [JsonPropertyName("toolCalls")]
    public required List<ToolCallRecord> ToolCalls { get; init; }

    [JsonPropertyName("finalResponse")]
    public required string FinalResponse { get; init; }

    [JsonPropertyName("expectedBehavior")]
    public required string ExpectedBehavior { get; init; }

    [JsonPropertyName("expectedChanged")]
    public required bool ExpectedChanged { get; init; } // ❶

    [JsonPropertyName("expectedFrom")]
    public string? ExpectedFrom { get; init; } // ❷

    [JsonPropertyName("expectedTo")]
    public string? ExpectedTo { get; init; }
}
```

❶ True when an unsafe direction change is expected; false when the sequence should be safe

❷ The direction names involved in the change, or null when no change is expected

Let's create three new evaluation test records, as shown in listing 11.22 (the redundant properties are replaced with an ellipsis):

Listing 11.22 Custom Data (JSONL)

```
{ "id": "robby-direction-change-001", "source": "Observations", "scenario": "direction-
change-detected",
...
"userInput": "Go back and forth 3 meters",
"toolCalls": [ { "name": "Backward", "arguments": { "distance": 3 }, "result": "Moved
backward 3 meters" }, { "name": "Forward", "arguments": { "distance": 3 }, "result": "Moved
```

```

forward 3 meters"}],
"finalResponse":"Moving backward 3 meters, then forward 3 meters.",
"expectedBehavior":"The evaluator should detect an unsafe direction change because
Backward is followed by Forward without Stop.",
"expectedChanged":true,
"expectedFrom":"Backward",
"expectedTo":"Forward"}

{"id":"robby-direction-change-002","source":"Observations","scenario":"no-
direction-change",
...
"userInput":"Move forward 5 meters then forward another 3 meters",
"toolCalls":[{"name":"Forward","arguments":{"distance":5},"result":"Moved forward
5 meters"}, {"name":"Forward","arguments":{"distance":3},"result":"Moved forward 3
meters"}],
"finalResponse":"Moved forward 5 meters, then forward another 3 meters.",
"expectedBehavior":"The evaluator should not detect a direction change because two
consecutive Forward calls keep the same linear direction.",
"expectedChanged":false,
"expectedFrom":null,
"expectedTo":null}

{"id":"robby-direction-change-003","source":"Observations","scenario":"non-linear-
command-resets-tracking",
...
"userInput":"Go forward 2 meters, stop, then go backward 2 meters",
"toolCalls":[{"name":"Forward","arguments":{"distance":2},"result":"Moved forward
2
meters"}, {"name":"Stop","arguments":{},"result":"Stopped"}, {"name":"Backward","arg
uments":{"distance":2},"result":"Moved backward 2 meters"}],
"finalResponse":"Moved forward 2 meters, stopped, then moved backward 2 meters.",
"expectedBehavior":"The evaluator should not detect an unsafe direction change
because Stop resets direction tracking before Backward.",
"expectedChanged":false,
"expectedFrom":null,
"expectedTo":null}

```

Three scenarios cover the evaluator's key behaviors:

- `robby-direction-change-001` detects an unsafe change: Backward is immediately followed by Forward without a Stop in between. `ExpectedChanged` is true, `ExpectedFrom` is Backward, `ExpectedTo` is Forward.
- `robby-direction-change-002` detects no change: two consecutive Forward calls keep the same linear direction. `ExpectedChanged` is false.
- `robby-direction-change-003` detects no change even though the direction is different: a Stop in between Forward and Backward resets the tracking, making the sequence safe. `ExpectedChanged` is false.

Listing 11.23 Direction Change Evaluator

```

public class DirectionChangeEvaluator : IEvaluator
{
    public const string DirectionChangeMetricName = "DirectionChange";

    private static readonly HashSet<string> KnownMoves =
        new(StringComparer.OrdinalIgnoreCase)
        {
            "Forward", "Backward", "TurnLeft", "TurnRight", "Stop"
        };

    public IReadOnlyCollection<string> EvaluationMetricNames =>
        [DirectionChangeMetricName];
}

```

```

public ValueTask<EvaluationResult> EvaluateAsync(
    IEnumerable<ChatMessage> messages,
    ChatResponse modelResponse,
    ChatConfiguration? chatConfiguration = null,
    IEnumerable<EvaluationContext>? additionalContext = null,
    CancellationToken cancellationToken = default)
{
    List<string> toolNames = [.. modelResponse.Messages
        .SelectMany(message => message.Contents)
        .OfType<FunctionCallContent>()
        .Select(toolCall => toolCall.Name)]; //❶

    (bool isSafe, string reason, _, _, _) =
        AnalyzeDirectionChangeSafety(toolNames); //❷

    BooleanMetric metric = new(DirectionChangeMetricName, isSafe, reason)
    {
        Interpretation = isSafe
            ? new EvaluationMetricInterpretation(
                EvaluationRating.Good, reason: reason)
            : new EvaluationMetricInterpretation(
                EvaluationRating.Unacceptable, failed: true, reason: reason)
    }; //❸

    return new ValueTask<EvaluationResult>(new EvaluationResult(metric));
}

public static (bool IsSafe, string Reason, bool Changed,
    string? From, string? To) AnalyzeDirectionChangeSafety(
    IReadOnlyList<string> toolNames)
{
    if (toolNames.Count == 0)
        return (false, "No tool calls were found in Robby's response.",
            false, null, null);

    List<string> unknownMoves = [.. toolNames
        .Where(n => !KnownMoves.Contains(n))
        .Distinct(StringComparer.OrdinalIgnoreCase)];

    if (unknownMoves.Count > 0)
        return (false, $"Unknown move calls found: {string.Join(", ",
            unknownMoves)}. ", false, null, null);

    string? previousLinearDirection = null;

    foreach (string toolName in toolNames)
    {
        if (toolName.Equals("stop", StringComparison.OrdinalIgnoreCase))
        {
            previousLinearDirection = null; //❹
            continue;
        }

        string? currentLinearDirection = toolName is "forward" or "backward"
            ? toolName
            : null;

        if (currentLinearDirection is null) continue; //❺

        if (previousLinearDirection is not null &&
            !previousLinearDirection.Equals(currentLinearDirection,
                StringComparison.OrdinalIgnoreCase))
        {
            return (false,

```

```

        $"Unsafe direction change detected: "
        + $"{previousLinearDirection} "
        + $"followed by {currentLinearDirection} without Stop.",
        true, previousLinearDirection, currentLinearDirection); //❹
    }

    previousLinearDirection = currentLinearDirection;
}

return (true, "No unsafe linear direction change was detected.",
        false, null, null);
}
}

```

- ❶ Tool call names are extracted directly from `FunctionCallContent` items in the response
- ❷ The static helper is shared between `EvaluateAsync` and the test, so both use identical logic
- ❸ No LLM is needed — the metric and its interpretation are set deterministically
- ❹ Stop resets direction tracking; the next `Forward` or `Backward` is treated as a fresh start
- ❺ `TurnLeft` and `TurnRight` do not affect linear direction tracking
- ❻ Returns the full change details so the test can assert `From` and `To` separately

Listing 11.24 shows the Direction Change Evaluator tests.

Listing 11.24 Direction Change Evaluator Test

```

[Theory]
[InlineData("robby-direction-change-001")]
[InlineData("robby-direction-change-002")]
[InlineData("robby-direction-change-003")]
public async Task
    DirectionChangeSafetyEvaluator_MatchesExpectedBehavior(string id)
{
    DirectionChangeRecord record =
        LoadDirectionChangeRecords("direction-change.jsonl")
        .Single(r => r.Id == id);

    List<string> toolNames =
        [.. record.ToolCalls.Select(toolCall => toolCall.Name)];

    (_, _, bool changed, string? from, string? to) =
        DirectionChangeEvaluator
        .AnalyzeDirectionChangeSafety(toolNames); //❶

    Assert.Equal(record.ExpectedChanged, changed); //❷
    Assert.Equal(record.ExpectedFrom, from); //❷
    Assert.Equal(record.ExpectedTo, to); //❷

    List<ChatMessage> history =
    [
        new(ChatRole.System, record.Agent.Instructions),
        new(ChatRole.User, record.UserInput)
    ];

    List<AIContent> toolCallContents = [.. record.ToolCalls
        .Select((tc, index) =>
        {
            AIFunctionArguments arguments = new(tc.Arguments.ToDictionary(
                kv => kv.Key,
                kv => kv.Value is JsonElement element
                    ? element.Deserialize<object>()
                    : kv.Value));

            return new FunctionCallContent($"call-{index + 1}", tc.Name, arguments);
        })];

    ChatResponse response = new(

```

```

new ChatMessage(ChatRole.Assistant, toolCallContents));

DirectionChangeEvaluator evaluator = new();
EvaluationResult result = await evaluator.EvaluateAsync(
    history, response, new ChatConfiguration(_chatClient));
BooleanMetric directionChangeSafety = result
    .Get<BooleanMetric>(DirectionChangeEvaluator.DirectionChangeMetricName);

Assert.NotNull(directionChangeSafety.Interpretation);
Assert.Equal(!record.ExpectedChanged, directionChangeSafety.Value); ❸
Assert.Equal(record.ExpectedChanged,
    directionChangeSafety.Interpretation.Failed); ❹
}

private static readonly JsonSerializerOptions JsonOptions =
    new() { PropertyNameCaseInsensitive = true };

private static IEnumerable<DirectionChangeRecord>
    LoadDirectionChangeRecords(string fileName) =>
        File.ReadLines(Path.Combine(AppContext.BaseDirectory, "Data", fileName))
            .Where(line => !string.IsNullOrEmpty(line))
            .Select((line, index) => JsonSerializer
                .Deserialize<DirectionChangeRecord>(line, JsonOptions)
                ?? throw new InvalidDataException(
                    $"Could not deserialize {fileName} line {index + 1}."));

```

- ❶ Verifies the direction-change analysis in isolation
- ❷ Checks change detected and both directions reported correctly
- ❸ Asserts that a direction is expected
- ❹ Asserts that the direction is changed

This test is two tests in one. The first block (the lines before building history) exercises `AnalyzeDirectionChangeSafety` directly with only the tool name list: no chat client, no LLM, and no `EvaluateAsync`. The second block exercises the full `IEvaluator` pipeline. Both blocks must pass for every record, which means the same theory verifies both the deterministic analysis logic and the evaluator wire-up.

Evaluation test output:

All three direction-change tests should pass.

- `robby-direction-change-001`: changed is true, from is Backward, to is Forward. `directionChangeSafety.Value` is false (unsafe). `Interpretation.Failed` is true.
- `robby-direction-change-002`: changed is false, from and to are null. `directionChangeSafety.Value` is true (safe). `Interpretation.Failed` is false.
- `robby-direction-change-003`: changed is false despite a direction change in the raw sequence because Stop reset tracking. `directionChangeSafety.Value` is true (safe). `Interpretation.Failed` is false.

PRACTICAL EXERCISE

Add a fourth record to `direction-change.jsonl` in which Robby calls `Forward`, `TurnLeft`, `TurnRight`, and then `Backward` without a `Stop`. Set `ExpectedChanged` to `true`, `ExpectedFrom` to `Forward`, and `ExpectedTo` to `"Backward"`. Verify that turns do not reset direction tracking. Only `Stop` does.

Summary

- Interpret agent traces, logs, and metrics emitted by Agent Framework and understand how spans connect agent runs, model calls, and tool invocations.
- Configure *OpenTelemetry* for Agent Framework agents and export traces, metrics, and logs

to OTLP backends such as *Console Exporter* or *Aspire Dashboard*.

- Use built-in HTTP client instrumentation to relate agent-level spans to underlying HTTP calls to OpenAI or Azure OpenAI, including latency, status codes, failures, and retries.
- Use `Microsoft.Extensions.AI.Evaluation` to move from exact-string tests to quality-based evaluation of agent responses (coherence, relevance, grounding, tool usage, and safety).
- Use built-in quality evaluators (intent resolution, coherence, relevance, fluency, groundedness, equivalence, completeness, retrieval) via `Microsoft.Extensions.AI.Evaluation` and assert on their `NumericMetric` ratings.
- Provide additional context to evaluators (`groundTruth` baselines, `retrievedContextChunks`, and tool definitions) and interpret their scores and pass thresholds.
- Evaluate agent/tool behavior using tool-aware evaluators such as `TaskAdherenceEvaluator` and `ToolCallAccuracyEvaluator` that inspect `FunctionCallContent`.
- Compose multiple evaluators (for example, coherence and equivalence) into a single `CompositeEvaluator` and assert on their combined results in your tests.
- Design and plug in custom evaluators that capture domain-specific rules, such as Robby's safety constraints or movement policies.
- Implement and test a fully custom, non-LLM evaluator (`DirectionChangeEvaluator`) that encodes a domain safety policy over tool sequences and exposes it as a `BooleanMetric`.

12

Orchestrating agents using sequential and concurrent patterns

This chapter covers

- Implementing Sequential patterns with specialized agents
- Implementing Concurrent patterns with agents and a custom aggregator
- Composing multiple agents with Sequential and Concurrent patterns

We saw in previous chapters that Robby handles different situations, such as ice on the road, fire risk, and a tree fifty meters ahead, quite well, but those were simple scenarios. Imagine Robby facing a complex navigation challenge that requires multiple AI tasks, from reading sensors to making safety decisions and controlling motors. Imagine an environmental specialist agent monitoring weather conditions, a safety expert assessing risks, or an engine controller executing precise movements. Each agent brings unique expertise, and together they form a collaborative intelligence far more capable than any single agent working alone.

This is the power of multi-agent orchestration, and we can use orchestration patterns to solve problems that are too complex for a single agent.

12.1 Understanding Core Concepts

Think of Agent Framework like a modern flight control operation: every mission is routed through the right controllers, systems, and checklists instead of relying on a single overworked pilot. A *workflow* is that route, the end-to-end path a user request takes as it flows through one or more agents, tools, and system components until a result is produced. A *workflow* is not a single agent; it is the coordination layer that decides which agents run, in what order, and how their outputs feed into one another.

A key advantage of Agent Framework is its model flexibility: a single workflow can route a mission through multiple agents, each powered by the best AI model for its specific task. Instead of one large model trying to handle every concern, you can mix fast, cost-effective models for routine steps with more capable models for complex reasoning or risk-sensitive decisions. This lets you balance latency, cost, and capability for individual steps within a workflow.

Specialization is not only about models; it is also about responsibilities. One agent might focus on understanding user intent, another on orchestrating tools or external APIs, and another on enforcing safety or policy checks before actions are applied. Because each agent has a narrow, well-defined role, you can evolve or replace it independently, replacing its chat client as long as its input and output remain compatible with the rest of the workflow.

Workflow pattern orchestration, enabled by Agent Framework, addresses several limitations of single-agent systems:

- Specialization vs. generalization: agents can specialize in their domains, leading to more accurate and reliable outcomes than a single generalist model.
- Safety and redundancy: if one agent fails or produces low-confidence results, other agents or alternative paths in the workflow can continue the work, creating a more robust system.
- Scalability and modularity: new capabilities can be added by introducing new agents or workflow branches without disrupting existing flows, as long as contracts between steps stay stable.

Note If you come from distributed systems, you can think of workflows as the “request path” through a set of agents, much like a request flow through microservices in an enterprise system. Microservices decompose monolithic applications into services focused on specific business capabilities; collaborative agents create specialized AI entities that solve complex problems within workflow. Although individual microservice endpoints are typically designed as stateless and focus on data processing, agents typically maintain conversational state and context. Still, the core idea of high cohesion and loose coupling applies to both and is a useful mental model for deciding when to split functionality into separate agents.

It’s like a jazz band: each musician (service or agent) has their own instrument and expertise, but they must coordinate to create something greater than the sum of their parts. This approach allows you to optimize performance, cost, and capability by matching the right model to the right job, which is a core principle of building effective multi-agent workflows.

12.1.1 Agent Workflow Builder

AgentWorkflowBuilder is a high-level factory that turns your agents into a concrete workflow graph without requiring you to touch executors or edges directly. In Agent Framework, executors are the processing units inside a workflow (such as agents or custom logic using conventional code) that receive typed messages, do work, and emit new messages or events; edges are the connections between executors that define how messages flow through the graph.

The workflow in figure 12.1 is modeled as a graph of executors (*Agents* or *Modules*) connected by edges that carry data and control between them. Each executor is a self-contained step in the workflow. An Agent can act as an AI-driven executor, while a Module represents conventional code, but both are treated uniformly as “nodes” in the orchestration graph.

Workflow Orchestration using Executors (Agent / Module) and Edges (Conditional)

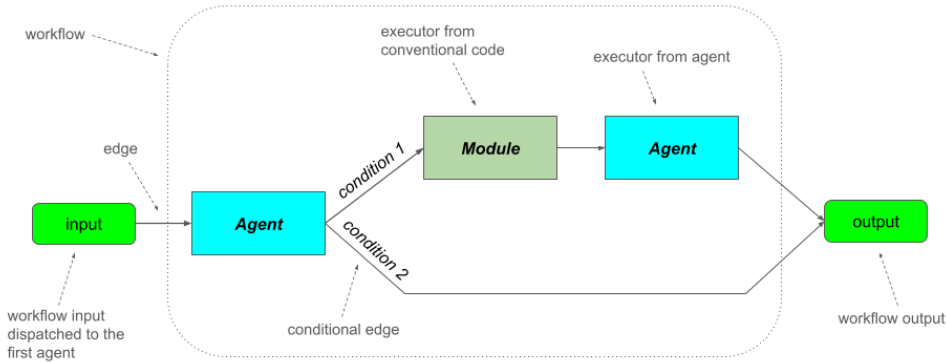


Figure 12.1 Workflow orchestration as a graph of executors and conditional edges.

The most intuitive orchestration patterns are *Sequential* and *Concurrent* workflows: sequential when each step depends on the previous one, and concurrent when independent agents can process the same input in parallel and their results are combined, as follows:

1. Sequential: the output of agent N becomes the input of agent N+1

```
Workflow AgentWorkflowBuilder
    .BuildSequential(params IEnumerable<IAgent> agents)
```

2. Concurrent: the same input fans out to all agents; the aggregator merges the results

```
Workflow AgentWorkflowBuilder
    .BuildConcurrent(IEnumerable<IAgent> agents,
        Func<IList<List<ChatMessage>>, List<ChatMessage>>? aggregator = null)
```

The optional `aggregator` is a C# function delegate (`Func`) that receives the chat message lists produced by the concurrent agents and returns the combined list that the workflow emits. Its combining logic is user-defined. You can omit it when the default aggregation behavior is sufficient.

Both methods return a `Workflow`. A workflow is a directed graph of executors, internal components that receive typed messages and forward them. Agents bind naturally as executors; the builder wires them together so you can focus only on personas and tools.

12.1.2 In-Process Execution

Agent Framework runs workflows in-process via `InProcessExecution.RunAsync`. This returns a `Run` object whose generated events can be inspected after the run, as in listing 12.1:

Listing 12.1 Non-streaming run example

```
await using Run run = await InProcessExecution
    .RunAsync(workflow, input: prompt); // ❶
foreach (WorkflowEvent evt in run.NewEvents) // ❷
{
    // handle events
}
```

- ❶ executes non-streaming run
- ❷ iterates through the workflow events of the run

Alternatively, use `InProcessExecution.RunStreamingAsync` to execute an in-process workflow while observing its events as they occur. It returns a `StreamingRun`, as listing 12.2 shows:

Listing 12.2 Streaming run example

```
await using StreamingRun run = await InProcessExecution
    .RunStreamingAsync(workflow, input: mission); //❶
await run
    .TrySendMessageAsync(new TurnToken(emitEvents: true)); //❷

await foreach (WorkflowEvent evt in run.WatchStreamAsync()) //❸
{
    // handle events
}
```

- ❶ executes streaming run
- ❷ sends a turn token explicitly and enable event emission
- ❸ iterates through the workflow events as they are streamed

Streaming execution allows us to react to events in real time rather than waiting for the workflow to complete. `RunStreamingAsync` enqueues the input and returns a `StreamingRun`, but it does not automatically send a `TurnToken`. Therefore, this example explicitly sends `new TurnToken(emitEvents: true)` to signal that the input phase is complete, trigger execution in workflows, and request streaming agent events. Without it, the workflow stalls waiting for a turn signal that never arrives.

12.2 Understanding Sequential Orchestration

Sequential orchestration is like a flight director running a strict pre-flight sequence: fuel confirmed, systems configured, runway assigned, then takeoff clearance. Each step builds on the previous one, and nothing can be skipped or reordered without violating the checklist.

Sequential orchestration is the most intuitive pattern: a pipeline where each agent's output becomes the next agent's input.

Use sequential orchestration when:

- The output of one agent is required before the next can proceed
- Each agent transforms or validates the previous output
- The overall flow is deterministic (fixed order) and easy to reason about

Figure 12.2 shows the functional diagram of the *Sequential* orchestrator, which operates as a pipeline: each agent's output becomes the next agent's input.

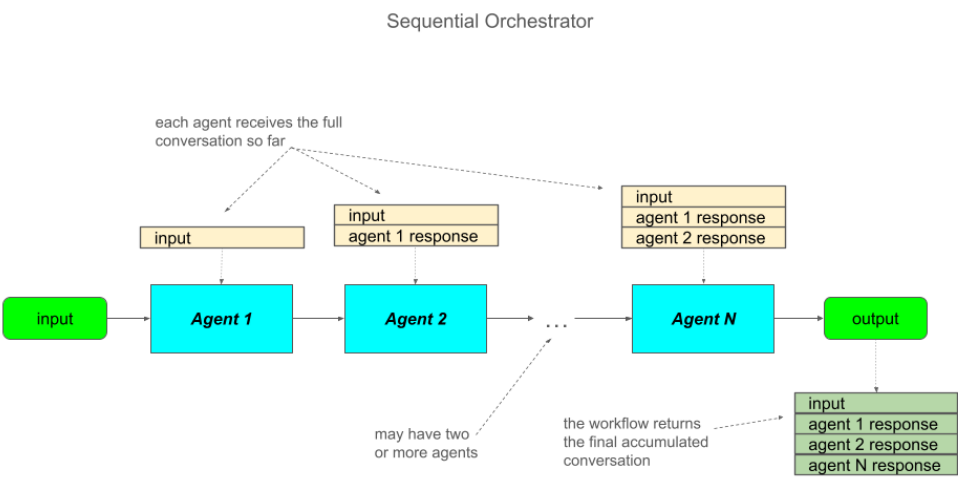


Figure 12.2 The Sequential orchestrator creates a pipeline where each agent's output becomes the next agent's input, ensuring data flows in a predictable order. Each agent must successfully complete before the next one begins.

Data flows in a predictable order, and each agent must complete successfully before the next one begins.

In this pattern, tasks have clear dependencies and the orchestrator maintains control over the entire workflow; agents cannot interfere with each other's operations. If an agent fails, the run fails and downstream stages do not execute. `BuildSequential` itself does not add retry, recovery, or conditional-routing behavior. We add those capabilities explicitly when the workflow requires them.

Representative use cases include step-by-step flight procedures, multi-stage validation pipelines, and decision flows where each stage must complete before the next can start.

12.2.1 Sequential Orchestration in Action

Assume the user sends Robby this mission: `There is a tree directly in front of the car. Avoid it and then come back to the original path.` The workflow does not send that command straight to the motors. It first gathers environmental facts, evaluates whether conditions are safe, and only then allows movement to be executed.

Let's use a workflow in which a natural-language mission is processed by three specialist agents in a fixed order:

1. `EnvironmentAgent` perceives. It reads the sensors (temperature, humidity, rain droplets, wind) and reports on the environment. It owns `SensorTools`.
2. `SafetyAgent` decides. It reads the environmental report and grants or denies clearance. It has no tools at all; its only job is judgment.
3. `MotorsAgent` acts. It turns the mission into moves and executes them with `MotorTools` or stops if clearance is denied.

Figure 12.3 shows the conceptual diagram of clearance-based movement execution:

Example: Clearance-Based Movement Execution

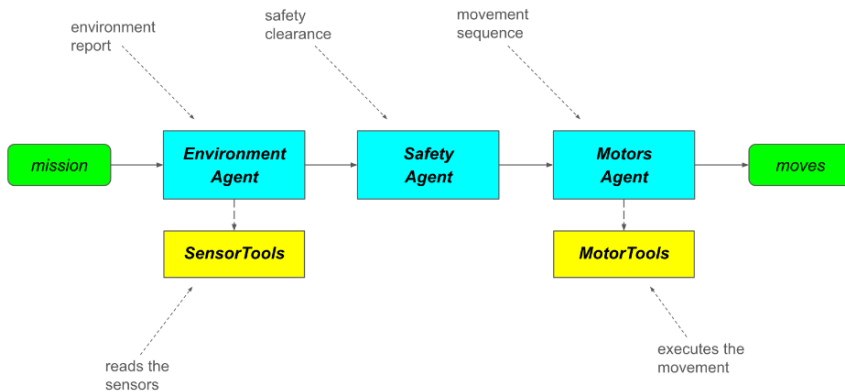


Figure 12.3 The diagram shows a Sequential Orchestrator in which a “Mission Command” request flows through three agents in order: Environmental, Safety, and Motors. Each agent’s output becomes the next agent’s input. For example, the environmental report produced by Environment Agent feeds Safety Agent, and the safety clearance produced by Safety Agent feeds Motors Agent, which produces a safe sequence of movements.

The mission flows through `EnvironmentAgent`, then `SafetyAgent`, then `MotorsAgent`, and each agent’s output becomes the next agent’s input. This is a good fit for sequential orchestration because each stage depends on the result of the previous one.

PRACTICAL EXAMPLE

Why not one agent? A single “god agent” for Robby would have to perceive, judge, and act with one prompt and one tool surface. Splitting the workflow into three agents gives you narrower prompts, less tool exposure, and clearer observability when something goes wrong.

The separation of responsibilities is deliberate. `EnvironmentAgent` is the only agent that can read sensors, `SafetyAgent` makes only the clearance decision, and `MotorsAgent` is the only agent that can invoke movement tools.

Listing 12.3 shows a *Sequential* orchestrator that uses three specialized agents.

Requirements (NuGet):

```
dotnet add package Microsoft.Agents.AI
dotnet add package Microsoft.Agents.AI.OpenAI
dotnet add package Microsoft.Agents.AI.Workflows
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
```

Code path: `SequentialOrchestration/Program.cs`

Listing 12.3 Sequential orchestrator agents’ initialization

```
// 'usings' and API key fetching omitted for brevity

var environmentAgent = new OpenAIClient(apiKey)
    .GetChatClient(model)
    .AsAIAgent("""
    ## PERSONA
    You are the EnvironmentAgent that reads sensors.

    ## ACTIONS
```

```

    Call SensorTools to read temperature, humidity, rain drops, and wind speed.

    ## OUTPUT TEMPLATE
    Respond only with the environment report, and make the rain status easy to
    identify.
    """
    name: "EnvironmentAgent",
    tools: [... SensorTools.AsAITools()]
); //❶

var safetyAgent = new OpenAIClient(apiKey)
    .GetChatClient(model)
    .AsAIAgent("""
    ## PERSONA
    You are the SafetyAgent that grants or denies mission clearance.

    ## ACTIONS
    Grant clearance unless the droplet level is Medium or High (rain detected),
    otherwise deny.

    ## OUTPUT TEMPLATE
    Respond with GRANTED or DENIED and a brief reason.
    """,
    "SafetyAgent"
); //❷

var motorsAgent = new OpenAIClient(apiKey)
    .GetChatClient(model)
    .AsAIAgent("""
    ## PERSONA
    You are the MotorsAgent that executes movement commands.

    ## ACTIONS
    If clearance is DENIED, call the Stop tool and then respond with
    "Mission stopped due to unsafe conditions."
    Otherwise, break the mission into moves
    (forward, backward, turn left, turn right, stop) and execute them
    using MotorTools.

    ## OUTPUT TEMPLATE
    Respond only with the executed movement sequence.
    """,
    "MotorsAgent",
    tools: [... MotorTools.AsAITools()]
); //❸

```

❶ defines **EnvironmentAgent** with sensor-reading tools

❷ defines **SafetyAgent** that grants or denies clearance

❸ defines **MotorsAgent** that executes movement commands

Now that we've initialized the agents, let's create and run the workflow (listing 12.4).

Listing 12.4 Building and running a sequential orchestrator workflow

```

var prompt = """
# MISSION COMMAND: Exploration Trip

There is a tree directly in front of the car.
Avoid it and then come back to the original path.
"""; //❶

var workflow = AgentWorkflowBuilder
    .BuildSequential(environmentAgent, safetyAgent, motorsAgent);

await using Run run = await InProcessExecution
    .RunAsync(workflow, input: prompt); //❷

```

```

foreach (WorkflowEvent evt in run.NewEvents) //❸
{
    switch (evt)
    {
        case ExecutorCompletedEvent completed: //❹
            Console.WriteLine($"[EXECUTOR] {completed.ExecutorId} completed.");
            break;

        case AgentResponseEvent response: //❺
            Console.WriteLine(response.Response.Text);
            break;

        case AgentResponseUpdateEvent update: //❻
            Console.Write(update.Update.Text);
            break;

        case WorkflowOutputEvent output:
            List<Microsoft.Extensions.AI.ChatMessage>? messages =
                output.As<List<Microsoft.Extensions.AI.ChatMessage>>();
            Console.WriteLine("\n[WORKFLOW OUTPUT] "
                + $"{messages?.LastOrDefault()?.Text}");
            break;

        case WorkflowErrorEvent error: //❼
            Console.WriteLine("\n[WORKFLOW ERROR] "
                + $" {error.Exception?.InnerException?.Message
                    ?? error.Exception?.Message ?? "unknown"}");
            break;

        case ExecutorFailedEvent failed: //❽
            Console.Error.WriteLine($" \n[EXECUTOR FAILED] {failed.Data?.Message}");
            break;
    }
}

```

- ❶ defines the mission text describing the tree avoidance task
- ❷ runs the sequential workflow with the mission input
- ❸ iterates over all workflow events from the run
- ❹ handles executor completion events for observability
- ❺ logs full agent response messages when they arrive
- ❻ logs streaming agent response updates token by token
- ❼ logs workflow-level errors emitted during execution
- ❽ logs failures for individual executors in the workflow

Code output (DENIED):

```

[02:14:28:890] SENSORS: READING Temperature: 26 Celsius degrees.
[02:14:29:416] SENSORS: READING Humidity: 39 %
[02:14:29:933] SENSORS: READING Droplet Level: Medium
[02:14:30:456] SENSORS: READING Wind speed: 14 kmph
[02:14:33:396] MOTORS: Stop
[EXECUTOR] EnvironmentAgent_a71f5ee81a664b0c9535097aa445fbf2 completed.
Environment Report
- Temperature: 26°C
- Humidity: 39%
- **RAIN STATUS: Medium**
- Wind Speed: 14 kmph
[EXECUTOR] EnvironmentAgent_a71f5ee81a664b0c9535097aa445fbf2 completed.
[EXECUTOR] SafetyAgent_f78ed979063e4ef09488bee8f8434046 completed.
[EXECUTOR] SafetyAgent_f78ed979063e4ef09488bee8f8434046 completed.
DENIED - Rain detected at Medium droplet level.
[EXECUTOR] SafetyAgent_f78ed979063e4ef09488bee8f8434046 completed.
[EXECUTOR] MotorsAgent_22ed62883adf4bb230685697688a69 completed.
[EXECUTOR] MotorsAgent_22ed62883adf4bb230685697688a69 completed.
Mission stopped due to unsafe conditions.
[EXECUTOR] MotorsAgent_22ed62883adf4bb230685697688a69 completed.

```

```
[EXECUTOR] OutputMessages completed.
[EXECUTOR] OutputMessages completed.
[WORKFLOW OUTPUT] Mission stopped due to unsafe conditions.
[EXECUTOR] OutputMessages completed.
```

EnvironmentAgent reports rain, SafetyAgent denies clearance, and MotorsAgent stops the robot. The workflow still completes normally because DENIED is treated as regular output.

Code output (GRANTED):

```
[02:12:11:852] SENSORS: READING Temperature: 83 Celsius degrees.
[02:12:12:365] SENSORS: READING Humidity: 85 %
[02:12:12:879] SENSORS: READING Droplet Level: Low
[02:12:13:402] SENSORS: READING Wind speed: 14 kmph
[02:12:17:397] MOTORS: TurnRight: 90°
[02:12:19:196] MOTORS: Forward: 1m
[02:12:20:895] MOTORS: TurnLeft: 90°
[02:12:22:603] MOTORS: Forward: 2m
[02:12:24:640] MOTORS: TurnLeft: 90°
[02:12:26:624] MOTORS: Forward: 1m
[EXECUTOR] EnvironmentAgent 24f1739a78c341beb82aa71c8fcf8fe9 completed.
Environment Report
- Temperature: 83
- Humidity: 85
- **RAIN STATUS: Low**
- Wind Speed: 14 kmph
[EXECUTOR] EnvironmentAgent 24f1739a78c341beb82aa71c8fcf8fe9 completed.
[EXECUTOR] SafetyAgent_1ef36793f0dc4e099909506f01e7476f completed.
[EXECUTOR] SafetyAgent_1ef36793f0dc4e099909506f01e7476f completed.
GRANTED - Droplet level is Low, so conditions are acceptable.
[EXECUTOR] SafetyAgent_1ef36793f0dc4e099909506f01e7476f completed.
[EXECUTOR] MotorsAgent_7a26fbab470a4d079725363e40299550 completed.
[EXECUTOR] MotorsAgent_7a26fbab470a4d079725363e40299550 completed.
Turn right 90°, forward 1 m, turn left 90°, forward 2 m, turn left 90°, forward 1
m
[EXECUTOR] MotorsAgent_7a26fbab470a4d079725363e40299550 completed.
[EXECUTOR] OutputMessages completed.
[EXECUTOR] OutputMessages completed.
[WORKFLOW OUTPUT] Turn right 90°, forward 1 m, turn left 90°, forward 2 m, turn
left 90°, forward 1 m
[EXECUTOR] OutputMessages completed.
```

EnvironmentAgent reports safe conditions, SafetyAgent grants clearance, and MotorsAgent executes the movement plan. The workflow completes normally with the movement sequence as output.

IMPORTANT Agent failure is a hard stop. In a *Sequential* pipeline, a failure in any executor stops the entire run. If SafetyAgent throws a network timeout or malformed tool response, MotorsAgent never fires. That behavior is appropriate here because if the sensor reading fails or clearance cannot be determined, the motors should never execute movement. For non-hard-stop behavior, we have to dive deeper into how workflows are built.

EXERCISE

Let's suppose that Robby must abort the mission immediately if he gets a DENIED report.

The previous sample (listing 12.4) treated the *Sequential* workflow's agent run as "successful" as long as it completed without throwing an exception, even if the response content was a DENIED report rather than a clearance.

We extend the sample by adding *Agent* middleware to SafetyAgent, so the workflow can treat a DENIED as a hard failure instead of just another text response.

Update the sample so that when the safety response contains DENIED, the middleware throws an exception and aborts that executor.

Because this failure happens while the workflow is running, use streaming execution to observe it in real time. Replace the non-streaming `RunAsync` path with `RunStreamingAsync`, then inspect the emitted events through `WatchStreamAsync`.

A run may produce two different types of errors:

- `ExecutorFailedEvent` means a specific executor failed. Here, that is the `SafetyAgent` stage failing because the middleware threw `InvalidOperationException("MISSION ABORT")`.
- `WorkflowErrorEvent` means the workflow emitted a workflow-level error event. This represents a broader workflow error than a single executor failure.

For Robby, either case means the mission must not continue, but they describe different failure scopes: one is executor-level, the other is workflow-level. Here, we focus on the executor.

Add middleware to the old agent definition, as shown in listing 12.5.

Listing 12.5 Agent with middleware

```
var safetyAgent = new OpenAIClient(apiKey)
    .GetChatClient(model)
    .AsAIAgent("""
        ## PERSONA
        You are the SafetyAgent that grants or denies mission clearance.

        ## ACTIONS
        Grant clearance unless the droplet level is Medium or High (rain detected),
        otherwise deny.

        ## OUTPUT TEMPLATE
        Respond with GRANTED or DENIED and a brief reason.
        """,
        "SafetyAgent")
    .AsBuilder()
    .Use(
        runFunc: AgentResponses.MissionAbort, //❶
        runStreamingFunc: null) //❷
    .Build();
```

❶ Registers `MissionAbort` for non-streaming agent runs

❷ No streaming middleware is registered

WARNING The `Use` method accepts separate callbacks for non-streaming and streaming agent runs. This exercise needs only the non-streaming callback, so `getStreamingResponseFunc` is `null`.

And add *Agent* middleware class (listing 12.6).

Code path: `SequentialOrchestration/AgentResponses.cs`

Listing 12.6 Mission Abort Response type middleware

```
public static class AgentResponses
{
    public static async Task<AgentResponse> MissionAbort(
        IEnumerable<ChatMessage> messages, AgentSession? session,
        AgentRunOptions? options, AIAgent innerAgent,
        CancellationToken cancellationToken) //❶
    {
        AgentResponse response = await innerAgent
            .RunAsync(messages, session, options, cancellationToken); //❷
        if (response.Text.Contains("DENIED") == true)
        {

```



```

        Console.WriteLine("\n*** MISSION ABORT ***");
        throw new InvalidOperationException("MISSION ABORT"); //❸
    }

    return response; //❹
}
}

```

- ❶ middleware signature wrapping the inner agent
- ❷ runs the inner agent to get its normal response
- ❸ throws to mark the mission as aborted on denial
- ❹ returns the original response when clearance is granted

We now have the new agent in place along with its middleware. The earlier version of the sample used the non-streaming execution code as in listing 12.7. `RunAsync` waits for the workflow to complete, so `run.NewEvents` is a completed event history: you can inspect what happened, but you cannot react while the run is in progress.

Listing 12.7 Non-streaming run execution

```

await using Run run = await InProcessExecution
    .RunAsync(workflow, input: prompt); //❶
foreach (WorkflowEvent evt in run.NewEvents) //❷
{
    // print workflow events
}

```

- ❶ runs and waits for the workflow to complete
- ❷ iterates through the workflow emitted events after execution has completed

To observe the middleware-triggered failure as soon as it occurs, replace listing 12.7 with the streaming execution code in listing 12.8.

Listing 12.8 Streaming run execution

```

await using StreamingRun run = await InProcessExecution
    .RunStreamingAsync(workflow, input: prompt); //❶
await run
    .TrySendMessageAsync(new TurnToken(emitEvents: true)); //❷
await foreach (WorkflowEvent evt in run.WatchStreamAsync()) //❸
{
    // handle workflow events
}

```

- ❶ runs the workflow in streaming mode
- ❷ explicitly sends turn token to trigger the run
- ❸ iterates through the workflow events as they are emitted

Because `MaintenanceTools/SensorTools` return randomized readings, re-running the app, a few times will naturally exercise both outcomes:

- Safety clearance contains `DENIED`: the middleware prints `*** MISSION ABORT ***`
 - the `SafetyAgent` executor fails
 - you should see an `ExecutorFailedEvent`
 - workflow should not continue to normal movement execution
- Safety clearance contains `GRANTED`:
 - the workflow ends with `MotorsAgent` producing the result

IMPORTANT Use streaming execution when the host application must propagate workflow events to the user or react to them as they occur. With `RunStreamingAsync`, `WatchStreamAsync()` yields events while the workflow is still executing, enabling live progress displays, token updates, alerts, approval prompts, or other immediate actions. With `RunAsync`, the workflow completes

first and `run.NewEvents` is available only afterward, so it supports post-run inspection but not real-time event handling.

12.2.2 Working with Agent Response Chaining Only

The first *Sequential* sample deliberately passes the growing conversation from one agent to the next. That behavior keeps the original mission available to every stage, but it also means that the context grows after every response. For a longer pipeline, later agents may receive details they do not need.

Agent Framework lets us choose between two chaining modes. With the default accumulated-conversation mode, each agent receives the original input and all responses produced so far. With response-only chaining, each agent receives only the immediately preceding agent response. The choice affects both the context sent between stages and the messages returned as the workflow output.

Think of the pipeline as a relay between Robby's agents. The full-conversation mode passes a growing mission folder containing the original command and every completed report. Response-only chaining passes only the latest handoff note. The smaller note is cheaper and more focused, but anything omitted from it is no longer available downstream.

TIP We can set `chainOnlyAgentResponses` to `true` in the *Sequential* orchestration builder `Workflow BuildSequential(string workflowName, bool chainOnlyAgentResponses, params IEnumerable<IAgent> agents)` when every agent produces a self-contained result that becomes the complete input for the next stage. This reduces context growth and prevents downstream agents from being influenced by earlier messages. Keep it `false` when later agents still need the original request (as in Robby's workflow, where `MotorsAgent` needs both the mission command and the safety decision).

Figure 12.4 shows response-only chaining, enabled by setting `chainOnlyAgentResponses` to `true`.

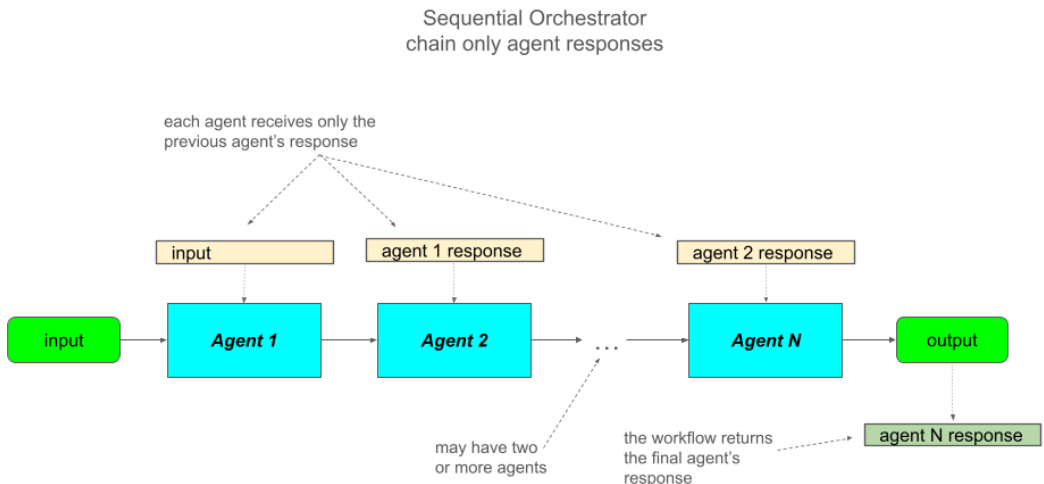


Figure 12.4 Response-only sequential chaining shows each agent receiving only the previous agent's response, while the workflow returns only the final agent's response.

Agent 1 still receives the original mission because it starts the pipeline. Agent 2 receives only Agent 1's response, and Agent N receives only Agent N-1's response. The terminal output executor also receives only the last response, so the workflow output no longer contains the original input or earlier agent responses.

IMPORTANT The `chainOnlyAgentResponses` option changes message flow; it does not summarize missing context. When it is true, each agent must produce a self-contained handoff containing everything the next agent needs.

PRACTICAL EXAMPLE

Robby's `EnvironmentAgent`, `SafetyAgent`, and `MotorsAgent` make the trade-off concrete. `SafetyAgent` can decide from the environment report alone, so response-only chaining is sufficient for that handoff. `MotorsAgent` is different: after a `GRANTED` decision, it needs both the safety verdict and the original tree-avoidance mission. If it receives only `GRANTED` it knows that movement is allowed but not which movement to execute.

Listing 12.9 shows a *Sequential* orchestrator as pipeline that uses three specialized agents.

Requirements (NuGet):

```
dotnet add package Microsoft.Agents.AI
dotnet add package Microsoft.Agents.AI.OpenAI
dotnet add package Microsoft.Agents.AI.Workflows
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
```

Code path: `SequentialOrchestrationAsPipeline/Program.cs`

Listing 12.9 Sequential orchestration pipeline agents' initialization

```
// 'usings' and API key fetching omitted for brevity

var environmentAgent = new OpenAIClient(apiKey)
    .GetChatClient(model)
    .AsAIAgent("""
    ## PERSONA
    You are the EnvironmentAgent that starts a mission by reading sensors and
    interpreting the mission command.

    ## ACTIONS
    Call SensorTools to read temperature, humidity, rain drops, and wind speed.
    Extract the destination, obstacles, and required outcome from the mission
    command.
    Include all mission details in your report, including the original command,
    because the next agent receives only your response.

    ## OUTPUT TEMPLATE
    Respond only with a self-contained report containing:
    - Mission objective
    - Known obstacles
    - Temperature
    - Humidity
    - Rain droplet level
    - Wind speed
    """,
    "EnvironmentAgent",
    tools: [.. SensorTools.AsAITools()]); //❶

var navigatorAgent = new OpenAIClient(apiKey)
    .GetChatClient(model)
    .AsAIAgent("""
    ## PERSONA
    You are the NavigatorAgent that transforms an environment report into a route
    plan.
```

```

    ## ACTIONS
    Use only the received environment report.
    Plan a safe route that completes the mission objective and avoids every known
    obstacle.
    Express the route as an ordered list using only: forward, backward, turn left,
    turn right, and stop.

    ## OUTPUT TEMPLATE
    Respond only with the ordered movement plan, with one movement per line.
    "",
    "NavigatorAgent"); //❷

var motorsAgent = new OpenAIClient(apiKey)
    .GetChatClient(model)
    .AsAIAgent("")
    ## PERSONA
    You are the MotorsAgent that executes an ordered movement plan.

    ## ACTIONS
    Execute each received movement in order using MotorTools.
    Do not reinterpret the mission or alter the route plan.

    ## OUTPUT TEMPLATE
    Respond only with the executed movement sequence.
    "",
    "MotorsAgent",
    tools: [.. MotorTools.AsAITools()]); //❸

var prompt = ""
    # MISSION COMMAND: Exploration Trip

    There is a tree directly in front of the car. Avoid it and then come back to the
    original path.
    "",

```

❶ defines **EnvironmentAgent** with sensor-reading tools

❷ defines **NavigatorAgent** that transforms the environment report into a movement plan

❸ defines **MotorsAgent** that executes movement commands

Now that we've initialized the agents, let's create and run the workflow. Listing 12.10 makes the chaining policy explicit and prints every text message in the workflow output, making the two modes easy to compare.

Listing 12.10 Selecting and observing the sequential chaining mode

```

Workflow workflow = AgentWorkflowBuilder.BuildSequential(
    "ProcessingPipeline",
    chainOnlyAgentResponses: true, //❶
    environmentAgent,
    navigatorAgent,
    motorsAgent); //❷

await using StreamingRun run = await InProcessExecution
    .RunStreamingAsync(workflow, input: prompt);
await run.TrySendMessageAsync(new TurnToken(emitEvents: true));

await foreach (WorkflowEvent evt in run.WatchStreamAsync())
{
    switch (evt)
    {
        case ExecutorCompletedEvent completed:
            Console.WriteLine($"[EXECUTOR] {completed.ExecutorId} completed.");
            break;

        case AgentResponseUpdateEvent update:

```

```

        Console.Write(update.Update.Text);
        break;

    case WorkflowOutputEvent output:
        List<Microsoft.Extensions.AI.ChatMessage>? messages = output
            .As<List<Microsoft.Extensions.AI.ChatMessage>>();
        var allAgentsMessages = messages?
            .Where(m => m.Role != ChatRole.Tool && !string.IsNullOrEmpty(m.Text))
            .Select(m => $"{m.Role}: {m.Text}"); //❸
        Console.WriteLine("\n[WORKFLOW OUTPUT] "
            + $"{string.Join("\n", allAgentsMessages!)}");
        break;

    case WorkflowErrorEvent error:
        Console.WriteLine("\n[WORKFLOW ERROR] " +
            $"{error.Exception?.InnerException?.Message
            ?? error.Exception?.Message
            ?? "unknown"}");
        break;

    case ExecutorFailedEvent failed:
        Console.WriteLine($"{"\n[EXECUTOR FAILED] {failed.Data?.Message}");
        break;
}
}

```

- ❶ keeps only last agent response in conversation by selecting true
- ❷ builds the sequential workflow with an explicit chaining policy
- ❸ Selects each non-tool and non-empty text messages returned by the workflow

Code output (chainOnlyAgentResponses: true):

```

[EXECUTOR] EnvironmentAgent_6f8261c61caf411ba281be2905f719cb completed.
[11:03:53:981] SENSORS: READING Temperature: 4 Celsius degrees.
[11:03:54:510] SENSORS: READING Humidity: 58 %
[11:03:55:016] SENSORS: READING Droplet Level: Medium
[11:03:55:535] SENSORS: READING Wind speed: 30 kmph
- Mission objective: Proceed on the exploration trip, avoid the tree directly in
front of the car, then return to the original path. Original command: "There is a
tree directly in front of the car. Avoid it and then come back to the original
path."
- Known obstacles: Tree directly in front of the car
- Temperature: 4
- Humidity: 58
- Rain droplet level: Medium
- Wind speed: 30
[EXECUTOR] EnvironmentAgent_6f8261c61caf411ba281be2905f719cb completed.
[EXECUTOR] NavigatorAgent_2db96454016c4787b8a3e017f6792b3b completed.
turn right
forward
turn left
forward
turn left
forward
turn right
stop
[EXECUTOR] NavigatorAgent_2db96454016c4787b8a3e017f6792b3b completed.
[EXECUTOR] MotorsAgent_efea1b0d46d3417f89aa6347937a516d completed.
[11:03:59:054] MOTORS: TurnRight: 90°
[11:04:00:929] MOTORS: Forward: 1m
[11:04:02:885] MOTORS: TurnLeft: 90°
[11:04:04:721] MOTORS: Forward: 1m
[11:04:06:700] MOTORS: TurnLeft: 90°
[11:04:08:710] MOTORS: Forward: 1m
[11:04:10:508] MOTORS: TurnRight: 90°
[11:04:12:499] MOTORS: Stop
turn right
forward

```

```

turn left
forward
turn left
forward
turn right
stop
[EXECUTOR] MotorsAgent_efealb0d46d3417f89aa6347937a516d completed.
[EXECUTOR] OutputMessages completed.

[WORKFLOW OUTPUT] turn right
forward
turn left
forward
turn left
forward
turn right
stop
[EXECUTOR] OutputMessages completed.

```

We collect all agent responses in `allAgentsMessages` for learning purposes only, because when `chainOnlyAgentResponses` is `true`, the workflow output retains only the final agent's response.

Choose response-only chaining when:

- Every agent response is a complete handoff for the next stage
- Earlier prompts and responses would add noise or unnecessary token cost
- The caller needs only the final agent response as the workflow output

Keep the full accumulated conversation when:

- A downstream agent still needs the original user request
- Later stages must inspect more than the immediately preceding response
- The caller needs complete conversation produced by the workflow

EXERCISE

With `chainOnlyAgentResponses` set to `true`, the workflow output retains only the most recent agent response. When it is set to `false`, the output retains the original mission together with the responses from all three agents.

Now change `chainOnlyAgentResponses` to `false` in listing 12.10 and run the same pipeline again.

You'll see that the output includes the input mission and every collected agent response.

IMPORTANT When `chainOnlyAgentResponses` is `true`, the workflow output contains only the last agents' responses, but when it is `false`, the workflow output contains the original input and the responses from all agents.

12.2.3 Working with Checkpoints

Long-running workflows may be interrupted by process failures, transient infrastructure problems, or planned pauses. Restarting the workflow from the beginning can repeat expensive model calls and tool operations. Agent Framework *checkpoints* let us capture the workflow's execution state and later resume from a completed stage.

IMPORTANT A checkpoint is a saved snapshot of a workflow at a known execution boundary. It records enough state to continue from that point later, rather than restarting the workflow and repeating stages that have already been completed.

A workflow executes in *super steps*. During a super step, one or more activated executors process their available messages. When those executors finish, Agent Framework emits a `SuperStepCompletedEvent`. If the run has defined a `CheckpointManager`, the framework automatically creates a checkpoint at the end of every super step.

For Robby's sequential workflow, a useful recovery point is immediately after `SafetyAgent` grants clearance but before `MotorsAgent` executes movement.

If execution must be resumed, the environment and safety stages do not need to run again. The checkpoint contains the messages and workflow state required to continue with the pending motors stage.

PRACTICAL EXAMPLE

Imagine Robby's workflow is interrupted after the safety assessment. By resuming from a checkpoint, Robby can recover quickly without repeating the completed environment and safety checks. Listing 12.11 shows how to enable checkpointing when starting the streaming run.

Requirements (NuGet):

```
dotnet add package Microsoft.Agents.AI
dotnet add package Microsoft.Agents.AI.OpenAI
dotnet add package Microsoft.Agents.AI.Workflows
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
```

Code path: `SequentialOrchestrationWithCheckpoints/Program.cs`

Listing 12.11 Starting a checkpointed sequential workflow

```
CheckpointManager checkpointManager = CheckpointManager.Default; // ❶

Workflow workflow = AgentWorkflowBuilder.BuildSequential(
    "SafeExecutionWithCheckpoints",
    environmentAgent,
    safetyAgent,
    motorsAgent); // ❷

await using StreamingRun run = await InProcessExecution.RunStreamingAsync(
    workflow,
    input: prompt,
    checkpointManager: checkpointManager); // ❸

await run.TrySendMessageAsync(new TurnToken(emitEvents: true)); // ❹
```

❶ creates the checkpoint manager used to store and retrieve checkpoints

❷ builds the named sequential workflow

❸ enables automatic checkpoint creation for the run

❹ starts workflow processing and requests event emission

IMPORTANT `CheckpointManager.Default` uses in-memory storage. It is suitable for this sample, but its checkpoints do not survive application termination. A production application that must recover after a process restart needs a persistent checkpoint store.

The framework creates a checkpoint after every super step, so the application must select the checkpoint that represents the desired recovery boundary. Listing 12.12 selects the checkpoint produced after `SafetyAgent` completes.

Listing 12.12 Capturing the checkpoint and motor execution

```
CheckpointInfo? checkpoint = await ProcessWorkflowAsync(run);

async Task<CheckpointInfo?> ProcessWorkflowAsync(StreamingRun run, bool
captureCheckpoint = true) // ❶
{
```

```

CheckpointInfo? checkpoint = null;
await foreach (WorkflowEvent evt in run.WatchStreamAsync())
{
    switch (evt)
    {
        case SuperStepCompletedEvent superStep:
            Console.WriteLine($"[SUPER STEP] {superStep.StepNumber} completed.");
            if (captureCheckpoint && checkpoint is null)
            {
                if (superStep.CompletionInfo!.ActivatedExecutors.Any(id =>
id.StartsWith("MotorsAgent", StringComparison.Ordinal)))
                {
                    checkpoint = superStep.CompletionInfo?.Checkpoint
                        ?? throw new InvalidOperationException("Checkpoint is null.");
                    Console.WriteLine($"*** CHECKPOINT CREATED ***"); // ❷
                }
            }
            break;

        case ExecutorCompletedEvent completed:
            Console.WriteLine($"[EXECUTOR] {completed.ExecutorId} completed.");
            break;

        case AgentResponseUpdateEvent update:
            Console.WriteLine(update.Update.Text);
            break;

        case WorkflowOutputEvent output:
            List<Microsoft.Extensions.AI.ChatMessage>? messages =
output.As<List<Microsoft.Extensions.AI.ChatMessage>>();
            Console.WriteLine("\n[WORKFLOW OUTPUT] "
                + $"{messages?.LastOrDefault()?.Text}");
            break;

        case WorkflowErrorEvent error:
            Console.WriteLine($"[WORKFLOW ERROR] "
                + $"{(error.Exception?.InnerException?.Message
                    ?? error.Exception?.Message
                    ?? "unknown")}");
            break;

        case ExecutorFailedEvent failed:
            Console.WriteLine($"[EXECUTOR FAILED] {failed.Data?.Message}");
            break;
    }
}

return checkpoint;
}

```

❶ defines the execution with the checkpoint creation

❷ sets a newer checkpoint

`CompletionInfo.ActivatedExecutors` identifies the executors that participated in the completed super step. In our example, the generated executor ID contains a unique suffix, so we use `StartsWith("MotorsAgent")` rather than comparing the complete identifier.

`CompletionInfo.Checkpoint` is a `CheckpointInfo` value that identifies the saved state. The checkpoint data itself remains managed by the `CheckpointManager`; therefore, the same manager must be supplied when the workflow is resumed.

After the original run completes, the sample creates a fresh workflow and resumes it from the selected checkpoint, as shown in listing 12.13.

Listing 12.13 Resuming from the checkpoint


```

if (checkpoint is not null)
{
    Console.WriteLine("*** RESUMING FROM CHECKPOINT ***");

    Workflow resumedWorkflow = AgentWorkflowBuilder.BuildSequential(
        "SafeExecutionFromCheckpoint",
        environmentAgent,
        safetyAgent,
        motorsAgent); //❶

    await using StreamingRun resumedRun = await InProcessExecution
        .ResumeStreamingAsync(
            resumedWorkflow,
            checkpoint,
            checkpointManager,
            CancellationToken.None); //❷

    _ = await ProcessWorkflowAsync(resumedRun, captureCheckpoint: false); //❸
}

```

- ❶ rebuilds the workflow with the same executor structure
- ❷ restores the saved state and continues pending execution
- ❸ calls again the execution

Code output:

```

[EXECUTOR] EnvironmentAgent_6793e0eb64e74949b03157df0fda7234 completed.
[12:43:54:173] SENSORS: READING Temperature: 26 Celsius degrees.
[12:43:54:685] SENSORS: READING Humidity: 64 %
[12:43:55:199] SENSORS: READING Droplet Level: None
[12:43:55:722] SENSORS: READING Wind speed: 29 kmph
Temperature: 26°C
Humidity: 64%
RAIN STATUS: No rain detected
Wind Speed: 29 km/h[EXECUTOR] EnvironmentAgent_6793e0eb64e74949b03157df0fda7234
completed.
[SUPER STEP] 0 completed.
[EXECUTOR] SafetyAgent_b4f9feeb90f84c31b62fc66ec9798b70 completed.
[EXECUTOR] SafetyAgent_b4f9feeb90f84c31b62fc66ec9798b70 completed.
GRANTED - No rain detected (droplet level: None).[EXECUTOR]
SafetyAgent_b4f9feeb90f84c31b62fc66ec9798b70 completed.
[SUPER STEP] 1 completed.
*** CHECKPOINT CREATED ***
[EXECUTOR] MotorsAgent_49ale28eb0ae43a9803c59896460de49 completed.
[EXECUTOR] MotorsAgent_49ale28eb0ae43a9803c59896460de49 completed.
[12:43:58:771] MOTORS: TurnRight: 90°
[12:44:00:925] MOTORS: Forward: 1m
[12:44:02:745] MOTORS: TurnLeft: 90°
[12:44:04:517] MOTORS: Forward: 2m
[12:44:06:180] MOTORS: TurnLeft: 90°
[12:44:08:195] MOTORS: Forward: 1m
Turn right 90°, forward 1m, turn left 90°, forward 2m, turn left 90°, forward
1m[EXECUTOR] MotorsAgent_49ale28eb0ae43a9803c59896460de49 completed.
[SUPER STEP] 2 completed.
[EXECUTOR] OutputMessages completed.
[EXECUTOR] OutputMessages completed.

[WORKFLOW OUTPUT] Turn right 90°, forward 1m, turn left 90°, forward 2m, turn left
90°, forward 1m
[EXECUTOR] OutputMessages completed.
[SUPER STEP] 3 completed.
*** RESUMING FROM CHECKPOINT ***
[EXECUTOR] MotorsAgent_49ale28eb0ae43a9803c59896460de49 completed.
[EXECUTOR] MotorsAgent_49ale28eb0ae43a9803c59896460de49 completed.
[12:44:11:440] MOTORS: TurnRight: 90°
[12:44:13:210] MOTORS: Forward: 1m
[12:44:15:569] MOTORS: TurnLeft: 90°

```

```
[12:44:17:244] MOTORS: Forward: 2m
[12:44:18:949] MOTORS: TurnLeft: 90°
[12:44:20:969] MOTORS: Forward: 1m
Turn right 90°, forward 1m, turn left 90°, forward 2m, turn left 90°, forward
1m[EXECUTOR] MotorsAgent_49a1e28eb0ae43a9803c59896460de49 completed.
[SUPER STEP] 0 completed.
[EXECUTOR] OutputMessages completed.
[EXECUTOR] OutputMessages completed.

[WORKFLOW OUTPUT] Turn right 90°, forward 1m, turn left 90°, forward 2m, turn left
90°, forward 1m
[EXECUTOR] OutputMessages completed.
[SUPER STEP] 1 completed.
```

The resumed run begins directly with `MotorsAgent`, demonstrating that the completed environment and safety stages were restored from the checkpoint rather than executed again. Robby can therefore recover more quickly by continuing from the pending movement stage.

This sample resumes on a fresh `StreamingRun` instead of restoring state on the run that just completed. Doing so keeps the resumed execution separate from any queued, unpublished state updates left by the original run's final super step.

IMPORTANT A checkpoint restores workflow state; it does not undo external side effects. In this demonstration, the original run completes and executes the motors' maneuvers before the earlier checkpoint is resumed, so resuming can execute the motor commands again. Production workflows must make side-effecting operations idempotent, record operation identifiers, or pause for approval before replaying them.

Checkpointing is especially useful when model calls are expensive, workflows contain many stages, or execution must pause for external input. It provides recovery at well-defined super-step boundaries without requiring every executor to repeat work that has already been completed.

WARNING Checkpoints are not limited to sequential orchestration. They can be enabled for any Agent Framework workflow executed with a checkpoint manager, including concurrent, composed, and custom workflows. Checkpoints are created at super-step boundaries determined by the workflow graph.

EXERCISE

Let's assume Robby wants to preserve the time spent assessing environmental conditions and invoking the sensor tools. Modify the workflow to capture a checkpoint right after `EnvironmentAgent`, allowing an interrupted mission to resume with `SafetyAgent` instead of repeating the sensor readings.

In listing 12.12 replace the line:

```
id => id.StartsWith("MotorsAgent", StringComparison.Ordinal)))
```

with:

```
id => id.StartsWith("SafetyAgent", StringComparison.Ordinal)))
```

The resumed run should now execute both `SafetyAgent` and `MotorsAgent`, allowing Robby to resume from an earlier checkpoint.

12.3 Understanding Concurrent Orchestration

Concurrent orchestration is like a flight director sending the same command to several specialist crews at once. One crew checks the engine, another verifies navigation, another reviews weather and communications, and each works in parallel from the same flight plan. The flight director waits for every report, then combines them into a single go or no-go decision for the flight.

Concurrent orchestration fans out the same input to multiple agents at the same time and then aggregates their outputs into a single result. It is useful when the agents do not depend on each other's intermediate results and when parallel execution is faster than evaluating them one by one.

Figure 12.5 shows the functional diagram of a *Concurrent* orchestration.

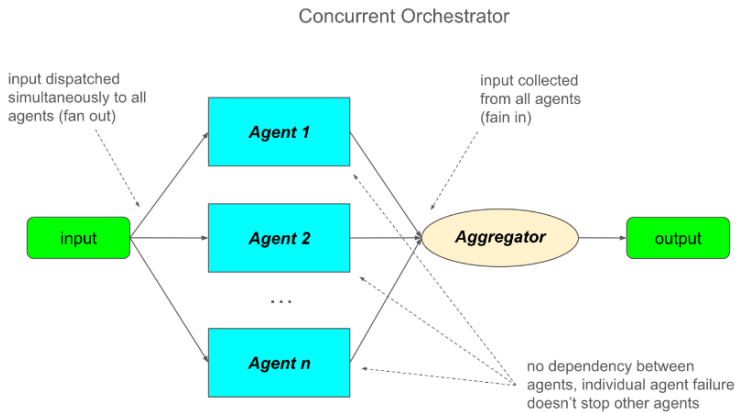


Figure 12.5 Concurrent Orchestrator sends the input to multiple agents simultaneously, allowing them to work in parallel. The agents operate independently without relying on each other, so the failure of one agent does not stop the others from completing their tasks. The individual results from each agent are then aggregated into a single, combined response.

The input is dispatched to multiple agents simultaneously, work proceeds independently, and an aggregator combines the results at the end. Unlike sequential orchestration, no agent waits for another agent's output before it starts.

This pattern is useful when:

- Multiple tasks are independent for the same input
- Waiting for tasks to complete serially would add unnecessary latency
- The outputs must be considered together before the workflow can continue

`BuildConcurrent` accepts an optional aggregator parameter, but in practice you almost always want to provide one. Without an explicit aggregation strategy, parallel execution does not automatically produce a meaningful combined decision.

IMPORTANT When the `aggregator` argument is omitted, `BuildConcurrent` uses a default aggregator that returns the last message from each nonempty branch message list. It preserves every response, but it does not interpret, reconcile, rank, or summarize them. Provide a custom aggregator when the workflow must produce a domain decision, such as Robby's single `GRANTED` or `DENIED` safety clearance.

12.3.1 Concurrent Orchestration in Action

Assume the user sends Robby another mission: `Exploration Trip: Assess the environmental conditions and ensure safety clearance`. The workflow has to assess clearance from multiple sources (agents). It gathers environment and maintenance reports in parallel, then aggregates them into a coherent clearance result.

- `MaintenanceAgent` checks the subsystems (battery, tires, motors) using AI tools in `MaintenanceTools`.
- `EnvironmentAgent` checks the surroundings (temperature, humidity, droplets, wind) via AI tools in `SensorTools`.
- `SafetyAggregator` aggregates reports into a coherent clearance.

The structure is simple: the mission fans out to both agents, both run independently, and the aggregator produces the final clearance.

Figure 12.6 illustrates this fan-out/fan-in shape and aggregation.

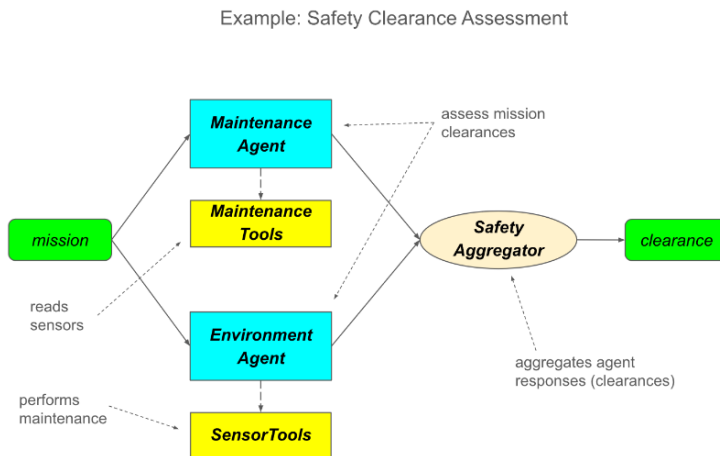


Figure 12.6 The diagram illustrates a Concurrent Orchestrator where “Mission Command” fans out to two independent agents: Maintenance and Environmental. Each agent runs in parallel on the same request, produces safety clearances, and sends its result directly to the Safety Aggregator. The structure highlights parallelism and independence: one agent’s outcome does not block the other.

The mission is sent to `MaintenanceAgent` and `EnvironmentAgent` at the same time, each produces its own clearance-oriented report, and both results flow into `SafetyAggregator`, which returns one final decision. This is exactly the kind of problem concurrent orchestration is meant to solve: independent checks that must be evaluated together.

PRACTICAL EXAMPLE

The key design choice is the aggregator. Running agents in parallel is only half of the pattern; the workflow also needs a rule for turning multiple reports into one decision.

Aggregation is much easier if each agent returns a structured response (as JSON) instead of simple text. Rather than scraping text for keywords like `GRANTED` or `DENIED`, we give every agent the same type of contract, a `Response` record containing a `ClearanceState` (see listing 12.14).

Code path: `ConcurrentOrchestration/ClearanceState.cs`

Listing 12.14 ClearanceState enumeration

```
public enum ClearanceState
{
    GRANTED,
    DENIED
}
```

The `Response` type in listing 12.15 is a user-defined record used to set the agent `ResponseFormat`. Each concurrent agent is instructed to return JSON that matches this model, including its clearance decision and reason. The aggregator deserializes each agent's JSON response into a `Response` object before applying its aggregation rule.

Code path: `ConcurrentOrchestration/Response.cs`

Listing 12.15 Response record type

```
public record Response
{
    [JsonPropertyName("clearance")]
    public required ClearanceState Clearance { get; init; }

    [JsonPropertyName("reason")]
    public required string Reason { get; init; }
}
```

Each concurrent agent returns a structured JSON response that matches the user-defined `Response` record. This gives `SafetyAggregator` predictable fields to deserialize and evaluate, rather than requiring it to infer decisions from raw text.

The aggregator receives one `List<ChatMessage>` per concurrent agent. It flattens those lists, deserializes the assistant messages into `Response` objects, and denies the combined clearance if any agent returns `DENIED`. Listing 12.16 shows this aggregation logic.

Code path: `ConcurrentOrchestration/SafetyAggregator.cs`

Listing 12.16 SafetyAggregator

```
public class SafetyAggregator
{
    public static JsonSerializerOptions
        JsonSerializerOptions { get; } = new(JsonSerializerDefaults.Web)
    {
        Converters = { new JsonStringEnumConverter() }
    }; //❶

    public static List<ChatMessage> AggregateClearances(
        IList<List<ChatMessage>> agentResponses) //❷
    {
        List<Response> responses = [.. agentResponses
            .SelectMany(messages => messages)
            .Where(message => message.Role == ChatRole.Assistant
                && !string.IsNullOrEmpty(message.Text))
            .Select(message => JsonSerializer
                .Deserialize<Response>(message.Text, JsonSerializerOptions)!)]; //❸

        string[] deniedReasons = [.. responses
            .Where(response => response.Clearance == ClearanceState.DENIED)
            .Select(response => response.Reason)]; //❹

        string summary = deniedReasons.Length > 0
            ? $"{ClearanceState.DENIED}: {string.Join(" ", deniedReasons)}"
            : $"{ClearanceState.GRANTED}: All clearances were granted."; //❺
    }
}
```

```
        return [new ChatMessage(ChatRole.Assistant, summary)]; //❹
    }
}
```

- ❶ configures JSON options with enum-as-string conversion
- ❷ aggregates multiple agents' chat responses into one message list
- ❸ flattens the per-agent message lists and deserializes each JSON response
- ❹ collects reasons from all DENIED responses into an array
- ❺ builds GRANTED or DENIED summary text from the reasons
- ❻ returns a List<ChatMessage> containing the aggregated assistant response

JsonStringEnumConverter is required because the agents emit enum values as JSON strings, for example GRANTED and DENIED. The converter allows to deserialize those strings into the ClearanceState enum.

The aggregator receives one message list per concurrent agent and returns a merged List<ChatMessage> containing the final assistant message.

Listing 12.17 shows *Concurrent* orchestration using an aggregation.

Requirements (NuGet):

```
dotnet add package Microsoft.Agents.AI
dotnet add package Microsoft.Agents.AI.OpenAI
dotnet add package Microsoft.Agents.AI.Workflows
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
```

Code path: ConcurrentOrchestration/Program.cs

Listing 12.17 Concurrent orchestrator agents initialization

```
// 'usings' and API key fetching omitted for brevity

ChatResponseFormatJson responseFormat =
    Microsoft.Extensions.AI.ChatResponseFormat
        .ForJsonSchema<Response>(SafetyAggregator.JsonSerializerOptions); //❶

var maintenanceAgent = new OpenAIClient(apiKey)
    .GetChatClient(model)
    .AsAIAgent(new ChatClientAgentOptions
    {
        Name = "MaintenanceAgent",
        ChatOptions = new ChatOptions
        {
            Instructions = """
                ## PERSONA
                You are the MaintenanceAgent that monitors maintenance conditions.

                ## ACTIONS
                Activate maintenance protocols for dangerous conditions detection.

                ## SAFETY THRESHOLDS
                Grant clearance unless ANY of the following hard limits are exceeded:
                - Battery health below 30%
                - Tire pressure below 28 PSI or above 36 PSI
                - Motor efficiency below 30%

                ## OUTPUT TEMPLATE
                Respond with the maintenance clearance using the requested JSON schema.
                """,
            ResponseFormat = responseFormat, //❷
            Tools = [.. MaintenanceTools.AsAITools()] //❸
        }
    }); //❹

var environmentAgent = new OpenAIClient(apiKey)
    .GetChatClient(model)
```

```

.AsAIAgent(new ChatClientAgentOptions
{
    Name = "EnvironmentAgent",
    ChatOptions = new ChatOptions
    {
        Instructions = """
            ## PERSONA
            You are the EnvironmentAgent that reads sensors.

            ## ACTIONS
            Read sensors for temperature, humidity, rain drops, and wind speed.

            ## SAFETY THRESHOLDS
            Grant clearance unless ANY of the following hard limits are exceeded:
            - Temperature above 100 Celsius
            - Humidity above 80%
            - Droplet level is Extreme
            - Wind speed above 100 kmph

            ## OUTPUT TEMPLATE
            Respond with the environment clearance using the requested JSON schema.
            """,
        ResponseFormat = responseFormat, //❷
        Tools = [.. SensorTools.AsAITools()] //❸
    }
}); //❹

```

- ❶ defines the JSON schema response format
- ❷ enforces JSON output for MaintenanceAgent
- ❸ gives MaintenanceAgent access to maintenance tools
- ❹ defines the maintenance agent
- ❺ enforces JSON output for EnvironmentAgent
- ❻ gives EnvironmentAgent access to sensor tools
- ❼ defines the environment agent

IMPORTANT We can request plain text, unrestricted JSON, or no response format and rely on prompt instructions. We may use JSON Schema because it defines the expected fields and types, making the response safer to deserialize and use for workflow routing.

Now that we've initialized the agents, let's create and run the workflow (listing 12.18).

Listing 12.18 Building and running a concurrent orchestrator workflow

```

var prompt = """
MISSION COMMAND: Exploration Trip

Assess the environment conditions and ensure safety clearance.
""";

var workflow = AgentWorkflowBuilder
    .BuildConcurrent([maintenanceAgent, environmentAgent],
        SafetyAggregator.AggregateClearances); //❶

await using Run run = await InProcessExecution
    .RunAsync(workflow, input: prompt); //❷
foreach (WorkflowEvent evt in run.NewEvents) //❸
{
    switch (evt)
    {
        case ExecutorCompletedEvent completed: //❹
            Console.WriteLine($"[EXECUTOR] {completed.ExecutorId} completed.");
            break;
    }
}

```

```

case AgentResponseEvent response: //❶
    Console.WriteLine(response.Response.Text);
    break;

case AgentResponseUpdateEvent update: //❷
    Console.Write(update.Update.Text);
    break;

case WorkflowOutputEvent output: //❸
    List<Microsoft.Extensions.AI.ChatMessage>? messages = output
        .As<List<Microsoft.Extensions.AI.ChatMessage>>();
    Console.WriteLine($"\\n[WORKFLOW OUTPUT] {messages?.LastOrDefault()?.Text}");
    break;

case WorkflowErrorEvent error: //❹
    Console.WriteLine("\\n[WORKFLOW ERROR] "
        + $"{error.Exception?.InnerException?.Message
            ?? error.Exception?.Message
            ?? "unknown"}");
    break;

case ExecutorFailedEvent failed: //❺
    Console.Error.WriteLine($"\\n[EXECUTOR FAILED] {failed.Data?.Message}");
    break;
}
}

```

- ❶ builds the concurrent workflow with an aggregator
- ❷ runs the concurrent workflow with the mission input
- ❸ iterates over workflow events from the run
- ❹ handles executor completion events
- ❺ logs complete agent responses
- ❻ logs streaming agent response updates
- ❼ handles the final workflow output event
- ❽ logs workflow-level errors
- ❾ logs executor-level failures

Code output (at least one clearance DENIED):

```

[02:20:07:370] MAINTENANCE: CHECKING battery health: 14%
[02:20:07:382] SENSORS: READING Temperature: -11 Celsius degrees.
[02:20:07:900] SENSORS: READING Humidity: 74 %
[02:20:07:900] MAINTENANCE: CHECKING tire pressure: 30 PSI
[02:20:08:395] MAINTENANCE: CHECKING motors. Status: 42%
[02:20:08:395] SENSORS: READING Droplet Level: High
[02:20:08:910] SENSORS: READING Wind speed: 2 kmph
[EXECUTOR] Start completed.
{"clearance":"DENIED","reason":"Battery health is below the 30% safety threshold
at 14%. Tire pressure is within range at 30 PSI, and motor efficiency is above
threshold at 42%, but clearance is denied because one hard limit was exceeded."}
[EXECUTOR] MaintenanceAgent_1e3aa408bd2249d483fe69a5282a2201 completed.
{"clearance":"GRANTED","reason":"Temperature is -11 C, humidity is 74%, droplet
level is High, and wind speed is 2 kmph. None of the denial thresholds were
exceeded."}
[EXECUTOR] EnvironmentAgent_166d6c7e68534b049fd0c92470e17f43 completed.
[EXECUTOR] Batchter/EnvironmentAgent_166d6c7e68534b049fd0c92470e17f43 completed.
[EXECUTOR] Batchter/EnvironmentAgent_166d6c7e68534b049fd0c92470e17f43 completed.
[EXECUTOR] Batchter/MaintenanceAgent_1e3aa408bd2249d483fe69a5282a2201 completed.
[EXECUTOR] Batchter/MaintenanceAgent_1e3aa408bd2249d483fe69a5282a2201 completed.
[EXECUTOR] Batchter/MaintenanceAgent_1e3aa408bd2249d483fe69a5282a2201 completed.
[EXECUTOR] ConcurrentEnd completed.
[WORKFLOW OUTPUT] DENIED: Battery health is below the 30% safety threshold at 14%.
Tire pressure is within range at 30 PSI, and motor efficiency is above threshold
at 42%, but clearance is denied because one hard limit was exceeded.
[EXECUTOR] ConcurrentEnd completed.

```


Code output (all clearances GRANTED):

```
[02:23:30:515] MAINTENANCE: CHECKING battery health: 69%
[02:23:31:029] MAINTENANCE: CHECKING tire pressure: 32 PSI
[02:23:31:146] SENSORS: READING Temperature: 36 Celsius degrees.
[02:23:31:545] MAINTENANCE: CHECKING motors. Status: 56%
[02:23:31:653] SENSORS: READING Humidity: 67 %
[02:23:32:169] SENSORS: READING Droplet Level: Low
[02:23:32:692] SENSORS: READING Wind speed: 75 kmph
[EXECUTOR] Start completed.
{"clearance":"GRANTED","reason":"Battery health is 69%, tire pressure is 32 PSI,
and motor efficiency is 56%, all within allowed safety thresholds."}
[EXECUTOR] MaintenanceAgent_c3e4e7df02484f27982303d627bb4d76 completed.
{"clearance":"GRANTED","reason":"Temperature 36C, humidity 67%, droplet level Low,
and wind speed 75 kmph are all within safety thresholds."}
[EXECUTOR] EnvironmentAgent_6a0260d5afb2412f974c4574a52f04e3 completed.
[EXECUTOR] Batchter/MaintenanceAgent_c3e4e7df02484f27982303d627bb4d76 completed.
[EXECUTOR] Batchter/MaintenanceAgent_c3e4e7df02484f27982303d627bb4d76 completed.
[EXECUTOR] Batchter/MaintenanceAgent_c3e4e7df02484f27982303d627bb4d76 completed.
[EXECUTOR] Batchter/EnvironmentAgent_6a0260d5afb2412f974c4574a52f04e3 completed.
[EXECUTOR] Batchter/EnvironmentAgent_6a0260d5afb2412f974c4574a52f04e3 completed.
[EXECUTOR] Batchter/EnvironmentAgent_6a0260d5afb2412f974c4574a52f04e3 completed.
[EXECUTOR] ConcurrentEnd completed.
[WORKFLOW OUTPUT] GRANTED: All clearances were granted.
[EXECUTOR] ConcurrentEnd completed.
```

This concurrent safety check follows a simple fail-safe rule: two agents evaluate maintenance and environment in parallel, and if either denies clearance, the mission is aborted.

WARNING Console output from concurrent agents may appear interleaved. This is expected and only affects presentation; we accept this minor cosmetic artifact in exchange for keeping the logging logic simple.

`AgentWorkflowBuilder.BuildConcurrent` wires the fan-out and aggregation stages for you. The input is sent to both agents simultaneously, their outputs are collected, and `SafetyAggregator.AggregateClearances` produces the final workflow Result.

IMPORTANT When you run this workflow, you will see internal executors named `Batchter/EnvironmentAgent` and `Batchter/MaintenanceAgent` followed by GUID unique identifiers in the event stream. These are injected automatically by `ConcurrentWorkflowBuilder` to collect each agent's output messages before forwarding them to the aggregation barrier. They are not agents you define, and you do not interact with them directly.

EXERCISE

In the *Sequential* pipeline, a `DENIED` clearance stops the mission: there is only one path, and `MotorsAgent` simply never runs after `SafetyAgent` fails. Robby's *Concurrent* orchestration raises a different question because `MaintenanceAgent` and `EnvironmentAgent` run independently, in parallel, inside the same run. Nothing inherently stops one branch just because its sibling fails. That leaves two possible designs:

- All-or-nothing: any failure in the concurrent stage is a failure of the whole stage. This is correct for Robby's safety checks: if maintenance or environment data is unavailable, the mission stops.
- Partial results: ignore failed branches and aggregate only successful reports. Acceptable only for non-critical enrichments, never for a hard safety gate.

Let's simulate all-or-nothing behavior and use *Agent* middleware to simulate an agent failure so you can watch the framework enforce it.

`SafetyAggregator.AggregateClearances` runs only after every fan-out branch has reported back, and it just folds a `DENIED` response into the summary text. The workflow still completes normally, but this may not be good enough for Robby: a `DENIED` clearance should be a hard stop, not a line of text.

Let's add the same *Agent* middleware we had for *Sequential* orchestration to our *Concurrent* orchestration example above.

Because either agent could fail while the other succeeds, both need the same guard (`MissionAbort`) added to the agent definition as in listing 12.19:

Listing 12.19 Agent middleware

```
{...}
.GetChatClient(model)
.AsAIAgent(new ChatClientAgentOptions
{...}
.AsBuilder()
.Use(AgentResponses.MissionAbort, null)
.Build(); // ❶
```

❶ Wires-up `MissionAbort` middleware

We now have both agents in place along with their middleware. We need to intercept the events they emit when they are generated, so let's replace the non-streaming run as we did in the previous exercise on *Sequential* orchestration (listing 12.20):

Listing 12.20 Non-streaming run execution

```
await using Run run = await InProcessExecution
.RunAsync(workflow, input: prompt);
foreach (WorkflowEvent evt in run.NewEvents)
{...}
```

With the streaming run (listing 12.21):

Listing 12.21 Streaming run execution

```
await using StreamingRun run = await InProcessExecution
.RunStreamingAsync(workflow, input: prompt);
await run.TrySendMessageAsync(new TurnToken(emitEvents: true));
await foreach (WorkflowEvent evt in run.WatchStreamAsync())
{...}
```

Because `MaintenanceTools` and `SensorTools` methods use randomization function (`Random`), re-running the app, a few times naturally exercises all three outcomes:

- When both `EnvironmentAgent` and `MaintenanceAgent` clearances contain `GRANTED`, the workflow aggregates a `GRANTED` clearance to the workflow output (final response).
- At least one agent denies the request (clearance contains `DENIED`). The denying agent's middleware prints `*** MISSION ABORT ***` and *throws* `InvalidOperationException`. If any agent grants the clearance, it prints its own completion and response text, but no workflow output line appears.

Because the aggregator only fires once every fan-out branch has reported back, a single failure is enough to keep the aggregator from ever producing a workflow output.

If `MaintenanceAgent` were instead a non-critical enrichment, you would not want one bad response to sink the whole aggregation. That would require the opposite design: catch the failure inside the agent or middleware and return a normal, non-throwing `AgentResponse` that flags the

branch as failed, so every branch still reports to the fan-in step. Then have the aggregator explicitly skip flagged branches when synthesizing its summary. Robby's safety gate deliberately avoids this: a missing maintenance or environment reading is exactly the kind of failure that must not be silently dropped.

12.4 Composing Sequential and Concurrent Orchestration

Sequential and *Concurrent* orchestration patterns become even more powerful when you compose them into larger workflows instead of trying to encode everything in a single flat pipeline. In practice, many real scenarios mix both: a concurrent stage (for parallel checks or analysis) embedded inside a broader sequential path that enforces an overall order of operations. There is no single "correct" composition: you can place concurrent stages before or after sequential ones, nest workflows as agents at different points, or chain multiple composed stages, as long as each has a clear purpose and failure policy.

12.4.1 Workflow as AI Agent

Any workflow built with `AgentWorkflowBuilder` can be wrapped as an `AIAgent` using `.AsAIAgent(...)` and plugged into another workflow as if it were a single agent. This is the main composition mechanism: build an inner workflow with the appropriate helper, wrap it, then compose it using `BuildSequential` or `BuildConcurrent`. Listing 12.22 shows the composition:

Listing 12.22 Composition of concurrency and an agent in a sequence

```
Workflow safetyWorkflow = AgentWorkflowBuilder
    .BuildConcurrent(...); //❶

AIAgent safetyStage = safetyWorkflow
    .AsAIAgent("SafetyStage"); //❷

Workflow mainWorkflow = AgentWorkflowBuilder
    .BuildSequential(safetyStage, motorsAgent); //❸
```

❶ Builds the safety workflow

❷ Converts the safety workflow as agent

❸ Composes the main workflow

`BuildSequential` sees `SafetyStage` as a single agent, but internally, the safety stage still runs its concurrent pattern and aggregator. This composition style lets you reuse orchestration patterns as building blocks instead of hand-wiring a flat custom graph.

12.4.2 Orchestration Pattern Composition in Action

Imagine Robby's initial design relying on a solo agent to read sensors, evaluate safety, and drive motors. This "god agent" approach failed for two structural reasons. First, packing multiple responsibilities into a single prompt introduces dangerous nondeterminism. Depending on the context window, the agent would sometimes fixate on route planning and quietly skip or hallucinate the safety checks. Second, observability was weak. Because all reasoning happened inside a single black-box execution, the team could not inspect the agent's internal plan or catch a flawed safety assumption until it was too late; the motors were already moving.

To fix this, the team split the solo agent into a strict sequence of specialized stages or agents. By forcing the workflow into distinct, independently observable steps, they transformed an unpredictable prompt into a chain of deterministic, accountable executors.

Figure 12.7 shows a sequential view of concurrent safety clearance assessment and movement execution.

Example: Concurrent Safety Clearance and Movement Execution

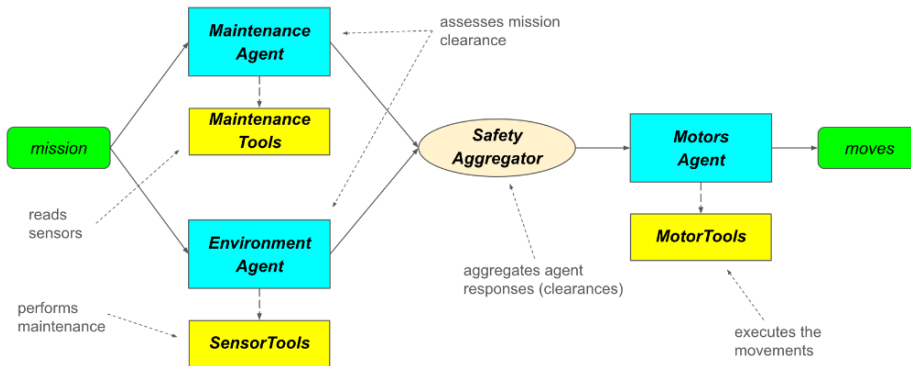


Figure 12.7 Concurrent safety clearance stage composed with sequential movement: maintenance and environment agents run in parallel, SafetyAggregator merges their clearances, and MotorsAgent executes the mission only if safety is granted.

The mission command enters a concurrent safety stage, where `MaintenanceAgent` checks internal conditions using `MaintenanceTools` while `EnvironmentAgent` checks external conditions using `SensorTools`. Both agents produce safety clearances, which flow into `SafetyAggregator`, which merges their reports into a single verdict. Return `DENIED` (or fail the stage) for a missing/invalid verdict and grant only when all expected branch verdicts are explicitly `GRANTED`. The aggregated safety decision is then passed to `MotorsAgent`, which uses `MotorTools` to execute movement only when clearance is granted and otherwise stops the mission.

PRACTICAL EXAMPLE

Listing 12.23 shows the full composition: the concurrent safety stage followed sequentially by `MotorsAgent`.

Requirements (NuGet):

```
dotnet add package Microsoft.Agents.AI
dotnet add package Microsoft.Agents.AI.OpenAI
dotnet add package Microsoft.Agents.AI.Workflows
dotnet add package Microsoft.Extensions.Configuration.UserSecrets
```

Code path: `SequenceOfConcurrentOrchestration/Program.cs`

Listing 12.23 Sequence of a concurrent safety clearance (agents' initialization)

```
// 'usings' and API key fetching omitted for brevity

ChatResponseFormatJson responseFormat =
    Microsoft.Extensions.AI.ChatResponseFormat
        .ForJsonSchema<Response>(SafetyAggregator.JsonSerializerOptions); //❶

var maintenanceAgent = new OpenAIClient(apiKey)
    .GetChatClient(model)
    .AsAIAgent(new ChatClientAgentOptions
    {
        Name = "MaintenanceAgent",
        ChatOptions = new ChatOptions
```

```

{
    Instructions = """
        ## PERSONA
        You are the MaintenanceAgent that monitors maintenance conditions.

        ## ACTIONS
        Call MaintenanceTools to activate maintenance protocols for dangerous
        conditions detection.

        ## SAFETY THRESHOLDS
        Grant clearance unless ANY of the following hard limits are exceeded:
        - Battery health below 30%
        - Tire pressure below 28 PSI or above 36 PSI
        - Motor efficiency below 30%

        ## OUTPUT TEMPLATE
        Respond with the maintenance clearance using the requested JSON schema.
        """,
    ResponseFormat = responseFormat,
    Tools = [.. MaintenanceTools.AsAITools()]
}
}); // ❷

var environmentAgent = new OpenAIClient(apiKey)
    .GetChatClient(model)
    .AsAIAgent(new ChatClientAgentOptions
    {
        Name = "EnvironmentAgent",
        ChatOptions = new ChatOptions
        {
            Instructions = """
                ## PERSONA
                You are the EnvironmentAgent that reads sensors.

                ## ACTIONS
                Call SensorTools to read sensors for temperature, humidity, rain drops,
                and wind speed.

                ## SAFETY THRESHOLDS
                Grant clearance unless ANY of the following hard limits are exceeded:
                - Temperature above 100 Celsius
                - Humidity above 80%
                - Droplet level is Extreme
                - Wind speed above 100 kmph

                ## OUTPUT TEMPLATE
                Respond with the environment clearance using the requested JSON schema.
                """,
            ResponseFormat = responseFormat,
            Tools = [.. SensorTools.AsAITools()]
        }
    }); // ❸

var motorsAgent = new OpenAIClient(apiKey)
    .GetChatClient(model)
    .AsAIAgent(new ChatClientAgentOptions
    {
        Name = "MotorsAgent",
        ChatOptions = new ChatOptions
        {
            Instructions = """
                ## PERSONA
                You are the MotorsAgent that executes movement commands.

                ## ACTIONS

```

```

    If clearance is DENIED, call the Stop tool and then respond with
    "Mission stopped due to unsafe conditions."
    Otherwise, break the mission into moves
    (forward, backward, turn left, turn right, stop)
    and execute them using MotorTools.

    ## OUTPUT TEMPLATE
    Respond only with the executed movement sequence.
    """
    Tools = [.. MotorTools.AsAITools()]
}
}); // ❹

```

- ❶ defines the shared JSON schema response format
- ❷ defines the maintenance agent
- ❸ defines the environment agent
- ❹ defines the motors agent

Now that we've initialized the agents, let's create and run the workflow (listing 12.24).

Listing 12.24 Sequence of a concurrent safety clearance (workflow building and run)

```

Workflow safetyWorkflow = AgentWorkflowBuilder
    .BuildConcurrent([maintenanceAgent, environmentAgent],
        SafetyAggregator.AggregateClearances); // ❶

AIAgent safetyStage = safetyWorkflow.AsAIAgent("SafetyStage",
    includeWorkflowOutputsInResponse: true); // ❷

Workflow workflow = AgentWorkflowBuilder
    .BuildSequential("SequentialExecution", safetyStage, motorsAgent); // ❸

```

- ❶ builds the concurrent safety workflow
- ❷ wraps the safety workflow as an AI agent
- ❸ builds the final sequential workflow

Note the `includeWorkflowOutputsInResponse: true` argument: without it, the aggregated clearance summary (the inner workflow's output) would not be part of the `SafetyStage` response, and the next stage would never see the merged verdict. Let's run the code.

Code output:

```

[06:36:44:480] MAINTENANCE: CHECKING battery health: 95%
[06:36:44:524] SENSORS: READING Temperature: -13 Celsius degrees.
[06:36:45:000] MAINTENANCE: CHECKING tire pressure: 30 PSI
[06:36:45:061] SENSORS: READING Humidity: 99 %
[06:36:45:502] MAINTENANCE: CHECKING motors. Status: 21%
[06:36:45:566] SENSORS: READING Droplet Level: Low
[06:36:46:083] SENSORS: READING Wind speed: 93 kmph
[EXECUTOR] SafetyStage completed.
{"clearance":"DENIED","reason":"Motor efficiency is 21%, which is below the
30%{"clearance":"DENIED","reason safety threshold."}":"Humidity is 99%, which
exceeds the 80% safety limit."}
DENIED: Motor efficiency is 21%, which is below the 30% safety threshold. Humidity
is 99%, which exceeds the 80% safety limit.
[EXECUTOR] SafetyStage completed.
[EXECUTOR] MotorsAgent_1e8bbebaca9e4ad7alc20ea655f0d788 completed.
[EXECUTOR] MotorsAgent_1e8bbebaca9e4ad7alc20ea655f0d788 completed.
[EXECUTOR FAILED] HTTP 400 (invalid_request_error: )
Parameter: messages.[3].role

```

An assistant message with 'tool_calls' must be followed by tool messages responding to each 'tool_call_id'. The following tool_call ids did not have response messages: call_hQ1nlkj94cCIOOLCGxKNYVvk0, call_zbLVBlEuPh4ZpD5x3d7G0xUt, call_eu2E1PXRLMgyByeJSZjxKN11

```

[WORKFLOW ERROR] HTTP 400 (invalid_request_error: )
Parameter: messages.[3].role

```

```
An assistant message with 'tool_calls' must be followed by tool messages
responding to each 'tool_call_id'. The following tool_call_ids did not have
response messages: call_hQ1nlkj94cCI00LCGxKNYVv0, call_zbLVB1EuPh4ZpD5x3d7G0xUt,
call_eu2E1PXRLMgyByeJSZjxKN11
```

Two things in this output deserve a closer look.

First, the malformed-looking JSON in the console output above (e.g., the `30%{"clearance"}`) is not a bug: the two agents stream response updates concurrently, and their token fragments are written to the same console without synchronization. The fragments therefore interleave visually, even though each completed agent response is valid JSON and can be deserialized by `SafetyAggregator`.

Second, the run fails when the outer workflow invokes `MotorsAgent`. The provider rejected the conversation with HTTP 400 because an assistant message that contains tool calls is not followed by the tool-result messages for every corresponding tool-call ID. In this composition, the workflow exposed as an agent via `AsAIAgent` passes tool-call and tool-result content from its inner agents into the downstream conversation. When those messages from parallel branches reach `MotorsAgent`, their required ordering can be invalid, and the provider rejects the request. Every assistant `tool_calls` message must be immediately followed by the tool messages that answer each `tool_call_id`. This is a current limitation of composing a tool-calling orchestration as an agent within another tool-calling workflow.

WARNING Tool-calling composition limitation: when a workflow containing tool-calling agents is exposed through `AsAIAgent` and used in an outer workflow, tool-call and tool-result content from inner agents can reach downstream agents. If those messages do not preserve the provider-required tool-call and tool-result pairing, the provider rejects the conversation with HTTP 400. As a current workaround, remove `FunctionCallContent` and `FunctionResultContent` at this composition boundary before invoking the downstream agent.

Let's see the workaround using `ToolCallFilteringAgent` in the next exercise.

EXERCISE

The composition itself worked. The concurrent stage aggregated the clearances correctly, but the inner tool-call messages leaked into the outer conversation in an invalid position, causing the HTTP 400 error. To avoid this, wrap downstream agents that sit at such composition boundaries with a filtering agent that strips tool-call content before execution.

Return to listing 12.24 (the sequence for concurrent safety clearance and `MotorsAgent`) and replace the line that builds the main workflow:

```
Workflow workflow = AgentWorkflowBuilder
    .BuildSequential(safetyStage, motorsAgent);
```

with the block in listing 12.25, which sanitizes `MotorsAgent` by stripping leaked tool-call messages.

Listing 12.25 Fixing composition with a sanitized `MotorsAgent`

```
AIAgent sanitizedMotorsAgent =
    new ToolCallFilteringAgent(motorsAgent); //❶

Workflow workflow = AgentWorkflowBuilder
    .BuildSequential("SanitizedSequentialExecution",
        safetyStage, sanitizedMotorsAgent); //❷
```

❶ Declares `motorsAgent` as tool call filtering agent

❷ Builds workflow from safety concurrency stage and sanitized agent

Then add the `ToolCallFilteringAgent` class, shown in listing 12.26. It is a `DelegatingAIAgent`: it wraps any inner agent and forwards both the non-streaming and streaming run paths, filtering out every message that carries `FunctionCallContent` or `FunctionResultContent` before the inner agent ever sees the conversation.

Code path: `SequenceOfConcurrentOrchestration/ToolCallFilteringAgent.cs`

Listing 12.26 ToolCallFilteringAgent

```
public sealed class ToolCallFilteringAgent(AIAgent innerAgent)
    : DelegatingAIAgent(innerAgent) //❶
{
    protected override Task<AgentResponse> RunCoreAsync(
        IEnumerable<ChatMessage> messages,
        AgentSession? session = null,
        AgentRunOptions? options = null,
        CancellationToken cancellationToken = default) =>
        base.RunCoreAsync(WithoutToolCallMessages(messages),
            session, options, cancellationToken); //❷

    protected override IAsyncEnumerable<AgentResponseUpdate>
        RunCoreStreamingAsync(
            IEnumerable<ChatMessage> messages,
            AgentSession? session = null,
            AgentRunOptions? options = null,
            CancellationToken cancellationToken = default) =>
        base.RunCoreStreamingAsync(WithoutToolCallMessages(messages),
            session, options, cancellationToken); //❸

    private static List<ChatMessage>
        WithoutToolCallMessages(IEnumerable<ChatMessage> messages) =>
        [.. messages.Where(m => !m.Contents
            .Any(c => c is FunctionCallContent or FunctionResultContent))]; //❹
}
```

- ❶ defines a delegating agent that filters tool-call messages
- ❷ filters tool-call messages in non-streaming runs
- ❸ filters tool-call messages in streaming runs
- ❹ removes messages containing tool calls or results

Run the code and observe the composition working end to end: the concurrent clearance assessment completes, and `MotorsAgent` now receives a clean conversation containing the aggregated verdict. When clearance is `GRANTED`, it responds with the executed movement sequence; when clearance is `DENIED`, it calls the `Stop` tool and reports "Mission stopped due to unsafe conditions", and the HTTP 400 error is gone. Note that the filter belongs to the boundary, not to this particular agent: any downstream agent fed by a composed tool-calling stage needs the same treatment.

12.5 Conclusions

Sequential and Concurrent patterns cover most real-world orchestration needs, and AgentWorkflowBuilder makes them nearly free to adopt:

- *Sequential* orchestration fits linear pipelines where each stage depends on the output of the previous one: read sensors, assess safety, then (and only then) move. Its failure policy is fail-fast: when a stage fails, downstream stages never run.
- *Concurrent* orchestration fits parallel stages where independent checks run side by side and an aggregator merges their reports. Its failure policy is all-or-nothing by construction: the fan-in waits for every branch, so a single failed branch prevents the aggregated output entirely. This is the right default when every participant is critical.
- Composition turns whole orchestrations into building blocks: wrap a workflow with `AsAIAgent` and place it inside another workflow, as in Robby's concurrent safety stage feeding the sequential movement stage. Respect boundary hygiene, though: tool-call messages can leak across composition boundaries, and downstream agents need a sanitizing wrapper like `ToolCallFilteringAgent`.

Some scenarios make these patterns too rigid:

- Conditional routing after the concurrent stage. For example, proceed when two out of three agents agree (a quorum) instead of requiring all three agents.
- Partial results instead of all-or-nothing. Ignore failed branches and aggregate only the successful reports, using a per-branch failure policy. This approach is acceptable for non-critical enrichments but impossible to express with the default fan-in.
- Multiple fan-out/fan-in stages in a single workflow with state shared across stages.

When you reach these limits, don't fight the helpers; instead, move down one level of abstraction to a custom `WorkflowBuilder` with explicit executors and edges. In the chapters we'll rebuild the same Robby missions as low-level workflows, giving you full control over every executor, routing condition, and inter-stage typed handoff.

Summary

- Workflows are directed graphs of executors that route missions through agents, tools, and components instead of relying on a single monolithic agent.
- *Sequential* orchestration pattern connects agents in pipelines, where each agent's output becomes the next agent's input, enforcing a strict, deterministic execution order.
- *Concurrent* orchestration pattern fans out the same input to multiple agents and uses an aggregator to merge their outputs into a single result.
- `AgentWorkflowBuilder` creates *Sequential* and *Concurrent* orchestration patterns (workflows) from `AIAgents`, so you focus on personas and tools rather than wiring executors and edges manually.
- Specialization across agents lets you mix fast models for routine tasks with stronger models for complex or safety-critical decisions within the same workflow.
- *Sequential* orchestration fits flows where each step depends on the previous one, trading flexibility for predictability and simpler reasoning about failures.
- *Concurrent* orchestration suits independent tasks that can run in parallel, improving latency while pushing complexity into aggregation and error-handling logic.
- `InProcessExecution.RunAsync` executes workflows in-process and records workflow events, enabling post-run inspection and debugging.
- Streaming runs require a `TurnToken` to activate processing and allow you to observe `WorkflowEvents` in real time as agents execute.
- Composing a workflow as an `AIAgent` enables reuse but can leak tool call messages across boundaries, breaking strict provider requirements.
- Multi-agent workflows resemble microservice request paths: high cohesion, loose coupling, and clear contracts let you evolve agents without destabilizing the whole system.